# biomedical-science-ai999x: capability guide

**Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `biomedical_science_ai999x.about()` for the same answer.

Every capability biomedical-science-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

## Quick start

```
import biomedical_science_ai999x

result = biomedical_science_ai999x.solve('bmed_pk_iv_bolus_concentration', D='0.0005 kg', V='0.04
m^3', k='2.77778e-05 1/s', t='21600 s')
result.show()
# C = 0.00686014 kg/m^3, fraction_remaining = 0.548811
```

## By topic

| topic | capabilities |
|---|---|
| Biomechanics | Static lever model of the forearm: muscle force and joint reaction, Poiseuille flow in a blood vessel: flow rate, mean velocity, wall shear stress, resistance, Two-element Windkessel: diastolic pressure decay, Windkessel models (two- and three-element): input impedance at a frequency, Kelvin-Voigt viscoelastic tissue model: creep strain under constant stress, Standard linear solid (Zener) tissue model: creep under constant stress, Standard linear solid (Zener) tissue model: stress relaxation at constant strain, Cardiac output by indicator dilution (Stewart-Hamilton), Single-compartment respiratory mechanics: resistance and compliance by least squares, Body mass index (a calculation, not medical advice), Basal metabolic rate by a stated regression (a calculation, not medical advice) |
| Diagnostic test statistics | ROC curve area of a diagnostic score (Mann-Whitney form), Optimal diagnostic cut-off by the Youden index, Sensitivity and specificity of a diagnostic score at a cut-off, Sensitivity and specificity with Wilson score intervals, Cohen's kappa for agreement between two raters or two tests |
| Epidemic models | SIR epidemic model: final size (final susceptible fraction and attack fraction), SEIR epidemic model: final size, SIR epidemic model: peak prevalence and the time of the peak, SEIR epidemic model: initial exponential growth rate from R0, Reproduction number from an epidemic growth rate (gamma-distributed generation interval), SIS epidemic model: prevalence at a time and the endemic level, Reed-Frost chain-binomial epidemic: distribution and mean of the final size, SEIR epidemic model: peak prevalence and the time of the peak |
| Epidemiological measures of association | Odds ratio of a 2x2 exposure-outcome table with Woolf (logit) interval, Relative risk (risk ratio) of a cohort 2x2 table with log (Katz) interval, Absolute risk reduction and number needed to treat with Wald intervals, Incidence rate ratio of two person-time groups with log-Wald interval, Standardized mortality (incidence) ratio with exact Poisson interval, Mantel-Haenszel pooled odds ratio over strata with Robins-Breslow-Greenland interval |
| Medical-imaging physics | X-ray attenuation through tissue layers (narrow monoenergetic beam), MRI spin-echo signal from T1, T2, TR and TE (simplified model), MRI inversion recovery: longitudinal magnetisation at TI and the null point, MRI spoiled gradient-echo signal (Ernst equation), Ultrasound reflection and transmission at a tissue interface (normal incidence), Ultrasound transmission through a matching layer (normal incidence), |

| topic | capabilities |
|---|---|
| | Imaging resolution: MTF of a Gaussian point-spread function, Diffusion MRI: apparent diffusion coefficient from signals at several b-values, MRI multi-echo T2 estimate (log-linear fit), Signal-to-noise ratio gained by signal averaging, MRI Ernst angle (the flip angle of the largest spoiled gradient-echo signal), CT number in Hounsfield units, Larmor (precession) frequency in a magnetic field, Ultrasound wavelength in tissue |
| Neuron models | Goldman-Hodgkin-Katz voltage equation: membrane potential from K, Na and Cl, Goldman-Hodgkin-Katz current equation: ionic current density through a membrane, Leaky integrate-and-fire neuron: firing rate under constant current, Passive membrane (leaky integrator): voltage at a time under constant current, Passive cable (dendrite or axon): steady-state voltage along it and input resistance, Hodgkin-Huxley model: steady-state gating variables at a membrane potential, Hodgkin-Huxley model: zero-current steady (resting) membrane potential, Hodgkin-Huxley model: potassium conductance after a voltage-clamp step, Postsynaptic potential of a passive membrane driven by an exponentially decaying synaptic current, FitzHugh-Nagumo neuron model: fixed point and its stability |
| Pharmacokinetic data analysis | Elimination rate constant and half-life from concentration data (log-linear fit), AUC from concentration-time data by the trapezoidal rule, Non-compartmental analysis of an IV bolus profile (AUC, lambda z, half-life, clearance, Vz), Absolute bioavailability from IV and oral concentration profiles (a model calculation, not medical advice) |
| Pharmacokinetic models | One-compartment IV bolus: concentration at a time (a model calculation, not medical advice), Area under the concentration curve of a one-compartment IV bolus (a model calculation, not medical advice), Oral dose, first-order absorption: concentration at a time (Bateman function) (a model calculation, not medical advice), Oral dose: time and height of the peak concentration (tmax, Cmax) (a model calculation, not medical advice), Repeated IV bolus doses: concentration after the nth dose (a model calculation, not medical advice), Repeated IV bolus dosing: steady-state peak, trough, average and accumulation factor (a model calculation, not medical advice), Repeated oral dosing: steady-state peak, its time and the trough (a model calculation, not medical advice), Constant-rate IV infusion: concentration at a time and steady state (a model calculation, not medical advice), Infusion of fixed duration: concentration during and after it (a model calculation, not medical advice), Loading dose and maintenance infusion rate of a one-compartment model (model calculation, not a dosing recommendation), Repeated short infusions: steady-state peak and trough (a model calculation, not medical advice), Constant infusion: time to reach a fraction of steady state (a model calculation, not medical advice), Michaelis-Menten (saturable) elimination: concentration at a time (a model calculation, not medical advice), Michaelis-Menten elimination: time to fall to a concentration (a model calculation, not medical advice), Michaelis-Menten elimination: steady-state concentration under a constant dosing rate (a model calculation, not medical advice), Two-compartment IV bolus: macro-constants and concentration from micro-constants (a model calculation, not medical advice), Two-compartment model: micro-constants, clearance and Vss from a biexponential curve (a model calculation, not medical advice), Hepatic clearance: well-stirred liver model (a model calculation, not medical advice), Hepatic clearance: parallel-tube (sinusoidal) liver model (a model calculation, not medical advice), Effect-compartment model: effect-site concentration and its peak after an IV bolus (a model calculation, not medical advice) |
| Survival analysis | Kaplan-Meier survival estimate at a time and median survival, Kaplan-Meier survival with log(-log) (exponential Greenwood) confidence interval, Nelson-Aalen cumulative hazard at a time, Log-rank test comparing two survival curves, Cox proportional-hazards model with one covariate: hazard ratio (Efron ties), Exponential survival model: hazard rate, mean and median survival (maximum likelihood), Hazard ratio of two exponential survival groups with Wald interval, Restricted mean survival time from the Kaplan-Meier curve |

## Outcomes

| outcome | meaning | what to do |
|---|---|---|
| SOLVED | a value with its unit, steps, assumptions and verification | use it; read the assumptions |

| outcome | meaning | what to do |
|---|---|---|
| NEEDS_INPUT | a required input is missing; `uncertainty['needs']` names it | add it (with its unit) and ask again |
| AMBIGUOUS | two readings or two physical branches give different answers | say which situation is meant |
| INVALID_INPUT | a unit of the wrong kind, an unknown input name, values with no valid solution | correct the input |
| OUT_OF_SCOPE | no certified capability computes this (or the structure is not modelled yet) | see the catalogue below |
| UNRECOGNIZED_FORMAT | the wording was not recognised; a supported template is offered in `uncertainty['try']` | use the template or solve() |
| INTERNAL_ERROR | a defect of the engine, never a property of the question | report it with the question |

## Result fields

| field | meaning |
|---|---|
| value, unit | the result in SI units |
| values | every output of a procedure (x1, x2, discriminant ...) |
| outcome | one of the seven outcomes above |
| steps | each equation used, its inputs (with where each came from) and its result |
| assumptions | conditions and declared defaults the answer relies on |
| verification | residual of every equation, dimensional consistency, independent checks |
| latex() | the derivation as LaTeX |
| show() | renders in a notebook, prints in a terminal |

## Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

## Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

| id | operation | inputs | outputs |
|---|---|---|---|
| bmed_pk_iv_bolus_concentration | One-compartment IV bolus: concentration at a time (a model calculation, not medical advice) | D, V, k, t | C, fraction_remaining |

| id | operation | inputs | outputs |
|---|---|---|---|
| bmed_pk_terminal_half_life_fit | Elimination rate constant and half-life from concentration data (log-linear fit) | t, C | k, t_half, C0 |
| bmed_pk_auc_iv_bolus | Area under the concentration curve of a one-compartment IV bolus (a model calculation, not medical advice) | D, CL, V, t | AUC_inf, AUC_0_t |
| bmed_pk_auc_trapezoid | AUC from concentration-time data by the trapezoidal rule | t, C, method | AUC |
| bmed_pk_nca_iv_bolus | Non-compartmental analysis of an IV bolus profile (AUC, lambda_z, half-life, clearance, Vz) | D, t, C, n_terminal | AUC_last, lambda_z, t_half, AUC_inf, CL, Vz |
| bmed_pk_oral_concentration | Oral dose, first-order absorption: concentration at a time (Bateman function) (a model calculation, not medical advice) | F, D, V, ka, k, t | C, AUC_0_t |
| bmed_pk_oral_peak | Oral dose: time and height of the peak concentration (tmax, Cmax) (a model calculation, not medical advice) | F, D, V, ka, k | t_max, C_max |
| bmed_pk_nth_dose_iv | Repeated IV bolus doses: concentration after the nth dose (a model calculation, not medical advice) | D, V, k, tau, n, t | C_max_n, C |
| bmed_pk_steady_state_iv | Repeated IV bolus dosing: steady-state peak, trough, average and accumulation factor (a model calculation, not medical advice) | D, V, k, tau | C_max_ss, C_min_ss, C_avg_ss, R_ac |
| bmed_pk_steady_state_oral | Repeated oral dosing: steady-state peak, its time and the trough (a model calculation, not medical advice) | F, D, V, ka, k, tau | C_max_ss, t_max_ss, C_min_ss |
| bmed_pk_infusion_concentration | Constant-rate IV infusion: concentration at a time and steady state (a model calculation, not medical advice) | R0, CL, V, t | C, C_ss |
| bmed_pk_short_infusion | Infusion of fixed duration: concentration during and after it (a model calculation, not medical advice) | R0, T_inf, CL, V, t | C, C_end |
| bmed_pk_loading_maintenance | Loading dose and maintenance infusion rate of a one-compartment model (model calculation, not a dosing recommendation) | C_target, V, CL | LD, R0 |
| bmed_pk_intermittent_infusion_steady_state | Repeated short infusions: steady-state peak and trough (a | R0, T_inf, tau, CL, V | C_max_ss, C_min_ss |

| id | operation | inputs | outputs |
|---|---|---|---|
| | model calculation, not medical advice) | | |
| bmed_pk_time_to_steady_state_fraction | Constant infusion: time to reach a fraction of steady state (a model calculation, not medical advice) | CL, V, f | t |
| bmed_pk_michaelis_menten_concentration | Michaelis-Menten (saturable) elimination: concentration at a time (a model calculation, not medical advice) | C0, Vmax, Km, V, t | C |
| bmed_pk_michaelis_menten_time | Michaelis-Menten elimination: time to fall to a concentration (a model calculation, not medical advice) | C0, C, Vmax, Km, V | t |
| bmed_pk_michaelis_menten_steady_state | Michaelis-Menten elimination: steady-state concentration under a constant dosing rate (a model calculation, not medical advice) | R0, Vmax, Km | C_ss |
| bmed_pk_two_compartment_bolus | Two-compartment IV bolus: macro-constants and concentration from micro-constants (a model calculation, not medical advice) | D, V1, k10, k12, k21, t | alpha, beta, A, B, C |
| bmed_pk_two_compartment_micro_constants | Two-compartment model: micro-constants, clearance and Vss from a biexponential curve (a model calculation, not medical advice) | D, A, alpha, B, beta | V1, k10, k12, k21, CL, Vss |
| bmed_pk_bioavailability_from_profiles | Absolute bioavailability from IV and oral concentration profiles (a model calculation, not medical advice) | D_iv, t_iv, C_iv, D_oral, t_oral, C_oral | AUC_iv, AUC_oral, F |
| bmed_pk_hepatic_clearance_well_stirred | Hepatic clearance: well-stirred liver model (a model calculation, not medical advice) | Q, fu, CL_int | CL_H, E |
| bmed_pk_hepatic_clearance_parallel_tube | Hepatic clearance: parallel-tube (sinusoidal) liver model (a model calculation, not medical advice) | Q, fu, CL_int | CL_H, E |
| bmed_pk_effect_site_concentration | Effect-compartment model: effect-site concentration and its peak after an IV bolus (a model calculation, not medical advice) | D, V, k, ke0, t | Ce, t_peak, Ce_peak |
| bmed_epi_sir_final_size | SIR epidemic model: final size (final susceptible fraction and attack fraction) | R0, s0, i0 | s_inf, attack |
| bmed_epi_seir_final_size | SEIR epidemic model: final size | R0, s0, e0, i0 | s_inf, attack |
| bmed_epi_sir_peak | SIR epidemic model: peak prevalence and the time of the | beta, gamma, s0, i0 | i_max, t_peak |

| id | operation | inputs | outputs |
|---|---|---|---|
| | peak | | |
| bmed_epi_seir_growth_rate | SEIR epidemic model: initial exponential growth rate from R0 | R0, sigma, gamma | r |
| bmed_epi_r_from_growth_rate | Reproduction number from an epidemic growth rate (gamma-distributed generation interval) | r, Tg_mean, Tg_sd | R |
| bmed_epi_sis_prevalence | SIS epidemic model: prevalence at a time and the endemic level | beta, gamma, i0, t | i, i_star |
| bmed_epi_reed_frost_final_size | Reed-Frost chain-binomial epidemic: distribution and mean of the final size | n, m, p | mean_final, P_no_spread, distribution |
| bmed_dx_roc_auc | ROC curve area of a diagnostic score (Mann-Whitney form) | scores_pos, scores_neg | AUC, U |
| bmed_dx_youden_cutoff | Optimal diagnostic cut-off by the Youden index | scores_pos, scores_neg | threshold, sensitivity, specificity, J |
| bmed_dx_threshold_sensitivity_specificity | Sensitivity and specificity of a diagnostic score at a cut-off | scores_pos, scores_neg, threshold | sensitivity, specificity, TP, FN, TN, FP |
| bmed_dx_sensitivity_specificity_ci | Sensitivity and specificity with Wilson score intervals | TP, FN, TN, FP, level | sensitivity, sens_lower, sens_upper, specificity, spec_lower, spec_upper |
| bmed_odds_ratio_ci | Odds ratio of a 2x2 exposure-outcome table with Woolf (logit) interval | a, b, c, d, level | OR, lower, upper |
| bmed_relative_risk_ci | Relative risk (risk ratio) of a cohort 2x2 table with log (Katz) interval | a, b, c, d, level | RR, lower, upper |
| bmed_nnt_ci | Absolute risk reduction and number needed to treat with Wald intervals | events_t, n_t, events_c, n_c, level | ARR, ARR_lower, ARR_upper, NNT, NNT_lower, NNT_upper |
| bmed_incidence_rate_ratio_ci | Incidence rate ratio of two person-time groups with log-Wald interval | events1, time1, events2, time2, level | IRR, lower, upper |
| bmed_smr_exact_ci | Standardized mortality (incidence) ratio with exact Poisson interval | observed, expected, level | SMR, lower, upper |
| bmed_mantel_haenszel_odds_ratio | Mantel-Haenszel pooled odds ratio over strata with Robins-Breslow-Greenland interval | tables, level | OR_MH, lower, upper |
| bmed_cohens_kappa | Cohen's kappa for agreement between two raters or two tests | counts | kappa, p_observed |
| bmed_km_survival_median | Kaplan-Meier survival estimate at a time and median survival | times, events, t | S_t, median |
| bmed_km_confidence_interval | Kaplan-Meier survival with log(-log) (exponential Greenwood) confidence interval | times, events, t, level | S_t, lower, upper |

| id | operation | inputs | outputs |
|---|---|---|---|
| bmed_nelson_aalen_hazard | Nelson-Aalen cumulative hazard at a time | times, events, t | H_t |
| bmed_logrank_test | Log-rank test comparing two survival curves | times1, events1, times2, events2 | chi_square, p_value |
| bmed_cox_hazard_ratio | Cox proportional-hazards model with one covariate: hazard ratio (Efron ties) | times, events, x, level | beta, HR, se, HR_lower, HR_upper |
| bmed_exponential_survival_fit | Exponential survival model: hazard rate, mean and median survival (maximum likelihood) | times, events | hazard, mean, median |
| bmed_exponential_hazard_ratio | Hazard ratio of two exponential survival groups with Wald interval | times1, events1, times2, events2, level | HR, lower, upper |
| bmed_restricted_mean_survival_time | Restricted mean survival time from the Kaplan-Meier curve | times, events, tau | RMST |
| bmed_elbow_lever_muscle_force | Static lever model of the forearm: muscle force and joint reaction | W, d_load, W_arm, d_arm, d_muscle, theta | F_muscle, R_x, R_y, R |
| bmed_poiseuille_vessel_flow | Poiseuille flow in a blood vessel: flow rate, mean velocity, wall shear stress, resistance | dP, r, L, mu | Q, v_mean, tau_wall, R_hyd |
| bmed_windkessel_diastolic_decay | Two-element Windkessel: diastolic pressure decay | P0, Pv, R, C, t | P |
| bmed_windkessel_input_impedance | Windkessel models (two- and three-element): input impedance at a frequency | model, R, C, f, Zc | Z_mod, phase_deg |
| bmed_kelvin_voigt_creep | Kelvin-Voigt viscoelastic tissue model: creep strain under constant stress | sigma, E, eta, t | strain |
| bmed_sls_creep | Standard linear solid (Zener) tissue model: creep under constant stress | sigma, E_inf, E1, eta, t | strain, strain_inf |
| bmed_sls_stress_relaxation | Standard linear solid (Zener) tissue model: stress relaxation at constant strain | eps0, E_inf, E1, eta, t | stress, stress_inf |
| bmed_indicator_dilution_cardiac_output | Cardiac output by indicator dilution (Stewart-Hamilton) | m, t, C | AUC, CO |
| bmed_respiratory_mechanics_fit | Single-compartment respiratory mechanics: resistance and compliance by least squares | P, Q, V | R, C, P0 |
| bmed_ghk_voltage | Goldman-Hodgkin-Katz voltage equation: membrane potential from K, Na and Cl | T, PK, PNa, PCl, K_o, K_i, Na_o, Na_i, Cl_o, Cl_i | V_m |
| bmed_ghk_current_density | Goldman-Hodgkin-Katz current equation: ionic current density through a membrane | P, z, C_in, C_out, V, T | I |

| id | operation | inputs | outputs |
|---|---|---|---|
| bmed_lif_firing_rate | Leaky integrate-and-fire neuron: firing rate under constant current | tau_m, R_m, I, V_rest, V_th, V_reset, t_ref | rate, ISI |
| bmed_lif_subthreshold_voltage | Passive membrane (leaky integrator): voltage at a time under constant current | tau_m, R_m, I, V_rest, V0, t | V |
| bmed_cable_steady_state | Passive cable (dendrite or axon): steady-state voltage along it and input resistance | Rm, Ri, d, x, V0, end, L | V_x, R_in |
| bmed_hh_gating_steady_state | Hodgkin-Huxley model: steady-state gating variables at a membrane potential | V | m_inf, h_inf, n_inf |
| bmed_hh_resting_potential | Hodgkin-Huxley model: zero-current steady (resting) membrane potential | gNa, gK, gL, ENa, EK, EL | V_rest |
| bmed_hh_potassium_conductance_clamp | Hodgkin-Huxley model: potassium conductance after a voltage-clamp step | gK_max, V_hold, V_step, t | g_K, n |
| bmed_postsynaptic_potential | Postsynaptic potential of a passive membrane driven by an exponentially decaying synaptic current | tau_m, tau_s, R_m, I0, t | V_t, t_peak, V_peak |
| bmed_fitzhugh_nagumo_fixed_point | FitzHugh-Nagumo neuron model: fixed point and its stability | a, b, eps, I | v_star, w_star, trace, determinant, stable |
| bmed_xray_attenuation_layers | X-ray attenuation through tissue layers (narrow monoenergetic beam) | I0, mu_rho, rho, x | T, I |
| bmed_mri_spin_echo_signal | MRI spin-echo signal from T1, T2, TR and TE (simplified model) | M0, TR, TE, T1, T2 | S |
| bmed_mri_inversion_recovery | MRI inversion recovery: longitudinal magnetisation at TI and the null point | T1, TR, TI | Mz_ratio, TI_null |
| bmed_mri_spoiled_gradient_echo | MRI spoiled gradient-echo signal (Ernst equation) | M0, TR, TE, T1, T2star, alpha | S |
| bmed_ultrasound_interface | Ultrasound reflection and transmission at a tissue interface (normal incidence) | Z1, Z2 | R, T, r_p, t_p |
| bmed_ultrasound_matching_layer | Ultrasound transmission through a matching layer (normal incidence) | Z1, Zm, Z2, d, c_m, f | T, kd |
| bmed_gaussian_psf_mtf | Imaging resolution: MTF of a Gaussian point-spread function | fwhm, f | MTF, f50 |
| bmed_mri_adc_fit | Diffusion MRI: apparent diffusion coefficient from signals at several b-values | b, S | ADC, S0 |
| bmed_mri_t2_fit | MRI multi-echo T2 estimate (log-linear fit) | TE, S | T2, S0 |

| id | operation | inputs | outputs |
|---|---|---|---|
| bmed_snr_averaging | Signal-to-noise ratio gained by signal averaging | SNR1, N, SNR_target | SNR, gain, N |
| bmed_mri_ernst_angle | MRI Ernst angle (the flip angle of the largest spoiled gradient-echo signal) | TR, T1 | alpha, alpha_deg, S_rel |
| bmed_epi_seir_peak | SEIR epidemic model: peak prevalence and the time of the peak | beta, sigma, gamma, s0, e0, i0 | t_peak, i_max, s_at_peak, e_at_peak |
| bmed_bmi | Body mass index (a calculation, not medical advice) | m, h | BMI, BMI_prime |
| bmed_bmr | Basal metabolic rate by a stated regression (a calculation, not medical advice) | m, h, age, sex, equation | BMR_kcal_day, BMR_W, BMR_MJ_day |
| bmed_hounsfield_units | CT number in Hounsfield units | mu, mu_water, mu_air | HU, mu_rel |
| bmed_larmor_frequency | Larmor (precession) frequency in a magnetic field | B, gamma_bar | f, f_MHz, omega |
| bmed_ultrasound_wavelength | Ultrasound wavelength in tissue | f, c | wavelength, wavelength_mm, period, k |

### One-compartment IV bolus: concentration at a time (a model calculation, not medical advice) — `bmed_pk_iv_bolus_concentration`

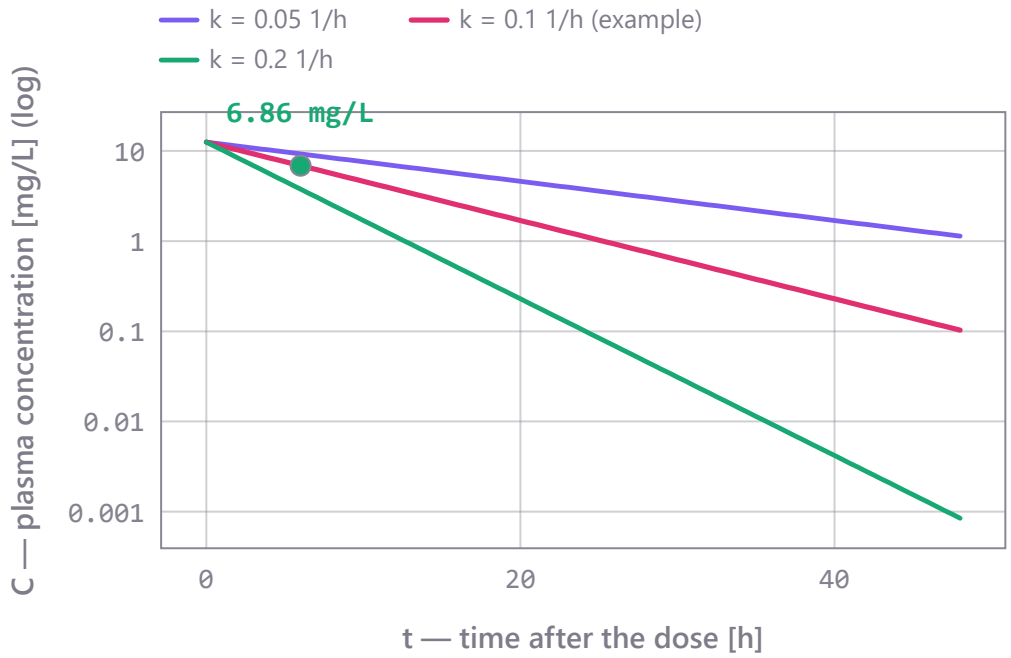
The plasma concentration of a one-compartment model a time t after an intravenous bolus dose, with first-order elimination; also the fraction of the dose remaining.

$$C(t) = \frac{D}{V}\, e^{-kt}$$

| input | meaning | condition |
|---|---|---|
| `D` | bolus dose | |
| `V` | volume of distribution | |
| `k` | elimination rate constant | |
| `t` | time after the dose | |

| output | meaning |
|---|---|
| `C` | concentration at time t |
| `fraction_remaining` | fraction of the dose still in the body |

- **Method:** C(t) = (D/V) exp(-k t)
- **Assumes:** one compartment, instantaneous mixing; first-order elimination; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




One-compartment intravenous bolus: plasma concentration over 48 h (log axis, so each line is straight) for D = 500 mg, V = 40 L and three elimination rate constants (the example k = 0.1 1/h, half and double), 97 runs each. Marked: at t = 6 h, C = 6.86015 mg/L, fraction of the dose remaining 0.548812.

```
biomedical_science_ai999x.solve('bmed_pk_iv_bolus_concentration', D=0.0005, V=0.04,
k=2.777777777777778e-05, t=21600.0)
# -> C = 0.00686015 kg/m^3, fraction_remaining = 0.548812   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_iv_bolus_concentration', D=0.001, V=0.05,
k=6.944444444444444e-05, t=5400.0)
# -> C = 0.0137458 kg/m^3, fraction_remaining = 0.687289   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| IV bolus concentration: D = 500 mg, V = 40 L, k = 0.1 1/h, t = 6 h | SOLVED | C = 0.00686015 kg/m^3, fraction_remaining = 0.548812 |
| IV bolus concentration: D = 1 g, V = 50 L, k = 0.25 1/h, t = 90 min | SOLVED | C = 0.0137458 kg/m^3, fraction_remaining = 0.687289 |
| IV bolus concentration: V = 40 L, k = 0.1 1/h, t = 6 h | NEEDS_INPUT | bmed_pk_iv_bolus_concentration needs D (bolus dose). Example: solve('bmed_pk_iv_bolus_concentration', D='0.0005 kg', V=' |
| IV bolus concentration: D = 500 mg, V = 40 L, k = 0.1 1/h, t = 6 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| IV bolus concentration: D = 0.5 g, V = 40000 mL, k = 0.1 1/h, t = 360 min | SOLVED | C = 0.00686015 kg/m^3, fraction_remaining = 0.548812 |
| IV bolus concentration: D = 500 mg, V = 40 L, k = -0.1 1/h, t = 6 h | INVALID_INPUT | One-compartment IV bolus: concentration at a time (a model calculation, not medical advice): the dose D, the volume V an |

| question | outcome | result |
|---|---|---|
| IV bolus concentration: D = 500 mg, V = 40 L, k = 0.1 1/h, t = 6 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Elimination rate constant and half-life from concentration data (log-linear fit) — `bmed_pk_terminal_half_life_fit`

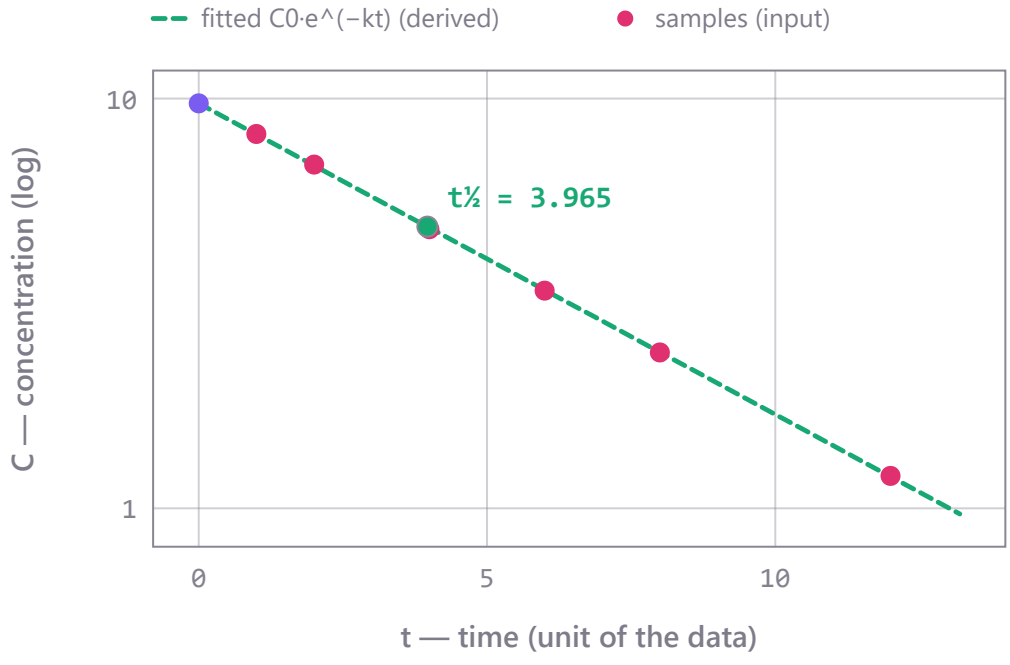
The first-order elimination rate constant, half-life and back-extrapolated concentration from a straight-line least-squares fit of ln C against time (mono-exponential decline).

$$\ln C = \ln C_0 - kt, \quad t_{1/2} = \frac{\ln 2}{k}$$

| input | meaning | condition |
|---|---|---|
| `t` | sampling times, in one time unit (results are per that unit) | |
| `C` | concentrations at those times, in one concentration unit (C0 is in that unit) | |

| output | meaning |
|---|---|
| `k` | elimination rate constant, per time unit of t |
| `t_half` | half-life, in the time unit of t |
| `C0` | back-extrapolated concentration at $t = 0$, in the unit of C |

- **Method:** least-squares line ln $C = ln\, C_0 - k$ t; t_half = ln $2/k$
- **Assumes:** mono-exponential decline over the sampled times; all concentrations positive; unweighted least squares on ln C; at least three points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Log-linear fit of the example's 6 concentration samples (log axis). The dashed line is C0·exp(−k t) drawn from the returned k = 0.174805 per time unit and C0 = 9.73505 (derived). Marked: the half-life t½ = 3.96526 where the line reaches C0/2; dot: back-extrapolated C0.

```
biomedical_science_ai999x.solve('bmed_pk_terminal_half_life_fit', t=[1, 2, 4, 6, 8, 12], C=[8.2, 6.9, 4.8, 3.4, 2.4, 1.2])
# -> k = 0.174805, t_half = 3.96526, C0 = 9.73505   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_terminal_half_life_fit', t=[0.5, 1, 3, 5], C=[95, 80, 41, 22])
# -> k = 0.325426, t_half = 2.12997, C0 = 110.832   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Terminal half-life fit: t = [1, 2, 4, 6, 8, 12], C = [8.2, 6.9, 4.8, 3.4, 2.4, 1.2] | SOLVED | k = 0.174805, t_half = 3.96526, C0 = 9.73505 |
| Terminal half-life fit: t = [0.5, 1, 3, 5], C = [95, 80, 41, 22] | SOLVED | k = 0.325426, t_half = 2.12997, C0 = 110.832 |
| Terminal half-life fit: C = [8.2, 6.9, 4.8, 3.4, 2.4, 1.2] | NEEDS_INPUT | bmed_pk_terminal_half_life_fit needs t (sampling times, in one time unit (results are per that unit)). Example: solve('b |
| Terminal half-life fit: t = [1, 2, 4, 6, 8, 12], C = [8.2, 6.9, 4.8, 3.4, 2.4, 1.2], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Terminal half-life fit: t = [1, 2, 4], C = [8.2, 9.1, 10.5] | INVALID_INPUT | Elimination rate constant and half-life from concentration data (log-linear fit): the concentrations do not decline (the |
| Terminal half-life fit: t = [1, 2, 4, 6], C = [8.2, 6.9, 4.8, 3.4] mg/L | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Area under the concentration curve of a one-compartment IV bolus (a model calculation, not medical advice) — `bmed_pk_auc_iv_bolus`

The area under the plasma concentration-time curve from 0 to infinity and from 0 to t after an IV bolus in a one-compartment model.

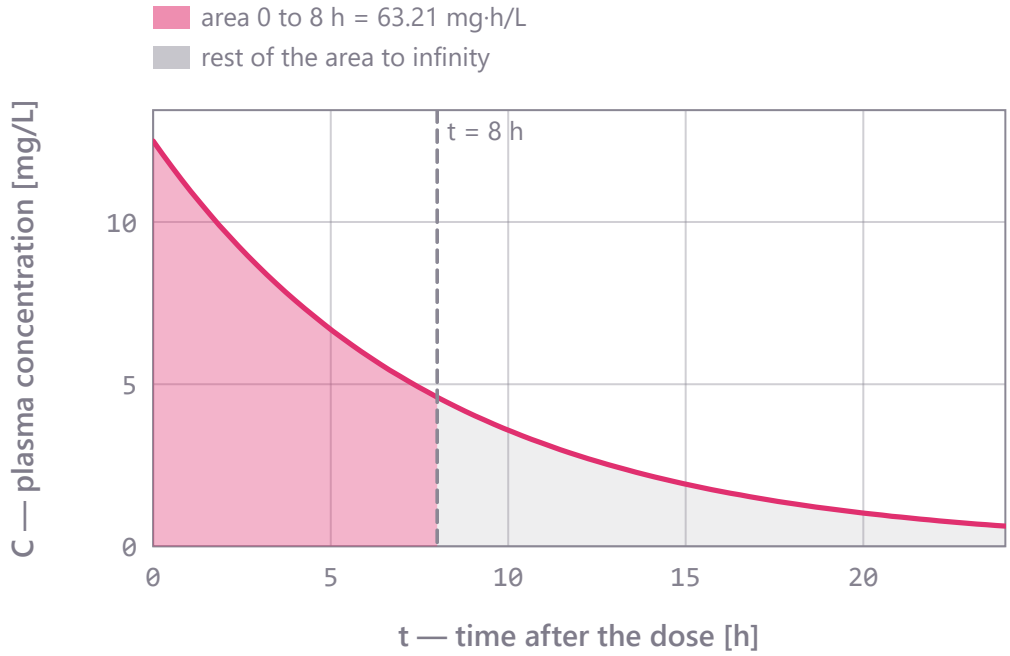
$$AUC_{0-\infty} = \frac{D}{CL}, \quad AUC_{0-t} = \frac{D}{CL}\left(1 - e^{-CL\,t/V}\right)$$

| input | meaning | condition |
|---|---|---|
| `D` | bolus dose | |
| `CL` | clearance | |
| `V` | volume of distribution | |
| `t` | end of the partial area | |

| output | meaning |
|---|---|
| `AUC_inf` | area from 0 to infinity |
| `AUC_0_t` | area from 0 to t |

- **Method:** AUC(0-inf) = D/CL; AUC(0-t) = (D/CL)(1 - exp(-CL t / V))
- **Assumes:** one compartment; first-order elimination $k = CL/V$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Area under the concentration curve of an intravenous bolus (D = 500 mg, CL = 5 L/h, V = 40 L): the curve is the package's bolus concentration with k = CL/V = 0.125 1/h, 91 runs over 24 h. Marked: AUC from 0 to 8 h = 63.2121 mg·h/L (shaded) of AUC to infinity = 100 mg·h/L.

```
biomedical_science_ai999x.solve('bmed_pk_auc_iv_bolus', D=0.0005, CL=1.3888888888888887e-06,
V=0.04, t=28800.0)
# -> AUC_inf = 360 kg*s/m^3, AUC_0_t = 227.563 kg*s/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_auc_iv_bolus', D=9.999999999999999e-05, CL=2e-06, V=0.03,
t=86400.0)
# -> AUC_inf = 50 kg*s/m^3, AUC_0_t = 49.8424 kg*s/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| AUC of an IV bolus: D = 500 mg, CL = 5 L/h, V = 40 L, t = 8 h | SOLVED | AUC_inf = 360 kg*s/m^3, AUC_0_t = 227.563 kg*s/m^3 |
| AUC of an IV bolus: D = 100 mg, CL = 120 mL/min, V = 30 L, t = 1 day | SOLVED | AUC_inf = 50 kg*s/m^3, AUC_0_t = 49.8424 kg*s/m^3 |
| AUC of an IV bolus: CL = 5 L/h, V = 40 L, t = 8 h | NEEDS_INPUT | bmed_pk_auc_iv_bolus needs D (bolus dose). Example: solve('bmed_pk_auc_iv_bolus', D='0.0005 kg', CL='1.38889e-06 m^3/s', |
| AUC of an IV bolus: D = 500 mg, CL = 5 L/h, V = 40 L, t = 8 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| AUC of an IV bolus: D = 500 mg, CL = 0 L/h, V = 40 L, t = 8 h | INVALID_INPUT | Area under the concentration curve of a one-compartment IV bolus (a model calculation, not medical advice): the dose D, |

### AUC from concentration-time data by the trapezoidal rule — `bmed_pk_auc_trapezoid`

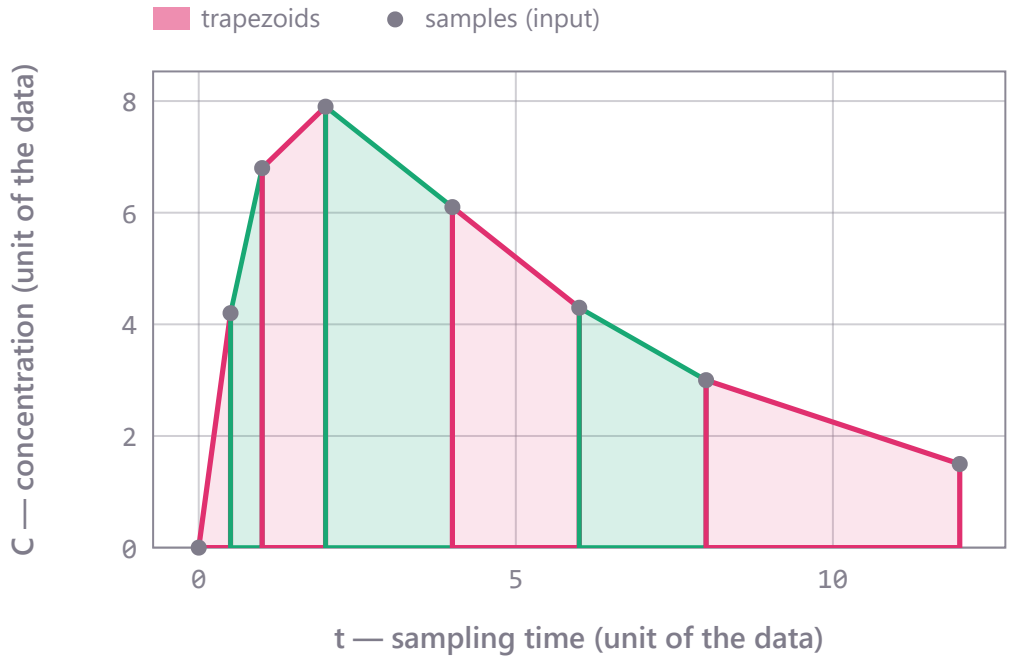
The area under a sampled concentration-time curve from the first to the last sample, by the linear trapezoidal rule or the linear-up/log-down rule (stated).

$$AUC = \sum_i \frac{(C_i + C_{i+1})\,\Delta t_i}{2} \quad \text{or} \quad \frac{(C_i - C_{i+1})\,\Delta t_i}{\ln(C_i / C_{i+1})}$$

| input | meaning | condition |
|---|---|---|
| `t` | sampling times, strictly increasing, in one time unit | |
| `C` | concentrations at those times, in one concentration unit | |
| `method` | linear trapezoids, or linear while rising and logarithmic while falling | |

| output | meaning |
|---|---|
| `AUC` | area from the first to the last sample, in (unit of C) x (unit of t) |

- **Method:** sum of segment areas: linear $(C_1 + C_2)\,dt/2$; log-down $(C_1 - C_2)$ dt / ln(C1/C2)
- **Assumes:** times strictly increasing; concentrations >= 0; log-down segments need both concentrations positive (otherwise linear)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The 8 concentration samples of the example (input) and the 7 trapezoids whose areas add up to the package's linear AUC = 51.85 (concentration × time units). With linear trapezoids while rising and logarithmic ones while falling the package gives AUC = 51.2462.

```
biomedical_science_ai999x.solve('bmed_pk_auc_trapezoid', t=[0, 0.5, 1, 2, 4, 6, 8, 12], C=[0, 4.2,
6.8, 7.9, 6.1, 4.3, 3.0, 1.5], method='linear')
# -> AUC = 51.85   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_auc_trapezoid', t=[0, 0.5, 1, 2, 4, 6, 8, 12], C=[0, 4.2,
6.8, 7.9, 6.1, 4.3, 3.0, 1.5], method='linear_up_log_down')
# -> AUC = 51.2462   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| AUC by the trapezoidal rule: t = [0, 0.5, 1, 2, 4, 6, 8, 12], C = [0, 4.2, 6.8, 7.9, 6.1, 4.3, 3, 1.5], method = linear | SOLVED | AUC = 51.85 |
| AUC by the trapezoidal rule: t = [0, 0.5, 1, 2, 4, 6, 8, 12], C = [0, 4.2, 6.8, 7.9, 6.1, 4.3, 3, 1.5], method = linear_up_log_down | SOLVED | AUC = 51.2462 |
| AUC by the trapezoidal rule: C = [0, 4.2, 6.8, 7.9, 6.1, 4.3, 3, 1.5], method = linear | NEEDS_INPUT | bmed_pk_auc_trapezoid needs t (sampling times, strictly increasing, in one time unit). Example: solve('bmed_pk_auc_trape |
| AUC by the trapezoidal rule: t = [0, 0.5, 1, 2, 4, 6, 8, 12], C = [0, 4.2, 6.8, 7.9, 6.1, 4.3, 3, 1.5], method = linear, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| AUC by the trapezoidal rule: t = [0, 1, 2], C = [0, 4, 3] | NEEDS_INPUT | bmed_pk_auc_trapezoid needs method (linear trapezoids, or linear while rising and logarithmic while falling). Example: s |
| AUC by the trapezoidal rule: t = [0, 2, 1], C = [0, 4, 3], method = linear | INVALID_INPUT | AUC from concentration-time data by the trapezoidal rule: the times must increase strictly. |

## Non-compartmental analysis of an IV bolus profile (AUC, lambda_z, half-life, clearance, Vz) — `bmed_pk_nca_iv_bolus`

Non-compartmental parameters from a concentration-time profile after an IV bolus: AUC to the last sample (linear trapezoids), terminal rate constant from the last n points, half-life, AUC to infinity, clearance and terminal volume.

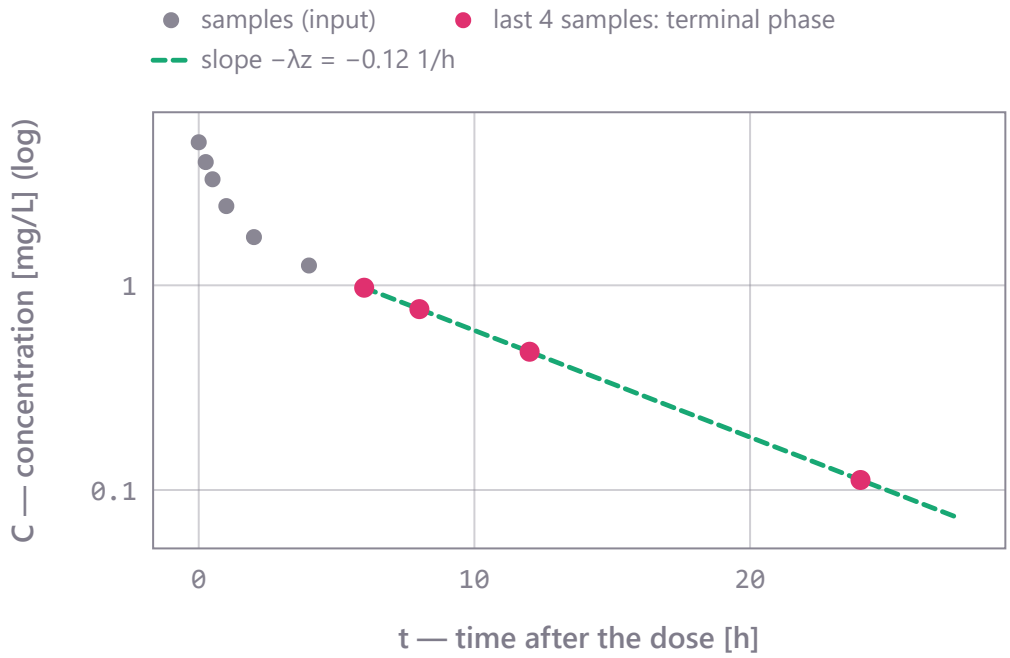
$$AUC_{0-\infty} = AUC_{0-t_{last}} + \frac{C_{last}}{\lambda_z}, \quad CL = \frac{D}{AUC_{0-\infty}}, \quad V_z = \frac{CL}{\lambda_z}$$

| input | meaning | condition |
|---|---|---|
| `D` | IV bolus dose | |
| `t` | sampling times in h, the first at 0 (measured or back-extrapolated C0) | |
| `C` | concentrations in mg/L at those times | |
| `n_terminal` | number of last points used for the terminal slope (3 to n - 1) | |

| output | meaning |
|---|---|
| `AUC_last` | AUC from 0 to the last sample, $mgh/L$ |
| `lambda_z` | terminal rate constant, $1/h$ |
| `t_half` | terminal half-life, h |
| `AUC_inf` | AUC from 0 to infinity (observed C_last / lambda_z added), $mgh/L$ |
| `CL` | clearance D / AUC_inf, L/h |

| output | meaning |
|---|---|
| Vz | terminal volume CL / lambda_z, L |

- **Method:** AUC by trapezoids; ln $C = a - lambda_z$ t on the last points; AUC_inf = AUC_last + C_last/lambda_z; CL = D/AUC_inf; $Vz = CL/lambda_z$
- **Assumes:** linear trapezoidal rule; lambda_z by unweighted least squares on ln C of the last n points; extrapolation with the observed last concentration; times in h, concentrations in mg/L
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Non-compartmental analysis of the example's 10 samples after an intravenous bolus of 100 mg (log axis). The dashed line has the package's terminal slope λz = 0.120166 1/h from the last 4 samples, drawn through the last sample (derived). Returned: t½ = 5.76825 h, AUC_last = 18.4825 mg·h/L, AUC_inf = 19.4145 mg·h/L, CL = 5.15078 L/h, Vz = 42.8639 L.

```
biomedical_science_ai999x.solve('bmed_pk_nca_iv_bolus', D=9.999999999999999e-05, t=[0, 0.25, 0.5,
1, 2, 4, 6, 8, 12, 24], C=[5.0, 4.0, 3.3, 2.44, 1.72, 1.25, 0.974, 0.766, 0.474, 0.112],
n_terminal=4)
# -> AUC_last = 18.4825, lambda_z = 0.120166, t_half = 5.76825, AUC_inf = 19.4145, CL = 5.15078, Vz
= 42.8639   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_nca_iv_bolus', D=0.00025, t=[0, 1, 2, 4, 8, 12], C=[12.5,
9.8, 7.9, 5.1, 2.2, 0.95], n_terminal=3)
# -> AUC_last = 53.9, lambda_z = 0.210067, t_half = 3.29965, AUC_inf = 58.4224, CL = 4.27918, Vz =
20.3706   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Non-compartmental analysis of an IV bolus: D = 100 mg, t = [0, 0.25, 0.5, 1, 2, 4, 6, 8, 12, 24], C = [5, 4, 3.3, 2.44, 1.72, 1.25, 0.974, 0.766, 0.474, 0.112], n_terminal = 4 | SOLVED | AUC_last = 18.4825, lambda_z = 0.120166, t_half = 5.76825, AUC_inf = 19.4145, CL = 5.15078, Vz = 42.8639 |

| question | outcome | result |
|---|---|---|
| Non-compartmental analysis of an IV bolus: D = 250 mg, t = [0, 1, 2, 4, 8, 12], C = [12.5, 9.8, 7.9, 5.1, 2.2, 0.95], n_terminal = 3 | SOLVED | AUC_last = 53.9, lambda_z = 0.210067, t_half = 3.29965, AUC_inf = 58.4224, CL = 4.27918, Vz = 20.3706 |
| Non-compartmental analysis of an IV bolus: t = [0, 0.25, 0.5, 1, 2, 4, 6, 8, 12, 24], C = [5, 4, 3.3, 2.44, 1.72, 1.25, 0.974, 0.766, 0.474, 0.112], n_terminal = 4 | NEEDS_INPUT | bmed_pk_nca_iv_bolus needs D (IV bolus dose). Example: solve('bmed_pk_nca_iv_bolus', D='0.0001 kg', t=[0, 0.25, 0.5, 1, |
| Non-compartmental analysis of an IV bolus: D = 100 mg, t = [0, 0.25, 0.5, 1, 2, 4, 6, 8, 12, 24], C = [5, 4, 3.3, 2.44, 1.72, 1.25, 0.974, 0.766, 0.474, 0.112], n_terminal = 4, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Non-compartmental analysis of an IV bolus: D = 100 mg, t = [1, 2, 4, 8, 12], C = [4, 3, 2, 1, 0.5], n_terminal = 3 | INVALID_INPUT | Non-compartmental analysis of an IV bolus profile (AUC, lambda_z, half-life, clearance, Vz): the first sample must be at |

### Oral dose, first-order absorption: concentration at a time (Bateman function) (a model calculation, not medical advice) — `bmed_pk_oral_concentration`

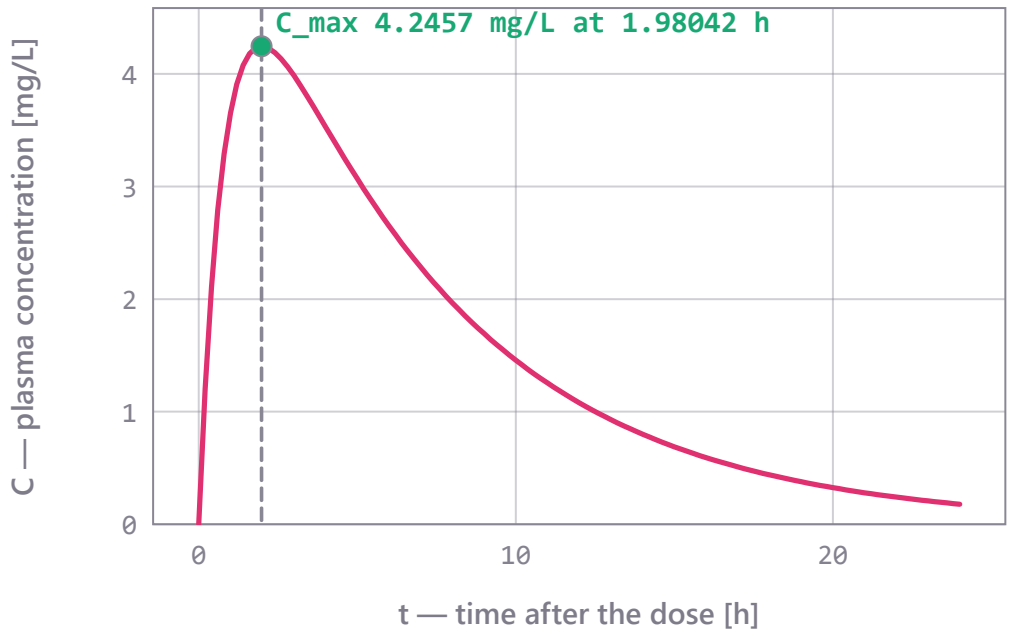
The plasma concentration and partial AUC of a one-compartment model with first-order absorption (rate ka) and elimination (rate k) after a single oral dose with bioavailability F.

$$C(t) = \frac{FDk_a}{V(k_a - k)}\left(e^{-kt} - e^{-k_a t}\right)$$

| input | meaning | condition |
|---|---|---|
| `F` | bioavailable fraction (0 < F <= 1; 80% or 0.8) | |
| `D` | oral dose | |
| `V` | volume of distribution | |
| `ka` | absorption rate constant | |
| `k` | elimination rate constant | |
| `t` | time after the dose | |

| output | meaning |
|---|---|
| `C` | concentration at time t |
| `AUC_0_t` | area under the curve from 0 to t |

- **Method:** Bateman function $C = F$ D ka / (V (ka - k)) (exp(-k t) - exp(-ka t)) and its integral
- **Assumes:** one compartment; first-order absorption and elimination; $ka \neq k$; no lag time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Plasma concentration after one oral dose (D = 250 mg, F = 0.8, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h), computed by the package at 120 times over 24 h; marked: t_max = 1.98042 h, C_max = 4.2457 mg/L.




C (concentration at time t) against t (time after the dose) from 1080 s to 108000 s on a logarithmic axis. One curve for each value of D (oral dose): 0.00025 kg, 0.000125 kg, 0.0005 kg. Other inputs as in the worked example: F = 0.8, V = 0.035 m^3, ka = 0.0003333333333333333 1/s, k = 4.1666666666666665e-05 1/s. 41 points per curve computed by biomedical-science-ai999x; marked: the worked example, t = 10800 s, C = 0.00398566 kg/m^3.

```
biomedical_science_ai999x.solve('bmed_pk_oral_concentration', F=0.8, D=0.00025, V=0.035,
ka=0.0003333333333333333, k=4.1666666666666665e-05, t=10800.0)
# -> C = 0.00398566 kg/m^3, AUC_0_t = 37.7397 kg*s/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_oral_concentration', F=0.6, D=4.9999999999999996e-05,
V=0.1, ka=1.388888888888889e-05, k=5.555555555555556e-05, t=43200.0)
# -> C = 4.58094e-05 kg/m^3, AUC_0_t = 1.61185 kg*s/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Oral dose concentration: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, t = 3 h | SOLVED | C = 0.00398566 kg/m^3, AUC_0_t = 37.7397 kg*s/m^3 |
| Oral dose concentration: F = 60%, D = 50 mg, V = 100 L, ka = 0.05 1/h, k = 0.2 1/h, t = 12 h | SOLVED | C = 4.58094e-05 kg/m^3, AUC_0_t = 1.61185 kg*s/m^3 |
| Oral dose concentration: D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, t = 3 h | NEEDS_INPUT | bmed_pk_oral_concentration needs F (bioavailable fraction (0 < F <= 1; 80% or 0.8)). Example: solve('bmed_pk_oral_concen |
| Oral dose concentration: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, t = 3 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Oral dose concentration: F = 80, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, t = 3 h | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Oral dose concentration: F = 0.8, D = 250 mg, V = 35 L, ka = 0.15 1/h, k = 0.15 1/h, t = 3 h | INVALID_INPUT | Oral dose, first-order absorption: concentration at a time (Bateman function) (a model calculation, not medical advice): |

## Oral dose: time and height of the peak concentration (tmax, Cmax) (a model calculation, not medical advice) — `bmed_pk_oral_peak`

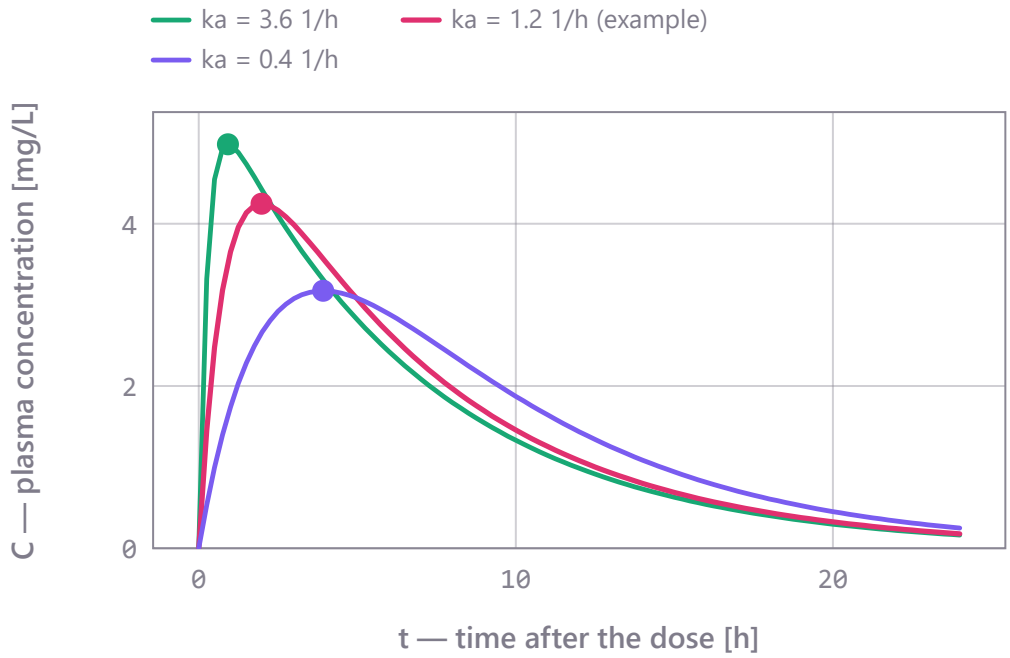
The time of the peak plasma concentration and the peak concentration of a one-compartment model with first-order absorption after a single oral dose.

$$t_{max} = \frac{\ln(k_a/k)}{k_a - k}, \quad C_{max} = \frac{FD}{V} e^{-kt_{max}}$$

| input | meaning | condition |
|---|---|---|
| `F` | bioavailable fraction $(0 < F \leq 1)$ | |
| `D` | oral dose | |
| `V` | volume of distribution | |
| `ka` | absorption rate constant | |
| `k` | elimination rate constant | |

| output | meaning |
|---|---|
| `t_max` | time of the peak |
| `C_max` | peak concentration |

- **Method:** $dC/dt = 0$: t_max = ln(ka/k)/(ka - k); C_max = (F D / V) exp(-k t_max)
- **Assumes:** one compartment; first-order absorption and elimination; $ka \neq k$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Oral dose (F = 0.8, D = 250 mg, V = 35 L, k = 0.15 1/h) for three absorption rate constants: concentration curves (97 runs each) with the peak the package returns marked on each — faster absorption gives an earlier, higher peak. Marked: ka = 3.6 1/h: t_max 0.921175 h, C_max 4.97683 mg/L; ka = 1.2 1/h: t_max 1.98042 h, C_max 4.2457 mg/L; ka = 0.4 1/h: t_max 3.92332 h, C_max 3.17235 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_oral_peak', F=0.8, D=0.00025, V=0.035,
ka=0.0003333333333333333, k=4.1666666666666665e-05)
# -> t_max = 7129.51 s, C_max = 0.0042457 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_oral_peak', F=1, D=0.0005, V=0.06, ka=0.00025,
k=2.4166666666666663e-05)
# -> t_max = 10346.1 s, C_max = 0.00648981 kg/m^3   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Oral dose peak: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h | SOLVED | t_max = 7129.51 s, C_max = 0.0042457 kg/m^3 |
| Oral dose peak: F = 1, D = 500 mg, V = 60 L, ka = 0.9 1/h, k = 0.087 1/h | SOLVED | t_max = 10346.1 s, C_max = 0.00648981 kg/m^3 |
| Oral dose peak: D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h | NEEDS_INPUT | bmed_pk_oral_peak needs F (bioavailable fraction (0 < F <= 1)). Example: solve('bmed_pk_oral_peak', F=0.8, D='0.00025 kg |
| Oral dose peak: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Repeated IV bolus doses: concentration after the nth dose (a model calculation, not medical advice) — `bmed_pk_nth_dose_iv`

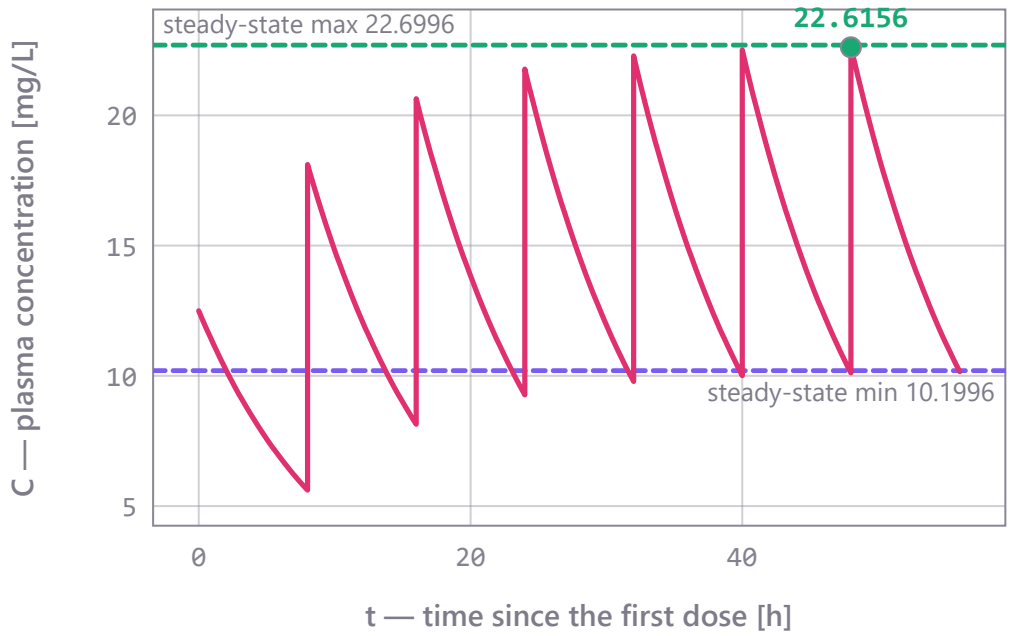
The peak concentration just after the nth of equal IV bolus doses given every tau, and the concentration a time t later, in a one-compartment model (superposition).

$$C_n(t) = \frac{D}{V}\,\frac{1 - e^{-nk\tau}}{1 - e^{-k\tau}}\,e^{-kt}$$

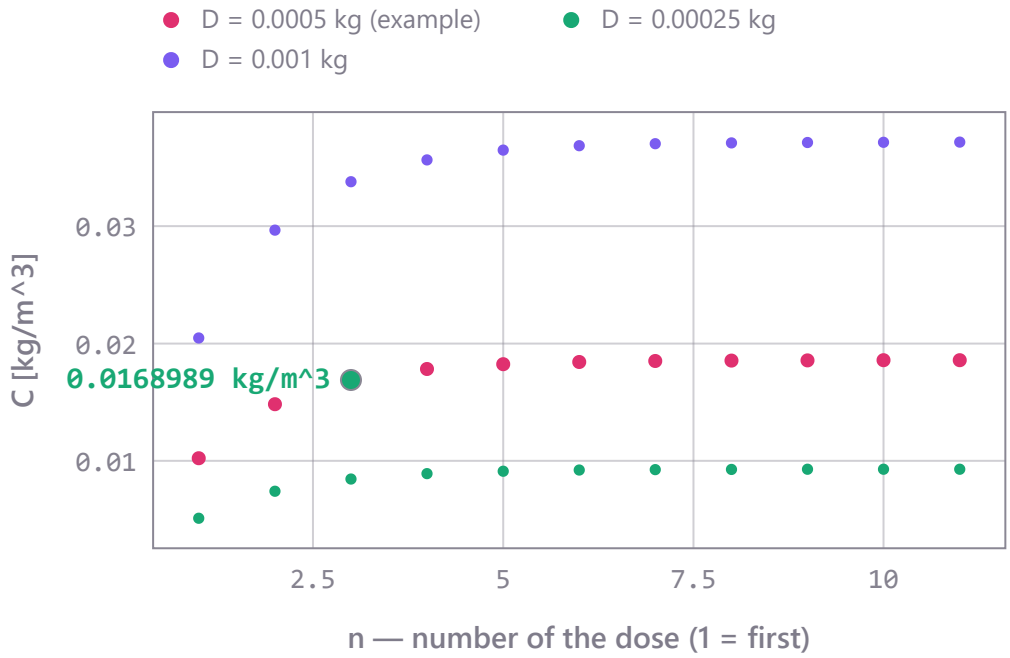
| input | meaning | condition |
|---|---|---|
| `D` | each bolus dose | |
| `V` | volume of distribution | |
| `k` | elimination rate constant | |
| `tau` | dosing interval | |
| `n` | number of the dose (1 = first) | |
| `t` | time after the nth dose (0 to tau) | |

| output | meaning |
|---|---|
| `C_max_n` | concentration just after the nth dose |
| `C` | concentration a time t after the nth dose |

- **Method:** superposition: $C = (D/V)\,(1 - r^n)/(1 - r)$ exp(-k t), $r = exp$(-k tau)
- **Assumes:** equal doses at equal intervals; linear one-compartment model; $0 \le t \le \tau$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Seven intravenous doses of 500 mg every 8 h (V = 40 L, k = 0.1 1/h): the concentration computed by the package at 31 times in each interval rises towards the steady state it returns (max 22.6996 mg/L, min 10.1996 mg/L, accumulation ratio 1.81597); marked: peak after dose 7, t = 48 h, C = 22.6156 mg/L.




C (concentration a time t after the nth dose) against n (number of the dose (1 = first)) from 1 to 11. One curve for each value of D (each bolus dose): 0.0005 kg, 0.00025 kg, 0.001 kg. Other inputs as in the worked example: V = 0.04 m^3, k = 2.777777777777778e-05 1/s, tau = 28800.0 s, t = 7200.0 s. 11 points per curve computed by biomedical-science-ai999x; marked: the worked example, n = 3, C = 0.0168989 kg/m^3.

```
biomedical_science_ai999x.solve('bmed_pk_nth_dose_iv', D=0.0005, V=0.04, k=2.777777777777778e-05,
tau=28800.0, n=3, t=7200.0)
# -> C_max_n = 0.0206403 kg/m^3, C = 0.0168989 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_nth_dose_iv', D=0.001, V=0.06, k=5.555555555555556e-05,
tau=43200.0, n=1, t=0.0)
# -> C_max_n = 0.0166667 kg/m^3, C = 0.0166667 kg/m^3   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Concentration after the nth IV dose: D = 500 mg, V = 40 L, k = 0.1 1/h, tau = 8 h, n = 3, t = 2 h | SOLVED | C_max_n = 0.0206403 kg/m^3, C = 0.0168989 kg/m^3 |
| Concentration after the nth IV dose: D = 1 g, V = 60 L, k = 0.2 1/h, tau = 12 h, n = 1, t = 0 h | SOLVED | C_max_n = 0.0166667 kg/m^3, C = 0.0166667 kg/m^3 |
| Concentration after the nth IV dose: V = 40 L, k = 0.1 1/h, tau = 8 h, n = 3, t = 2 h | NEEDS_INPUT | bmed_pk_nth_dose_iv needs D (each bolus dose). Example: solve('bmed_pk_nth_dose_iv', D='0.0005 kg', V='0.04 m^3', k='2.7 |
| Concentration after the nth IV dose: D = 500 mg, V = 40 L, k = 0.1 1/h, tau = 8 h, n = 3, t = 2 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Concentration after the nth IV dose: D = 500 mg, V = 40 L, k = 0.1 1/h, tau = 8 h, n = 3, t = 9 h | INVALID_INPUT | Repeated IV bolus doses: concentration after the nth dose (a model calculation, not medical advice): t is the time after |
| Concentration after the nth IV dose: D = 500 mg, V = 40 L, k = 0.1 1/h, | INVALID_INPUT | n: expected a whole number written exactly, got '2.5'. |

| question | outcome | result |
|---|---|---|
| tau = 8 h, n = 2.5, t = 2 h | | |

### Repeated IV bolus dosing: steady-state peak, trough, average and accumulation factor (a model calculation, not medical advice) — `bmed_pk_steady_state_iv`

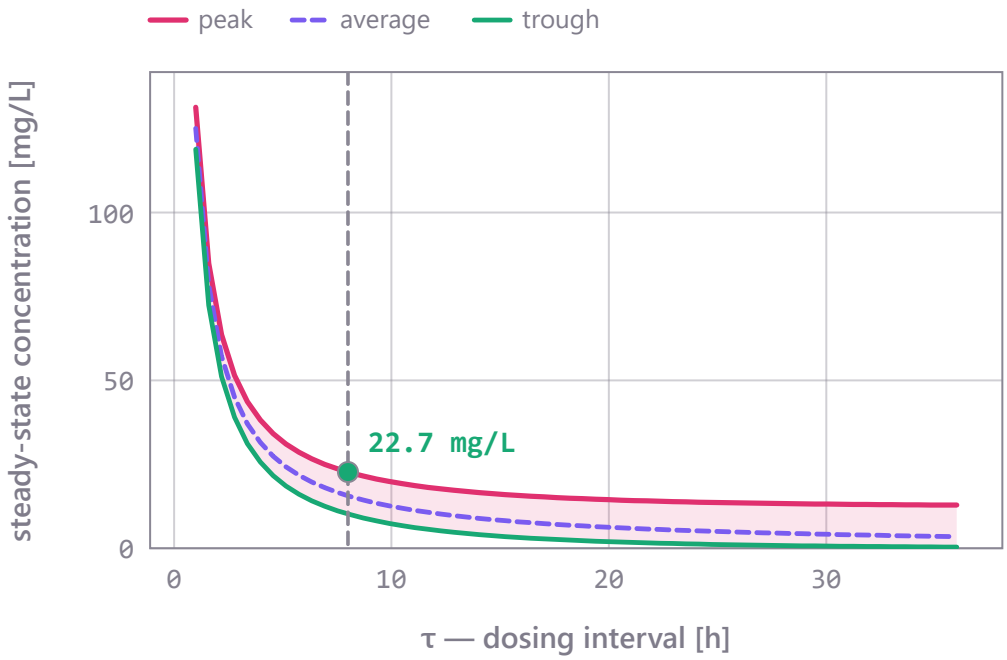
The steady-state peak, trough and average concentrations and the accumulation factor of a one-compartment model given equal IV bolus doses every tau.

$$C_{max,ss} = \frac{D/V}{1 - e^{-k\tau}}, \quad C_{min,ss} = C_{max,ss}\, e^{-k\tau}, \quad \bar{C}_{ss} = \frac{D}{Vk\tau}$$

| input | meaning | condition |
|---|---|---|
| `D` | each bolus dose | |
| `V` | volume of distribution | |
| `k` | elimination rate constant | |
| `tau` | dosing interval | |

| output | meaning |
|---|---|
| `C_max_ss` | steady-state peak (just after a dose) |
| `C_min_ss` | steady-state trough (just before a dose) |
| `C_avg_ss` | average concentration over an interval at steady state |
| `R_ac` | accumulation factor: steady-state peak over first-dose peak |

- **Method:** geometric series: C_max,ss = (D/V)/(1 - exp(-k tau)); C_min,ss = C_max,ss exp(-k tau); C_avg = D/(V k tau)
- **Assumes:** equal doses at equal intervals; linear one-compartment model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Steady state of repeated intravenous doses (D = 500 mg, V = 40 L, k = 0.1 1/h) against the dosing interval, 60 runs per output: peak, average and trough. Marked: the example τ = 8 h, peak 22.6996 mg/L, trough 10.1996 mg/L, average 15.625 mg/L, accumulation factor 1.81597.

```
biomedical_science_ai999x.solve('bmed_pk_steady_state_iv', D=0.0005, V=0.04, k=2.777777777777778e-05, tau=28800.0)
# -> C_max_ss = 0.0226996 kg/m^3, C_min_ss = 0.0101996 kg/m^3, C_avg_ss = 0.015625 kg/m^3, R_ac = 1.81597   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_steady_state_iv', D=0.00025, V=0.025, k=1.6055555555555554e-05, tau=43200.0)
# -> C_max_ss = 0.0199909 kg/m^3, C_min_ss = 0.00999095 kg/m^3, C_avg_ss = 0.0144175 kg/m^3, R_ac = 1.99909   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Steady state of repeated IV doses: D = 500 mg, V = 40 L, k = 0.1 1/h, tau = 8 h | SOLVED | C_max_ss = 0.0226996 kg/m^3, C_min_ss = 0.0101996 kg/m^3, C_avg_ss = 0.015625 kg/m^3, R_ac = 1.81597 |
| Steady state of repeated IV doses: D = 250 mg, V = 25 L, k = 0.0578 1/h, tau = 12 h | SOLVED | C_max_ss = 0.0199909 kg/m^3, C_min_ss = 0.00999095 kg/m^3, C_avg_ss = 0.0144175 kg/m^3, R_ac = 1.99909 |
| Steady state of repeated IV doses: V = 40 L, k = 0.1 1/h, tau = 8 h | NEEDS_INPUT | bmed_pk_steady_state_iv needs D (each bolus dose). Example: solve('bmed_pk_steady_state_iv', D='0.0005 kg', V='0.04 m^3' |
| Steady state of repeated IV doses: D = 500 mg, V = 40 L, k = 0.1 1/h, tau = 8 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Repeated oral dosing: steady-state peak, its time and the trough (a model calculation, not medical advice) — `bmed_pk_steady_state_oral`

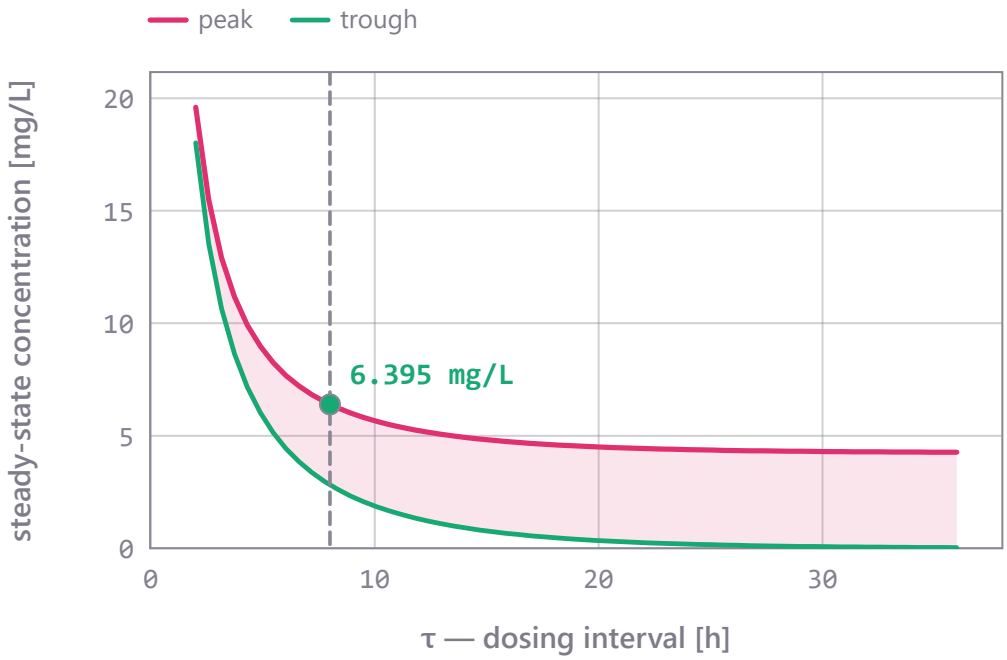
The steady-state peak concentration, the time of the peak after a dose and the trough of a one-compartment model with first-order absorption given equal oral doses every tau.

$$C_{ss}(t) = \frac{F D k_a}{V(k_a - k)} \left[ \frac{e^{-kt}}{1 - e^{-k\tau}} - \frac{e^{-k_a t}}{1 - e^{-k_a \tau}} \right]$$

| input | meaning | condition |
|---|---|---|
| `F` | bioavailable fraction $(0 < F \leq 1)$ | |
| `D` | each oral dose | |
| `V` | volume of distribution | |
| `ka` | absorption rate constant | |
| `k` | elimination rate constant | |
| `tau` | dosing interval | |

| output | meaning |
|---|---|
| `C_max_ss` | steady-state peak |
| `t_max_ss` | time of the peak after a dose |
| `C_min_ss` | steady-state trough (at the dosing times) |

- **Method:** superposition to steady state: C_ss(t) = F D ka/(V(ka - k)) [exp(-k $t)/(1 - exp$(-k tau)) - exp(-ka $t)/(1 - exp$(-ka tau))]; $dC/dt = 0$ for the peak
- **Assumes:** equal doses at equal intervals; linear one-compartment model, first-order absorption; $ka \neq k$; the steady-state peak lies inside the interval
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Steady state of repeated oral doses (F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h) against the dosing interval, 60 runs each. Marked: the example τ = 8 h, peak 6.39474 mg/L at 1.63917 h after a dose, trough 2.81433 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_steady_state_oral', F=0.8, D=0.00025, V=0.035,
ka=0.0003333333333333333, k=4.1666666666666665e-05, tau=28800.0)
# -> C_max_ss = 0.00639474 kg/m^3, t_max_ss = 5901.01 s, C_min_ss = 0.00281433 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_steady_state_oral', F=0.5, D=9.999999999999999e-05,
V=0.07, ka=0.0005555555555555556, k=2.777777777777778e-05, tau=86400.0)
# -> C_max_ss = 0.000674329 kg/m^3, t_max_ss = 5495.93 s, C_min_ss = 7.50141e-05 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Steady state of repeated oral doses: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, tau = 8 h | SOLVED | C_max_ss = 0.00639474 kg/m^3, t_max_ss = 5901.01 s, C_min_ss = 0.00281433 kg/m^3 |
| Steady state of repeated oral doses: F = 0.5, D = 100 mg, V = 70 L, ka = 2 1/h, k = 0.1 1/h, tau = 24 h | SOLVED | C_max_ss = 0.000674329 kg/m^3, t_max_ss = 5495.93 s, C_min_ss = 7.50141e-05 kg/m^3 |
| Steady state of repeated oral doses: D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, tau = 8 h | NEEDS_INPUT | bmed_pk_steady_state_oral needs F (bioavailable fraction (0 < F <= 1)). Example: solve('bmed_pk_steady_state_oral', F=0. |
| Steady state of repeated oral doses: F = 0.8, D = 250 mg, V = 35 L, ka = 1.2 1/h, k = 0.15 1/h, tau = 8 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Constant-rate IV infusion: concentration at a time and steady state (a model calculation, not medical advice) — `bmed_pk_infusion_concentration`

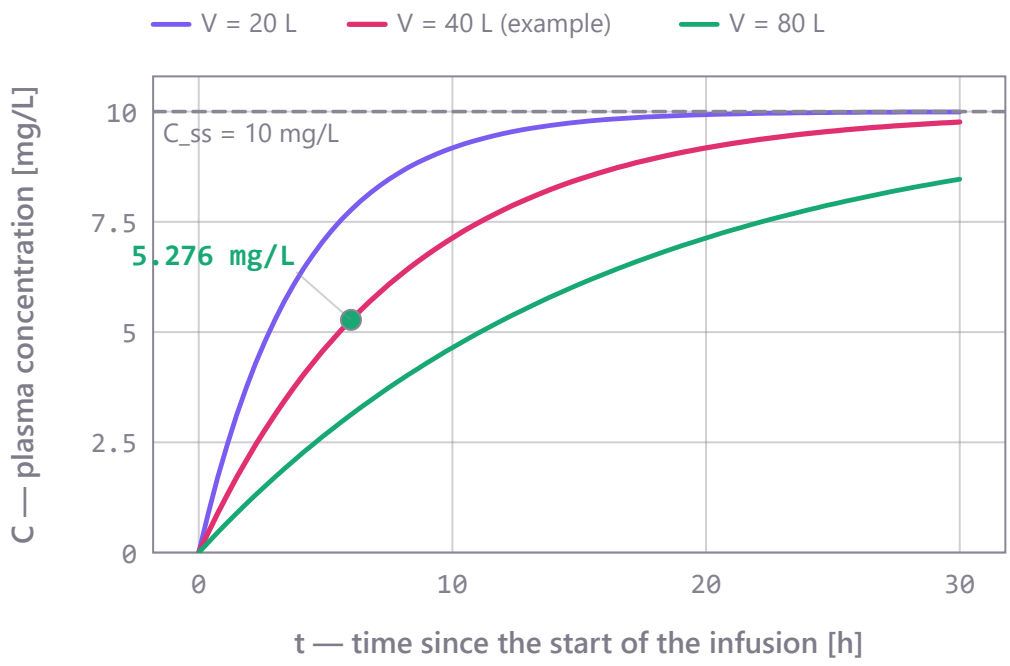
The plasma concentration of a one-compartment model during a constant-rate intravenous infusion started at $t = 0$, and the steady-state concentration it approaches.

$$C(t) = \frac{R_0}{CL}\left(1 - e^{-CL\,t/V}\right), \quad C_{ss} = \frac{R_0}{CL}$$

| input | meaning | condition |
|---|---|---|
| `R0` | infusion rate | |
| `CL` | clearance | |
| `V` | volume of distribution | |
| `t` | time since the start of the infusion | |

| output | meaning |
|---|---|
| `C` | concentration at time t |
| `C_ss` | steady-state concentration $R_0/CL$ |

- **Method:** $C = (R_0/CL)$(1 - exp(-CL t / V))
- **Assumes:** one compartment, first-order elimination; infusion still running at t; no drug at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Constant-rate infusion (R0 = 50 mg/h, CL = 5 L/h): the concentration rises towards the steady state R0/CL = 10 mg/L, sooner for a smaller volume of distribution (three volumes, 81 runs each; C_ss does not depend on V). Marked: example at t = 6 h, C = 5.27633 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_infusion_concentration', R0=1.3888888888888889e-08,
CL=1.3888888888888887e-06, V=0.04, t=21600.0)
# -> C = 0.00527633 kg/m^3, C_ss = 0.01 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_infusion_concentration', R0=3.3333333333333334e-08,
CL=1.6666666666666667e-06, V=0.02, t=86400.0)
# -> C = 0.0199851 kg/m^3, C_ss = 0.02 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Infusion concentration: R0 = 50 mg/h, CL = 5 L/h, V = 40 L, t = 6 h | SOLVED | C = 0.00527633 kg/m^3, C_ss = 0.01 kg/m^3 |
| Infusion concentration: R0 = 2 mg/min, CL = 100 mL/min, V = 20 L, t = 1 day | SOLVED | C = 0.0199851 kg/m^3, C_ss = 0.02 kg/m^3 |
| Infusion concentration: CL = 5 L/h, V = 40 L, t = 6 h | NEEDS_INPUT | bmed_pk_infusion_concentration needs R0 (infusion rate). Example: solve('bmed_pk_infusion_concentration', R0='1.38889e-0 |
| Infusion concentration: R0 = 50 mg/h, CL = 5 L/h, V = 40 L, t = 6 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Infusion of fixed duration: concentration during and after it (a model calculation, not medical advice) — `bmed_pk_short_infusion`

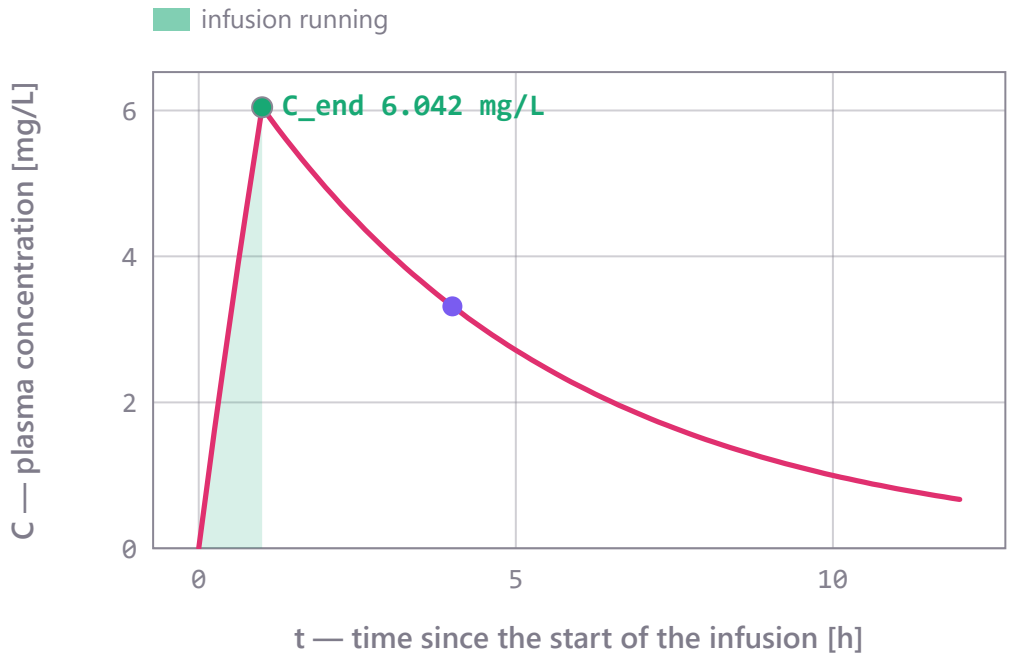
The plasma concentration of a one-compartment model at time t after the start of a constant-rate infusion lasting T_inf (during or after the infusion), and the concentration at the end of the infusion.

$$C(t) = \begin{cases} \frac{R_0}{CL}\left(1 - e^{-kt}\right) & t \le T \\ \frac{R_0}{CL}\left(1 - e^{-kT}\right) e^{-k(t-T)} & t > T \end{cases}$$

| input | meaning | condition |
|---|---|---|
| `R0` | infusion rate | |
| `T_inf` | duration of the infusion | |
| `CL` | clearance | |
| `V` | volume of distribution | |
| `t` | time since the start of the infusion | |

| output | meaning |
|---|---|
| `C` | concentration at time t |
| `C_end` | concentration at the end of the infusion |

- **Method:** during: $(R_0/CL)$(1 - exp(-k t)); after: C_end exp(-k (t - T_inf)), $k = CL/V$
- **Assumes:** one compartment, first-order elimination; single infusion, no drug before it
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




A short infusion of R0 = 200 mg/h for T_inf = 1 h (CL = 6 L/h, V = 30 L): concentration over 12 h, 97 runs. Marked: at the end of the infusion C_end = 6.04231 mg/L; dot: the example t = 4 h, C = 3.31609 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_short_infusion', R0=5.5555555555555555e-08, T_inf=3600.0,
CL=1.6666666666666667e-06, V=0.03, t=14400.0)
# -> C = 0.00331609 kg/m^3, C_end = 0.00604231 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_short_infusion', R0=2.7777777777777777e-08, T_inf=7200.0,
CL=1.111111111111111e-06, V=0.05, t=5400.0)
# -> C = 0.00282699 kg/m^3, C_end = 0.00369641 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Short infusion concentration: R0 = 200 mg/h, T_inf = 1 h, CL = 6 L/h, V = 30 L, t = 4 h | SOLVED | C = 0.00331609 kg/m^3, C_end = 0.00604231 kg/m^3 |
| Short infusion concentration: R0 = 100 mg/h, T_inf = 2 h, CL = 4 L/h, V = 50 L, t = 1.5 h | SOLVED | C = 0.00282699 kg/m^3, C_end = 0.00369641 kg/m^3 |
| Short infusion concentration: T_inf = 1 h, CL = 6 L/h, V = 30 L, t = 4 h | NEEDS_INPUT | bmed_pk_short_infusion needs R0 (infusion rate). Example: solve('bmed_pk_short_infusion', R0='5.55556e-08 kg/s', T_inf=' |
| Short infusion concentration: R0 = 200 mg/h, T_inf = 1 h, CL = 6 L/h, V = 30 L, t = 4 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Loading dose and maintenance infusion rate of a one-compartment model (model calculation, not a dosing recommendation) — `bmed_pk_loading_maintenance`

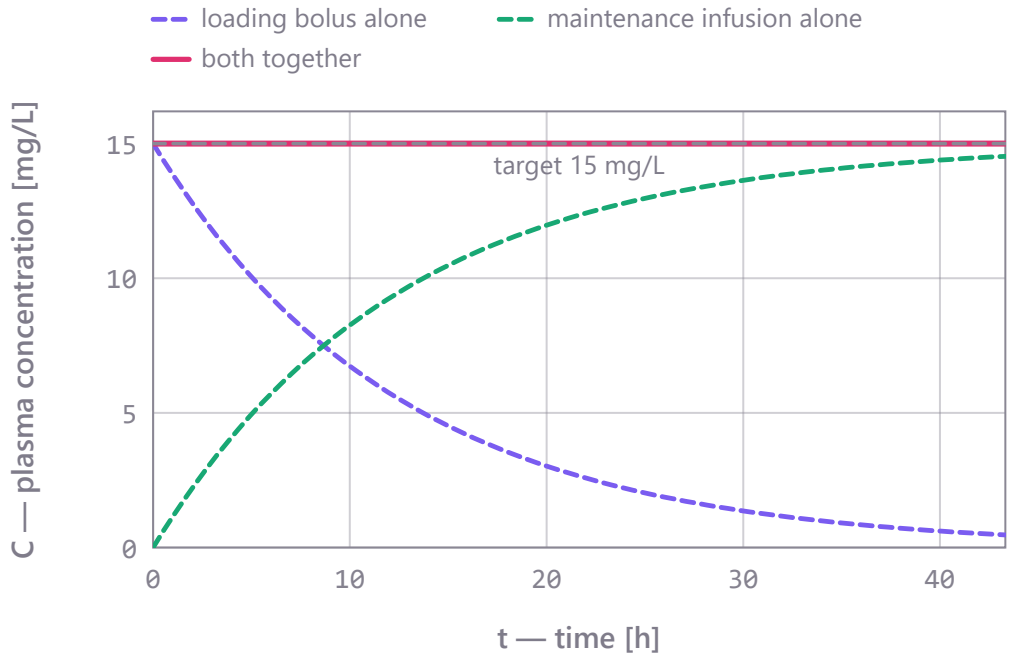
For a one-compartment model, the bolus and the constant infusion rate that together hold the model concentration at a target value from the start (model calculation only, not a dosing recommendation).

$$LD = C_{target} V, \quad R_0 = C_{target}\, CL$$

| input | meaning | condition |
|---|---|---|
| `C_target` | target model concentration | |
| `V` | volume of distribution | |
| `CL` | clearance | |

| output | meaning |
|---|---|
| `LD` | loading bolus of the model |
| `R0` | maintenance infusion rate of the model |

- **Method:** $LD = C$ V fills the volume; $R_0 = C$ CL replaces what is cleared
- **Assumes:** one compartment, first-order elimination; bolus and infusion start together
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Loading dose and maintenance infusion for a target of 15 mg/L (V = 50 L, CL = 4 L/h): the package returns LD = 750 mg and R0 = 60 mg/h. Drawn: the package's bolus concentration of LD (k = CL/V), its infusion concentration at R0, and their sum (superposition, derived), which holds the target from the start.

```
biomedical_science_ai999x.solve('bmed_pk_loading_maintenance', C_target=0.015, V=0.05,
CL=1.111111111111111e-06)
# -> LD = 0.00075 kg, R0 = 1.66667e-08 kg/s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_loading_maintenance', C_target=0.002,
V=0.35000000000000003, CL=8.333333333333334e-06)
# -> LD = 0.0007 kg, R0 = 1.66667e-08 kg/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Loading dose and maintenance rate model: C_target = 15 mg/L, V = 50 L, CL = 4 L/h | SOLVED | LD = 0.00075 kg, R0 = 1.66667e-08 kg/s |
| Loading dose and maintenance rate model: C_target = 2 mg/L, V = 350 L, CL = 0.5 L/min | SOLVED | LD = 0.0007 kg, R0 = 1.66667e-08 kg/s |
| Loading dose and maintenance rate model: V = 50 L, CL = 4 L/h | NEEDS_INPUT | bmed_pk_loading_maintenance needs C_target (target model concentration). Example: solve('bmed_pk_loading_maintenance', C |
| Loading dose and maintenance rate model: C_target = 15 mg/L, V = 50 L, CL = 4 L/h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Repeated short infusions: steady-state peak and trough (a model calculation, not medical advice) — `bmed_pk_intermittent_infusion_steady_state`

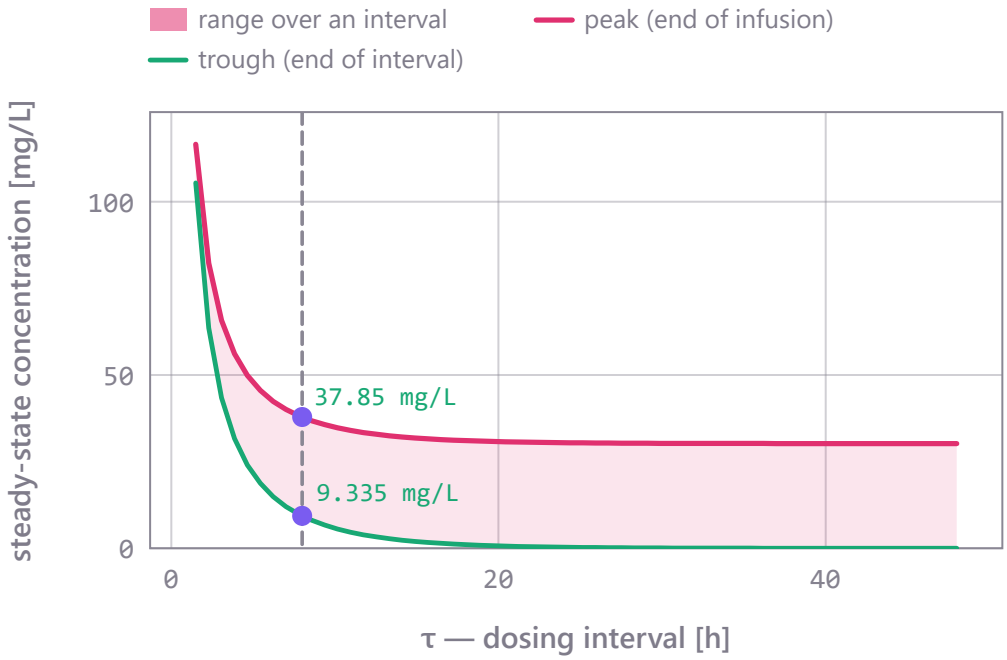
The steady-state peak (end of infusion) and trough (end of interval) of a one-compartment model given infusions of duration T_inf at rate R0 every tau.

$$C_{max,ss} = \frac{R_0}{CL}\,\frac{1 - e^{-kT}}{1 - e^{-k\tau}}, \quad C_{min,ss} = C_{max,ss}\, e^{-k(\tau - T)}$$

| input | meaning | condition |
|---|---|---|
| `R0` | infusion rate | |
| `T_inf` | duration of each infusion | |
| `tau` | dosing interval | |
| `CL` | clearance | |
| `V` | volume of distribution | |

| output | meaning |
|---|---|
| `C_max_ss` | steady-state concentration at the end of an infusion |
| `C_min_ss` | steady-state concentration at the end of an interval |

- **Method:** C_max,$ss = (R_0/CL)$(1 - exp(-k T))/(1 - exp(-k tau)); C_min,ss = C_max,ss exp(-k(tau - T)), $k = CL/V$
- **Assumes:** one compartment, first-order elimination; T_inf < tau; identical infusions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Steady-state peak and trough of repeated infusions (R0 = 1000 mg/h for 1 h each, CL = 6 L/h, V = 30 L) against the dosing interval, 60 runs each: a longer interval lowers both and widens the swing. Marked: the example τ = 8 h, peak 37.8542 mg/L, trough 9.33472 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_intermittent_infusion_steady_state',
R0=2.7777777777777776e-07, T_inf=3600.0, tau=28800.0, CL=1.6666666666666667e-06, V=0.03)
# -> C_max_ss = 0.0378542 kg/m^3, C_min_ss = 0.00933472 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_intermittent_infusion_steady_state',
R0=1.3888888888888888e-07, T_inf=1800.0, tau=43200.0, CL=1.3888888888888887e-06, V=0.02)
# -> C_max_ss = 0.012366 kg/m^3, C_min_ss = 0.000697641 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Intermittent infusion steady state: R0 = 1000 mg/h, T_inf = 1 h, tau = 8 h, CL = 6 L/h, V = 30 L | SOLVED | C_max_ss = 0.0378542 kg/m^3, C_min_ss = 0.00933472 kg/m^3 |
| Intermittent infusion steady state: R0 = 500 mg/h, T_inf = 30 min, tau = 12 h, CL = 5 L/h, V = 20 L | SOLVED | C_max_ss = 0.012366 kg/m^3, C_min_ss = 0.000697641 kg/m^3 |
| Intermittent infusion steady state: T_inf = 1 h, tau = 8 h, CL = 6 L/h, V = 30 L | NEEDS_INPUT | bmed_pk_intermittent_infusion_steady_state needs R0 (infusion rate). Example: solve('bmed_pk_intermittent_infusion_stead |
| Intermittent infusion steady state: R0 = 1000 mg/h, T_inf = 1 h, tau = 8 h, CL = 6 L/h, V = 30 L, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Intermittent infusion steady state: R0 = 1000 mg/h, T_inf = 9 h, tau = 8 h, CL = 6 L/h, V = 30 L | INVALID_INPUT | Repeated short infusions: steady-state peak and trough (a model calculation, not medical advice): the infusion must be s |

### Constant infusion: time to reach a fraction of steady state (a model calculation, not medical advice) — `bmed_pk_time_to_steady_state_fraction`

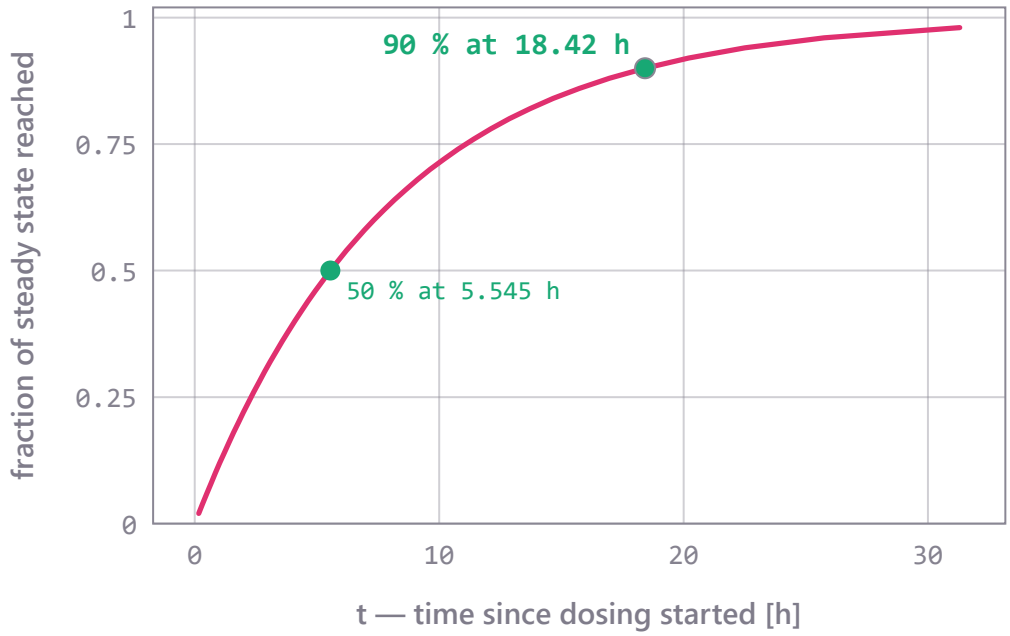
The time after the start of a constant-rate infusion at which a one-compartment model reaches a stated fraction of its steady-state concentration.

$$t_f = -\frac{\ln(1-f)}{CL/V}$$

| input | meaning | condition |
|---|---|---|
| `CL` | clearance | |
| `V` | volume of distribution | |
| `f` | fraction of steady state (90% or 0.9) | |

| output | meaning |
|---|---|
| `t` | time to reach f of steady state |

- **Method:** $1 - exp$(-k t) = f: $t = -ln$(1 - f)/k, $k = CL/V$
- **Assumes:** one compartment, first-order elimination; no drug at the start; $0 < f < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Time to reach a fraction of the steady state under constant input (CL = 5 L/h, V = 40 L), 49 runs from 2 % to 98 %, drawn as fraction against time. Marked: 50 % after 5.54518 h (one half-life) and the example 90 % after 18.4207 h.

```
biomedical_science_ai999x.solve('bmed_pk_time_to_steady_state_fraction', CL=1.3888888888888887e-06,
V=0.04, f=0.9)
# -> t = 66314.5 s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_time_to_steady_state_fraction', CL=5.555555555555555e-07,
V=0.1, f=0.5)
# -> t = 124766 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Time to fraction of steady state: CL = 5 L/h, V = 40 L, f = 90% | SOLVED | t = 66314.5 s |
| Time to fraction of steady state: CL = 2 L/h, V = 100 L, f = 0.5 | SOLVED | t = 124766 s |
| Time to fraction of steady state: V = 40 L, f = 90% | NEEDS_INPUT | bmed_pk_time_to_steady_state_fraction needs CL (clearance). Example: solve('bmed_pk_time_to_steady_state_fraction', CL=' |
| Time to fraction of steady state: CL = 5 L/h, V = 40 L, f = 90%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Time to fraction of steady state: CL = 5 L/h, V = 40 L, f = 100% | INVALID_INPUT | Constant infusion: time to reach a fraction of steady state (a model calculation, not medical advice): the fraction f mu |

## Michaelis-Menten (saturable) elimination: concentration at a time (a model calculation, not medical advice) — `bmed_pk_michaelis_menten_concentration`

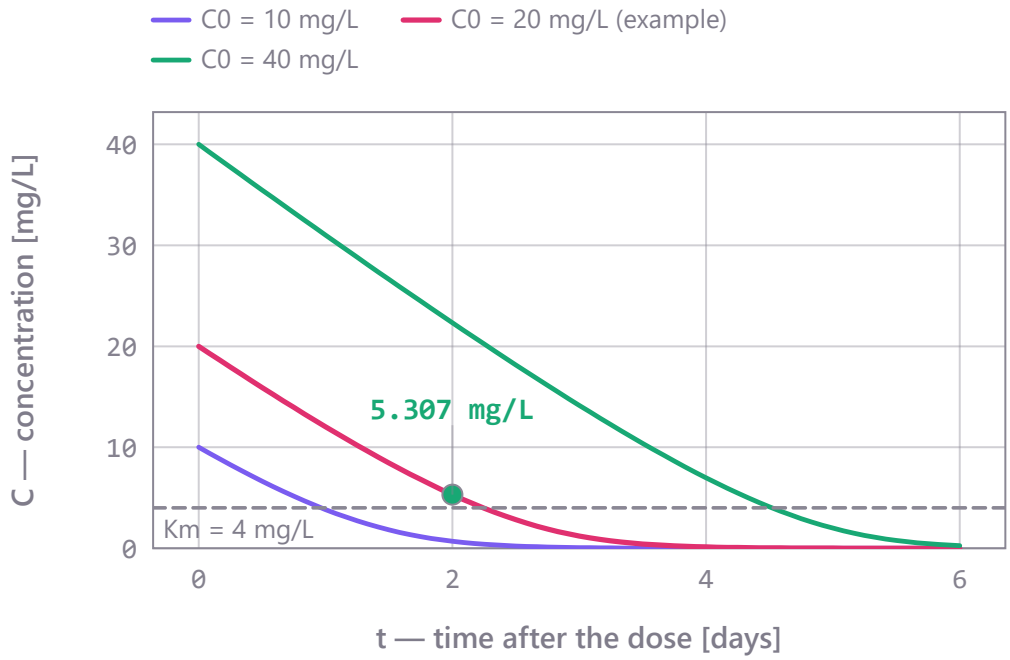
The concentration a time t after an IV bolus in a one-compartment model with saturable Michaelis-Menten elimination, from the closed form with the Lambert W function.

$$C(t) = K_m\, W\!\left(\frac{C_0}{K_m}\exp\frac{C_0 - V_{max}t/V}{K_m}\right)$$

| input | meaning | condition |
|---|---|---|
| `C0` | initial concentration | |
| `Vmax` | maximum elimination rate (amount per time) | |
| `Km` | Michaelis constant | |
| `V` | volume of distribution | |
| `t` | time after the dose | |

| output | meaning |
|---|---|
| `C` | concentration at time t |

- **Method:** V dC/dt = -Vmax C/(Km + C) integrates to $C = Km$ W($(C_0/Km)$ exp((C0 - Vmax t/V)/Km)); W found from w + ln $w = L$ by Newton steps
- **Assumes:** one compartment; elimination rate Vmax C/(Km + C); no other elimination
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Saturable (Michaelis–Menten) elimination: concentration over 6 days for Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L and three starting concentrations, 73 runs each. Far above Km the fall is nearly linear (zero order), below Km it becomes exponential. Marked: example at t = 2 days, C = 5.3069 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_concentration', C0=0.02,
Vmax=5.787037037037037e-09, Km=0.004, V=0.05, t=172800.0)
# -> C = 0.0053069 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_concentration', C0=0.005,
Vmax=2.7777777777777776e-09, Km=0.01, V=0.02, t=28800.0)
# -> C = 0.0037847 kg/m^3   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Michaelis-Menten concentration: C0 = 20 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L, t = 2 day | SOLVED | C = 0.0053069 kg/m^3 |
| Michaelis-Menten concentration: C0 = 5 mg/L, Vmax = 10 mg/h, Km = 10 mg/L, V = 20 L, t = 8 h | SOLVED | C = 0.0037847 kg/m^3 |
| Michaelis-Menten concentration: Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L, t = 2 day | NEEDS_INPUT | bmed_pk_michaelis_menten_concentration needs C0 (initial concentration). Example: solve('bmed_pk_michaelis_menten_concen |
| Michaelis-Menten concentration: C0 = 20 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L, t = 2 day, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Michaelis-Menten elimination: time to fall to a concentration (a model calculation, not medical advice) — `bmed_pk_michaelis_menten_time`

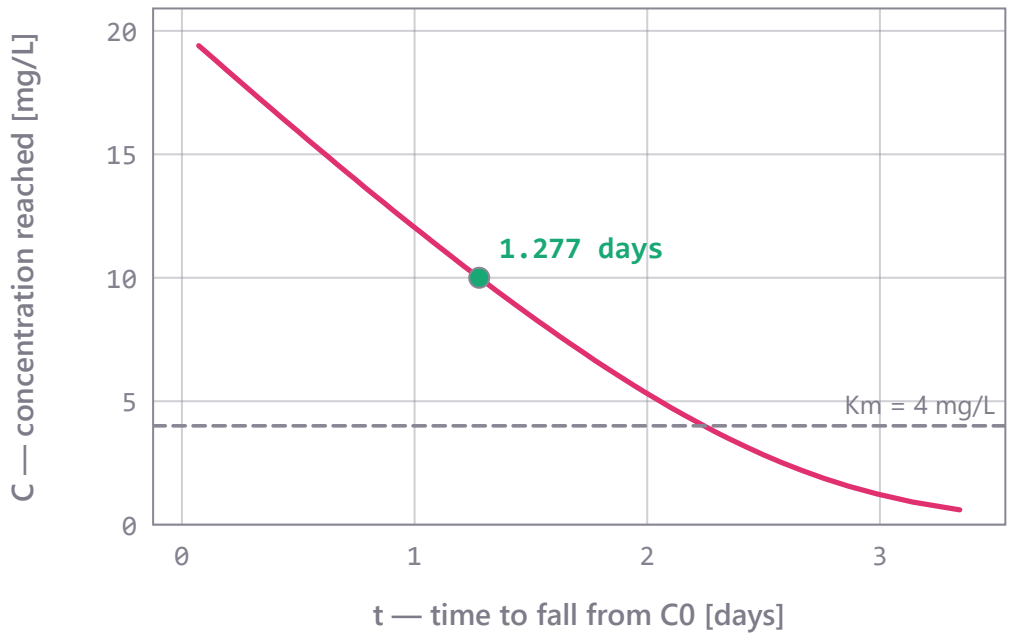
The time for the concentration of a one-compartment model with Michaelis-Menten elimination to fall from C0 to C.

$$t = \frac{V}{V_{max}}\left[(C_0 - C) + K_m \ln\frac{C_0}{C}\right]$$

| input | meaning | condition |
|---|---|---|
| `C0` | initial concentration | |
| `C` | target concentration (< C0) | |
| `Vmax` | maximum elimination rate | |
| `Km` | Michaelis constant | |
| `V` | volume of distribution | |

| output | meaning |
|---|---|
| `t` | time to fall from C0 to C |

- **Method:** $t = V$ [$(C_0 - C) + Km$ ln(C0/C)] / Vmax
- **Assumes:** one compartment; Michaelis-Menten elimination only; $C < C_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Time for the concentration to fall from C0 = 20 mg/L to each lower value under saturable elimination (Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L), 60 runs, drawn as concentration against time. Marked: the example target 10 mg/L is reached after t = 1.27726 days.

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_time', C0=0.02, C=0.01,
Vmax=5.787037037037037e-09, Km=0.004, V=0.05)
# -> t = 110355 s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_time', C0=0.03, C=0.003,
Vmax=5.555555555555555e-09, Km=0.005, V=0.04)
# -> t = 277293 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Michaelis-Menten time to concentration: C0 = 20 mg/L, C = 10 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L | SOLVED | t = 110355 s |
| Michaelis-Menten time to concentration: C0 = 30 mg/L, C = 3 mg/L, Vmax = 20 mg/h, Km = 5 mg/L, V = 40 L | SOLVED | t = 277293 s |
| Michaelis-Menten time to concentration: C = 10 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L | NEEDS_INPUT | bmed_pk_michaelis_menten_time needs C0 (initial concentration). Example: solve('bmed_pk_michaelis_menten_time', C0='0.02 |
| Michaelis-Menten time to concentration: C0 = 20 mg/L, C = 10 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Michaelis-Menten time to concentration: C0 = 10 mg/L, C = 20 mg/L, Vmax = 500 mg/day, Km = 4 mg/L, V = 50 L | INVALID_INPUT | Michaelis-Menten elimination: time to fall to a concentration (a model calculation, not medical advice): the target conc |

## Michaelis-Menten elimination: steady-state concentration under a constant dosing rate (a model calculation, not medical advice) — `bmed_pk_michaelis_menten_steady_state`

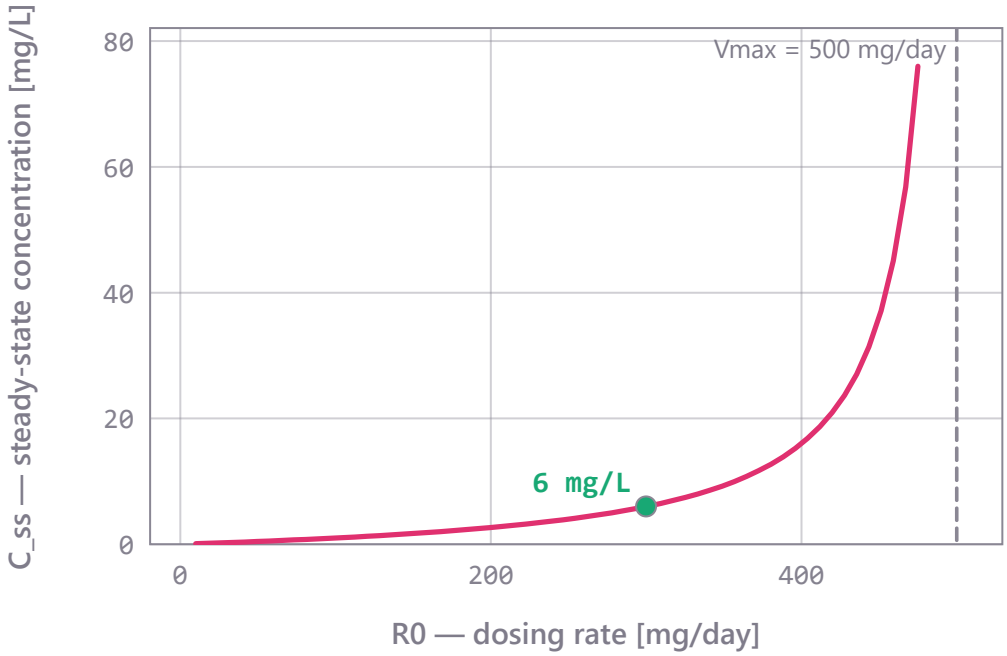
The steady-state concentration of a one-compartment model with Michaelis-Menten elimination given a constant dosing rate below Vmax.

$$C_{ss} = \frac{K_m R_0}{V_{max} - R_0}$$

| input | meaning | condition |
|---|---|---|
| `R0` | constant dosing rate (amount per time) | |
| `Vmax` | maximum elimination rate | |
| `Km` | Michaelis constant | |

| output | meaning |
|---|---|
| `C_ss` | steady-state concentration |

- **Method:** R0 = Vmax C/(Km + C): $C_ss = Km$ R0/(Vmax - R0)
- **Assumes:** Michaelis-Menten elimination only; R0 < Vmax (otherwise no steady state)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Steady-state concentration under saturable elimination against the dosing rate (Vmax = 500 mg/day, Km = 4 mg/L), 60 runs from 2 % to 95 % of Vmax: the concentration grows ever faster as the rate approaches Vmax, where no steady state exists. Marked: the example R0 = 300 mg/day gives C_ss = 6 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_steady_state', R0=3.472222222222222e-09,
Vmax=5.787037037037037e-09, Km=0.004)
# -> C_ss = 0.006 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_michaelis_menten_steady_state', R0=4.166666666666667e-09,
Vmax=5.555555555555555e-09, Km=0.005)
# -> C_ss = 0.015 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Michaelis-Menten steady state: R0 = 300 mg/day, Vmax = 500 mg/day, Km = 4 mg/L | SOLVED | C_ss = 0.006 kg/m^3 |
| Michaelis-Menten steady state: R0 = 15 mg/h, Vmax = 20 mg/h, Km = 5 mg/L | SOLVED | C_ss = 0.015 kg/m^3 |
| Michaelis-Menten steady state: Vmax = 500 mg/day, Km = 4 mg/L | NEEDS_INPUT | bmed_pk_michaelis_menten_steady_state needs R0 (constant dosing rate (amount per time)). Example: solve('bmed_pk_michael |
| Michaelis-Menten steady state: R0 = 300 mg/day, Vmax = 500 mg/day, Km = 4 mg/L, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Michaelis-Menten steady state: R0 = 600 mg/day, Vmax = 500 mg/day, Km = 4 mg/L | INVALID_INPUT | Michaelis-Menten elimination: steady-state concentration under a constant dosing rate (a model calculation, not medical |

### Two-compartment IV bolus: macro-constants and concentration from micro-constants (a model calculation, not medical advice) — `bmed_pk_two_compartment_bolus`

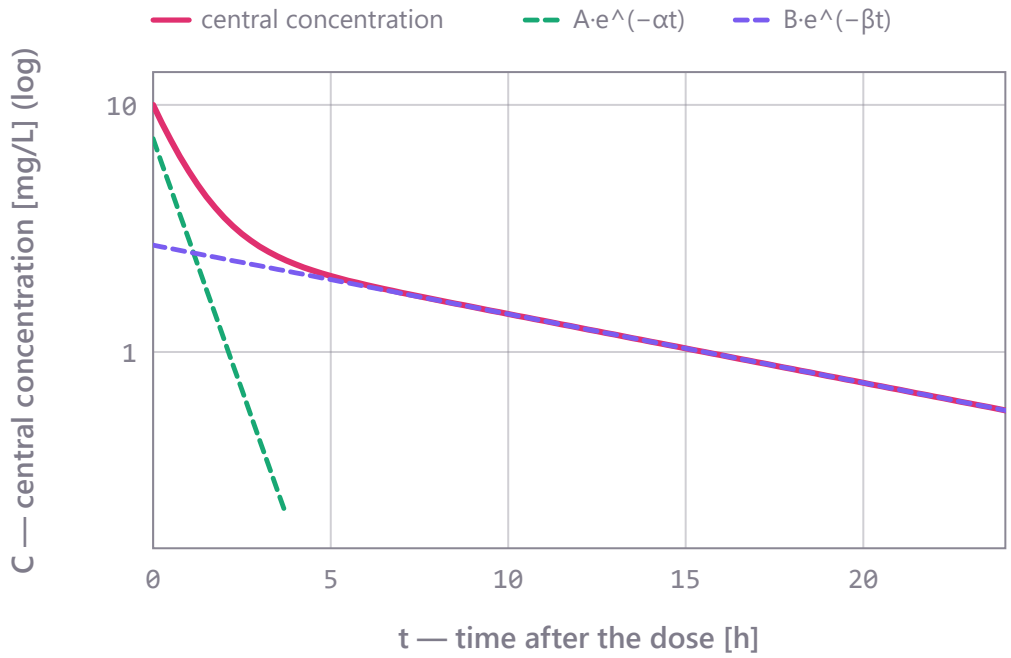
The hybrid rate constants alpha and beta, the coefficients A and B and the central concentration at time t of a two-compartment model after an IV bolus, from the volume V1 and the micro-constants k10, k12, k21.

$$C(t) = Ae^{-\alpha t} + Be^{-\beta t}, \quad \alpha\beta = k_{10}k_{21},\ \alpha + \beta = k_{10} + k_{12} + k_{21}$$

| input | meaning | condition |
|---|---|---|
| `D` | bolus dose | |
| `V1` | central volume | |
| `k10` | elimination rate constant from the central compartment | |
| `k12` | transfer rate constant central to peripheral | |
| `k21` | transfer rate constant peripheral to central | |
| `t` | time after the dose | |

| output | meaning |
|---|---|
| `alpha` | fast (distribution) rate constant |
| `beta` | slow (terminal) rate constant |
| `A` | coefficient of exp(-alpha t) |
| `B` | coefficient of exp(-beta t) |
| `C` | central concentration at time t |

- **Method:** alpha, beta roots of $x^2 - (k10 + k12 + k21)\,x + k10\,k21 = 0$; $A = (D/V_1)(\alpha - k21)/(\alpha - \beta)$, $B = (D/V_1)(k21 - \beta)/(\alpha - \beta)$
- **Assumes:** linear two-compartment model, elimination from the central compartment; $\alpha > \beta$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Two-compartment intravenous bolus (D = 100 mg, V1 = 10 L, k10 = 0.2, k12 = 0.5, k21 = 0.3 1/h): central concentration over 24 h (log axis, 97 runs) and its two exponential phases drawn from the returned A = 7.29416 mg/L, α = 0.93589 1/h (distribution) and B = 2.70584 mg/L, β = 0.0641101 1/h (terminal; derived). Marked in the legend; C(4 h) = 2.26643 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_two_compartment_bolus', D=9.999999999999999e-05, V1=0.01,
k10=5.555555555555556e-05, k12=0.0001388888888888889, k21=8.333333333333333e-05, t=14400.0)
# -> alpha = 0.000259969 1/s, beta = 1.78084e-05 1/s, A = 0.00729416 kg/m^3, B = 0.00270584 kg/m^3,
C = 0.00226643 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_two_compartment_bolus', D=0.0005, V1=0.02,
k10=2.777777777777778e-05, k12=0.0003333333333333333, k21=0.00011111111111111112, t=1800.0)
# -> alpha = 0.000465593 1/s, beta = 6.629e-06 1/s, A = 0.0193088 kg/m^3, B = 0.00569119 kg/m^3, C
= 0.0139756 kg/m^3   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Two-compartment bolus: D = 100 mg, V1 = 10 L, k10 = 0.2 1/h, k12 = 0.5 1/h, k21 = 0.3 1/h, t = 4 h | SOLVED | alpha = 0.000259969 1/s, beta = 1.78084e-05 1/s, A = 0.00729416 kg/m^3, B = 0.00270584 kg/m^3, C = 0.00226643 kg/m^3 |
| Two-compartment bolus: D = 500 mg, V1 = 20 L, k10 = 0.1 1/h, k12 = 1.2 1/h, k21 = 0.4 1/h, t = 30 min | SOLVED | alpha = 0.000465593 1/s, beta = 6.629e-06 1/s, A = 0.0193088 kg/m^3, B = 0.00569119 kg/m^3, C = 0.0139756 kg/m^3 |
| Two-compartment bolus: V1 = 10 L, k10 = 0.2 1/h, k12 = 0.5 1/h, k21 = 0.3 1/h, t = 4 h | NEEDS_INPUT | bmed_pk_two_compartment_bolus needs D (bolus dose). Example: solve('bmed_pk_two_compartment_bolus', D='0.0001 kg', V1='0 |
| Two-compartment bolus: D = 100 mg, V1 = 10 L, k10 = 0.2 1/h, k12 = 0.5 1/h, k21 = 0.3 1/h, t = 4 h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Two-compartment model: micro-constants, clearance and Vss from a biexponential curve (a model calculation, not medical advice) — `bmed_pk_two_compartment_micro_constants`

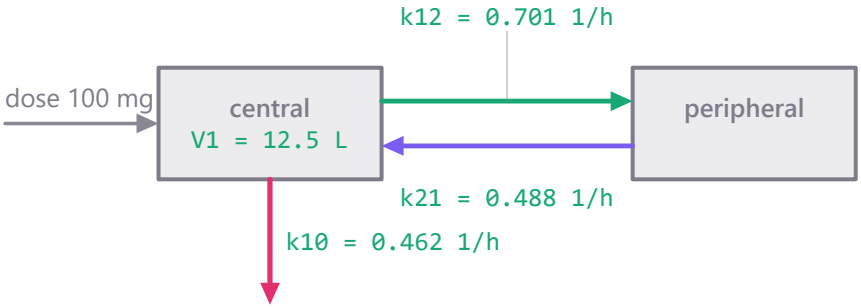
The central volume, the micro rate constants k10, k12, k21, the clearance and the steady-state volume of a two-compartment model from the biexponential fit $C = A$ exp(-alpha t) + B exp(-beta t) of an IV bolus.

$$k_{21} = \frac{A\beta + B\alpha}{A + B}, \quad k_{10} = \frac{\alpha\beta}{k_{21}}, \quad k_{12} = \alpha + \beta - k_{21} - k_{10}$$

| input | meaning | condition |
|---|---|---|
| `D` | bolus dose | |
| `A` | coefficient of the fast exponential | |
| `alpha` | fast rate constant | |
| `B` | coefficient of the slow exponential | |
| `beta` | slow rate constant | |

| output | meaning |
|---|---|
| `V1` | central volume |
| `k10` | elimination rate constant |
| `k12` | central to peripheral |
| `k21` | peripheral to central |
| `CL` | clearance |
| `Vss` | volume of distribution at steady state |

- **Method:** $V_1 = D/(A + B)$; k21 = (A beta + B alpha)/(A + B); $k10 = \alpha\ \beta/k21$; $k12 = \alpha + \beta - k21 - k10$; $CL = k10$ V1; Vss = V1 $(1 + k12/k21)$
- **Assumes:** IV bolus, linear two-compartment model, elimination from the central compartment; $\alpha > \beta > 0$, $A > 0$, $B > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The two-compartment model recovered from the bi-exponential coefficients of the example (D = 100 mg, A = 6 mg/L, α = 1.5 1/h, B = 2 mg/L, β = 0.15 1/h). Marked: V1 = 12.5 L, k10 = 0.461538 1/h, k12 = 0.700962 1/h, k21 = 0.4875 1/h; also returned CL = 5.76923 L/h and Vss = 30.4734 L.

```
biomedical_science_ai999x.solve('bmed_pk_two_compartment_micro_constants', D=9.999999999999999e-05,
A=0.006, alpha=0.0004166666666666664, B=0.002, beta=4.1666666666666665e-05)
# -> V1 = 0.0125 m^3, k10 = 0.000128205 1/s, k12 = 0.000194712 1/s, k21 = 0.000135417 1/s, CL =
1.60256e-06 m^3/s, Vss = 0.0304734 m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_two_compartment_micro_constants', D=0.0005, A=0.02,
alpha=0.0008333333333333333, B=0.005, beta=5.555555555555556e-05)
# -> V1 = 0.02 m^3, k10 = 0.000219298 1/s, k12 = 0.00045848 1/s, k21 = 0.000211111 1/s, CL =
4.38596e-06 m^3/s, Vss = 0.0634349 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Two-compartment micro-constants: D = 100 mg, A = 6 mg/L, alpha = 1.5 1/h, B = 2 mg/L, beta = 0.15 1/h | SOLVED | V1 = 0.0125 m^3, k10 = 0.000128205 1/s, k12 = 0.000194712 1/s, k21 = 0.000135417 1/s, CL = 1.60256e-06 m^3/s, Vss = 0.0304734 m^3 |
| Two-compartment micro-constants: D = 500 mg, A = 20 mg/L, alpha = 3 1/h, B = 5 mg/L, beta = 0.2 1/h | SOLVED | V1 = 0.02 m^3, k10 = 0.000219298 1/s, k12 = 0.00045848 1/s, k21 = 0.000211111 1/s, CL = 4.38596e-06 m^3/s, Vss = 0.0634349 m^3 |
| Two-compartment micro-constants: A = 6 mg/L, alpha = 1.5 1/h, B = 2 mg/L, beta = 0.15 1/h | NEEDS_INPUT | bmed_pk_two_compartment_micro_constants needs D (bolus dose). Example: solve('bmed_pk_two_compartment_micro_constants', |
| Two-compartment micro-constants: D = 100 mg, A = 6 mg/L, alpha = 1.5 1/h, B = 2 mg/L, beta = 0.15 1/h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Two-compartment micro-constants: D = 100 mg, A = | INVALID_INPUT | Two-compartment model: micro-constants, clearance and Vss from a biexponential curve (a model calculation, not medical a |

| question | outcome | result |
|---|---|---|
| 6 mg/L, alpha = 0.1 1/h, B = 2 mg/L, beta = 0.15 1/h | | |

### Absolute bioavailability from IV and oral concentration profiles (a model calculation, not medical advice) — `bmed_pk_bioavailability_from_profiles`

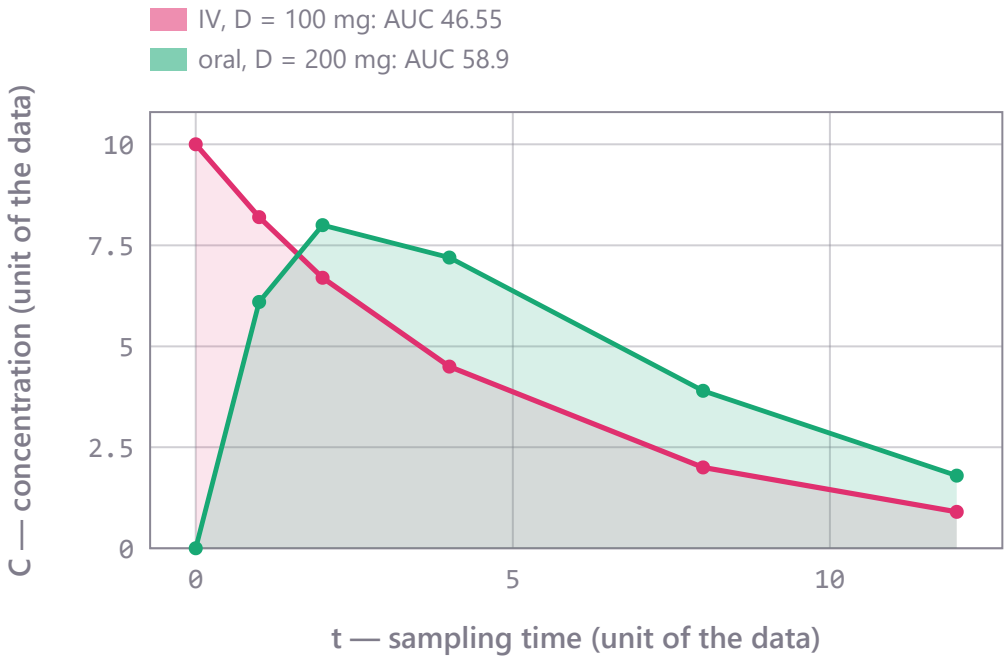
The absolute bioavailability F from the dose-normalised areas under an oral and an IV concentration-time profile (linear trapezoids from the first to the last sample).

$$F = \frac{AUC_{oral}\, D_{iv}}{AUC_{iv}\, D_{oral}}$$

| input | meaning | condition |
|---|---|---|
| `D_iv` | IV dose | |
| `t_iv` | IV sampling times (one time unit for both profiles) | |
| `C_iv` | IV concentrations (one concentration unit for both profiles) | |
| `D_oral` | oral dose | |
| `t_oral` | oral sampling times | |
| `C_oral` | oral concentrations | |

| output | meaning |
|---|---|
| `AUC_iv` | IV area (units of the lists) |
| `AUC_oral` | oral area (units of the lists) |
| `F` | absolute bioavailability (AUC_oral/D_oral)/(AUC_iv/D_iv) |

- **Method:** F = (AUC_oral D_iv)/(AUC_iv D_oral) with linear-trapezoid areas
- **Assumes:** linear pharmacokinetics (dose-proportional areas); both profiles sampled to a comparable end; areas to the last sample, no extrapolation; the same units in both profiles
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The intravenous and oral concentration profiles of the example (inputs) with their areas computed by the package: AUC_iv = 46.55, AUC_oral = 58.9. Dose-normalised, their ratio is the absolute bioavailability F = 0.632653 (marked in the legend).

```
biomedical_science_ai999x.solve('bmed_pk_bioavailability_from_profiles', D_iv=9.999999999999999e-05, t_iv=[0, 1, 2, 4, 8, 12], C_iv=[10, 8.2, 6.7, 4.5, 2.0, 0.9], D_oral=0.00019999999999999998, t_oral=[0, 1, 2, 4, 8, 12], C_oral=[0, 6.1, 8.0, 7.2, 3.9, 1.8])
# -> AUC_iv = 46.55, AUC_oral = 58.9, F = 0.632653   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_bioavailability_from_profiles', D_iv=4.9999999999999996e-05, t_iv=[0, 2, 4, 8], C_iv=[5, 3.5, 2.4, 1.2], D_oral=4.9999999999999996e-05, t_oral=[0, 2, 4, 8], C_oral=[0, 2.2, 2.0, 1.1])
# -> AUC_iv = 21.6, AUC_oral = 12.6, F = 0.583333   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Bioavailability from profiles: D_iv = 100 mg, t_iv = [0, 1, 2, 4, 8, 12], C_iv = [10, 8.2, 6.7, 4.5, 2, 0.9], D_oral = 200 mg, t_oral = [0, 1, 2, 4, 8, 12], C_oral = [0, 6.1, 8, 7.2, 3.9, 1.8] | SOLVED | AUC_iv = 46.55, AUC_oral = 58.9, F = 0.632653 |
| Bioavailability from profiles: D_iv = 50 mg, t_iv = [0, 2, 4, 8], C_iv = [5, 3.5, 2.4, 1.2], D_oral = 50 mg, t_oral = [0, 2, 4, 8], C_oral = [0, 2.2, 2, 1.1] | SOLVED | AUC_iv = 21.6, AUC_oral = 12.6, F = 0.583333 |
| Bioavailability from profiles: t_iv = [0, 1, 2, 4, 8, 12], C_iv = [10, 8.2, 6.7, 4.5, 2, 0.9], D_oral = 200 mg, t_oral = [0, 1, 2, 4, 8, 12], C_oral = [0, 6.1, 8, 7.2, 3.9, 1.8] | NEEDS_INPUT | bmed_pk_bioavailability_from_profiles needs D_iv (IV dose). Example: solve('bmed_pk_bioavailability_from_profiles', D_iv |
| Bioavailability from profiles: D_iv = 100 mg, t_iv = [0, 1, 2, 4, 8, 12], C_iv = [10, 8.2, 6.7, 4.5, 2, 0.9], D_oral = 200 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

| question | outcome | result |
|---|---|---|
| mg, t_oral = [0, 1, 2, 4, 8, 12], C_oral = [0, 6.1, 8, 7.2, 3.9, 1.8], zz = 1 | | |

### Hepatic clearance: well-stirred liver model (a model calculation, not medical advice) — `bmed_pk_hepatic_clearance_well_stirred`

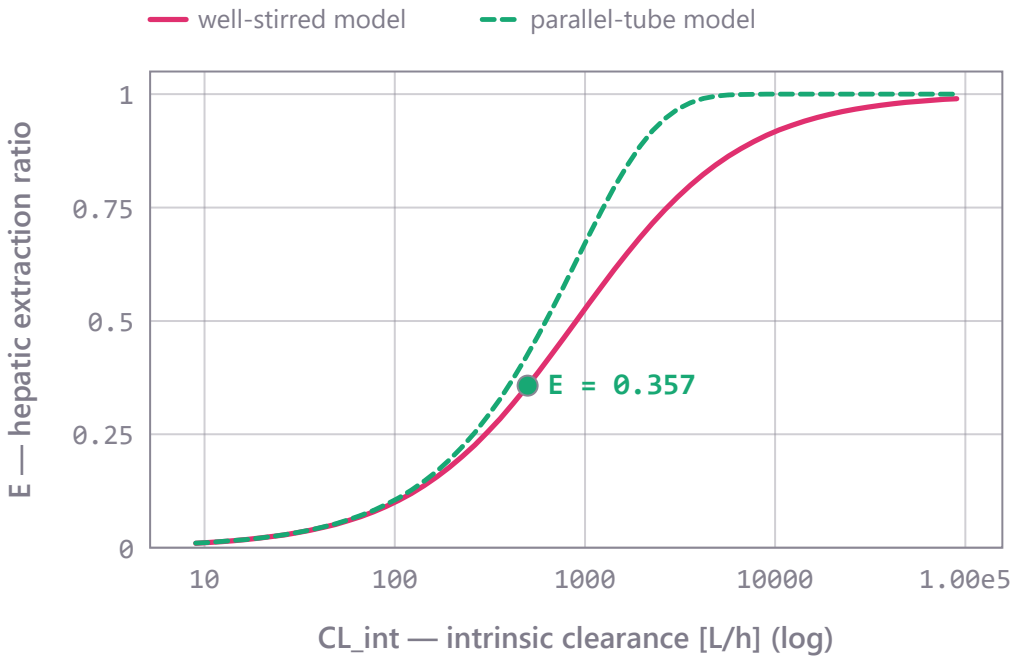
The hepatic clearance and extraction ratio of the well-stirred (venous equilibrium) liver model from hepatic blood flow, unbound fraction and intrinsic clearance.

$$CL_H = \frac{Q\, f_u\, CL_{int}}{Q + f_u\, CL_{int}}$$

| input | meaning | condition |
|---|---|---|
| `Q` | hepatic blood flow | |
| `fu` | unbound fraction in blood $(0 < fu \le 1)$ | |
| `CL_int` | intrinsic clearance of unbound drug | |

| output | meaning |
|---|---|
| `CL_H` | hepatic clearance |
| `E` | extraction ratio |

- **Method:** $CL_H = Q$ fu CL_int / (Q + fu CL_int); $E = CL_H/Q$
- **Assumes:** well-stirred liver: outflow concentration equals the liver concentration; linear elimination
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Hepatic extraction ratio against intrinsic clearance (log axis, 61 runs per model) for hepatic blood flow Q = 90 L/h and unbound fraction fu = 0.1, this capability's well-stirred model beside the parallel-tube model (dashed). Marked: the example CL_int = 500 L/h gives E = 0.357143, hepatic clearance 32.1429 L/h.

```
biomedical_science_ai999x.solve('bmed_pk_hepatic_clearance_well_stirred', Q=2.4999999999999998e-05,
fu=0.1, CL_int=0.0001388888888888889)
# -> CL_H = 8.92857e-06 m^3/s, E = 0.357143   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_hepatic_clearance_well_stirred', Q=2.5e-05, fu=0.5,
CL_int=1e-05)
# -> CL_H = 4.16667e-06 m^3/s, E = 0.166667   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Well-stirred hepatic clearance: Q = 90 L/h, fu = 0.1, CL_int = 500 L/h | SOLVED | CL_H = 8.92857e-06 m^3/s, E = 0.357143 |
| Well-stirred hepatic clearance: Q = 1.5 L/min, fu = 50%, CL_int = 0.6 L/min | SOLVED | CL_H = 4.16667e-06 m^3/s, E = 0.166667 |
| Well-stirred hepatic clearance: fu = 0.1, CL_int = 500 L/h | NEEDS_INPUT | bmed_pk_hepatic_clearance_well_stirred needs Q (hepatic blood flow). Example: solve('bmed_pk_hepatic_clearance_well_stir |
| Well-stirred hepatic clearance: Q = 90 L/h, fu = 0.1, CL_int = 500 L/h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Hepatic clearance: parallel-tube (sinusoidal) liver model (a model calculation, not medical advice) — `bmed_pk_hepatic_clearance_parallel_tube`

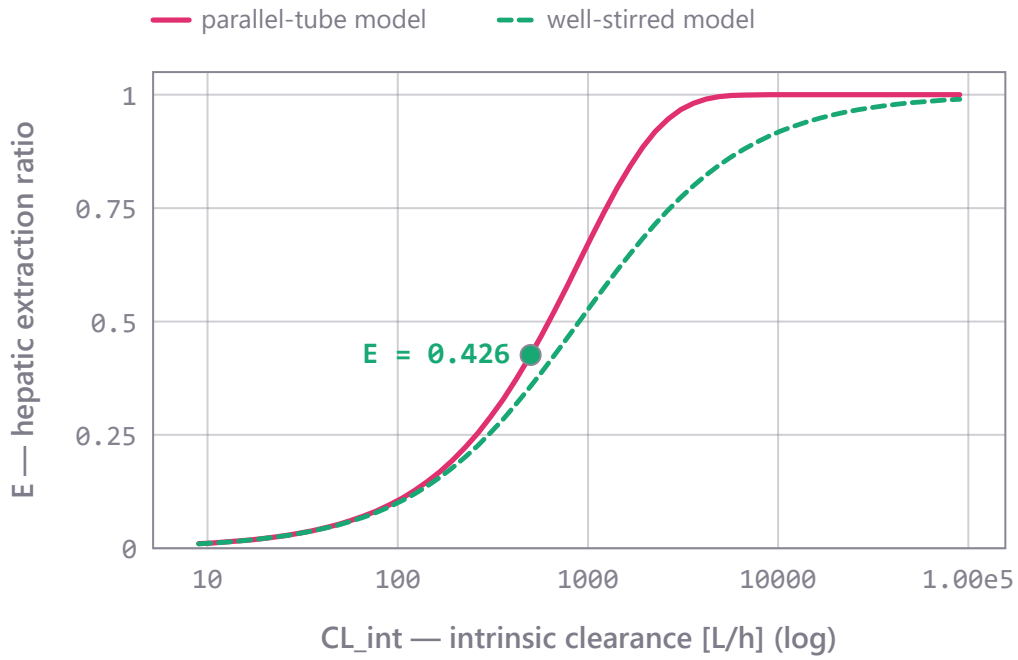
The hepatic clearance and extraction ratio of the parallel-tube liver model, in which the concentration falls along the sinusoids, from blood flow, unbound fraction and intrinsic clearance.

$$CL_H = Q\left(1 - e^{-f_u CL_{int}/Q}\right)$$

| input | meaning | condition |
|---|---|---|
| `Q` | hepatic blood flow | |
| `fu` | unbound fraction in blood $(0 < fu \leq 1)$ | |
| `CL_int` | intrinsic clearance of unbound drug | |

| output | meaning |
|---|---|
| `CL_H` | hepatic clearance |
| `E` | extraction ratio |

- **Method:** $E = 1 - exp$(-fu CL_int / Q); $CL_H = Q$ E
- **Assumes:** plug flow along identical sinusoids, enzymes evenly distributed; linear elimination
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Hepatic extraction ratio against intrinsic clearance (log axis, 61 runs per model) for hepatic blood flow Q = 90 L/h and unbound fraction fu = 0.1, this capability's parallel-tube model beside the well-stirred model (dashed). Marked: the example CL_int = 500 L/h gives E = 0.426247, hepatic clearance 38.3622 L/h.

```
biomedical_science_ai999x.solve('bmed_pk_hepatic_clearance_parallel_tube', Q=2.4999999999999998e-05, fu=0.1, CL_int=0.0001388888888888889)
# -> CL_H = 1.06562e-05 m^3/s, E = 0.426247   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_hepatic_clearance_parallel_tube', Q=2.5e-05, fu=0.5, CL_int=1e-05)
# -> CL_H = 4.53173e-06 m^3/s, E = 0.181269   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Parallel-tube hepatic clearance: Q = 90 L/h, fu = 0.1, CL_int = 500 L/h | SOLVED | CL_H = 1.06562e-05 m^3/s, E = 0.426247 |
| Parallel-tube hepatic clearance: Q = 1.5 L/min, fu = 50%, CL_int = 0.6 L/min | SOLVED | CL_H = 4.53173e-06 m^3/s, E = 0.181269 |
| Parallel-tube hepatic clearance: fu = 0.1, CL_int = 500 L/h | NEEDS_INPUT | bmed_pk_hepatic_clearance_parallel_tube needs Q (hepatic blood flow). Example: solve('bmed_pk_hepatic_clearance_parallel |
| Parallel-tube hepatic clearance: Q = 90 L/h, fu = 0.1, CL_int = 500 L/h, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Effect-compartment model: effect-site concentration and its peak after an IV bolus (a model calculation, not medical advice) — `bmed_pk_effect_site_concentration`

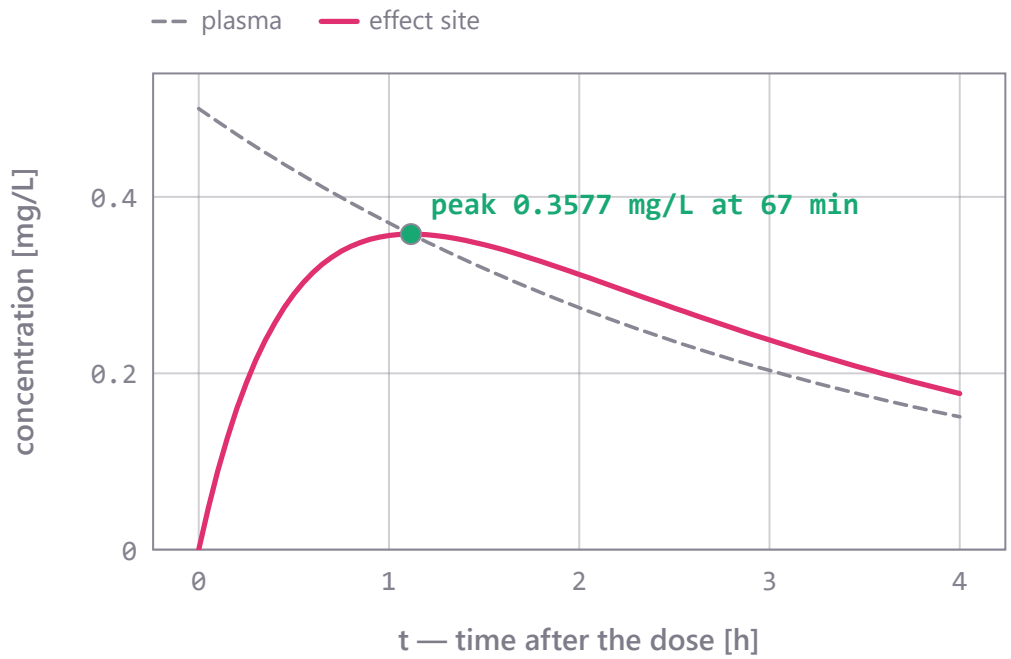
The effect-site concentration of a one-compartment model linked to an effect compartment (rate ke0) after an IV bolus, the time of its peak and the peak value.

$$C_e(t) = \frac{D}{V}\,\frac{k_{e0}}{k_{e0}-k}\left(e^{-kt} - e^{-k_{e0}t}\right), \quad t_{peak} = \frac{\ln(k_{e0}/k)}{k_{e0}-k}$$

| input | meaning | condition |
|---|---|---|
| `D` | bolus dose | |
| `V` | volume of distribution | |
| `k` | elimination rate constant | |
| `ke0` | effect-site equilibration rate constant | |
| `t` | time after the dose | |

| output | meaning |
|---|---|
| `Ce` | effect-site concentration at time t |
| `t_peak` | time of the effect-site peak |
| `Ce_peak` | peak effect-site concentration |

- **Method:** dCe/dt = ke0 (C - Ce): $Ce = (D/V)\,ke0/(ke0 - k)$ (exp(-k t) - exp(-ke0 t)); peak where $Ce = C$
- **Assumes:** one compartment plus an effect compartment of negligible volume; $ke0 \neq k$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Effect-site concentration after an intravenous bolus (D = 10 mg, V = 20 L, k = 0.3 1/h, ke0 = 2 1/h), 81 runs over 4 h, beside the plasma concentration of the same bolus (the package's one-compartment bolus). The effect site lags the plasma. Marked: t_peak = 66.9572 min, Ce_peak = 0.357746 mg/L.

```
biomedical_science_ai999x.solve('bmed_pk_effect_site_concentration', D=9.999999999999999e-06,
V=0.02, k=8.333333333333333e-05, ke0=0.0005555555555555556, t=1800.0)
# -> Ce = 0.000289899 kg/m^3, t_peak = 4017.43 s, Ce_peak = 0.000357746 kg/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_pk_effect_site_concentration', D=9.999999999999999e-05,
V=0.05, k=2.777777777777778e-05, ke0=0.0001388888888888889, t=14400.0)
# -> Ce = 0.00133746 kg/m^3, t_peak = 14484.9 s, Ce_peak = 0.00133748 kg/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Effect-site concentration: D = 10 mg, V = 20 L, k = 0.3 1/h, ke0 = 2 1/h, t = 30 min | SOLVED | Ce = 0.000289899 kg/m^3, t_peak = 4017.43 s, Ce_peak = 0.000357746 kg/m^3 |
| Effect-site concentration: D = 100 mg, V = 50 L, k = 0.1 1/h, ke0 = 0.5 1/h, t = 4 h | SOLVED | Ce = 0.00133746 kg/m^3, t_peak = 14484.9 s, Ce_peak = 0.00133748 kg/m^3 |
| Effect-site concentration: V = 20 L, k = 0.3 1/h, ke0 = 2 1/h, t = 30 min | NEEDS_INPUT | bmed_pk_effect_site_concentration needs D (bolus dose). Example: solve('bmed_pk_effect_site_concentration', D='1e-05 kg' |
| Effect-site concentration: D = 10 mg, V = 20 L, k = 0.3 1/h, ke0 = 2 1/h, t = 30 min, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## SIR epidemic model: final size (final susceptible fraction and attack fraction) — `bmed_epi_sir_final_size`

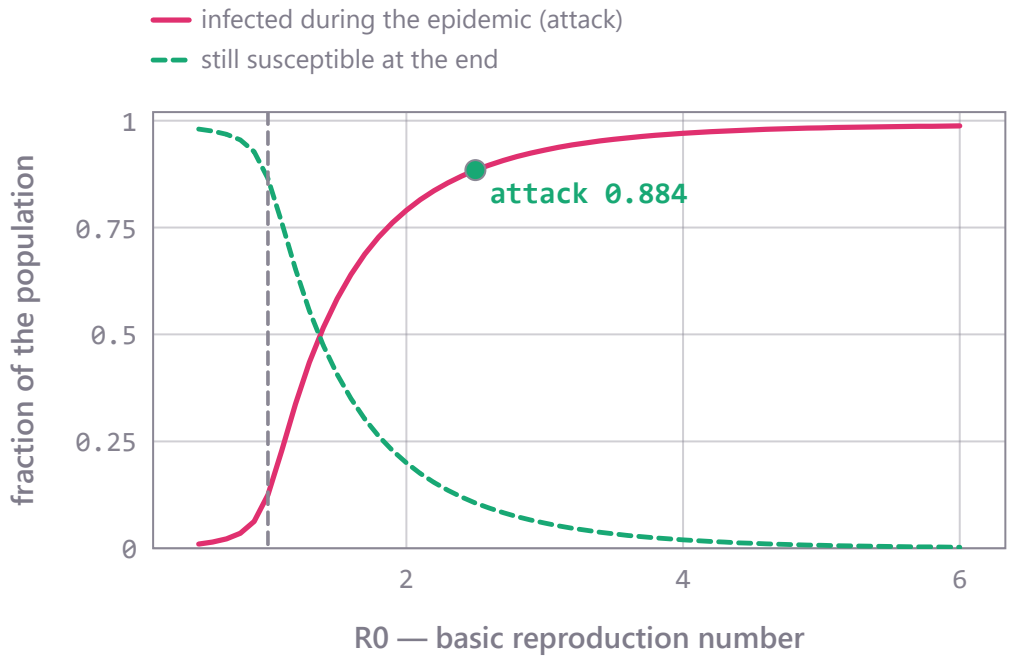
The fraction still susceptible when an SIR epidemic ends and the fraction infected during it, from R0 and the initial susceptible and infected fractions (the final-size relation).

$$\ln \frac{s_0}{s_\infty} = R_0 \left( s_0 + i_0 - s_\infty \right)$$

| input | meaning | condition |
|---|---|---|
| `R0` | basic reproduction number beta/gamma | |
| `s0` | initial susceptible fraction | |
| `i0` | initial infected fraction (> 0) | |

| output | meaning |
|---|---|
| `s_inf` | susceptible fraction at the end |
| `attack` | fraction infected during the epidemic, s0 - s_inf |

- **Method:** final-size relation ln(s0/s_inf) = R0 (s0 + i0 - s_inf), solved by bisection on (0, min(s0, 1/R0)]
- **Assumes:** closed population, no births or deaths, homogeneous mixing; fractions of the population; the rest of the population $(1 - s_0 - i_0)$ is immune
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Final size of an SIR epidemic against R0 (s0 = 0.99, i0 = 0.01), 56 runs from 0.5 to 6: below R0 = 1 almost nobody beyond the initial cases is infected, above it the attack fraction rises steeply. Marked: the example R0 = 2.5, attack 0.884106, s_inf = 0.105894.

```
biomedical_science_ai999x.solve('bmed_epi_sir_final_size', R0=2.5, s0=0.99, i0=0.01)
# -> s_inf = 0.105894, attack = 0.884106   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_sir_final_size', R0=1.5, s0=0.7, i0=0.001)
# -> s_inf = 0.618785, attack = 0.0812154   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SIR final size: R0 = 2.5, s0 = 0.99, i0 = 0.01 | SOLVED | s_inf = 0.105894, attack = 0.884106 |
| SIR final size: R0 = 1.5, s0 = 0.7, i0 = 0.001 | SOLVED | s_inf = 0.618785, attack = 0.0812154 |
| SIR final size: s0 = 0.99, i0 = 0.01 | NEEDS_INPUT | bmed_epi_sir_final_size needs R0 (basic reproduction number beta/gamma). Example: solve('bmed_epi_sir_final_size', R0=2. |
| SIR final size: R0 = 2.5, s0 = 0.99, i0 = 0.01, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| SIR final size: R0 = 2.5, s0 = 99%, i0 = 1% | SOLVED | s_inf = 0.105894, attack = 0.884106 |
| SIR final size: R0 = 2.5, s0 = 0.99, i0 = 0 | INVALID_INPUT | SIR epidemic model: final size (final susceptible fraction and attack fraction): i0 must be a fraction between 0 and 1. |

## SEIR epidemic model: final size — `bmed_epi_seir_final_size`

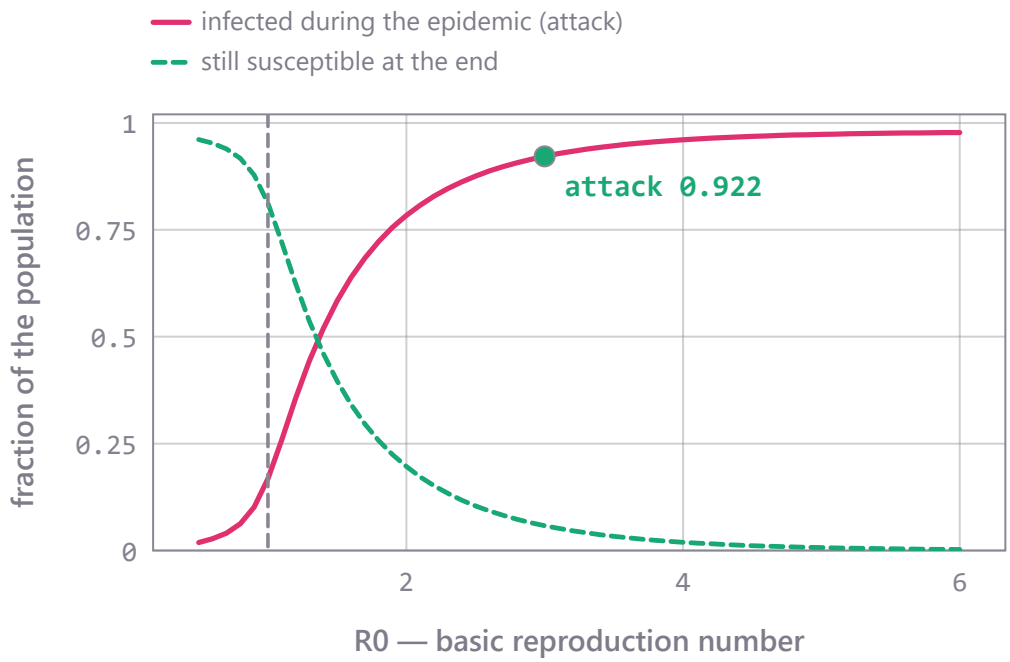
The final susceptible fraction and the attack fraction of an SEIR epidemic (the latent stage does not change the final size) from R0 and the initial susceptible, exposed and infectious fractions.

$$\ln \frac{s_0}{s_\infty} = R_0 \left( s_0 + e_0 + i_0 - s_\infty \right)$$

| input | meaning | condition |
|---|---|---|
| `R0` | basic reproduction number beta/gamma | |
| `s0` | initial susceptible fraction | |
| `e0` | initial exposed (latent) fraction | |
| `i0` | initial infectious fraction | |

| output | meaning |
|---|---|
| `s_inf` | susceptible fraction at the end |
| `attack` | fraction infected during the epidemic, s0 - s_inf |

- **Method:** final-size relation ln(s0/s_inf) = R0 (s0 + e0 + i0 - s_inf), solved by bisection
- **Assumes:** closed population, homogeneous mixing; exponentially distributed latent and infectious periods; $e_0 + i_0 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Final size of an SEIR epidemic against R0 (s0 = 0.98, e0 = 0.01, i0 = 0.01), 56 runs from 0.5 to 6: below R0 = 1 almost nobody beyond the initial cases is infected, above it the attack fraction rises steeply. Marked: the example R0 = 3, attack 0.921922, s_inf = 0.058078.

```
biomedical_science_ai999x.solve('bmed_epi_seir_final_size', R0=3, s0=0.98, e0=0.01, i0=0.01)
# -> s_inf = 0.058078, attack = 0.921922   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_seir_final_size', R0=1.8, s0=0.6, e0=0.002, i0=0)
# -> s_inf = 0.493974, attack = 0.106026   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SEIR final size: R0 = 3, s0 = 0.98, e0 = 0.01, i0 = 0.01 | SOLVED | s_inf = 0.058078, attack = 0.921922 |

| question | outcome | result |
|---|---|---|
| SEIR final size: R0 = 1.8, s0 = 0.6, e0 = 0.002, i0 = 0 | SOLVED | s_inf = 0.493974, attack = 0.106026 |
| SEIR final size: s0 = 0.98, e0 = 0.01, i0 = 0.01 | NEEDS_INPUT | bmed_epi_seir_final_size needs R0 (basic reproduction number beta/gamma). Example: solve('bmed_epi_seir_final_size', R0= |
| SEIR final size: R0 = 3, s0 = 0.98, e0 = 0.01, i0 = 0.01, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### SIR epidemic model: peak prevalence and the time of the peak — `bmed_epi_sir_peak`

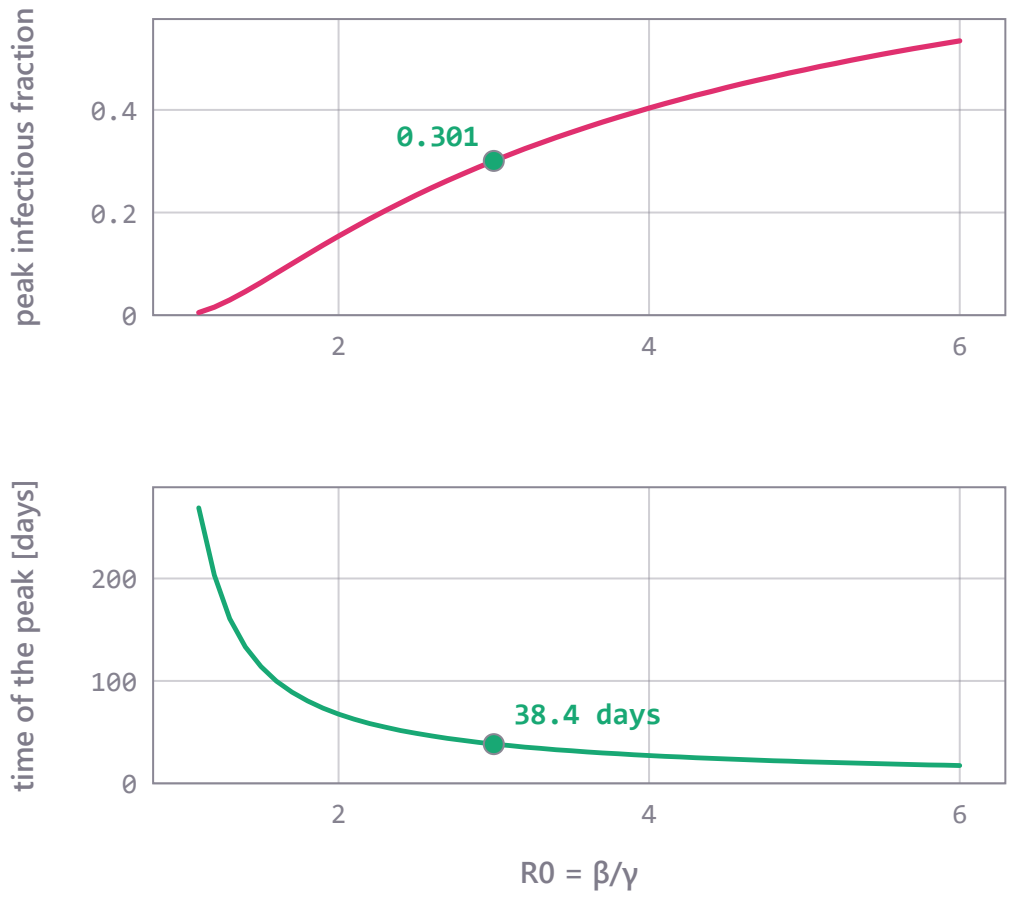
The largest infected fraction of an SIR epidemic and the time at which it occurs, from the transmission and recovery rates and the initial fractions.

$$i_{max} = s_0 + i_0 - \frac{1 + \ln(R_0 s_0)}{R_0}, \quad t_{peak} = \int_{1/R_0}^{s_0} \frac{ds}{\beta s\, i(s)}$$

| input | meaning | condition |
|---|---|---|
| `beta` | transmission rate | |
| `gamma` | recovery rate | |
| `s0` | initial susceptible fraction | |
| `i0` | initial infected fraction (> 0) | |

| output | meaning |
|---|---|
| `i_max` | peak infected fraction |
| `t_peak` | time of the peak |

- **Method:** i_max = s0 + i0 - (1 + ln(R0 s0))/R0 from the first integral of the SIR system; t_peak = integral over s from $1/R_0$ to s0 of ds / (beta s i(s)) (mpmath quadrature)
- **Assumes:** closed population, homogeneous mixing, frequency-dependent transmission; R0 $s_0 > 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Peak of an SIR epidemic against R0 = β/γ (β varied, infectious period 10 days, s0 = 0.999, i0 = 0.001), 50 runs from 1.1 to 6: a higher R0 gives a higher, earlier peak. Marked: the example β = 0.3 1/day (R0 = 3), peak fraction 0.300796 at 38.3553 days.

```
biomedical_science_ai999x.solve('bmed_epi_sir_peak', beta=3.472222222222222e-06,
gamma=1.1574074074074074e-06, s0=0.999, i0=0.001)
# -> i_max = 0.300796, t_peak = 3.3139e+06 s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_sir_peak', beta=5.787037037037037e-06,
gamma=2.3148148148148148e-06, s0=0.9, i0=0.01)
# -> i_max = 0.185628, t_peak = 1.56867e+06 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SIR peak: beta = 0.3 1/day, gamma = 0.1 1/day, s0 = 0.999, i0 = 0.001 | SOLVED | i_max = 0.300796, t_peak = 3.3139e+06 s |
| SIR peak: beta = 0.5 1/day, gamma = 0.2 1/day, s0 = 0.9, i0 = 0.01 | SOLVED | i_max = 0.185628, t_peak = 1.56867e+06 s |
| SIR peak: gamma = 0.1 1/day, s0 = 0.999, i0 = 0.001 | NEEDS_INPUT | bmed_epi_sir_peak needs beta (transmission rate). Example: solve('bmed_epi_sir_peak', beta='3.47222e-06 1/s', gamma='1.1 |
| SIR peak: beta = 0.3 1/day, gamma = 0.1 1/day, s0 = 0.999, i0 = 0.001, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

| question | outcome | result |
|---|---|---|
| SIR peak: beta = 0.1 1/day, gamma = 0.2 1/day, s0 = 0.999, i0 = 0.001 | INVALID_INPUT | SIR epidemic model: peak prevalence and the time of the peak: no epidemic peak: R0 s0 must exceed 1 (prevalence only fal |

### SEIR epidemic model: initial exponential growth rate from R0 — `bmed_epi_seir_growth_rate`

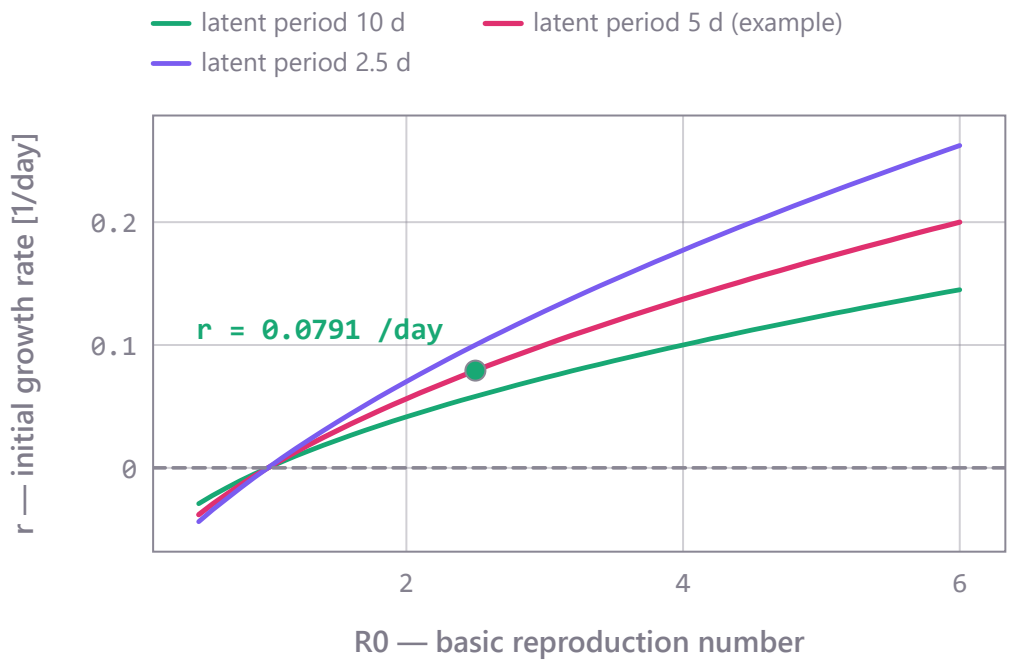
The early exponential growth (or decline) rate of an SEIR epidemic in a fully susceptible population, from R0, the rate of leaving the latent stage and the recovery rate.

$$r = \frac{-(\sigma+\gamma) + \sqrt{(\sigma+\gamma)^2 + 4\sigma\gamma(R_0-1)}}{2}$$

| input | meaning | condition |
|---|---|---|
| `R0` | basic reproduction number beta/gamma | |
| `sigma` | rate of leaving the latent stage (1/latent period) | |
| `gamma` | recovery rate (1/infectious period) | |

| output | meaning |
|---|---|
| `r` | initial growth rate (negative when R0 < 1) |

- **Method:** largest root of $r^2 + (\sigma+\gamma)$ r - sigma gamma $(R_0 - 1) = 0$
- **Assumes:** fully susceptible population at the start; exponential latent and infectious periods
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Initial growth rate of an SEIR epidemic against R0 for an infectious period of 10 days (γ = 0.1 1/day) and three latent periods (1/σ), 56 runs each: negative below R0 = 1, slower with a longer latent period. Marked: example R0 = 2.5, r = 0.0791288 per day (doubling time 8.75973 days, derived).

```
biomedical_science_ai999x.solve('bmed_epi_seir_growth_rate', R0=2.5, sigma=2.3148148148148148e-06,
gamma=1.1574074074074074e-06)
# -> r = 9.15842e-07 1/s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_seir_growth_rate', R0=0.8, sigma=5.787037037037037e-06,
gamma=2.8935185185185184e-06)
# -> r = -4.04667e-07 1/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SEIR growth rate: R0 = 2.5, sigma = 0.2 1/day, gamma = 0.1 1/day | SOLVED | r = 9.15842e-07 1/s |
| SEIR growth rate: R0 = 0.8, sigma = 0.5 1/day, gamma = 0.25 1/day | SOLVED | r = -4.04667e-07 1/s |
| SEIR growth rate: sigma = 0.2 1/day, gamma = 0.1 1/day | NEEDS_INPUT | bmed_epi_seir_growth_rate needs R0 (basic reproduction number beta/gamma). Example: solve('bmed_epi_seir_growth_rate', R |
| SEIR growth rate: R0 = 2.5, sigma = 0.2 1/day, gamma = 0.1 1/day, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Reproduction number from an epidemic growth rate (gamma-distributed generation interval) — `bmed_epi_r_from_growth_rate`

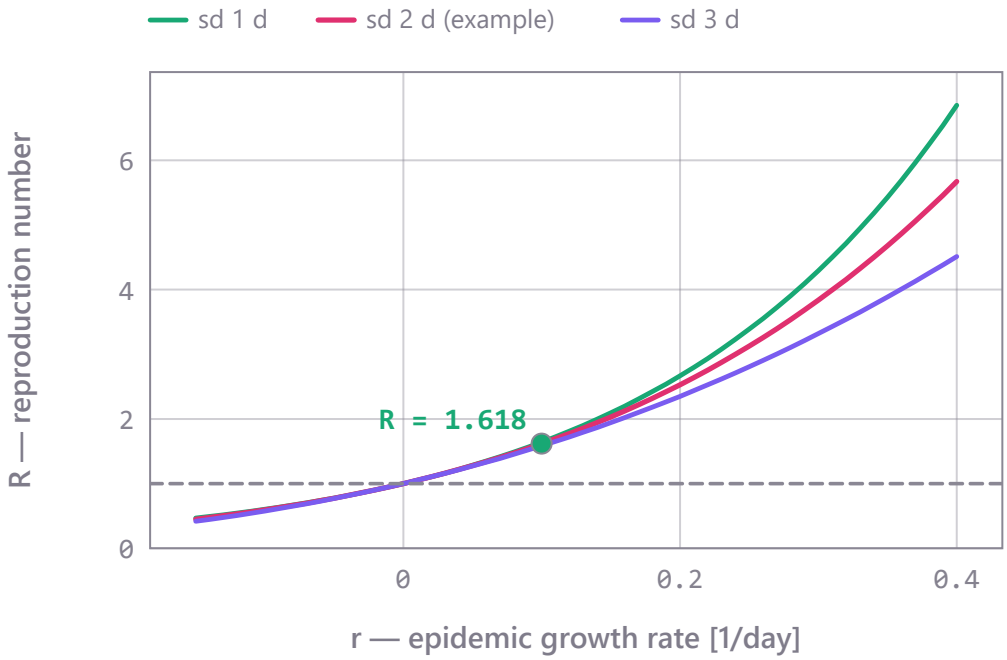
The reproduction number implied by an observed exponential growth rate when the generation interval follows a gamma distribution of stated mean and standard deviation (Wallinga-Lipsitch).

$$R = \left(1 + r\,\frac{s^2}{\mu}\right)^{\mu^2/s^2}$$

| input | meaning | condition |
|---|---|---|
| `r` | exponential growth rate (negative for a decline) | |
| `Tg_mean` | mean generation interval | |
| `Tg_sd` | standard deviation of the generation interval | |

| output | meaning |
|---|---|
| `R` | reproduction number |

- **Method:** $R = 1/M$(-r), M the moment generating function of the gamma distribution: R = (1 + r theta)^k, k = mean^2/sd^2, theta = sd^2/mean
- **Assumes:** exponential growth or decline; gamma-distributed generation interval
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Reproduction number implied by an observed growth rate, for a generation interval of mean 5 days and three standard deviations (56 runs each): R = 1 at zero growth; a wider interval lowers R for the same growth. Marked: the example r = 0.1 per day gives R = 1.6177.

```
biomedical_science_ai999x.solve('bmed_epi_r_from_growth_rate', r=1.1574074074074074e-06,
Tg_mean=432000.0, Tg_sd=172800.0)
# -> R = 1.6177   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_r_from_growth_rate', r=-5.787037037037037e-07,
Tg_mean=561600.0, Tg_sd=345600.0)
# -> R = 0.70694   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Reproduction number from growth rate: r = 0.1 1/day, Tg_mean = 5 day, Tg_sd = 2 day | SOLVED | R = 1.6177 |
| Reproduction number from growth rate: r = -0.05 1/day, Tg_mean = 6.5 day, Tg_sd = 4 day | SOLVED | R = 0.70694 |
| Reproduction number from growth rate: Tg_mean = 5 day, Tg_sd = 2 day | NEEDS_INPUT | bmed_epi_r_from_growth_rate needs r (exponential growth rate (negative for a decline)). Example: solve('bmed_epi_r_from_ |
| Reproduction number from growth rate: r = 0.1 1/day, Tg_mean = 5 day, Tg_sd = 2 day, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## SIS epidemic model: prevalence at a time and the endemic level — `bmed_epi_sis_prevalence`

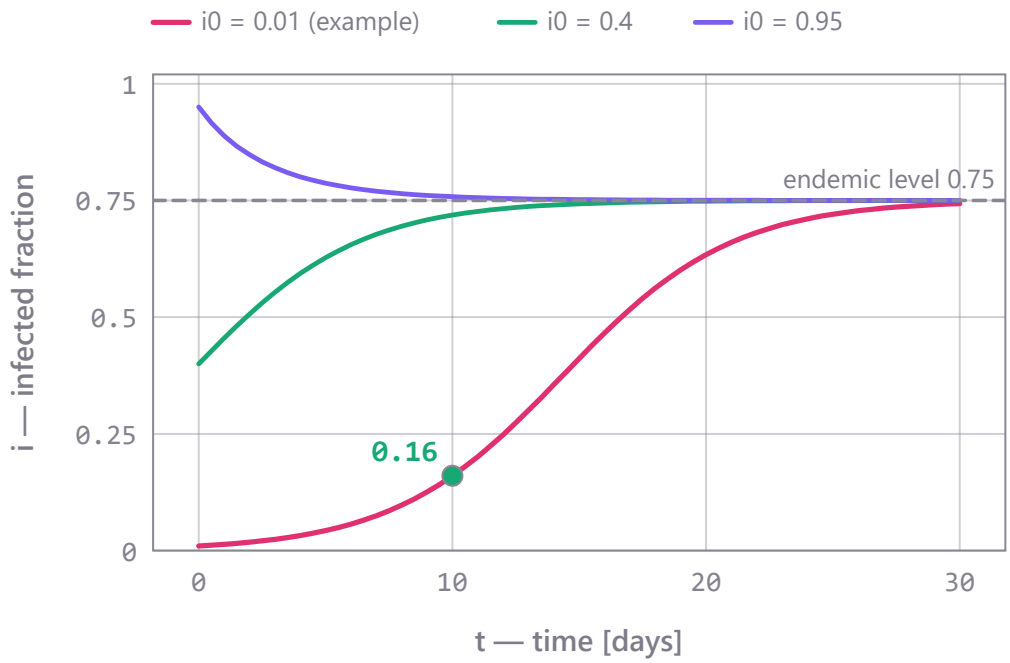
The infected fraction of an SIS model (no immunity after recovery) at time t and the endemic equilibrium it approaches.

$$i(t) = \frac{i_0\, e^{rt}}{1 + i_0\, \beta\, (e^{rt} - 1)/r}, \quad r = \beta - \gamma, \quad i^* = 1 - \gamma/\beta$$

| input | meaning | condition |
|---|---|---|
| `beta` | transmission rate | |
| `gamma` | recovery rate | |
| `i0` | initial infected fraction | |
| `t` | time | |

| output | meaning |
|---|---|
| `i` | infected fraction at time t |
| `i_star` | endemic infected fraction (0 when beta <= gamma) |

- **Method:** logistic solution of $di/dt = \beta$ i $(1 - i) - \gamma$ i: $i = i_0$ e^{rt} / (1 + i0 beta (e^{rt} - 1)/r), $r = \beta - \gamma$
- **Assumes:** closed population, homogeneous mixing; recovered individuals are susceptible again
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




SIS infection (β = 0.4 1/day, γ = 0.1 1/day): the infected fraction over 30 days from three starting fractions, 61 runs each; every start settles on the returned endemic level i* = 0.75. Marked: the example at t = 10 days, i = 0.160111.

```
biomedical_science_ai999x.solve('bmed_epi_sis_prevalence', beta=4.6296296296296e-06,
gamma=1.1574074074074e-06, i0=0.01, t=864000.0)
# -> i = 0.160111, i_star = 0.75   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_sis_prevalence', beta=1.1574074074074e-06,
gamma=2.3148148148148e-06, i0=0.3, t=432000.0)
# -> i = 0.162748, i_star = 0   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SIS prevalence: beta = 0.4 1/day, gamma = 0.1 1/day, i0 = 0.01, t = 10 day | SOLVED | i = 0.160111, i_star = 0.75 |
| SIS prevalence: beta = 0.1 1/day, gamma = 0.2 1/day, i0 = 0.3, t = 5 day | SOLVED | i = 0.162748, i_star = 0 |
| SIS prevalence: gamma = 0.1 1/day, i0 = 0.01, t = 10 day | NEEDS_INPUT | bmed_epi_sis_prevalence needs beta (transmission rate). Example: solve('bmed_epi_sis_prevalence', beta='4.62963e-06 1/s' |
| SIS prevalence: beta = 0.4 1/day, gamma = 0.1 1/day, i0 = 0.01, t = 10 day, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Reed-Frost chain-binomial epidemic: distribution and mean of the final size — `bmed_epi_reed_frost_final_size`

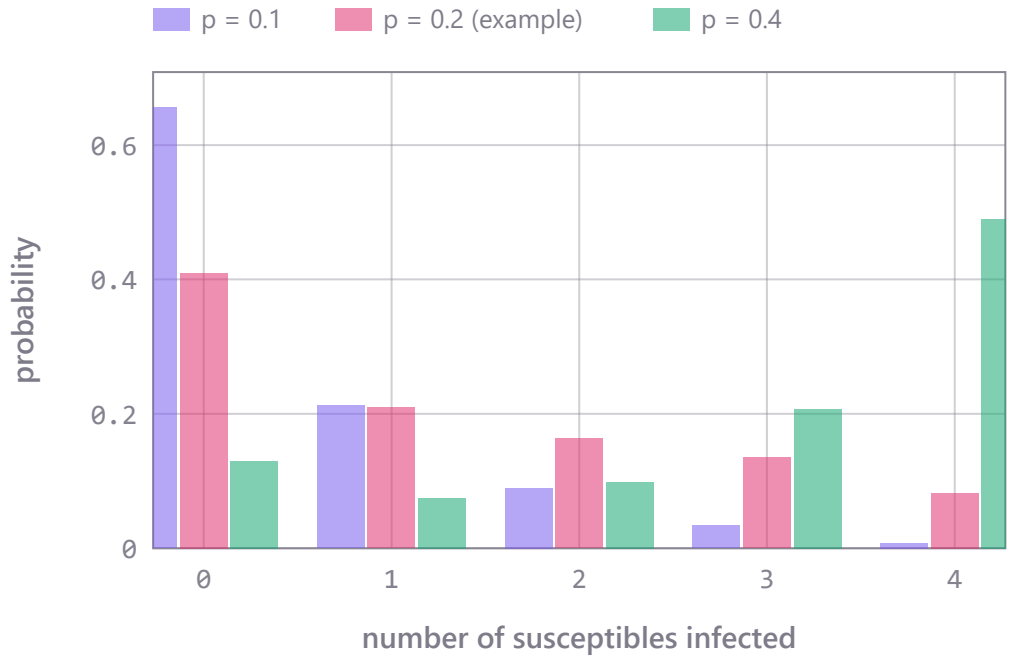
The probability distribution of the number of susceptibles eventually infected in a Reed-Frost chain-binomial epidemic in a small closed group, its mean and the chance of no spread.

$$S_{t+1} \sim \mathrm{Bin}\big(S_t, (1-p)^{I_t}\big), \quad I_{t+1} = S_t - S_{t+1}$$

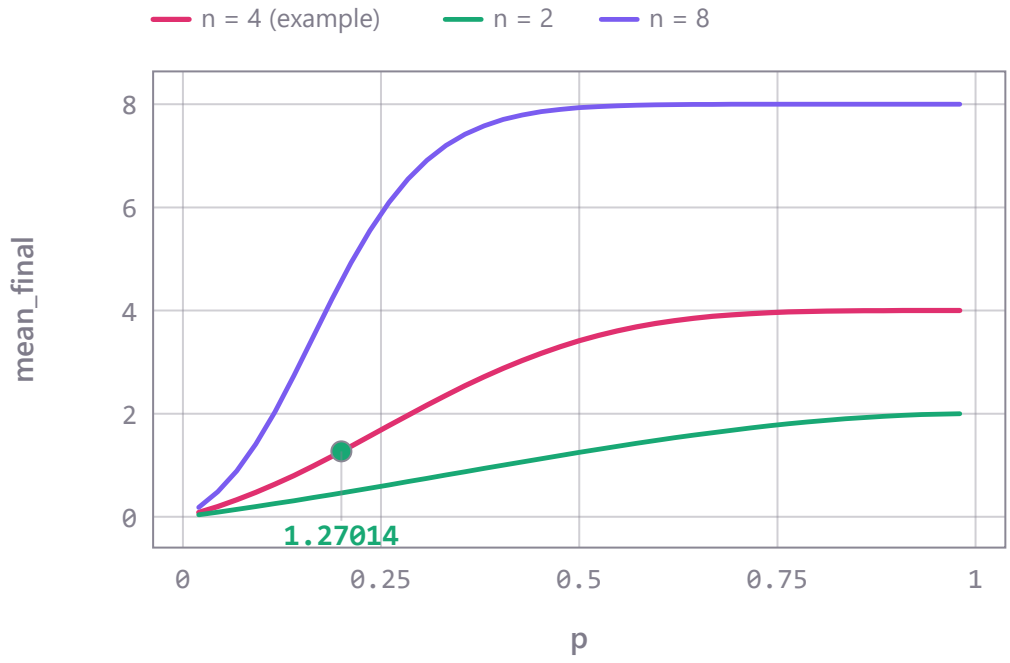
| input | meaning | condition |
|---|---|---|
| `n` | initial number of susceptibles (1 to 30) | |
| `m` | initial number of infectives (1 to 30) | |
| `p` | probability that one infective infects one given susceptible in a generation $(0 < p < 1)$ | |

| output | meaning |
|---|---|
| `mean_final` | expected number of susceptibles infected |
| `P_no_spread` | probability that no susceptible is infected |
| `distribution` | probabilities of 0, 1, ..., n susceptibles infected |

- **Method:** Markov chain over (susceptibles, infectives): a susceptible escapes i infectives with probability $(1-p)^i$; new infections are binomial; probabilities summed over all paths
- **Assumes:** discrete generations, each infective infectious for one generation; independent contacts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Reed–Frost chain-binomial final size: probability that 0, 1, ..., 4 of 4 susceptibles are infected, starting from 1 infective(s), for three per-contact infection probabilities (one run each). A small p mostly gives no spread; a larger p shifts the mass to a large outbreak. Marked: p = 0.1: mean 0.525363, no spread 0.6561; p = 0.2: mean 1.27014, no spread 0.4096; p = 0.4: mean 2.85302, no spread 0.1296.




mean_final (expected number of susceptibles infected) against p (probability that one infective infects one given susceptible in a generation (0 < p < 1)) from 0.02 to 0.98. One curve for each value of n (initial number of susceptibles (1 to 30)): 4, 2, 8. Other inputs as in the worked example: m = 1. 41 points per curve computed by biomedical-science-ai999x; marked: the worked example, p = 0.2, mean_final = 1.27014.

```
biomedical_science_ai999x.solve('bmed_epi_reed_frost_final_size', n=4, m=1, p=0.2)
# -> mean_final = 1.27014, P_no_spread = 0.4096, distribution = [0.4096, 0.2097152, 0.163577856,
0.1351614464, 0.0819454976]   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_reed_frost_final_size', n=10, m=2, p=0.05)
# -> mean_final = 1.53511, P_no_spread = 0.358486, distribution = [0.358485922409, 0.244085487499,
0.159570440671, 0.10291044883, 0.0640931869702, 0.0374640511926, 0.0198552532867, 0.00911594235476,
0.00338709781558, 0.000902915019467, 0.000129253952282]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Reed-Frost final size: n = 4, m = 1, p = 0.2 | SOLVED | mean_final = 1.27014, P_no_spread = 0.4096, distribution = [0.4096, 0.2097152, 0.163577856, 0.1351614464, 0.0819454976] |
| Reed-Frost final size: n = 10, m = 2, p = 0.05 | SOLVED | mean_final = 1.53511, P_no_spread = 0.358486, distribution = [0.358485922409, 0.244085487499, 0.159570440671, 0.10291044883, 0.0640931869702, 0.0374640511926, 0.0198552532867, 0.00911594235476, 0.00338709781558, 0.000902915019467, 0.000129253952282] |
| Reed-Frost final size: m = 1, p = 0.2 | NEEDS_INPUT | bmed_epi_reed_frost_final_size needs n (initial number of susceptibles (1 to 30)). Example: solve('bmed_epi_reed_frost_f |
| Reed-Frost final size: n = 4, m = 1, p = 0.2, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Reed-Frost final size: n = 4.5, m = 1, p = 0.2 | INVALID_INPUT | n: expected a whole number written exactly, got '4.5'. |
| Reed-Frost final size: n = 4, m = 1, p = 1.2 | INVALID_INPUT | Reed-Frost chain-binomial epidemic: distribution and mean of the final size: p must be a fraction between 0 and 1. |

## ROC curve area of a diagnostic score (Mann-Whitney form) — `bmed_dx_roc_auc`

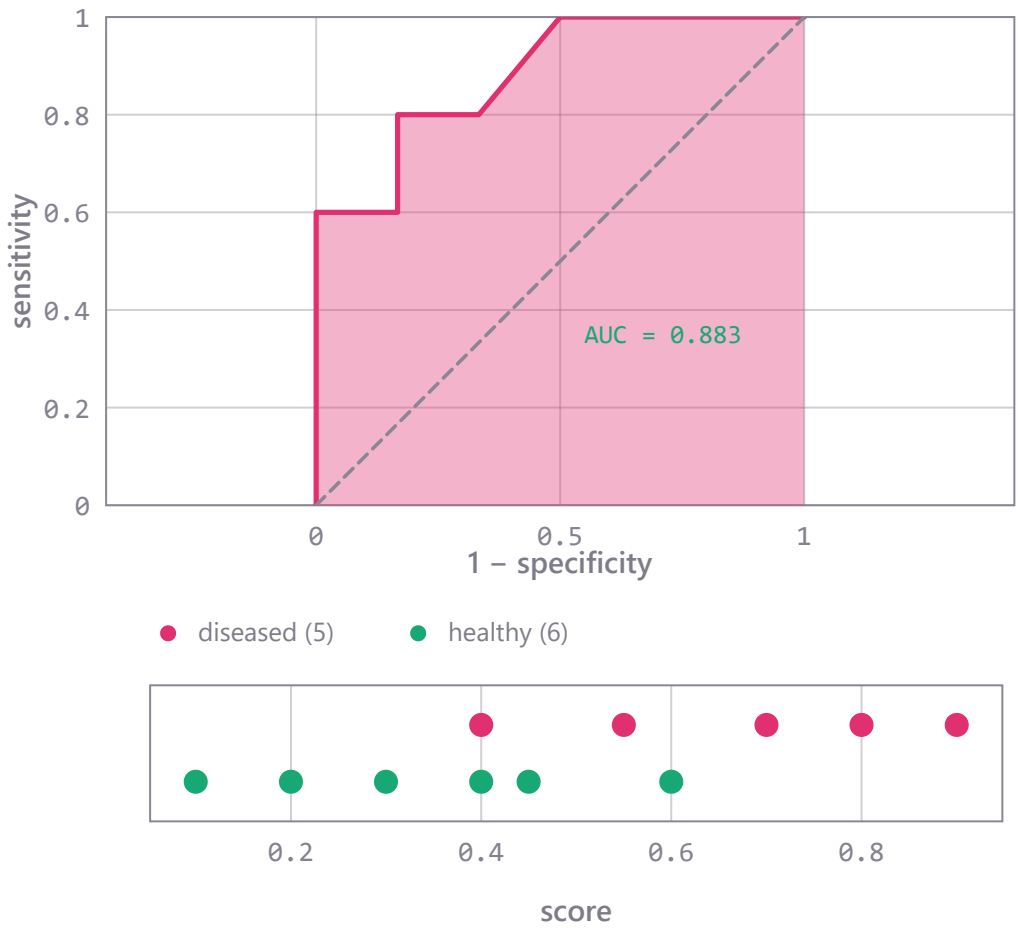
The area under the ROC curve of a continuous diagnostic score from the scores of diseased and non-diseased subjects: the probability that a diseased subject scores higher (ties count one half), with the Mann-Whitney U.

$$AUC = \frac{1}{n_1 n_0} \sum_{i,j} \left[ \mathbb{1}(x_i > y_j) + \tfrac{1}{2}\mathbb{1}(x_i = y_j) \right]$$

| input | meaning | condition |
|---|---|---|
| `scores_pos` | scores of the diseased (condition-positive) subjects | |
| `scores_neg` | scores of the non-diseased subjects | |

| output | meaning |
|---|---|
| `AUC` | area under the ROC curve |
| `U` | Mann-Whitney U of the diseased group |

- **Method:** AUC = U/(n1 n0), U = number of (diseased, non-diseased) pairs with the diseased score higher + half the ties
- **Assumes:** higher scores indicate disease; ties count one half
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Scores of the example's 5 diseased and 6 healthy subjects (bottom, input) and the ROC curve they give (top; sensitivity and specificity from the package at each of 12 thresholds). Marked: the package's area under the curve AUC = 0.883333, equal to the Mann–Whitney U = 26.5 divided by 5 × 6 pairs.

```
biomedical_science_ai999x.solve('bmed_dx_roc_auc', scores_pos=[0.9, 0.8, 0.7, 0.55, 0.4],
scores_neg=[0.6, 0.45, 0.4, 0.3, 0.2, 0.1])
# -> AUC = 0.883333, U = 26.5   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_dx_roc_auc', scores_pos=[3, 4, 5, 5], scores_neg=[1, 2, 3,
5])
# -> AUC = 0.78125, U = 12.5   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| ROC AUC: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1] | SOLVED | AUC = 0.883333, U = 26.5 |
| ROC AUC: scores_pos = [3, 4, 5, 5], scores_neg = [1, 2, 3, 5] | SOLVED | AUC = 0.78125, U = 12.5 |
| ROC AUC: scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1] | NEEDS_INPUT | bmed_dx_roc_auc needs scores_pos (scores of the diseased (condition-positive) subjects). Example: solve('bmed_dx_roc_auc |
| ROC AUC: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Optimal diagnostic cut-off by the Youden index — `bmed_dx_youden_cutoff`

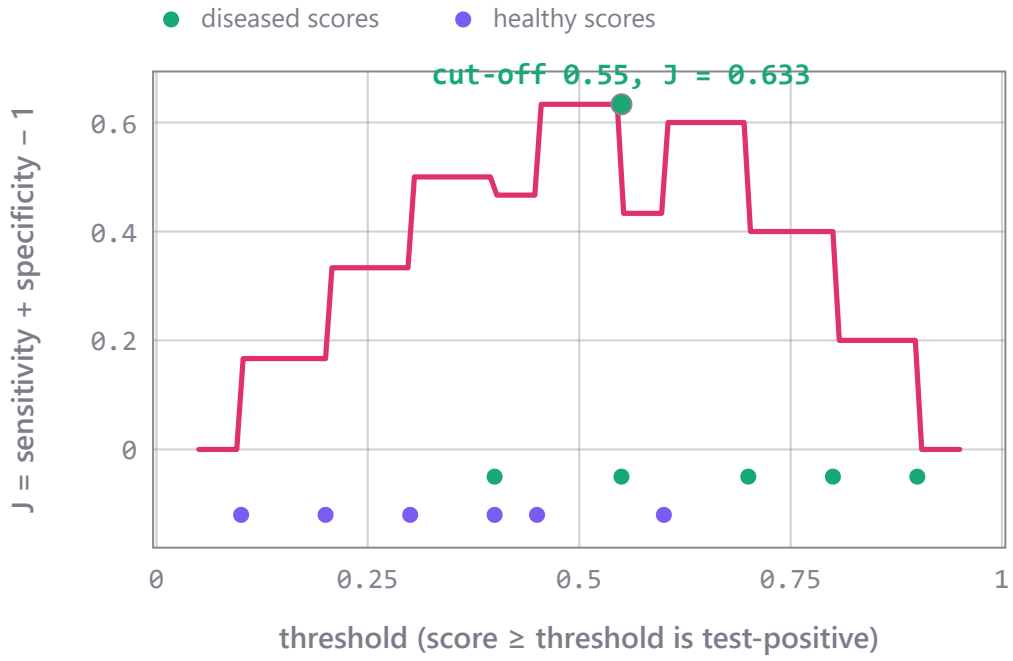
The score threshold that maximises the Youden index J = sensitivity + specificity - 1 over all observed cut-offs, with the sensitivity and specificity there.

$$J(c) = \mathrm{Se}(c) + \mathrm{Sp}(c) - 1, \quad c^* = \arg\max_c J(c)$$

| input | meaning | condition |
|---|---|---|
| `scores_pos` | scores of the diseased subjects | |
| `scores_neg` | scores of the non-diseased subjects | |

| output | meaning |
|---|---|
| `threshold` | optimal cut-off (score >= threshold is test-positive) |
| `sensitivity` | sensitivity at the cut-off |
| `specificity` | specificity at the cut-off |
| `J` | Youden index at the cut-off |

- **Method:** for every observed score as threshold: $J = TP/n_1 - FP/n_0$ (exact fractions); the maximum, highest threshold first
- **Assumes:** higher scores indicate disease; cut-offs at the observed scores; ties in J: the highest threshold (most specific); some cut-off has $J > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Youden index J against the threshold for the example's scores (dots below the curve, input), J computed from the package's sensitivity and specificity at 121 thresholds. Marked: the returned optimal cut-off 0.55 with J = 0.633333 (sensitivity 0.8, specificity 0.833333).

```
biomedical_science_ai999x.solve('bmed_dx_youden_cutoff', scores_pos=[0.9, 0.8, 0.7, 0.55, 0.4],
scores_neg=[0.6, 0.45, 0.4, 0.3, 0.2, 0.1])
# -> threshold = 0.55, sensitivity = 0.8, specificity = 0.833333, J = 0.633333   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_dx_youden_cutoff', scores_pos=[12, 15, 18, 22, 25, 30],
scores_neg=[8, 10, 11, 14, 16, 19, 21])
# -> threshold = 22, sensitivity = 0.5, specificity = 1, J = 0.5   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Youden cut-off: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1] | SOLVED | threshold = 0.55, sensitivity = 0.8, specificity = 0.833333, J = 0.633333 |
| Youden cut-off: scores_pos = [12, 15, 18, 22, 25, 30], scores_neg = [8, 10, 11, 14, 16, 19, 21] | SOLVED | threshold = 22, sensitivity = 0.5, specificity = 1, J = 0.5 |
| Youden cut-off: scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1] | NEEDS_INPUT | bmed_dx_youden_cutoff needs scores_pos (scores of the diseased subjects). Example: solve('bmed_dx_youden_cutoff', scores |
| Youden cut-off: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Sensitivity and specificity of a diagnostic score at a cut-off —

`bmed_dx_threshold_sensitivity_specificity`

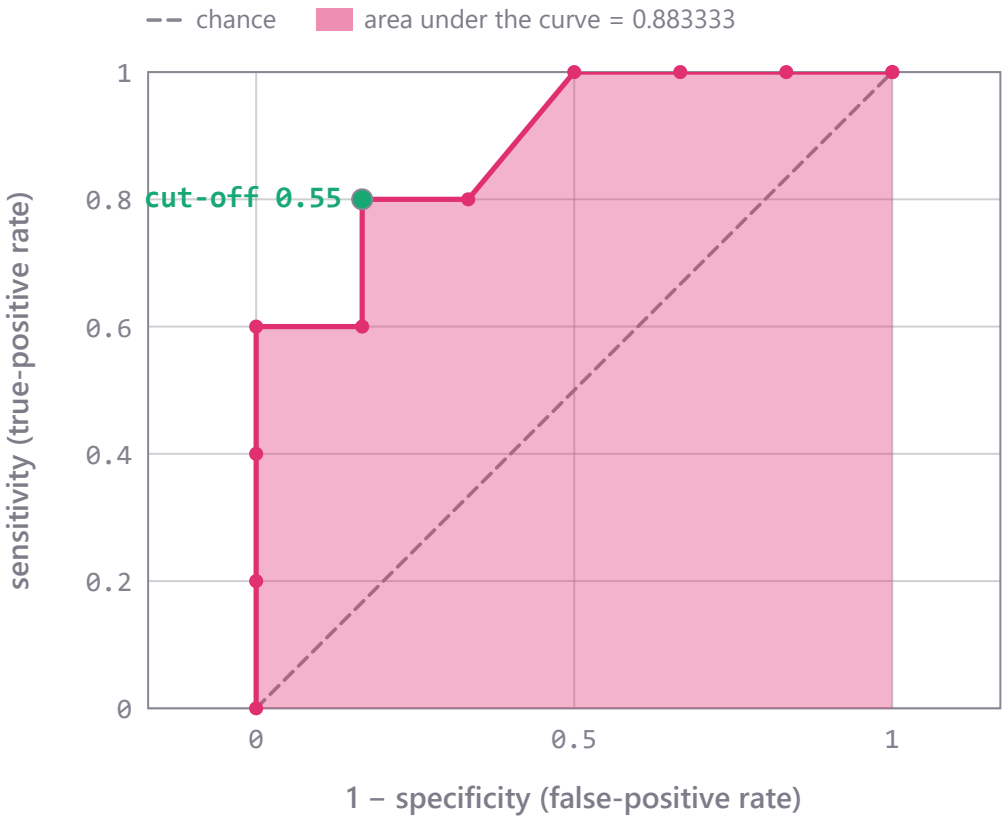
The confusion-matrix counts, sensitivity and specificity of a continuous diagnostic score when scores at or above a threshold are called positive.

$$\mathrm{Se} = \frac{TP}{TP + FN}, \quad \mathrm{Sp} = \frac{TN}{TN + FP}$$

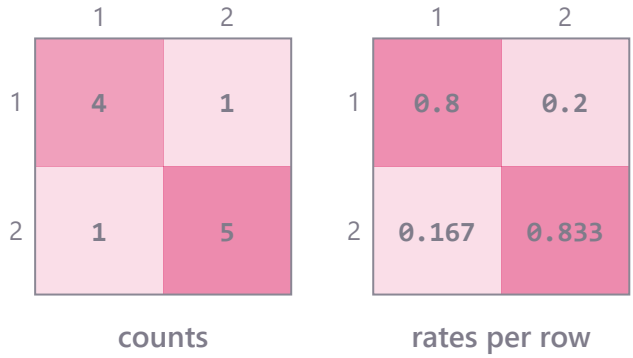
| input | meaning | condition |
|---|---|---|
| `scores_pos` | scores of the diseased subjects | |
| `scores_neg` | scores of the non-diseased subjects | |
| `threshold` | cut-off: a score >= threshold is test-positive | |

| output | meaning |
|---|---|
| `sensitivity` | TP/(TP + FN) |
| `specificity` | TN/(TN + FP) |
| `TP` | true positives |
| `FN` | false negatives |
| `TN` | true negatives |
| `FP` | false positives |

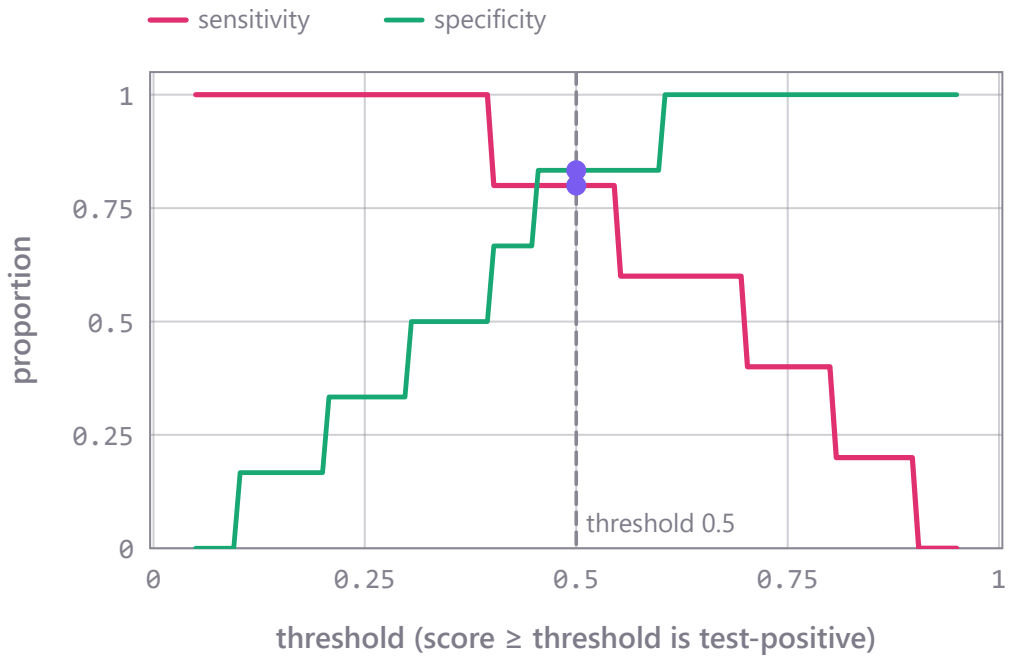
- **Method:** count the 2x2 table at the cut-off; sensitivity = TP/(TP + FN), specificity = TN/(TN + FP)
- **Assumes:** a score at or above the threshold is positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




ROC curve of a test with 5 diseased and 6 healthy scores: sensitivity and specificity computed by the package at 12 thresholds; area under the curve 0.883333. Marked: the Youden-optimal cut-off 0.55 (sensitivity 0.8, specificity 0.833333, J = 0.633333).




The 2 × 2 table (rows: diseased, healthy; columns: test positive, test negative) with the same table as rates per row (right), for the test at its Youden-optimal cut-off 0.55, counted by the package from 5 diseased and 6 healthy scores: TP = 4, FN = 1, FP = 1, TN = 5. Sensitivity 0.8 (95 % interval 0.375535–0.963776), specificity 0.833333 (0.436497–0.969947).




Sensitivity and specificity of the example's scores (5 diseased, 6 healthy) at 121 thresholds from 0.05 to 0.95: raising the threshold trades sensitivity for specificity, in steps at each observed score. Marked: the example threshold 0.5, sensitivity 0.8 (TP = 4, FN = 1), specificity 0.833333 (TN = 5, FP = 1).

```
biomedical_science_ai999x.solve('bmed_dx_threshold_sensitivity_specificity', scores_pos=[0.9, 0.8, 0.7, 0.55, 0.4], scores_neg=[0.6, 0.45, 0.4, 0.3, 0.2, 0.1], threshold=0.5)
# -> sensitivity = 0.8, specificity = 0.833333, TP = 4, FN = 1, TN = 5, FP = 1   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_dx_threshold_sensitivity_specificity', scores_pos=[12, 15, 18, 22, 25, 30], scores_neg=[8, 10, 11, 14, 16, 19, 21], threshold=15)
# -> sensitivity = 0.833333, specificity = 0.571429, TP = 5, FN = 1, TN = 4, FP = 3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Sensitivity and specificity at a cut-off: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1], threshold = 0.5 | SOLVED | sensitivity = 0.8, specificity = 0.833333, TP = 4, FN = 1, TN = 5, FP = 1 |
| Sensitivity and specificity at a cut-off: scores_pos = [12, 15, 18, 22, 25, 30], scores_neg = [8, 10, 11, 14, 16, 19, 21], threshold = 15 | SOLVED | sensitivity = 0.833333, specificity = 0.571429, TP = 5, FN = 1, TN = 4, FP = 3 |
| Sensitivity and specificity at a cut-off: scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1], threshold = 0.5 | NEEDS_INPUT | bmed_dx_threshold_sensitivity_specificity needs scores_pos (scores of the diseased subjects). Example: solve('bmed_dx_th |
| Sensitivity and specificity at a cut-off: scores_pos = [0.9, 0.8, 0.7, 0.55, 0.4], scores_neg = [0.6, 0.45, 0.4, 0.3, 0.2, 0.1], threshold = 0.5, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Sensitivity and specificity with Wilson score intervals — `bmed_dx_sensitivity_specificity_ci`

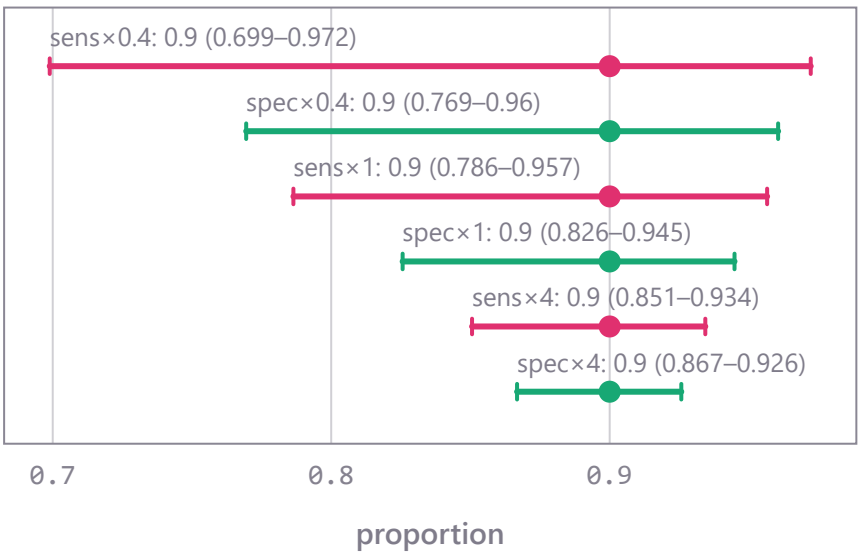
Sensitivity and specificity of a diagnostic test from its 2x2 counts, each with a Wilson score confidence interval at the stated level.

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

| input | meaning | condition |
|---|---|---|
| `TP` | true positives | |
| `FN` | false negatives | |
| `TN` | true negatives | |
| `FP` | false positives | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `sensitivity` | TP/(TP + FN) |
| `sens_lower` | lower Wilson limit |
| `sens_upper` | upper Wilson limit |
| `specificity` | TN/(TN + FP) |
| `spec_lower` | lower Wilson limit |
| `spec_upper` | upper Wilson limit |

- **Method:** Wilson interval (p + z^2/2n +- z sqrt(p(1 - p)/n + z^2/4n^2)) / $(1 + z^2/n)$
- **Assumes:** independent subjects; Wilson score interval (no continuity correction); level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Sensitivity and specificity with their 95 % Wilson intervals for the example's counts (TP = 45, FN = 5, TN = 90, FP = 10) and the same proportions with the counts scaled by 0.4, 1, 4 (the label gives the factor): more data, narrower intervals, the estimate unchanged. Marked at the example: sens 0.9 (0.786398–0.956524); spec 0.9 (0.825634–0.944771).

```
biomedical_science_ai999x.solve('bmed_dx_sensitivity_specificity_ci', TP=45, FN=5, TN=90, FP=10,
level=0.95)
# -> sensitivity = 0.9, sens_lower = 0.786398, sens_upper = 0.956524, specificity = 0.9, spec_lower
= 0.825634, spec_upper = 0.944771   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_dx_sensitivity_specificity_ci', TP=18, FN=2, TN=70, FP=30,
level=0.9)
# -> sensitivity = 0.9, sens_lower = 0.738337, sens_upper = 0.966337, specificity = 0.7, spec_lower
= 0.620168, spec_upper = 0.769295   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Sensitivity and specificity intervals: TP = 45, FN = 5, TN = 90, FP = 10, level = 95% | SOLVED | sensitivity = 0.9, sens_lower = 0.786398, sens_upper = 0.956524, specificity = 0.9, spec_lower = 0.825634, spec_upper = 0.944771 |
| Sensitivity and specificity intervals: TP = 18, FN = 2, TN = 70, FP = 30, level = 0.9 | SOLVED | sensitivity = 0.9, sens_lower = 0.738337, sens_upper = 0.966337, specificity = 0.7, spec_lower = 0.620168, spec_upper = 0.769295 |
| Sensitivity and specificity intervals: FN = 5, TN = 90, FP = 10, level = 95% | NEEDS_INPUT | bmed_dx_sensitivity_specificity_ci needs TP (true positives). Example: solve('bmed_dx_sensitivity_specificity_ci', TP=45 |
| Sensitivity and specificity intervals: TP = 45, FN = 5, TN = 90, FP = 10, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Sensitivity and specificity intervals: TP = 45, FN = 5, TN = 90, FP = 10 | NEEDS_INPUT | bmed_dx_sensitivity_specificity_ci needs level (confidence level (95% or 0.95)). Example: solve('bmed_dx_sensitivity_spe |

## Odds ratio of a 2x2 exposure-outcome table with Woolf (logit) interval — `bmed_odds_ratio_ci`

The odds ratio of an exposure-outcome 2x2 table (case-control or cohort) with the Woolf logit confidence interval at the stated level.

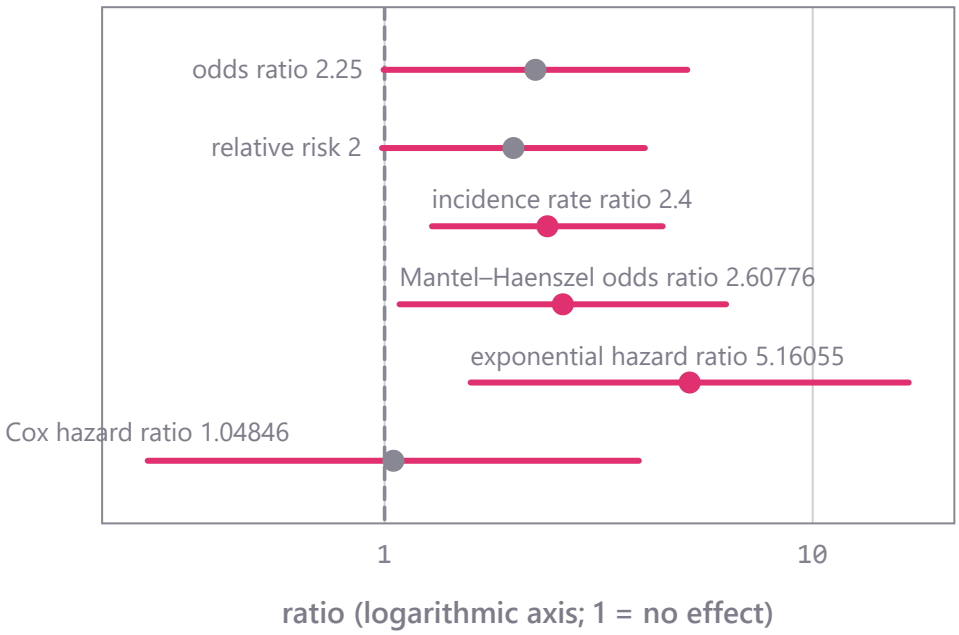
$$OR = \frac{ad}{bc}, \quad \mathrm{SE}(\ln OR) = \sqrt{\tfrac{1}{a} + \tfrac{1}{b} + \tfrac{1}{c} + \tfrac{1}{d}}$$

| input | meaning | condition |
|---|---|---|
| `a` | exposed with the outcome | |
| `b` | exposed without the outcome | |
| `c` | unexposed with the outcome | |
| `d` | unexposed without the outcome | |
| `level` | confidence level (95% or 0.95) | |

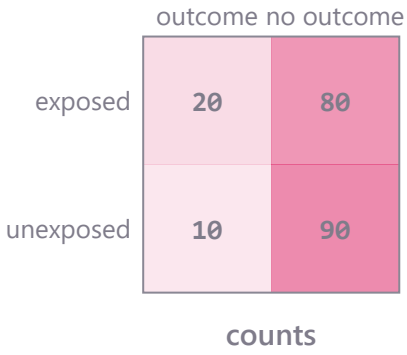
| output | meaning |
|---|---|
| `OR` | odds ratio ad/(bc) |
| `lower` | lower confidence limit |

| output | meaning |
|---|---|
| `upper` | upper confidence limit |

- **Method:** $OR = ad/(bc)$; SE(ln OR) = sqrt(1/a + 1/b + 1/c + 1/d)
- **Assumes:** no zero cell (no continuity correction); Woolf interval exp(ln OR +- z SE); level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Forest plot of six effect measures, each computed by the package on its own worked example, with 95 % intervals: odds ratio 2.25 (0.994295–5.09155); relative risk 2 (0.986563–4.05448); incidence rate ratio 2.4 (1.29133–4.46051); Mantel–Haenszel odds ratio 2.60776 (1.08405–6.27313); exponential hazard ratio 5.16055 (1.58923–16.7573); Cox hazard ratio 1.04846 (0.279864–3.92787). An interval that crosses 1 (grey point) is compatible with no effect.




The example's 2 × 2 table (rows: exposed, unexposed; columns: with, without the outcome): a = 20, b = 80, c = 10, d = 90. Returned: odds ratio ad/(bc) = 2.25, 95 % interval 0.994295–5.09155.

```
biomedical_science_ai999x.solve('bmed_odds_ratio_ci', a=20, b=80, c=10, d=90, level=0.95)
# -> OR = 2.25, lower = 0.994295, upper = 5.09155   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_odds_ratio_ci', a=45, b=55, c=30, d=120, level=0.99)
# -> OR = 3.27273, lower = 1.56471, upper = 6.84519   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Odds ratio with confidence interval: a = 20, b = 80, c = 10, d = 90, level = 95% | SOLVED | OR = 2.25, lower = 0.994295, upper = 5.09155 |
| Odds ratio with confidence interval: a = 45, b = 55, c = 30, d = 120, level = 0.99 | SOLVED | OR = 3.27273, lower = 1.56471, upper = 6.84519 |
| Odds ratio with confidence interval: b = 80, c = 10, d = 90, level = 95% | NEEDS_INPUT | bmed_odds_ratio_ci needs a (exposed with the outcome). Example: solve('bmed_odds_ratio_ci', a=20, b=80, c=10, d=90, leve |
| Odds ratio with confidence interval: a = 20, b = 80, c = 10, d = 90, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Odds ratio with confidence interval: a = 20, b = 0, c = 10, d = 90, level = 95% | INVALID_INPUT | b: must be at least 1, got 0. |

## Relative risk (risk ratio) of a cohort 2x2 table with log (Katz) interval — `bmed_relative_risk_ci`

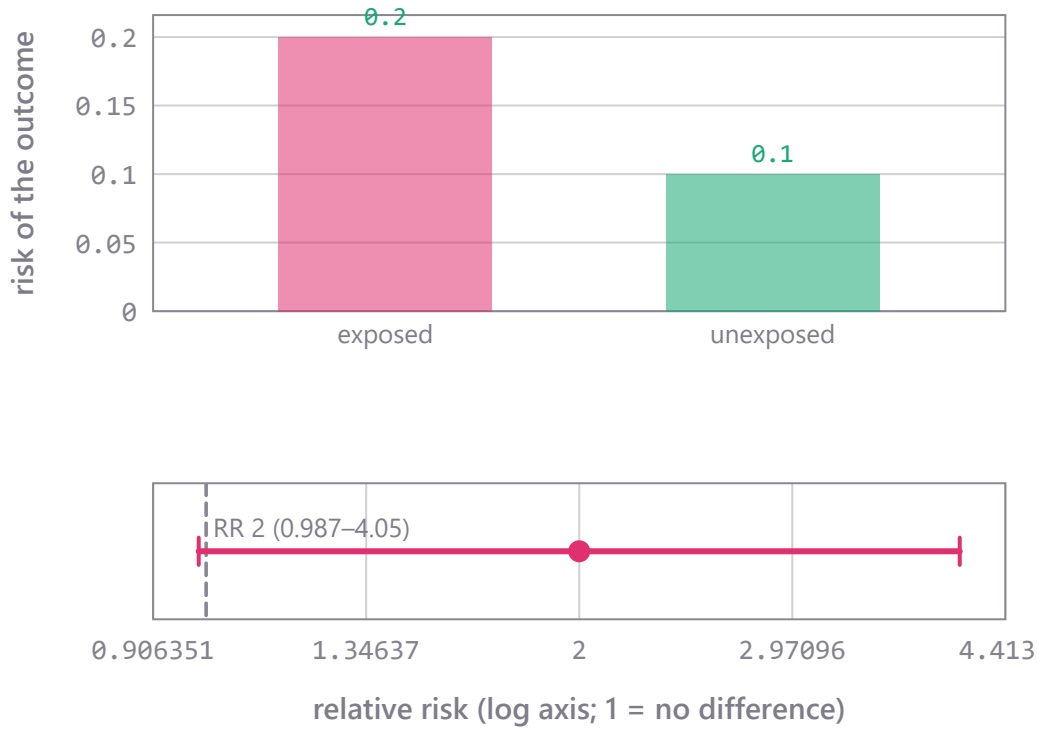
The risk ratio of exposed to unexposed in a cohort 2x2 table with the log-scale (Katz) confidence interval at the stated level.

$$RR = \frac{a/(a+b)}{c/(c+d)}, \quad \mathrm{SE}(\ln RR) = \sqrt{\tfrac{1}{a} - \tfrac{1}{a+b} + \tfrac{1}{c} - \tfrac{1}{c+d}}$$

| input | meaning | condition |
|---|---|---|
| `a` | exposed with the outcome | |
| `b` | exposed without the outcome | |
| `c` | unexposed with the outcome | |
| `d` | unexposed without the outcome | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `RR` | risk ratio [a/(a + b)]/[c/(c + d)] |
| `lower` | lower confidence limit |
| `upper` | upper confidence limit |

- **Method:** RR = [a/(a + b)]/[c/(c + d)]; SE(ln RR) = sqrt($1/a - 1/(a+b) + 1/c - 1/(c+d)$)
- **Assumes:** cohort (risks estimable); no zero cell; log interval exp(ln RR +- z SE); level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Risk of the outcome in the exposed (20 of 100) and unexposed (10 of 100) groups of the example (risks = events / group size, derived from the inputs), and the package's relative risk 2 with its 95 % interval 0.986563–4.05448 (bottom).

```
biomedical_science_ai999x.solve('bmed_relative_risk_ci', a=20, b=80, c=10, d=90, level=0.95)
# -> RR = 2, lower = 0.986563, upper = 4.05448   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_relative_risk_ci', a=45, b=55, c=30, d=120, level=0.9)
# -> RR = 2.25, lower = 1.62671, upper = 3.11211   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Relative risk with confidence interval: a = 20, b = 80, c = 10, d = 90, level = 95% | SOLVED | RR = 2, lower = 0.986563, upper = 4.05448 |
| Relative risk with confidence interval: a = 45, b = 55, c = 30, d = 120, level = 0.9 | SOLVED | RR = 2.25, lower = 1.62671, upper = 3.11211 |
| Relative risk with confidence interval: b = 80, c = 10, d = 90, level = 95% | NEEDS_INPUT | bmed_relative_risk_ci needs a (exposed with the outcome). Example: solve('bmed_relative_risk_ci', a=20, b=80, c=10, d=90 |
| Relative risk with confidence interval: a = 20, b = 80, c = 10, d = 90, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Absolute risk reduction and number needed to treat with Wald intervals — `bmed_nnt_ci`

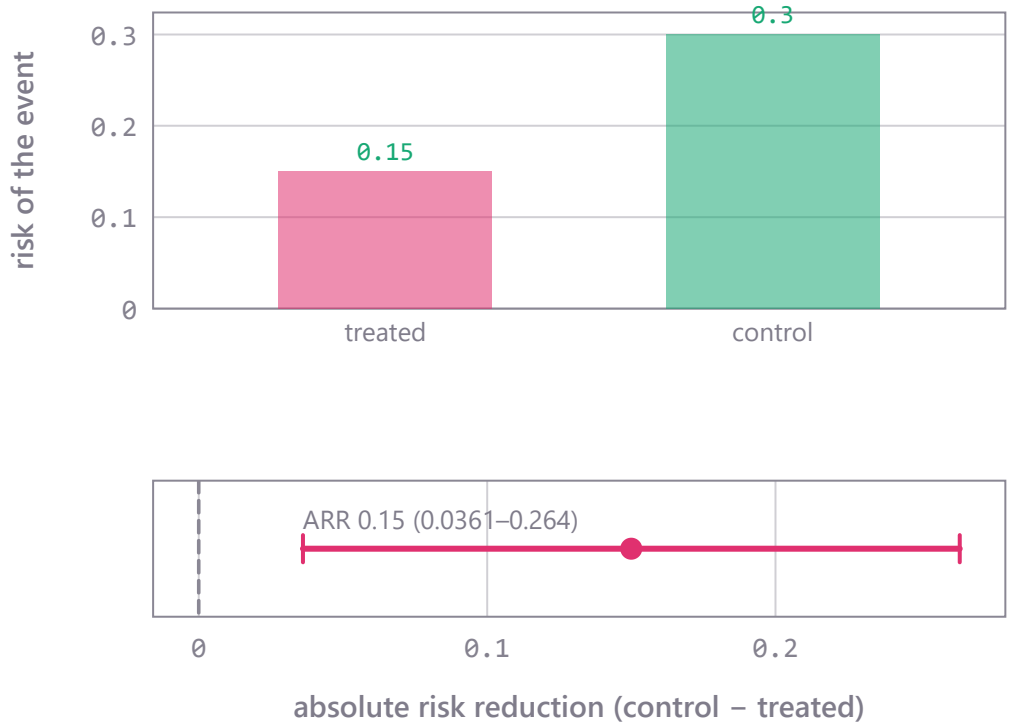
The absolute risk reduction (control risk minus treated risk) of a two-arm trial with its Wald confidence interval, and the number needed to treat with the interval obtained by inverting it.

$$ARR = p_c - p_t, \quad NNT = \frac{1}{ARR}, \quad \mathrm{SE} = \sqrt{\frac{p_t(1-p_t)}{n_t} + \frac{p_c(1-p_c)}{n_c}}$$

| input | meaning | condition |
|---|---|---|
| `events_t` | events in the treated group | |
| `n_t` | size of the treated group | |
| `events_c` | events in the control group | |
| `n_c` | size of the control group | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `ARR` | absolute risk reduction |
| `ARR_lower` | lower limit |
| `ARR_upper` | upper limit |
| `NNT` | number needed to treat 1/ARR |
| `NNT_lower` | 1/ARR_upper |
| `NNT_upper` | 1/ARR_lower |

- **Method:** ARR = pc - pt; $SE = sqrt$(pt(1 - pt)/nt + pc(1 - pc)/nc); NNT = 1/ARR with limits 1/upper, 1/lower
- **Assumes:** unpooled Wald interval for the difference of proportions; the interval excludes zero (a finite NNT interval); level stated; NNT not rounded
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Event risk in the treated (15 of 100) and control (30 of 100) groups of the example (risks derived from the inputs) and the package's absolute risk reduction ARR = 0.15 with its 95 % interval 0.0361364–0.263864. Returned: number needed to treat NNT = 1/ARR = 6.66667 (interval 3.78984–27.6729).

```
biomedical_science_ai999x.solve('bmed_nnt_ci', events_t=15, n_t=100, events_c=30, n_c=100,
level=0.95)
# -> ARR = 0.15, ARR_lower = 0.0361364, ARR_upper = 0.263864, NNT = 6.66667, NNT_lower = 3.78984,
NNT_upper = 27.6729   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_nnt_ci', events_t=40, n_t=400, events_c=80, n_c=410,
level=0.9)
# -> ARR = 0.095122, ARR_lower = 0.0545621, ARR_upper = 0.135682, NNT = 10.5128, NNT_lower =
7.37019, NNT_upper = 18.3277   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Number needed to treat: events_t = 15, n_t = 100, events_c = 30, n_c = 100, level = 95% | SOLVED | ARR = 0.15, ARR_lower = 0.0361364, ARR_upper = 0.263864, NNT = 6.66667, NNT_lower = 3.78984, NNT_upper = 27.6729 |
| Number needed to treat: events_t = 40, n_t = 400, events_c = 80, n_c = 410, level = 0.9 | SOLVED | ARR = 0.095122, ARR_lower = 0.0545621, ARR_upper = 0.135682, NNT = 10.5128, NNT_lower = 7.37019, NNT_upper = 18.3277 |
| Number needed to treat: n_t = 100, events_c = 30, n_c = 100, level = 95% | NEEDS_INPUT | bmed_nnt_ci needs events_t (events in the treated group). Example: solve('bmed_nnt_ci', events_t=15, n_t=100, events_c=3 |
| Number needed to treat: events_t = 15, n_t = 100, events_c = 30, n_c = 100, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Number needed to treat: events_t = 28, n_t = 100, events_c = 30, n_c = 100, level = 95% | INVALID_INPUT | Absolute risk reduction and number needed to treat with Wald intervals: the interval for the risk reduction does not exc |

## Incidence rate ratio of two person-time groups with log-Wald interval — `bmed_incidence_rate_ratio_ci`

The ratio of the incidence rates (events per person-time) of two groups with the log-scale Wald confidence interval at the stated level.

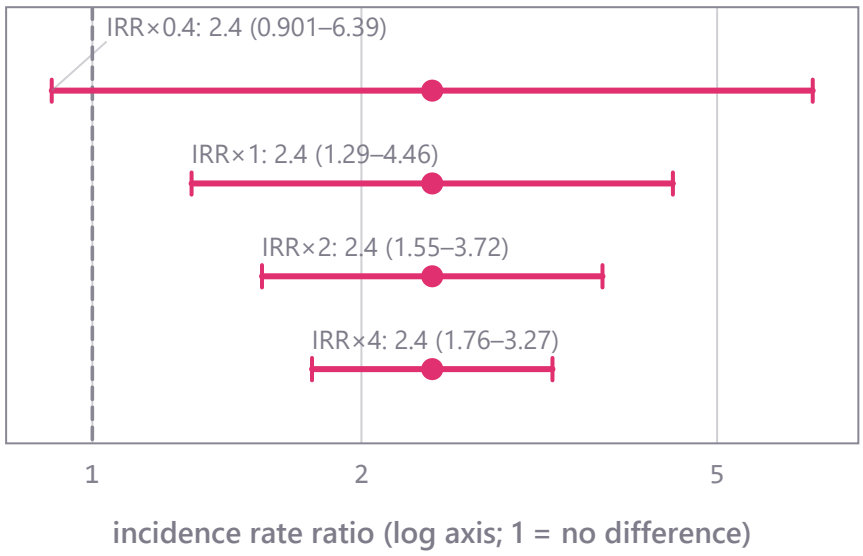
$$IRR = \frac{e_1/T_1}{e_2/T_2}, \quad \mathrm{SE}(\ln IRR) = \sqrt{\frac{1}{e_1} + \frac{1}{e_2}}$$

| input | meaning | condition |
|---|---|---|
| `events1` | events in group 1 | |
| `time1` | person-time of group 1 (the same unit for both groups) | |
| `events2` | events in group 2 (reference) | |
| `time2` | person-time of group 2 | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `IRR` | incidence rate ratio group 1 / group 2 |
| `lower` | lower limit |

| output | meaning |
|---|---|
| `upper` | upper limit |

- **Method:** IRR = (e1/T1)/(e2/T2); SE(ln IRR) = sqrt(1/e1 + 1/e2)
- **Assumes:** Poisson counts; log-Wald interval; the same person-time unit for both groups; level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The incidence rate ratio with its 95 % interval for the example's counts (events1 = 30, time1 = 1000, events2 = 15, time2 = 1200) and the same proportions with the counts scaled by 0.4, 1, 2, 4 (the label gives the factor): more data, narrower intervals, the estimate unchanged. Marked at the example: IRR 2.4 (1.29133–4.46051).

```
biomedical_science_ai999x.solve('bmed_incidence_rate_ratio_ci', events1=30, time1=1000, events2=15,
time2=1200, level=0.95)
# -> IRR = 2.4, lower = 1.29133, upper = 4.46051   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_incidence_rate_ratio_ci', events1=12, time1=2500.5,
events2=40, time2=4100, level=0.9)
# -> IRR = 0.491902, lower = 0.286257, upper = 0.845279   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Incidence rate ratio: events1 = 30, time1 = 1000, events2 = 15, time2 = 1200, level = 95% | SOLVED | IRR = 2.4, lower = 1.29133, upper = 4.46051 |
| Incidence rate ratio: events1 = 12, time1 = 2500.5, events2 = 40, time2 = 4100, level = 0.9 | SOLVED | IRR = 0.491902, lower = 0.286257, upper = 0.845279 |
| Incidence rate ratio: time1 = 1000, events2 = 15, time2 = 1200, level = 95% | NEEDS_INPUT | bmed_incidence_rate_ratio_ci needs events1 (events in group 1). Example: solve('bmed_incidence_rate_ratio_ci', events1=3 |
| Incidence rate ratio: events1 = 30, time1 = 1000, events2 = 15, time2 = 1200, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Standardized mortality (incidence) ratio with exact Poisson interval — `bmed_smr_exact_ci`

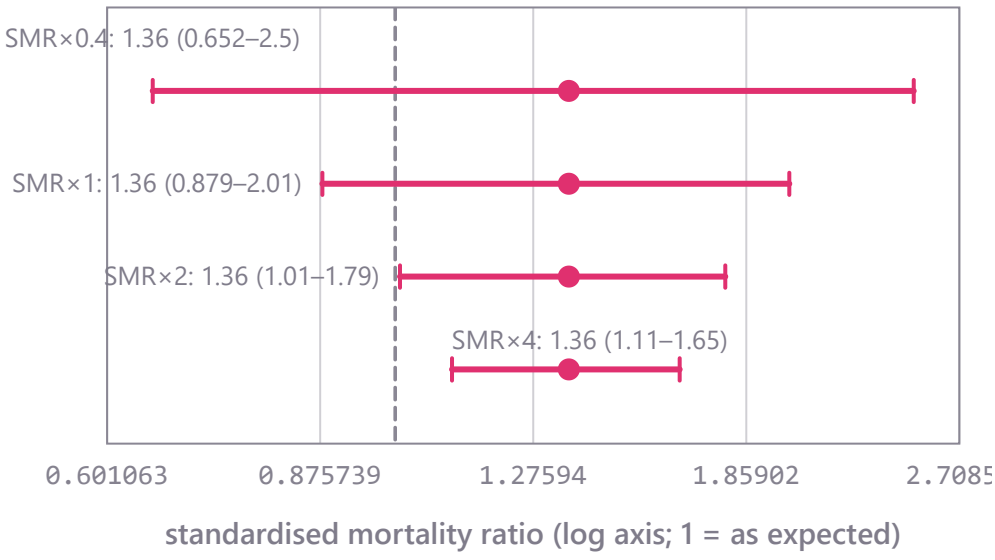
The standardized mortality or incidence ratio observed/expected with the exact (Garwood) Poisson confidence interval at the stated level.

$$\mathrm{SMR} = \frac{O}{E}, \quad P(X \geq O; \lambda_L) = \tfrac{\alpha}{2},\ P(X \leq O; \lambda_U) = \tfrac{\alpha}{2}$$

| input | meaning | condition |
|---|---|---|
| `observed` | observed number of deaths or cases (0 to 1000) | |
| `expected` | expected number from reference rates | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `SMR` | observed / expected |
| `lower` | lower exact limit |
| `upper` | upper exact limit |

- **Method:** limits lambda with P($X \geq O$; lambda) = alpha/2 and P($X \leq O$; lambda) = alpha/2 (bisection on the Poisson sums), divided by E
- **Assumes:** observed count Poisson; expected count treated as known; exact (Garwood) limits; level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The standardised mortality ratio with its exact 95 % interval for the example's counts (observed = 25, expected = 18.4) and the same proportions with the counts scaled by 0.4, 1, 2, 4 (the label gives the factor): more data, narrower intervals, the estimate unchanged. Marked at the example: SMR 1.3587 (0.879276–2.0057).

```
biomedical_science_ai999x.solve('bmed_smr_exact_ci', observed=25, expected=18.4, level=0.95)
# -> SMR = 1.3587, lower = 0.879276, upper = 2.0057   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_smr_exact_ci', observed=3, expected=7.5, level=0.9)
# -> SMR = 0.4, lower = 0.109026, upper = 1.03382   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Standardized mortality ratio: observed = 25, expected = 18.4, level = 95% | SOLVED | SMR = 1.3587, lower = 0.879276, upper = 2.0057 |
| Standardized mortality ratio: observed = 3, expected = 7.5, level = 0.9 | SOLVED | SMR = 0.4, lower = 0.109026, upper = 1.03382 |
| Standardized mortality ratio: expected = 18.4, level = 95% | NEEDS_INPUT | bmed_smr_exact_ci needs observed (observed number of deaths or cases (0 to 1000)). Example: solve('bmed_smr_exact_ci', o |
| Standardized mortality ratio: observed = 25, expected = 18.4, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Mantel-Haenszel pooled odds ratio over strata with Robins-Breslow-Greenland interval — `bmed_mantel_haenszel_odds_ratio`

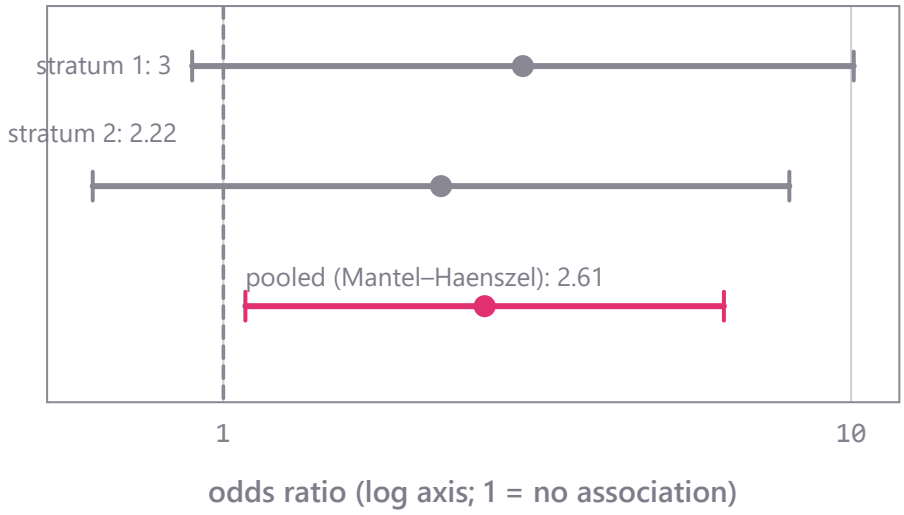
The Mantel-Haenszel odds ratio pooled over strata of 2x2 tables (adjusting for a confounder), with the Robins-Breslow-Greenland confidence interval at the stated level.

$$OR_{MH} = \frac{\sum_i a_i d_i / n_i}{\sum_i b_i c_i / n_i}$$

| input | meaning | condition |
|---|---|---|
| `tables` | one row [a, b, c, d] per stratum: exposed cases, exposed non-cases, unexposed cases, unexposed non-cases | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `OR_MH` | Mantel-Haenszel pooled odds ratio |
| `lower` | lower limit |
| `upper` | upper limit |

- **Method:** OR_MH = sum(a d / n) / sum(b c / n); variance of ln OR_MH by Robins, Breslow and Greenland
- **Assumes:** a common odds ratio across strata; Robins-Breslow-Greenland variance; level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Odds ratio of each stratum of the example (package's odds ratio with its 95 % interval) and the Mantel–Haenszel pooled odds ratio over the 2 strata: stratum 1 [10, 20, 5, 30] OR 3 (0.891475–10.0956); stratum 2 [8, 12, 6, 20] OR 2.22222 (0.619258–7.97449). Marked: pooled OR_MH = 2.60776 (1.08405–6.27313).

```
biomedical_science_ai999x.solve('bmed_mantel_haenszel_odds_ratio', tables=[[10, 20, 5, 30], [8, 12, 6, 20]], level=0.95)
# -> OR_MH = 2.60776, lower = 1.08405, upper = 6.27313   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mantel_haenszel_odds_ratio', tables=[[25, 75, 10, 90], [40, 60, 20, 80], [5, 45, 2, 48]], level=0.9)
# -> OR_MH = 2.78404, lower = 1.86901, upper = 4.14705   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Mantel-Haenszel odds ratio: tables = [[10, 20, 5, 30], [8, 12, 6, 20]], level = 95% | SOLVED | OR_MH = 2.60776, lower = 1.08405, upper = 6.27313 |
| Mantel-Haenszel odds ratio: tables = [[25, 75, 10, 90], [40, 60, 20, 80], [5, 45, 2, 48]], level = 0.9 | SOLVED | OR_MH = 2.78404, lower = 1.86901, upper = 4.14705 |
| Mantel-Haenszel odds ratio: level = 95% | NEEDS_INPUT | bmed_mantel_haenszel_odds_ratio needs tables (one row [a, b, c, d] per stratum: exposed cases, exposed non-cases, unexpo |
| Mantel-Haenszel odds ratio: tables = [[10, 20, 5, 30], [8, 12, 6, 20]], level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Cohen's kappa for agreement between two raters or two tests — `bmed_cohens_kappa`

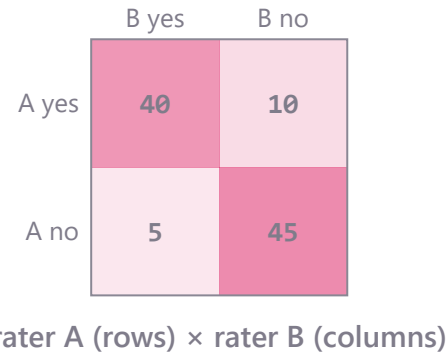
Cohen's kappa (chance-corrected agreement) and the observed agreement of two raters or two diagnostic tests classifying the same subjects, from their k x k agreement table.

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad p_e = \sum_i \frac{n_{i\cdot}\, n_{\cdot i}}{n^2}$$

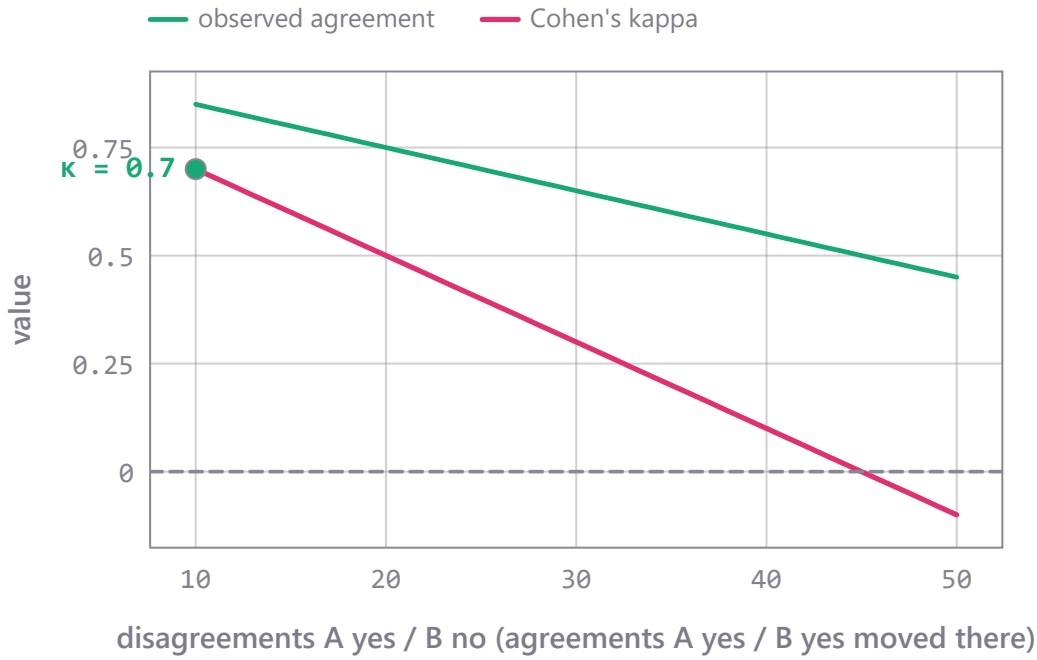
| input | meaning | condition |
|---|---|---|
| `counts` | agreement table: rows = categories given by rater A, columns = categories given by rater B | |

| output | meaning |
|---|---|
| `kappa` | Cohen's kappa |
| `p_observed` | observed proportion of agreement |

- **Method:** $\kappa = (p_o - p_e)/(1 - p_e)$, p_e = sum of products of row and column margins / n^2
- **Assumes:** the same subjects rated by both; unweighted kappa; the same categories in rows and columns
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The example's agreement table of two raters (100 subjects; rows: rater A, columns: rater B; agreements on the diagonal). Returned: observed agreement 0.85, Cohen's kappa 0.7 (agreement beyond chance).




Kappa and observed agreement as cases move from the agreement cell (A yes, B yes) to the disagreement cell (A yes, B no) of the example table [[40, 10], [5, 45]] (100 subjects), one run per table. Kappa falls faster than raw agreement because chance agreement is removed. Marked: the example, κ = 0.7, agreement 0.85.

```
biomedical_science_ai999x.solve('bmed_cohens_kappa', counts=[[40, 10], [5, 45]])
# -> kappa = 0.7, p_observed = 0.85   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_cohens_kappa', counts=[[20, 5, 1], [4, 30, 6], [0, 3, 31]])
# -> kappa = 0.710542, p_observed = 0.81   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Cohen's kappa: counts = [[40, 10], [5, 45]] | SOLVED | kappa = 0.7, p_observed = 0.85 |
| Cohen's kappa: counts = [[20, 5, 1], [4, 30, 6], [0, 3, 31]] | SOLVED | kappa = 0.710542, p_observed = 0.81 |
| Cohen's kappa: | NEEDS_INPUT | bmed_cohens_kappa needs counts (agreement table: rows = categories given by rater A, columns = categories given by rater |
| Cohen's kappa: counts = [[40, 10], [5, 45]], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Kaplan-Meier survival estimate at a time and median survival — `bmed_km_survival_median`

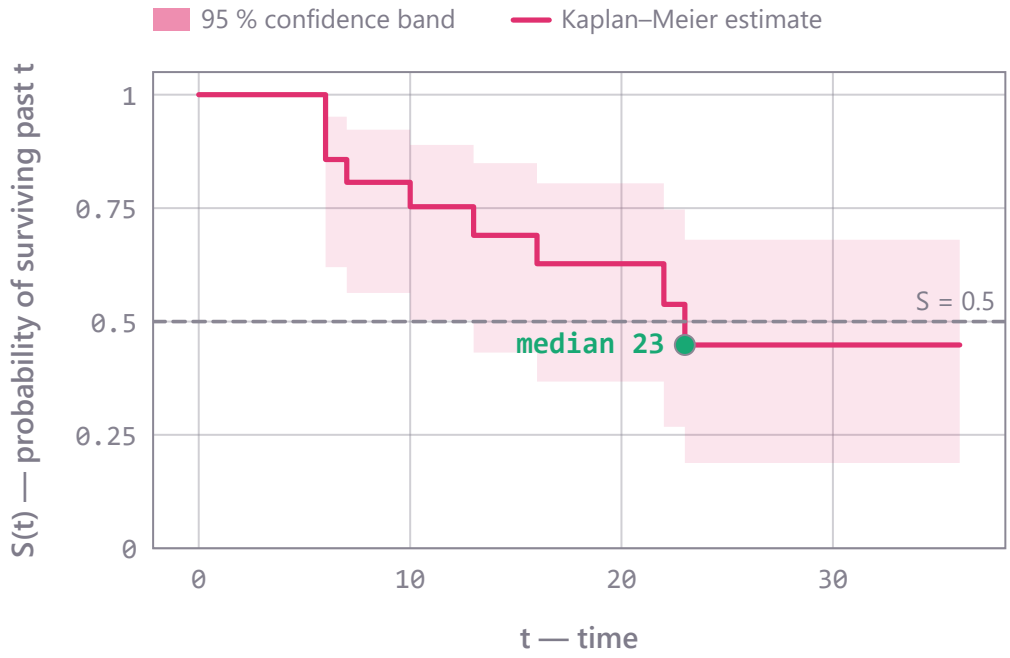
The Kaplan-Meier (product-limit) estimate of the survival probability at time t and the median survival time from follow-up times with event or censoring indicators.

$$\hat{S}(t) = \prod_{u_i \le t} \left(1 - \frac{d_i}{n_i}\right)$$

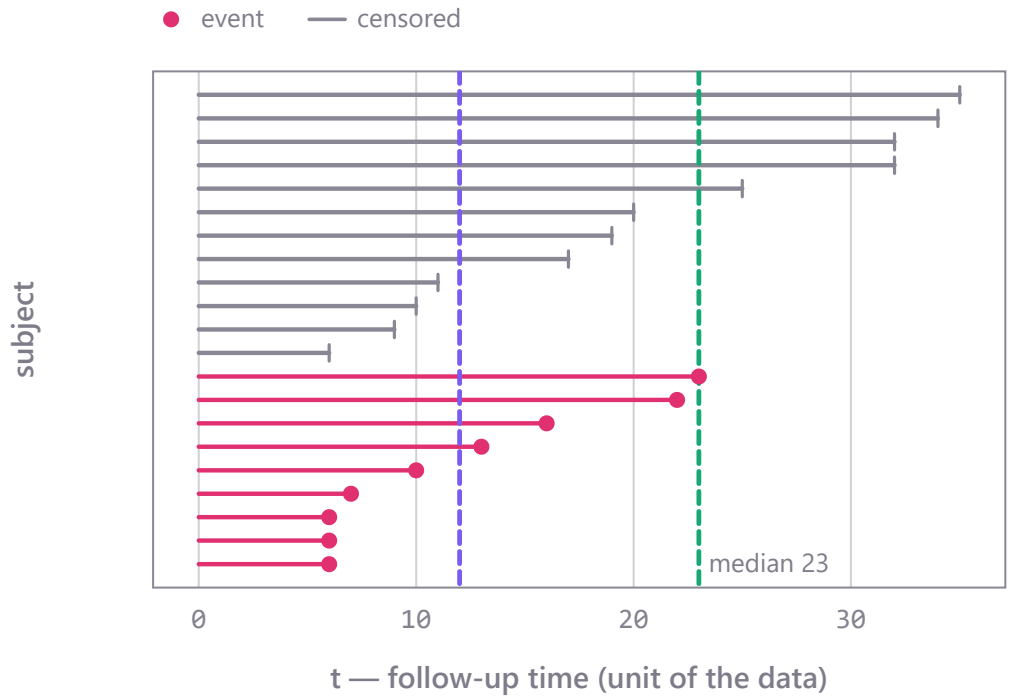
| input | meaning | condition |
|---|---|---|
| `times` | follow-up times (one time unit; results in that unit) | |
| `events` | 1 = event observed, 0 = censored, one per time | |
| `t` | time at which S(t) is reported | |

| output | meaning |
|---|---|
| `S_t` | Kaplan-Meier survival probability at t |
| `median` | median survival: first time with $S \le 0.5$ (none when not reached) |

- **Method:** S(t) = product over event times $u \le t$ of $(1 - d/n)$, exact fractions; median = first event time with $S \le 1/2$
- **Assumes:** non-informative censoring; a censoring at an event time counts as at risk at that time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kaplan–Meier survival estimate for 21 subjects (9 events, the rest censored), with its pointwise 95 % confidence band, both computed by the package at every whole time from 0 to 36. S(12) = 0.752941; marked: median survival t = 23, S = 0.448179.




The example's 21 subjects as follow-up lines from 0 to their time (dot: event observed, 9; bar end: censored, 12), the data the Kaplan–Meier estimate is computed from. Marked: the returned median survival 23 and the example time t = 12, where S = 0.752941.

```
biomedical_science_ai999x.solve('bmed_km_survival_median', times=[6, 6, 6, 7, 10, 13, 16, 22, 23,
6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0], t=12)
# -> S_t = 0.752941, median = 23   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_km_survival_median', times=[2, 3, 3, 5, 8, 9, 12, 15],
events=[1, 0, 1, 1, 0, 1, 1, 0], t=6)
# -> S_t = 0.6, median = 9   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Kaplan-Meier survival: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12 | SOLVED | S_t = 0.752941, median = 23 |
| Kaplan-Meier survival: times = [2, 3, 3, 5, 8, 9, 12, 15], events = [1, 0, 1, 1, 0, 1, 1, 0], t = 6 | SOLVED | S_t = 0.6, median = 9 |
| Kaplan-Meier survival: events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12 | NEEDS_INPUT | bmed_km_survival_median needs times (follow-up times (one time unit; results in that unit)). Example: solve('bmed_km_sur |
| Kaplan-Meier survival: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Kaplan-Meier survival: times = [6, 7, 10], events = [1, 2, 1], t = 8 | INVALID_INPUT | Kaplan-Meier survival estimate at a time and median survival: event indicators are 1 (event) or 0 (censored). |

## Kaplan-Meier survival with log(-log) (exponential Greenwood) confidence interval — `bmed_km_confidence_interval`

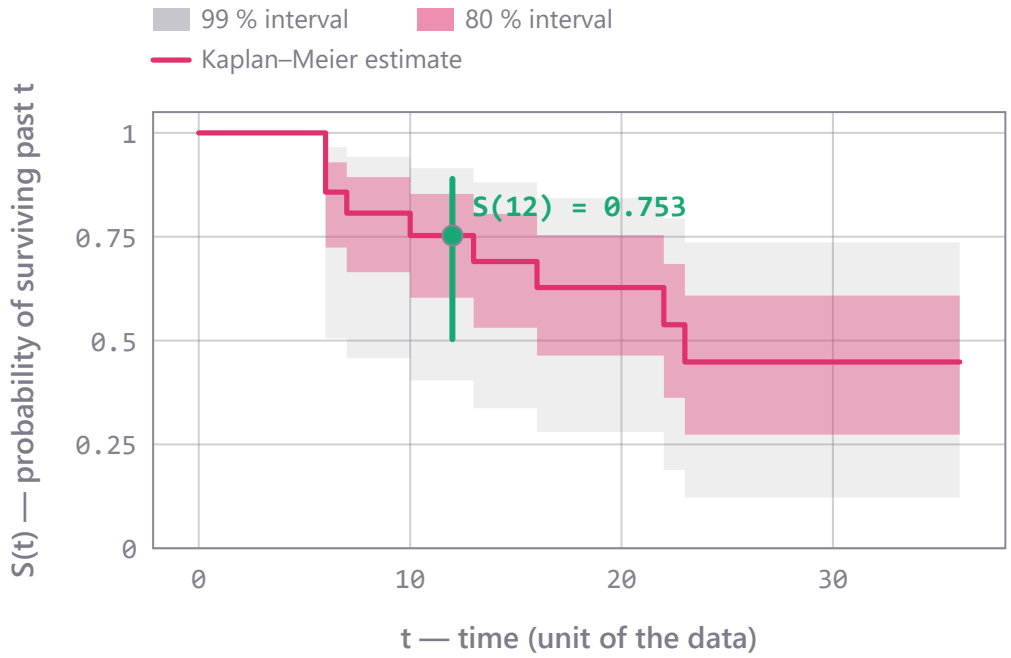
The Kaplan-Meier survival estimate at time t with its confidence interval on the log(-log) scale (Greenwood variance; the exponential Greenwood interval) at the stated level.

$$\hat{S}^{\exp\left(\pm z\sqrt{V}/|\ln \hat{S}|\right)}, \quad V = \sum_{u_i \le t} \frac{d_i}{n_i(n_i - d_i)}$$

| input | meaning | condition |
|---|---|---|
| `times` | follow-up times (one time unit) | |
| `events` | 1 = event, 0 = censored | |
| `t` | time at which S(t) is reported | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `S_t` | Kaplan-Meier estimate at t |
| `lower` | lower confidence limit |
| `upper` | upper confidence limit |

- **Method:** Greenwood sum V = sum d/(n(n - d)); limits $S^{exp}$(+-z sqrt(V)/|ln S|)
- **Assumes:** non-informative censoring; an event at or before t and S(t) > 0; level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kaplan–Meier estimate of the example's 21 follow-up times with its pointwise 80 % and 99 % confidence bands, both computed by the package at every step time. Marked: at t = 12 the 95 % interval 0.5032–0.889362 around S = 0.752941.

```
biomedical_science_ai999x.solve('bmed_km_confidence_interval', times=[6, 6, 6, 7, 10, 13, 16, 22,
23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0], t=12, level=0.95)
# -> S_t = 0.752941, lower = 0.5032, upper = 0.889362   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_km_confidence_interval', times=[2, 3, 3, 5, 8, 9, 12, 15],
events=[1, 0, 1, 1, 0, 1, 1, 0], t=8, level=0.9)
# -> S_t = 0.6, lower = 0.258167, upper = 0.82473   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Kaplan-Meier confidence interval: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12, level = 95% | SOLVED | S_t = 0.752941, lower = 0.5032, upper = 0.889362 |
| Kaplan-Meier confidence interval: times = [2, 3, 3, 5, 8, 9, 12, 15], events = [1, 0, 1, 1, 0, 1, 1, 0], t = 8, level = 0.9 | SOLVED | S_t = 0.6, lower = 0.258167, upper = 0.82473 |
| Kaplan-Meier confidence interval: events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12, level = 95% | NEEDS_INPUT | bmed_km_confidence_interval needs times (follow-up times (one time unit)). Example: solve('bmed_km_confidence_interval', |
| Kaplan-Meier confidence interval: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12, level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Kaplan-Meier confidence interval: times = [2, 3, 5], events = [1, 1, 1], t = 1, level = 95% | INVALID_INPUT | Kaplan-Meier survival with log(-log) (exponential Greenwood) confidence interval: no event at or before t: S(t) = 1 and |

### Nelson-Aalen cumulative hazard at a time — `bmed_nelson_aalen_hazard`

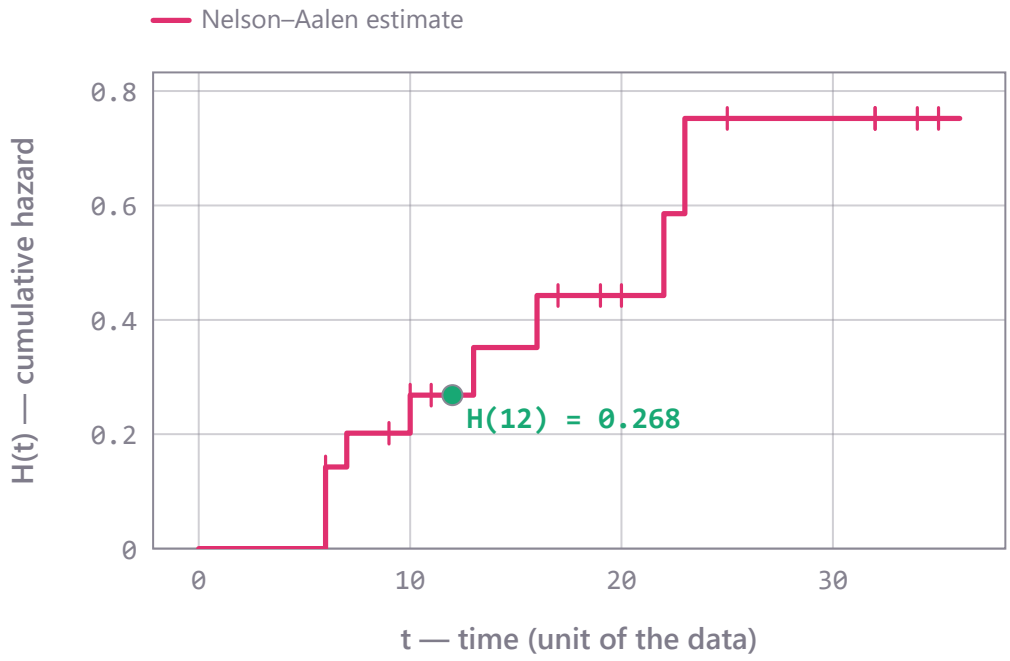
The Nelson-Aalen estimate of the cumulative hazard at time t from follow-up times with event or censoring indicators.

$$\hat{H}(t) = \sum_{u_i \le t} \frac{d_i}{n_i}$$

| input | meaning | condition |
|---|---|---|
| `times` | follow-up times (one time unit) | |
| `events` | 1 = event, 0 = censored | |
| `t` | time at which H(t) is reported | |

| output | meaning |
|---|---|
| `H_t` | cumulative hazard at t |

- **Method:** H(t) = sum over event times $u \le t$ of d/n
- **Assumes:** non-informative censoring; tied events contribute d/n (no tie smoothing)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Nelson–Aalen cumulative hazard of the example's 21 follow-up times (9 events; ticks: censored), the package's estimate at every step time: each event adds 1/(number at risk). Marked: H(12) = 0.268347.

```
biomedical_science_ai999x.solve('bmed_nelson_aalen_hazard', times=[6, 6, 6, 7, 10, 13, 16, 22, 23,
6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0], t=12)
# -> H_t = 0.268347   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_nelson_aalen_hazard', times=[2, 3, 3, 5, 8, 9, 12, 15],
events=[1, 0, 1, 1, 0, 1, 1, 0], t=10)
# -> H_t = 0.80119   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Nelson-Aalen cumulative hazard: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12 | SOLVED | H_t = 0.268347 |
| Nelson-Aalen cumulative hazard: times = [2, 3, 3, 5, 8, 9, 12, 15], events = [1, 0, 1, 1, 0, 1, 1, 0], t = 10 | SOLVED | H_t = 0.80119 |
| Nelson-Aalen cumulative hazard: events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12 | NEEDS_INPUT | bmed_nelson_aalen_hazard needs times (follow-up times (one time unit)). Example: solve('bmed_nelson_aalen_hazard', times |
| Nelson-Aalen cumulative hazard: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], t = 12, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Log-rank test comparing two survival curves — `bmed_logrank_test`

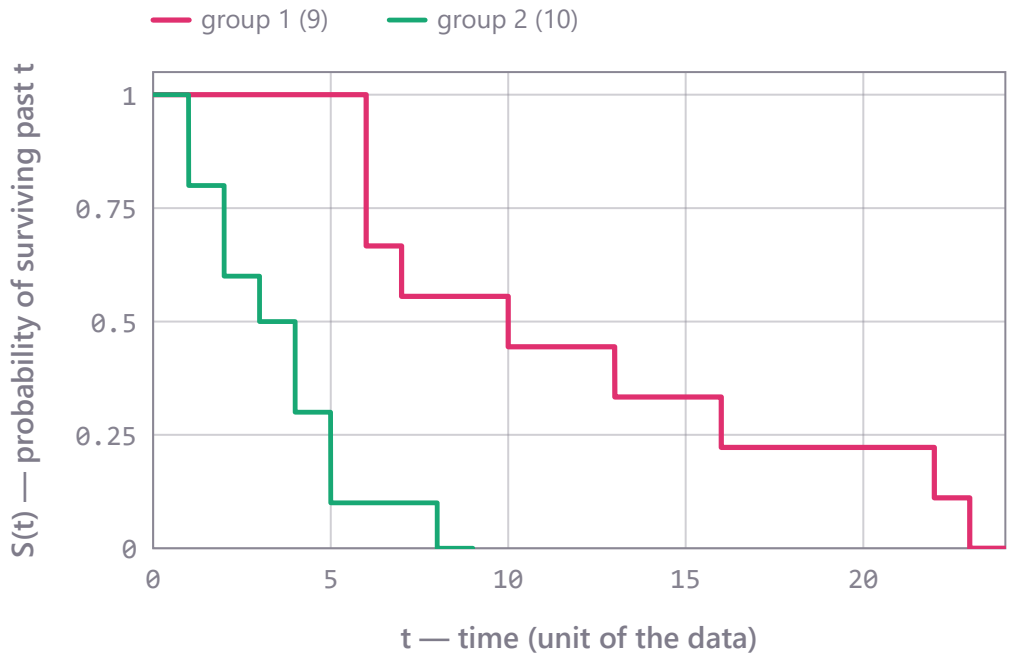
The log-rank (Mantel-Cox) chi-square statistic and p-value for equal survival in two groups from follow-up times and event indicators.

$$\chi^2 = \frac{(\sum_i (d_{1i} - e_{1i}))^2}{\sum_i v_i}$$

| input | meaning | condition |
|---|---|---|
| `times1` | follow-up times, group 1 | |
| `events1` | 1 = event, 0 = censored, group 1 | |
| `times2` | follow-up times, group 2 | |
| `events2` | 1 = event, 0 = censored, group 2 | |

| output | meaning |
|---|---|
| `chi_square` | log-rank chi-square (1 degree of freedom) |
| `p_value` | two-sided p-value |

- **Method:** at each event time: expected events of group 1 and the hypergeometric variance; $chi2 = (O_1 - E_1)^2 / V$
- **Assumes:** non-informative censoring; hypergeometric variance at each event time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kaplan–Meier curves of the example's two groups (9 and 10 subjects; ticks mark censored times), every step the package's survival estimate. Returned: log-rank $\chi^2$ = 13.85 (1 degree of freedom), p = 0.000197999.

```
biomedical_science_ai999x.solve('bmed_logrank_test', times1=[6, 6, 6, 7, 10, 13, 16, 22, 23], events1=[1, 1, 1, 1, 1, 1, 1, 1, 1], times2=[1, 1, 2, 2, 3, 4, 4, 5, 5, 8], events2=[1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
# -> chi_square = 13.85, p_value = 0.000197999   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_logrank_test', times1=[5, 8, 12, 20, 25], events1=[1, 1, 0, 1, 0], times2=[3, 4, 9, 10, 14], events2=[1, 1, 1, 0, 1])
# -> chi_square = 1.3895, p_value = 0.238489   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Log-rank test: times1 = [6, 6, 6, 7, 10, 13, 16, 22, 23], events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [1, 1, 2, 2, 3, 4, 4, 5, 5, 8], events2 = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1] | SOLVED | chi_square = 13.85, p_value = 0.000197999 |
| Log-rank test: times1 = [5, 8, 12, 20, 25], events1 = [1, 1, 0, 1, 0], times2 = [3, 4, 9, 10, 14], events2 = [1, 1, 1, 0, 1] | SOLVED | chi_square = 1.3895, p_value = 0.238489 |
| Log-rank test: events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [1, 1, 2, 2, 3, 4, 4, 5, 5, 8], events2 = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1] | NEEDS_INPUT | bmed_logrank_test needs times1 (follow-up times, group 1). Example: solve('bmed_logrank_test', times1=[6, 6, 6, 7, 10, 1 |
| Log-rank test: times1 = [6, 6, 6, 7, 10, 13, 16, 22, 23], events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [1, 1, 2, 2, 3, 4, 4, 5, 5, 8], events2 = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Cox proportional-hazards model with one covariate: hazard ratio (Efron ties) — `bmed_cox_hazard_ratio`

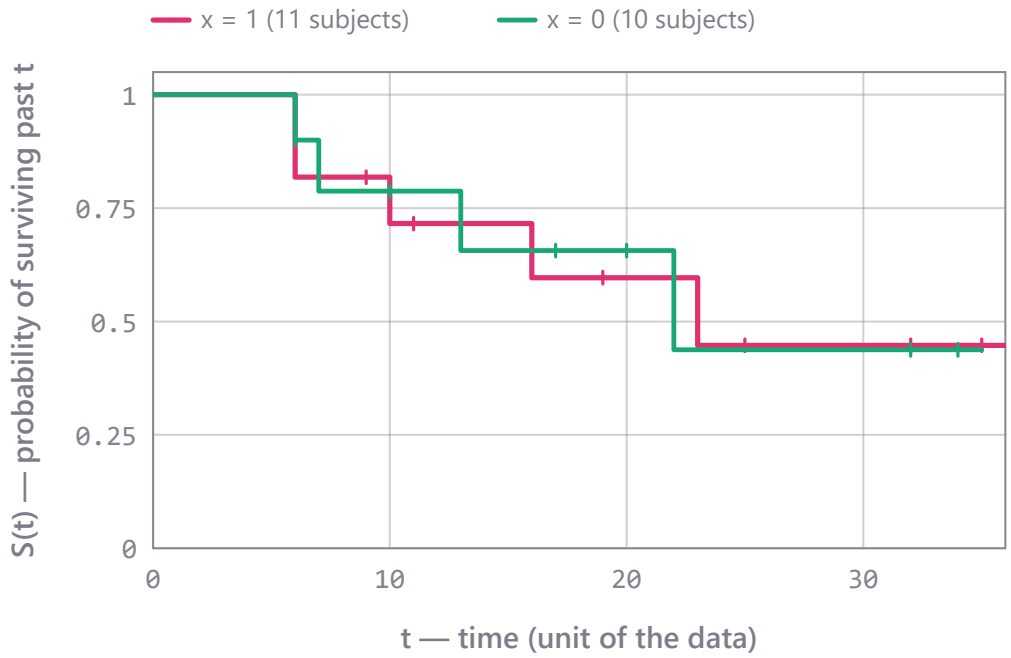
The log hazard ratio, hazard ratio, its standard error and Wald confidence interval of a Cox proportional-hazards model with a single covariate, by maximum partial likelihood with Efron's handling of ties.

$$\ell(\beta) = \sum_j \left[ \beta \sum_{i \in D_j} x_i - \sum_{l=0}^{d_j - 1} \ln \left( \sum_{R_j} e^{\beta x} - \frac{l}{d_j} \sum_{D_j} e^{\beta x} \right) \right]$$

| input | meaning | condition |
|---|---|---|
| `times` | follow-up times | |
| `events` | 1 = event, 0 = censored | |
| `x` | covariate value of each subject (e.g. 1 = treated, 0 = control) | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `beta` | log hazard ratio per unit of x |
| `HR` | hazard ratio exp(beta) |
| `se` | standard error of beta |
| `HR_lower` | lower Wald limit of the hazard ratio |
| `HR_upper` | upper Wald limit of the hazard ratio |

- **Method:** Newton-Raphson on the Efron partial log-likelihood (step halving); SE from the observed information
- **Assumes:** proportional hazards; Efron approximation for tied event times; a finite maximum (no monotone likelihood); level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kaplan–Meier curves of the example's two covariate groups (x = 1 and x = 0; 21 subjects, ticks mark censored times), each step the package's survival estimate for that group. The proportional-hazards fit returns HR = 1.04846 per unit of x (95 % interval 0.279864–3.92787, β = 0.0473228, se 0.673876).

```
biomedical_science_ai999x.solve('bmed_cox_hazard_ratio', times=[6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], x=[1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1], level=0.95)
# -> beta = 0.0473228, HR = 1.04846, se = 0.673876, HR_lower = 0.279864, HR_upper = 3.92787 (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_cox_hazard_ratio', times=[2, 3, 3, 5, 8, 9, 12, 15, 4, 7, 11, 14, 18, 21], events=[1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0], x=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], level=0.9)
# -> beta = 0.690016, HR = 1.99375, se = 0.737627, HR_lower = 0.592578, HR_upper = 6.70803 (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Cox hazard ratio: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], x = [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1], level = 95% | SOLVED | beta = 0.0473228, HR = 1.04846, se = 0.673876, HR_lower = 0.279864, HR_upper = 3.92787 |
| Cox hazard ratio: times = [2, 3, 3, 5, 8, 9, 12, 15, 4, 7, 11, 14, 18, 21], events = [1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0], x = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], level = 0.9 | SOLVED | beta = 0.690016, HR = 1.99375, se = 0.737627, HR_lower = 0.592578, HR_upper = 6.70803 |
| Cox hazard ratio: events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], x = [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1], level = 95% | NEEDS_INPUT | bmed_cox_hazard_ratio needs times (follow-up times). Example: solve('bmed_cox_hazard_ratio', times=[6, 6, 6, 7, 10, 13, |
| Cox hazard ratio: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: |

| question | outcome | result |
|---|---|---|
| 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], x = [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], level = 95%, zz = 1 | | biomedical_science_ai999x.templat |

## Exponential survival model: hazard rate, mean and median survival (maximum likelihood)

— `bmed_exponential_survival_fit`

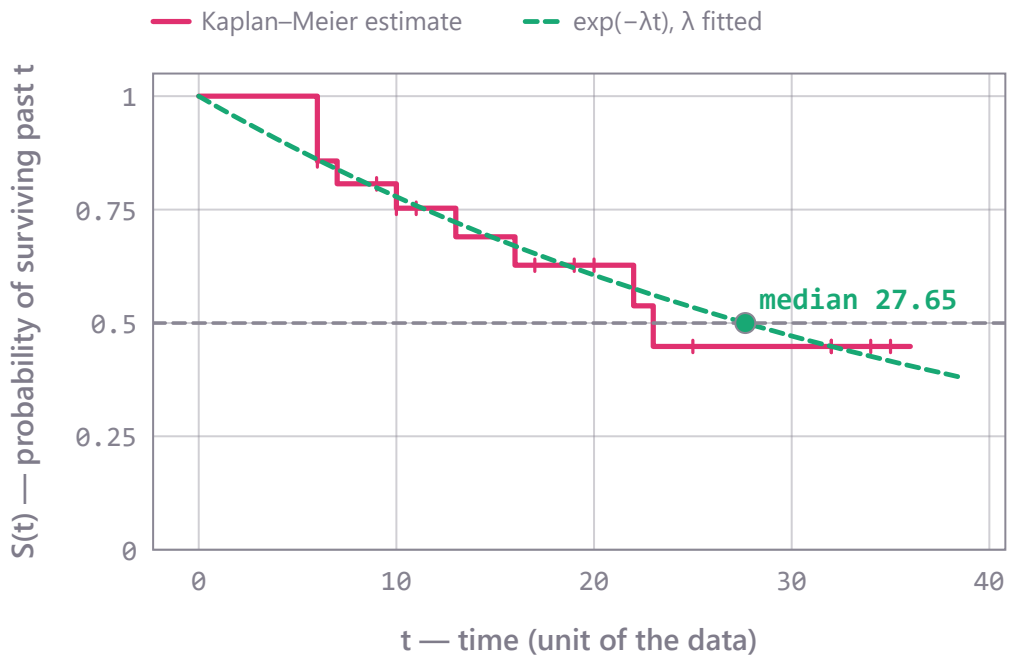
The maximum-likelihood constant hazard of an exponential survival model from censored follow-up data, with the implied mean and median survival times.

$$\hat{\lambda} = \frac{d}{\sum_i t_i}, \quad \tilde{t} = \frac{\ln 2}{\hat{\lambda}}$$

| input | meaning | condition |
|---|---|---|
| `times` | follow-up times, all positive (one time unit) | |
| `events` | 1 = event, 0 = censored | |

| output | meaning |
|---|---|
| `hazard` | events per unit time |
| `mean` | mean survival 1/hazard |
| `median` | median survival ln 2 / hazard |

- **Method:** hazard = (number of events) / (total follow-up time)
- **Assumes:** constant hazard; non-informative censoring; at least one event
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Exponential survival fitted to the example's 21 follow-up times (9 events; ticks: censored): the returned hazard λ = 0.0250696 per time unit gives the dashed curve exp(−λt) (derived), beside the package's Kaplan–Meier steps. Marked: median ln2/λ = 27.6489; mean 1/λ = 39.8889.

```
biomedical_science_ai999x.solve('bmed_exponential_survival_fit', times=[6, 6, 6, 7, 10, 13, 16, 22,
23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0])
# -> hazard = 0.0250696, mean = 39.8889, median = 27.6489   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_exponential_survival_fit', times=[2.5, 3.1, 4.0, 7.2, 8.8],
events=[1, 1, 0, 1, 0])
# -> hazard = 0.117188, mean = 8.53333, median = 5.91486   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Exponential survival fit: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] | SOLVED | hazard = 0.0250696, mean = 39.8889, median = 27.6489 |
| Exponential survival fit: times = [2.5, 3.1, 4, 7.2, 8.8], events = [1, 1, 0, 1, 0] | SOLVED | hazard = 0.117188, mean = 8.53333, median = 5.91486 |
| Exponential survival fit: events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] | NEEDS_INPUT | bmed_exponential_survival_fit needs times (follow-up times, all positive (one time unit)). Example: solve('bmed_exponent |
| Exponential survival fit: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Hazard ratio of two exponential survival groups with Wald interval —

`bmed_exponential_hazard_ratio`

The ratio of the maximum-likelihood constant hazards of two groups (group 1 relative to group 2) with the log-scale Wald confidence interval.

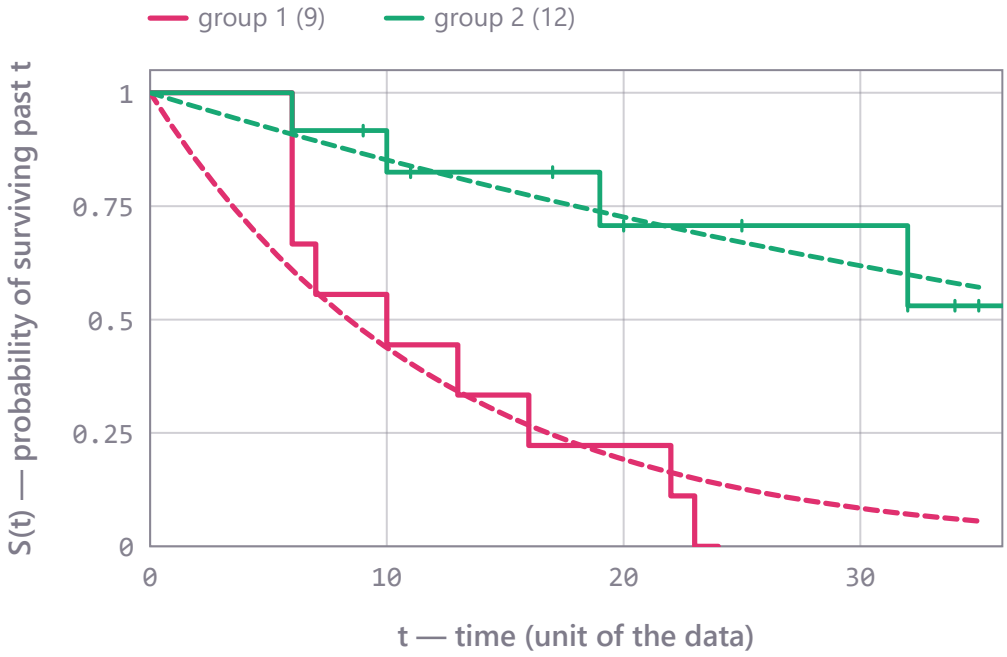
$$HR = \frac{d_1/\sum t_1}{d_2/\sum t_2}, \quad \mathrm{SE}(\ln HR) = \sqrt{\tfrac{1}{d_1} + \tfrac{1}{d_2}}$$

| input | meaning | condition |
|---|---|---|
| `times1` | follow-up times, group 1 (positive) | |
| `events1` | 1 = event, 0 = censored, group 1 | |
| `times2` | follow-up times, group 2 (reference) | |
| `events2` | 1 = event, 0 = censored, group 2 | |
| `level` | confidence level (95% or 0.95) | |

| output | meaning |
|---|---|
| `HR` | hazard ratio group 1 / group 2 |
| `lower` | lower limit |
| `upper` | upper limit |

- **Method:** hazard_g = d_g / sum t_g; SE(ln HR) = sqrt(1/d1 + 1/d2)

- **Assumes:** constant hazard in each group; at least one event per group; level stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kaplan–Meier curves of the example's two groups (9 and 12 subjects; ticks mark censored times), every step the package's survival estimate. Returned: exponential hazard ratio (group 1 / group 2) 5.16055, 95 % interval 1.58923–16.7573; dashed: exp(−λt) from each group's returned exponential hazard (derived).

```
biomedical_science_ai999x.solve('bmed_exponential_hazard_ratio', times1=[6, 6, 6, 7, 10, 13, 16, 22, 23], events1=[1, 1, 1, 1, 1, 1, 1, 1, 1], times2=[6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events2=[1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0], level=0.95)
# -> HR = 5.16055, lower = 1.58923, upper = 16.7573   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_exponential_hazard_ratio', times1=[5, 8, 12, 20, 25], events1=[1, 1, 0, 1, 0], times2=[3, 4, 9, 10, 14], events2=[1, 1, 1, 0, 1], level=0.9)
# -> HR = 0.428571, lower = 0.122019, upper = 1.50528   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Exponential hazard ratio: times1 = [6, 6, 6, 7, 10, 13, 16, 22, 23], events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events2 = [1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0], level = 95% | SOLVED | HR = 5.16055, lower = 1.58923, upper = 16.7573 |
| Exponential hazard ratio: times1 = [5, 8, 12, 20, 25], events1 = [1, 1, 0, 1, 0], times2 = [3, 4, 9, 10, 14], events2 = [1, 1, 1, 0, 1], level = 0.9 | SOLVED | HR = 0.428571, lower = 0.122019, upper = 1.50528 |
| Exponential hazard ratio: events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events2 = [1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0], level = 95% | NEEDS_INPUT | bmed_exponential_hazard_ratio needs times1 (follow-up times, group 1 (positive)). Example: solve('bmed_exponential_hazar |

| question | outcome | result |
|---|---|---|
| Exponential hazard ratio: times1 = [6, 6, 6, 7, 10, 13, 16, 22, 23], events1 = [1, 1, 1, 1, 1, 1, 1, 1, 1], times2 = [6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events2 = [1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0], level = 95%, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Restricted mean survival time from the Kaplan-Meier curve —

`bmed_restricted_mean_survival_time`

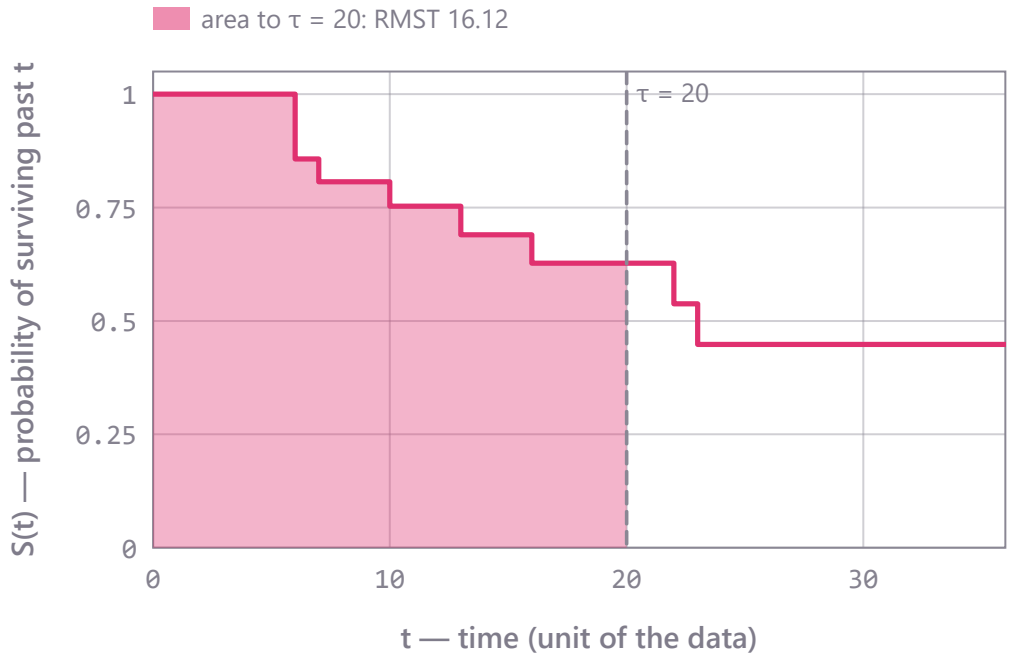
The restricted mean survival time up to tau: the area under the Kaplan-Meier survival curve from 0 to tau.

$$\mathrm{RMST}(\tau) = \int_0^{\tau} \hat{S}(t)\,dt$$

| input | meaning | condition |
|---|---|---|
| `times` | follow-up times (one time unit) | |
| `events` | 1 = event, 0 = censored | |
| `tau` | restriction time (in the unit of the times) | |

| output | meaning |
|---|---|
| `RMST` | restricted mean survival time, in the time unit |

- **Method:** RMST = integral from 0 to tau of the Kaplan-Meier step function
- **Assumes:** non-informative censoring; the Kaplan-Meier step function integrated exactly
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Restricted mean survival time: the area under the Kaplan–Meier curve of the example's 21 follow-up times from 0 to τ = 20 (shaded; the curve is the package's estimate at every step). Marked: RMST = 16.1165 time units, the expected survival time within the first 20.

```
biomedical_science_ai999x.solve('bmed_restricted_mean_survival_time', times=[6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], tau=20)
# -> RMST = 16.1165   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_restricted_mean_survival_time', times=[2, 3, 3, 5, 8, 9, 12, 15], events=[1, 0, 1, 1, 0, 1, 1, 0], tau=10)
# -> RMST = 7.175   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Restricted mean survival time: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], tau = 20 | SOLVED | RMST = 16.1165 |
| Restricted mean survival time: times = [2, 3, 3, 5, 8, 9, 12, 15], events = [1, 0, 1, 1, 0, 1, 1, 0], tau = 10 | SOLVED | RMST = 7.175 |
| Restricted mean survival time: events = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], tau = 20 | NEEDS_INPUT | bmed_restricted_mean_survival_time needs times (follow-up times (one time unit)). Example: solve('bmed_restricted_mean_s |
| Restricted mean survival time: times = [6, 6, 6, 7, 10, 13, 16, 22, 23, 6, 9, 10, 11, 17, 19, 20, 25, 32, 32, 34, 35], events = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], tau = 20, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Static lever model of the forearm: muscle force and joint reaction —

`bmed_elbow_lever_muscle_force`

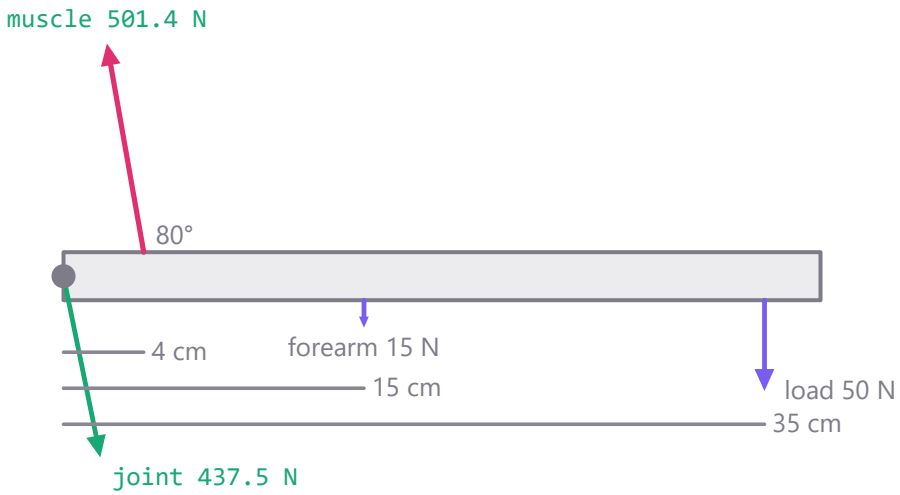
For a horizontal forearm held in static equilibrium, the elbow-flexor muscle force and the joint reaction force from the hand load, the forearm weight, their distances from the elbow and the muscle's insertion distance and angle.

$$F_m = \frac{W d_L + W_a d_a}{d_m \sin\theta}, \quad R_x = F_m \cos\theta, \quad R_y = W + W_a - F_m \sin\theta$$

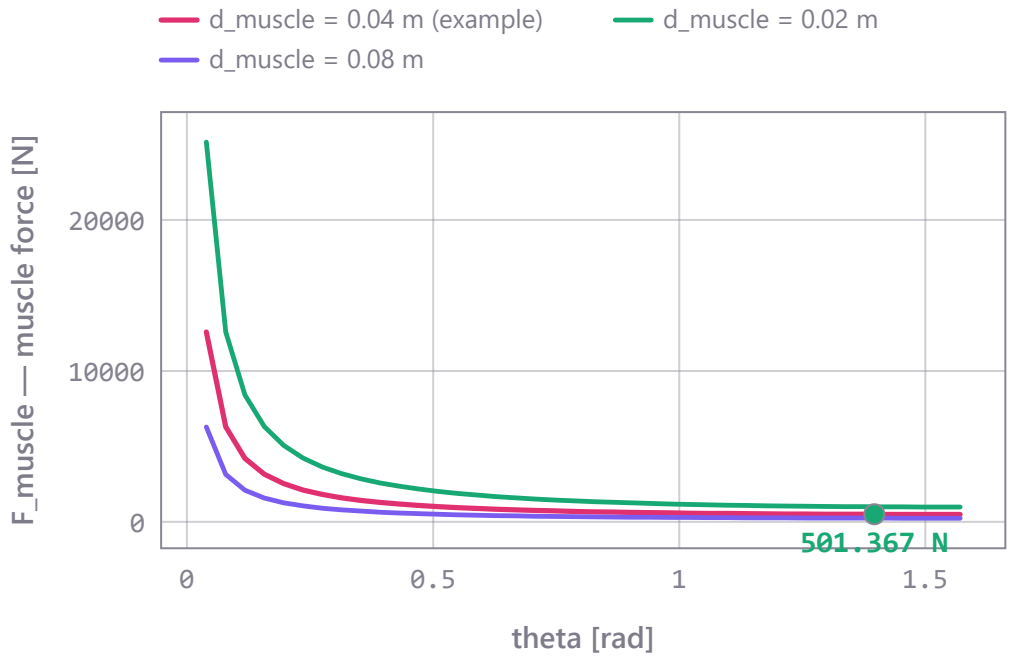
| input | meaning | condition |
|---|---|---|
| `W` | load held in the hand (weight) | |
| `d_load` | distance of the load from the elbow | |
| `W_arm` | weight of the forearm and hand | |
| `d_arm` | distance of the forearm's centre of mass from the elbow | |
| `d_muscle` | distance of the muscle insertion from the elbow | |
| `theta` | angle between the muscle and the forearm | |

| output | meaning |
|---|---|
| `F_muscle` | muscle force |
| `R_x` | joint reaction along the forearm (towards the hand positive) |
| `R_y` | joint reaction, upward positive |
| `R` | magnitude of the joint reaction |

- **Method:** moments about the elbow: F d_m sin(theta) = W d_load + W_arm d_arm; force balances give $R_x = F$ cos(theta), R_y = W + W_arm - F sin(theta)
- **Assumes:** static equilibrium in a vertical plane; forearm horizontal from the elbow, loads vertical; one muscle pulling up and back towards the elbow at angle theta; $0 < \theta < 180$ deg
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Free-body diagram of the forearm as a lever about the elbow (dot): load W = 50 N at 35 cm, forearm weight 15 N at 15 cm, muscle inserted at 4 cm pulling at 80° to the forearm (the example). Muscle and joint arrows are drawn to one scale, the two weights to a larger one. Returned: muscle force 501.367 N, joint reaction R = 437.5 N (R_x = 87.0614 N along the forearm, R_y = -428.75 N).




F_muscle (muscle force) against theta (angle between the muscle and the forearm) from 0 rad to 1.5708 rad. One curve for each value of d_muscle (distance of the muscle insertion from the elbow): 0.04 m, 0.02 m, 0.08 m. Other inputs as in the worked example: W = 50.0 N, d_load = 0.35000000000000003 m, W_arm = 15.0 N, d_arm = 0.15 m. 40 points per curve computed by biomedical-science-ai999x; marked: the worked example, theta = 1.39626 rad, F_muscle = 501.367 N.

```
biomedical_science_ai999x.solve('bmed_elbow_lever_muscle_force', W=50.0,
d_load=0.35000000000000003, W_arm=15.0, d_arm=0.15, d_muscle=0.04, theta=1.3962634015954636)
# -> F_muscle = 501.367 N, R_x = 87.0614 N, R_y = -428.75 N, R = 437.5 N   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_elbow_lever_muscle_force', W=100.0, d_load=0.3, W_arm=20.0,
d_arm=0.14, d_muscle=0.05, theta=1.5707963267948966)
# -> F_muscle = 656 N, R_x = 0 N, R_y = -536 N, R = 536 N   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Forearm lever muscle force: W = 50 N, d_load = 35 cm, W_arm = 15 N, d_arm = 15 cm, d_muscle = 4 cm, theta = 80 deg | SOLVED | F_muscle = 501.367 N, R_x = 87.0614 N, R_y = -428.75 N, R = 437.5 N |
| Forearm lever muscle force: W = 100 N, d_load = 0.3 m, W_arm = 20 N, d_arm = 0.14 m, d_muscle = 0.05 m, theta = 90 deg | SOLVED | F_muscle = 656 N, R_x = 0 N, R_y = -536 N, R = 536 N |
| Forearm lever muscle force: d_load = 35 cm, W_arm = 15 N, d_arm = 15 cm, d_muscle = 4 cm, theta = 80 deg | NEEDS_INPUT | bmed_elbow_lever_muscle_force needs W (load held in the hand (weight)). Example: solve('bmed_elbow_lever_muscle_force', |
| Forearm lever muscle force: W = 50 N, d_load = 35 cm, W_arm = 15 N, d_arm = 15 cm, d_muscle = 4 cm, theta = 80 deg, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Forearm lever muscle force: W = 50 N, d_load = 35 cm, W_arm = 15 N, | INVALID_INPUT | Static lever model of the forearm: muscle force and joint reaction: the muscle angle must lie between 0 and 180 |

| question | outcome | result |
|---|---|---|
| d_arm = 15 cm, d_muscle = 4 cm, theta = 190 deg | | degrees. |

### Poiseuille flow in a blood vessel: flow rate, mean velocity, wall shear stress, resistance — `bmed_poiseuille_vessel_flow`

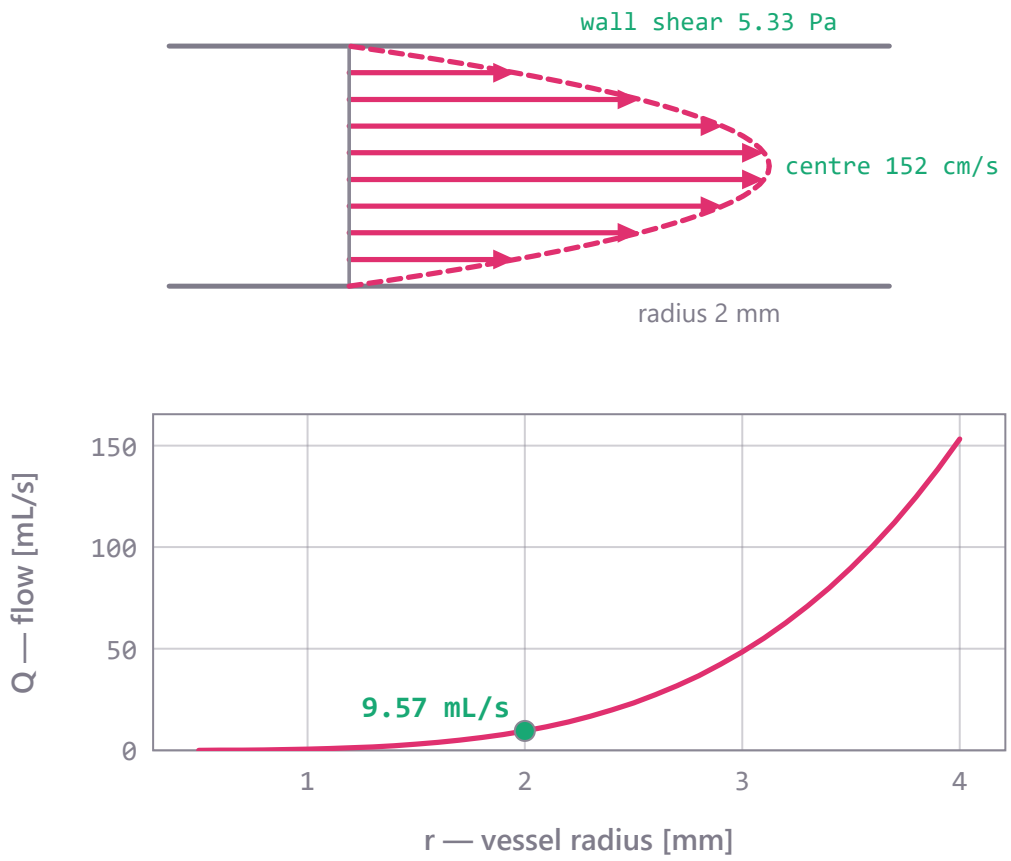
Laminar (Hagen-Poiseuille) flow of a Newtonian fluid through a rigid vessel: volume flow rate, mean velocity, wall shear stress and hydraulic resistance from the pressure drop, radius, length and viscosity.

$$Q = \frac{\pi r^4 \Delta P}{8 \mu L}, \quad \tau_w = \frac{4 \mu Q}{\pi r^3}, \quad R = \frac{8 \mu L}{\pi r^4}$$

| input | meaning | condition |
|---|---|---|
| `dP` | pressure drop along the vessel | |
| `r` | vessel radius | |
| `L` | vessel length | |
| `mu` | viscosity of the blood (Newtonian) | |

| output | meaning |
|---|---|
| `Q` | volume flow rate |
| `v_mean` | mean velocity |
| `tau_wall` | wall shear stress |
| `R_hyd` | hydraulic resistance |

- **Method:** $Q = pi\ r^4$ dP / (8 mu L); v = Q / (pi r^2); $tau_w = 4$ mu Q / (pi r^3); $R = 8$ mu L / (pi r^4)
- **Assumes:** steady laminar flow, Newtonian fluid; rigid straight cylindrical vessel, fully developed flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Poiseuille flow in a vessel (Δp = 533.29 Pa over L = 10 cm, viscosity 3.5 mPa·s): top, the parabolic velocity profile drawn from the returned mean velocity 76.1842 cm/s (centre velocity twice the mean, derived; arrows not to the length scale of the vessel); bottom, flow against radius, 36 runs — it grows with the fourth power of the radius. Marked: Q = 9.57359 mL/s at r = 2 mm, hydraulic resistance 5.5704e+07 Pa·s/m³.

```
biomedical_science_ai999x.solve('bmed_poiseuille_vessel_flow', dP=533.28954966, r=0.002, L=0.1,
mu=0.0035)
# -> Q = 9.57359e-06 m^3/s, v_mean = 0.761842 m/s, tau_wall = 5.3329 Pa, R_hyd = 5.57042e+07
Pa*s/m^3   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_poiseuille_vessel_flow', dP=50.0, r=9.999999999999999e-05,
L=0.005, mu=0.004)
# -> Q = 9.81748e-11 m^3/s, v_mean = 0.003125 m/s, tau_wall = 0.5 Pa, R_hyd = 5.09296e+11 Pa*s/m^3
(SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Poiseuille vessel flow: dP = 4 mmHg, r = 2 mm, L = 10 cm, mu = 3.5 mPa*s | SOLVED | Q = 9.57359e-06 m^3/s, v_mean = 0.761842 m/s, tau_wall = 5.3329 Pa, R_hyd = 5.57042e+07 Pa*s/m^3 |
| Poiseuille vessel flow: dP = 50 Pa, r = 100 um, L = 5 mm, mu = 0.004 Pa*s | SOLVED | Q = 9.81748e-11 m^3/s, v_mean = 0.003125 m/s, tau_wall = 0.5 Pa, R_hyd = 5.09296e+11 Pa*s/m^3 |
| Poiseuille vessel flow: r = 2 mm, L = 10 cm, mu = 3.5 mPa*s | NEEDS_INPUT | bmed_poiseuille_vessel_flow needs dP (pressure drop along the vessel). Example: solve('bmed_poiseuille_vessel_flow', dP= |

| question | outcome | result |
|---|---|---|
| Poiseuille vessel flow: dP = 4 mmHg, r = 2 mm, L = 10 cm, mu = 3.5 mPa*s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Two-element Windkessel: diastolic pressure decay — `bmed_windkessel_diastolic_decay`

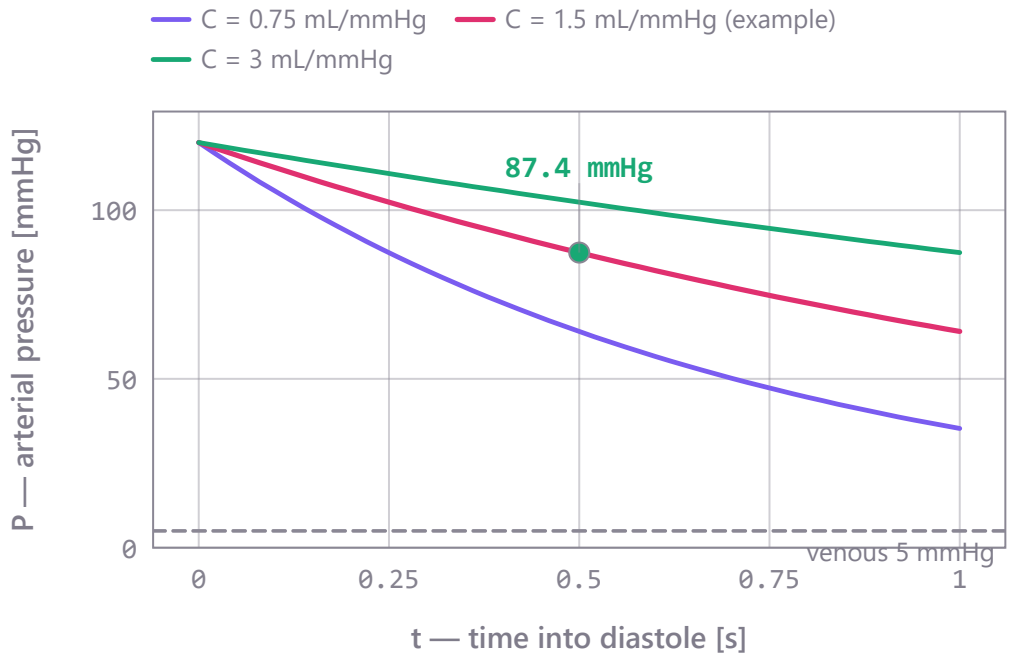
The arterial pressure of a two-element Windkessel model during diastole (no inflow), decaying from P0 towards the venous pressure with time constant RC.

$$P(t) = P_v + (P_0 - P_v)\, e^{-t/(RC)}$$

| input | meaning | condition |
|---|---|---|
| `P0` | pressure at the start of diastole | |
| `Pv` | venous (outflow) pressure | |
| `R` | peripheral resistance | |
| `C` | arterial compliance | |
| `t` | time into diastole | |

| output | meaning |
|---|---|
| `P` | arterial pressure at time t |

- **Method:** C dP/dt = -(P - Pv)/R: $P = Pv + (P_0 - Pv)$ exp(-t/(R C))
- **Assumes:** no inflow during diastole; linear resistance and compliance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Two-element Windkessel: arterial pressure falling through diastole from P0 = 120 mmHg towards the venous pressure (R = 1 mmHg·s/mL) for three compliances, 51 runs each over 1 s (1 mmHg = 133.322 Pa). Marked: example at t = 0.5 s, P = 87.4011 mmHg.

```
biomedical_science_ai999x.solve('bmed_windkessel_diastolic_decay', P0=15998.6864898,
Pv=666.611937075, R=133322387.41500002, C=1.1250923637684845e-08, t=0.5)
# -> P = 11652.5 Pa   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_windkessel_diastolic_decay', P0=13332.238741500001, Pv=0.0,
R=159986864.89800003, C=8.25067733430222e-09, t=0.8)
# -> P = 7272.68 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Windkessel diastolic decay: P0 = 120 mmHg, Pv = 5 mmHg, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, t = 0.5 s | SOLVED | P = 11652.5 Pa |
| Windkessel diastolic decay: P0 = 100 mmHg, Pv = 0 mmHg, R = 1.2 mmHg*s/mL, C = 1.1 mL/mmHg, t = 0.8 s | SOLVED | P = 7272.68 Pa |
| Windkessel diastolic decay: Pv = 5 mmHg, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, t = 0.5 s | NEEDS_INPUT | bmed_windkessel_diastolic_decay needs P0 (pressure at the start of diastole). Example: solve('bmed_windkessel_diastolic_ |
| Windkessel diastolic decay: P0 = 120 mmHg, Pv = 5 mmHg, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, t = 0.5 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Windkessel models (two- and three-element): input impedance at a frequency — `bmed_windkessel_input_impedance`

The modulus and phase of the input impedance of a two-element (R, C) or three-element (Zc, R, C) Windkessel model of the arterial tree at a stated frequency.

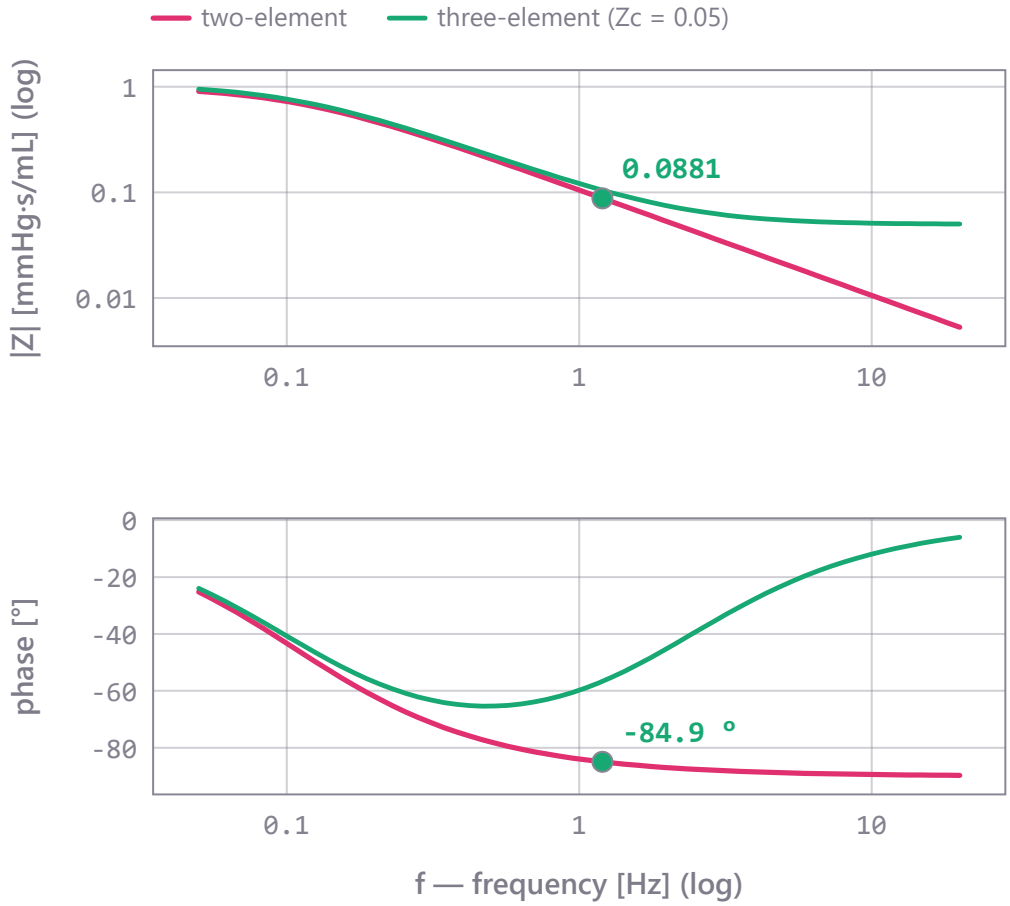
$$Z(\omega) = Z_c + \frac{R}{1 + j\omega RC}$$

| input | meaning | condition |
|---|---|---|
| `model` | Windkessel model | |
| `R` | peripheral resistance | |
| `C` | arterial compliance | |
| `f` | frequency | |
| `Zc` | characteristic impedance of the aorta (three-element model only) | |

| output | meaning |
|---|---|
| `Z_mod` | impedance modulus |
| `phase_deg` | impedance phase in degrees |

- **Method:** Z = Zc + R/(1 + j 2 pi f R C) (Zc = 0 for two elements); modulus and phase

- **Assumes:** linear model, sinusoidal steady state; Zc only for the three-element model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Input impedance of the arterial Windkessel (R = 1 mmHg·s/mL, C = 1.5 mL/mmHg) against frequency, modulus (top) and phase (bottom), 60 runs per line, for the two-element model and a three-element model with a chosen characteristic impedance Zc = 0.05 R: the third element levels the modulus off at high frequency and brings the phase back towards 0. Marked: example f = 1.2 Hz, |Z| = 0.0880758 mmHg·s/mL, phase -84.9471°.

```
biomedical_science_ai999x.solve('bmed_windkessel_input_impedance', model='two_element',
R=133322387.41500002, C=1.1250923637684845e-08, f=1.2)
# -> Z_mod = 1.17425e+07 Pa*s/m^3, phase_deg = -84.9471   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_windkessel_input_impedance', model='three_element',
R=133322387.41500002, C=1.1250923637684845e-08, f=2.0, Zc=6666119.370750002)
# -> Z_mod = 9.96556e+06 Pa*s/m^3, phase_deg = -45.0521   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Windkessel impedance: model = two_element, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, f = 1.2 Hz | SOLVED | Z_mod = 1.17425e+07 Pa*s/m^3, phase_deg = -84.9471 |
| Windkessel impedance: model = three_element, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, f = 2 Hz, Zc = 0.05 mmHg*s/mL | SOLVED | Z_mod = 9.96556e+06 Pa*s/m^3, phase_deg = -45.0521 |

| question | outcome | result |
|---|---|---|
| Windkessel impedance: R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, f = 1.2 Hz | NEEDS_INPUT | bmed_windkessel_input_impedance needs model (Windkessel model). Example: solve('bmed_windkessel_input_impedance', model= |
| Windkessel impedance: model = two_element, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, f = 1.2 Hz, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Windkessel impedance: model = three_element, R = 1 mmHg*s/mL, C = 1.5 mL/mmHg, f = 2 Hz | INVALID_INPUT | Windkessel models (two- and three-element): input impedance at a frequency: the three-element model needs the characteri |

## Kelvin-Voigt viscoelastic tissue model: creep strain under constant stress — `bmed_kelvin_voigt_creep`

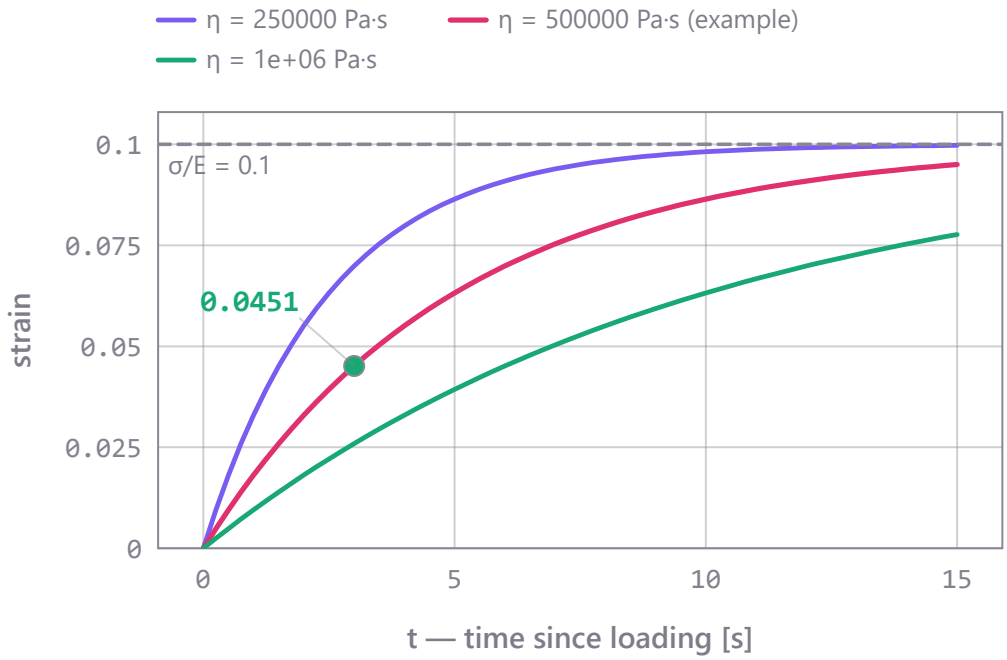
The strain of a Kelvin-Voigt (spring parallel to dashpot) tissue model a time t after a constant stress is applied.

$$\varepsilon(t) = \frac{\sigma}{E}\left(1 - e^{-Et/\eta}\right)$$

| input | meaning | condition |
|---|---|---|
| `sigma` | applied constant stress | |
| `E` | spring modulus | |
| `eta` | dashpot viscosity $(Pas)$ | |
| `t` | time since loading | |

| output | meaning |
|---|---|
| `strain` | strain at time t |

- **Method:** eta de/dt + E $e = \sigma$: $e = (\sigma/E)$(1 - exp(-E t/eta))
- **Assumes:** linear viscoelasticity; stress applied as a step at $t = 0$, no initial strain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Kelvin–Voigt creep under a constant stress σ = 10000 Pa (spring E = 100000 Pa) for three dashpot viscosities, 61 runs each: the strain creeps up towards σ/E (dashed, from the inputs), slower for a more viscous dashpot. Marked: the example t = 3 s, strain 0.0451188.

```
biomedical_science_ai999x.solve('bmed_kelvin_voigt_creep', sigma=10000.0, E=100000.0, eta=500000.0,
t=3.0)
# -> strain = 0.0451188   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_kelvin_voigt_creep', sigma=2000000.0, E=1000000000.0,
eta=5000000000.0, t=10.0)
# -> strain = 0.00172933   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Kelvin-Voigt creep: sigma = 10 kPa, E = 100 kPa, eta = 500 kPa*s, t = 3 s | SOLVED | strain = 0.0451188 |
| Kelvin-Voigt creep: sigma = 2 MPa, E = 1 GPa, eta = 5 GPa*s, t = 10 s | SOLVED | strain = 0.00172933 |
| Kelvin-Voigt creep: E = 100 kPa, eta = 500 kPa*s, t = 3 s | NEEDS_INPUT | bmed_kelvin_voigt_creep needs sigma (applied constant stress). Example: solve('bmed_kelvin_voigt_creep', sigma='10000 Pa |
| Kelvin-Voigt creep: sigma = 10 kPa, E = 100 kPa, eta = 500 kPa*s, t = 3 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Standard linear solid (Zener) tissue model: creep under constant stress — `bmed_sls_creep`

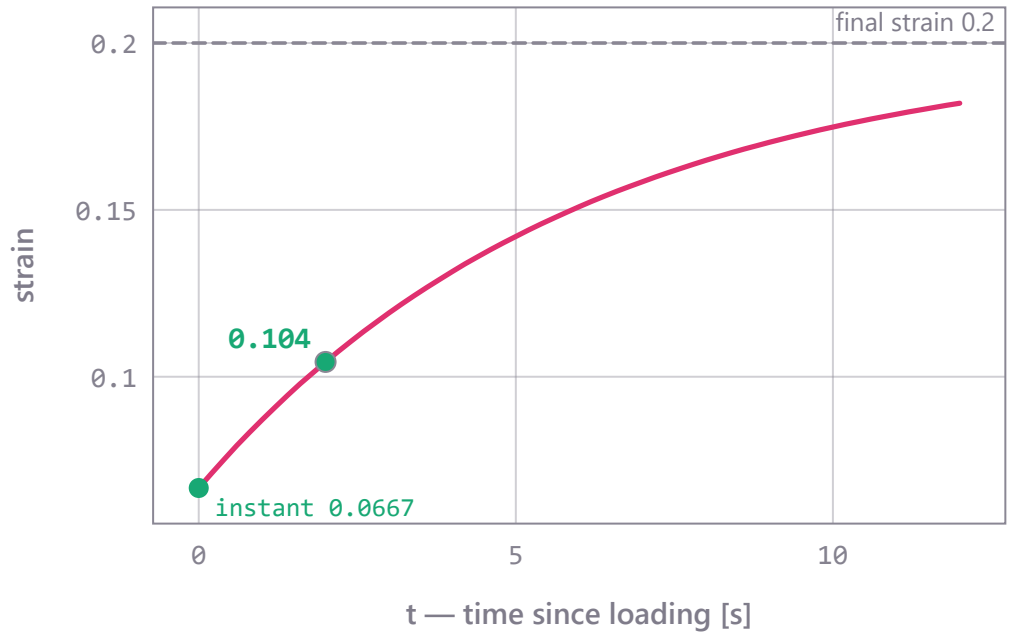
The creep strain of a standard linear solid (a spring E_inf parallel to a Maxwell arm E1-eta) a time t after a constant stress is applied, and the final strain.

$$\varepsilon(t) = \frac{\sigma}{E_\infty} - \left(\frac{\sigma}{E_\infty} - \frac{\sigma}{E_\infty + E_1}\right) e^{-t/\tau_c},\ \tau_c = \frac{\eta(E_\infty + E_1)}{E_\infty E_1}$$

| input | meaning | condition |
|---|---|---|
| `sigma` | applied constant stress | |
| `E_inf` | equilibrium (parallel) spring modulus | |
| `E1` | spring modulus of the Maxwell arm | |
| `eta` | dashpot viscosity of the Maxwell arm | |
| `t` | time since loading | |

| output | meaning |
|---|---|
| `strain` | strain at time t |
| `strain_inf` | final strain sigma/E_inf |

- **Method:** e(t) = sigma/E_inf - (sigma/E_inf - sigma/(E_inf + E1)) exp(-t/tau_c), $tau_c = \eta$ (E_inf + E1)/(E_inf E1)
- **Assumes:** linear viscoelasticity; step stress at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Standard linear solid under a constant stress σ = 10000 Pa (E∞ = 50000 Pa, Maxwell arm E1 = 100000 Pa, η = 200000 Pa·s), 61 runs: an instantaneous elastic strain (dot) then creep to the final strain σ/E∞ = 0.2. Marked: the example t = 2 s, strain 0.104462.

```
biomedical_science_ai999x.solve('bmed_sls_creep', sigma=10000.0, E_inf=50000.0, E1=100000.0,
eta=200000.0, t=2.0)
# -> strain = 0.104462, strain_inf = 0.2   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_sls_creep', sigma=1000000.0, E_inf=200000000.0,
E1=800000000.0, eta=2000000000.0, t=1.0)
# -> strain = 0.00130753, strain_inf = 0.005   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Standard linear solid creep: sigma = 10 kPa, E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 2 s | SOLVED | strain = 0.104462, strain_inf = 0.2 |
| Standard linear solid creep: sigma = 1 MPa, E_inf = 200 MPa, E1 = 800 MPa, eta = 2 GPa*s, t = 1 s | SOLVED | strain = 0.00130753, strain_inf = 0.005 |
| Standard linear solid creep: E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 2 s | NEEDS_INPUT | bmed_sls_creep needs sigma (applied constant stress). Example: solve('bmed_sls_creep', sigma='10000 Pa', E_inf='50000 Pa |
| Standard linear solid creep: sigma = 10 kPa, E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 2 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Standard linear solid (Zener) tissue model: stress relaxation at constant strain — `bmed_sls_stress_relaxation`

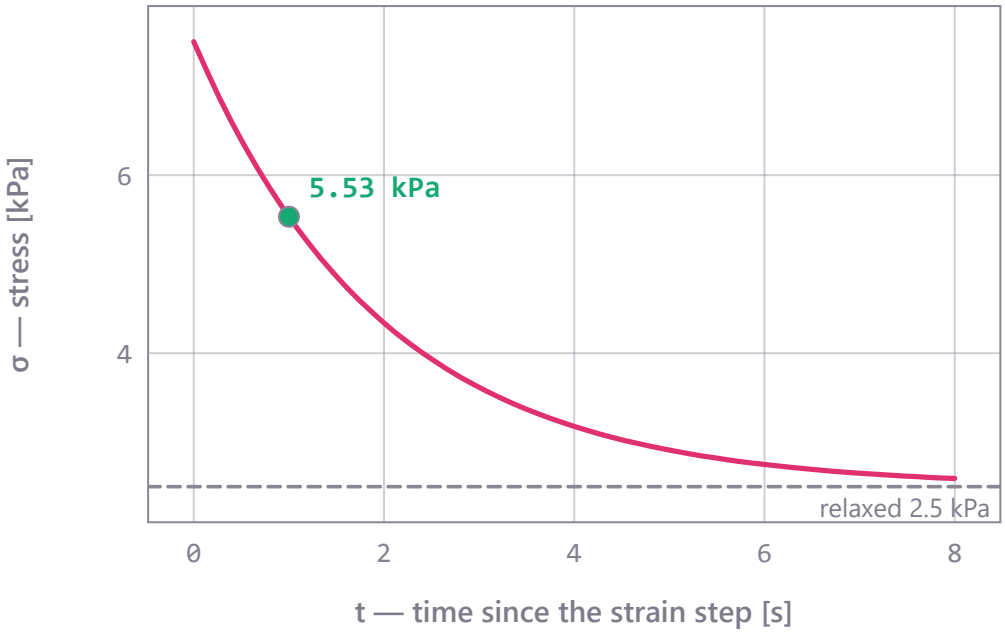
The stress of a standard linear solid a time t after a step strain eps0 is imposed, and the relaxed (equilibrium) stress.

$$\sigma(t) = \varepsilon_0 \left( E_\infty + E_1 e^{-E_1 t/\eta} \right)$$

| input | meaning | condition |
|---|---|---|
| `eps0` | imposed step strain (a pure number, e.g. 0.05) | |
| `E_inf` | equilibrium spring modulus | |
| `E1` | spring modulus of the Maxwell arm | |
| `eta` | dashpot viscosity of the Maxwell arm | |
| `t` | time since the strain step | |

| output | meaning |
|---|---|
| `stress` | stress at time t |
| `stress_inf` | relaxed stress E_inf eps0 |

- **Method:** sigma(t) = eps0 (E_inf + E1 exp(-E1 t/eta))
- **Assumes:** linear viscoelasticity; step strain at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Stress relaxation of a standard linear solid after a step strain ε0 = 0.05 (E∞ = 50000 Pa, E1 = 100000 Pa, η = 200000 Pa·s), 61 runs: the stress jumps to (E∞ + E1)ε0 and relaxes to E∞ε0 = 2.5 kPa. Marked: example t = 1 s, σ = 5.53265 kPa.

```
biomedical_science_ai999x.solve('bmed_sls_stress_relaxation', eps0=0.05, E_inf=50000.0,
E1=100000.0, eta=200000.0, t=1.0)
# -> stress = 5532.65 Pa, stress_inf = 2500 Pa   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_sls_stress_relaxation', eps0=0.01, E_inf=200000000.0,
E1=800000000.0, eta=2000000000.0, t=5.0)
# -> stress = 3.08268e+06 Pa, stress_inf = 2e+06 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Standard linear solid relaxation: eps0 = 0.05, E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 1 s | SOLVED | stress = 5532.65 Pa, stress_inf = 2500 Pa |
| Standard linear solid relaxation: eps0 = 0.01, E_inf = 200 MPa, E1 = 800 MPa, eta = 2 GPa*s, t = 5 s | SOLVED | stress = 3.08268e+06 Pa, stress_inf = 2e+06 Pa |
| Standard linear solid relaxation: E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 1 s | NEEDS_INPUT | bmed_sls_stress_relaxation needs eps0 (imposed step strain (a pure number, e.g. 0.05)). Example: solve('bmed_sls_stress_ |
| Standard linear solid relaxation: eps0 = 0.05, E_inf = 50 kPa, E1 = 100 kPa, eta = 200 kPa*s, t = 1 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Cardiac output by indicator dilution (Stewart-Hamilton) —

`bmed_indicator_dilution_cardiac_output`

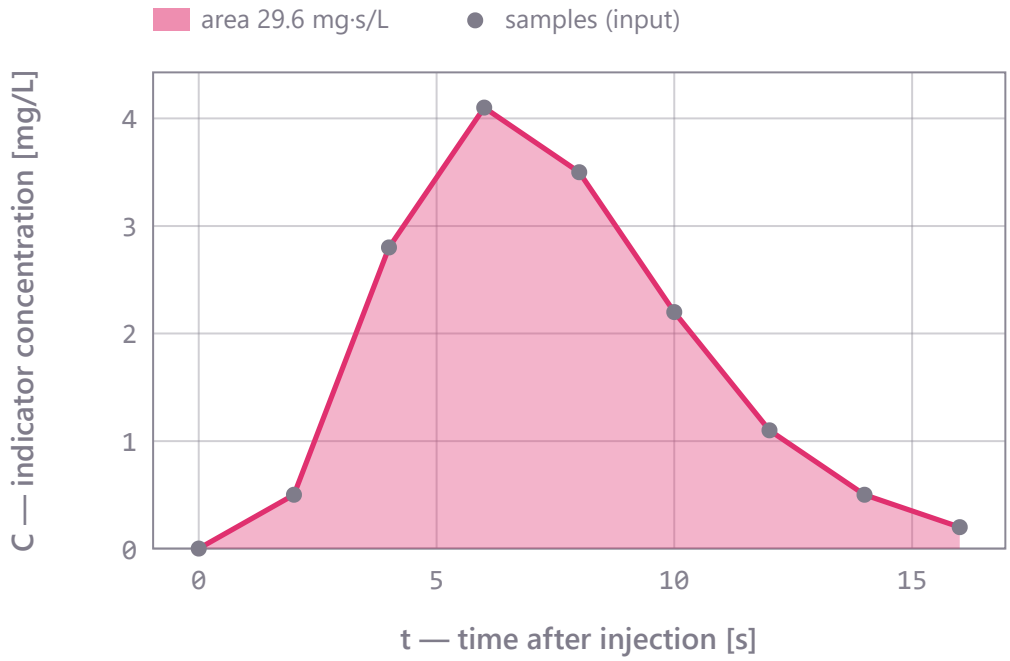
The cardiac output (volume flow) from the mass of an injected indicator and its sampled first-pass concentration curve, by the Stewart-Hamilton principle with a trapezoidal area.

$$CO = \frac{m}{\int_0^\infty C(t)\,dt}$$

| input | meaning | condition |
|---|---|---|
| `m` | mass of indicator injected | |
| `t` | sampling times in s, strictly increasing | |
| `C` | indicator concentrations in mg/L (first pass, recirculation excluded) | |

| output | meaning |
|---|---|
| `AUC` | area under the concentration curve, $mgs/L$ |
| `CO` | cardiac output m / AUC |

- **Method:** CO = m / integral of C dt (linear trapezoidal rule)
- **Assumes:** complete mixing, no indicator loss; first-pass curve (recirculation removed by the user); times in s, concentrations in mg/L; linear trapezoids
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Indicator-dilution curve: the example's 9 concentration samples after injecting m = 5 mg of indicator (input), with the area under them computed by the package, AUC = 29.6 mg·s/L. Marked: cardiac output m/AUC = 10.1351 L/min (0.000168919 $m^3/s$).

```
biomedical_science_ai999x.solve('bmed_indicator_dilution_cardiac_output', m=4.9999999999999996e-06,
t=[0, 2, 4, 6, 8, 10, 12, 14, 16], C=[0, 0.5, 2.8, 4.1, 3.5, 2.2, 1.1, 0.5, 0.2])
# -> AUC = 29.6, CO = 0.000168919 m^3/s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_indicator_dilution_cardiac_output', m=9.999999999999999e-06,
t=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10], C=[0, 0, 1.2, 4.5, 6.8, 6.0, 4.1, 2.5, 1.3, 0.6, 0.2])
# -> AUC = 27.1, CO = 0.000369004 m^3/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Indicator dilution cardiac output: m = 5 mg, t = [0, 2, 4, 6, 8, 10, 12, 14, 16], C = [0, 0.5, 2.8, 4.1, 3.5, 2.2, 1.1, 0.5, 0.2] | SOLVED | AUC = 29.6, CO = 0.000168919 m^3/s |
| Indicator dilution cardiac output: m = 10 mg, t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10], C = [0, 0, 1.2, 4.5, 6.8, 6, 4.1, 2.5, 1.3, 0.6, 0.2] | SOLVED | AUC = 27.1, CO = 0.000369004 m^3/s |
| Indicator dilution cardiac output: t = [0, 2, 4, 6, 8, 10, 12, 14, 16], C = [0, 0.5, 2.8, 4.1, 3.5, 2.2, 1.1, 0.5, 0.2] | NEEDS_INPUT | bmed_indicator_dilution_cardiac_output needs m (mass of indicator injected). Example: solve('bmed_indicator_dilution_car |
| Indicator dilution cardiac output: m = 5 mg, t = [0, 2, 4, 6, 8, 10, 12, 14, 16], C = [0, 0.5, 2.8, 4.1, 3.5, 2.2, 1.1, 0.5, 0.2], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Single-compartment respiratory mechanics: resistance and compliance by least squares — `bmed_respiratory_mechanics_fit`

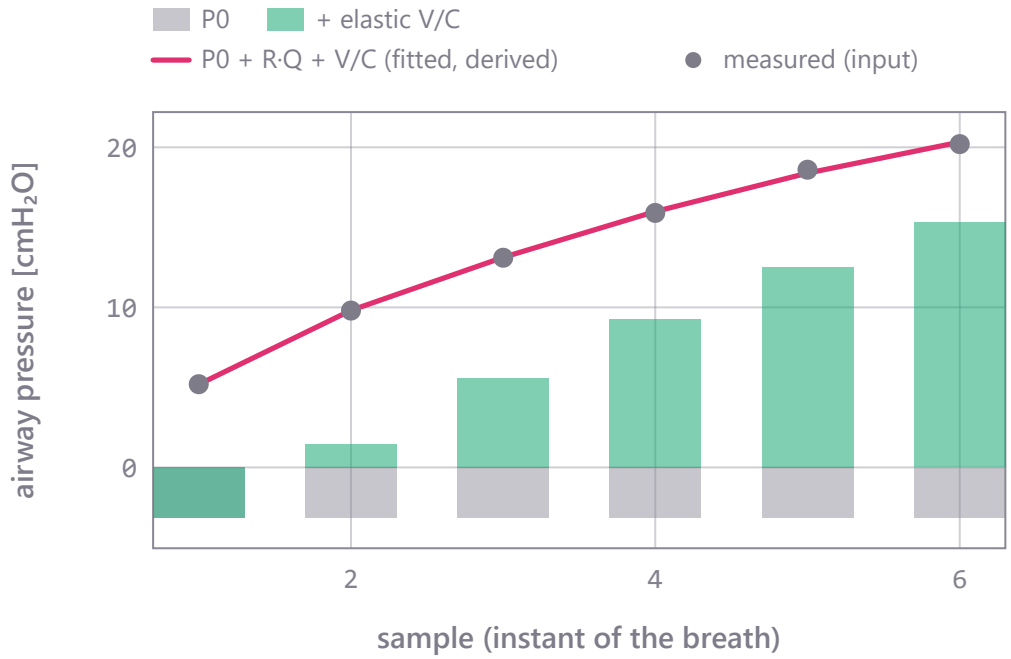
Airway resistance, respiratory compliance and the end-expiratory pressure of the single-compartment equation of motion $P = R\,Q + V/C + P_0$, fitted by least squares to sampled pressure, flow and volume.

$$P = R\dot{V} + \frac{V}{C} + P_0$$

| input | meaning | condition |
|---|---|---|
| `P` | airway pressures in cmH2O | |
| `Q` | flows in L/s at the same instants | |
| `V` | volumes above end-expiration in L at the same instants | |

| output | meaning |
|---|---|
| `R` | resistance, cmH2O*s/L |
| `C` | compliance, L/cmH2O |
| `P0` | pressure at zero flow and volume, cmH2O |

- **Method:** multiple linear regression of P on Q, V and a constant; C = 1/elastance
- **Assumes:** linear single-compartment model; pressures in cmH2O, flows in L/s, volumes in L; ordinary least squares
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Single-compartment fit of the example's 6 samples of pressure, flow and volume (inputs): the line is P0 + R·Q + V/C from the returned resistance R = 16.7141 $cmH_2O·s/L$, compliance C = 0.0216558 $L/cmH_2O$ and P0 = -3.16919 $cmH_2O$ (derived); bars split it into P0 and the elastic part V/C, the gap to the line being the resistive part R·Q.

```
biomedical_science_ai999x.solve('bmed_respiratory_mechanics_fit', P=[5.2, 9.8, 13.1, 15.9, 18.6,
20.2], Q=[0.5, 0.5, 0.45, 0.4, 0.35, 0.3], V=[0.0, 0.1, 0.19, 0.27, 0.34, 0.4])
# -> R = 16.7141, C = 0.0216558, P0 = -3.16919   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_respiratory_mechanics_fit', P=[8, 12.5, 16, 19.2, 22, 24.1,
25.5], Q=[0.8, 0.75, 0.7, 0.6, 0.5, 0.4, 0.3], V=[0, 0.08, 0.15, 0.22, 0.27, 0.32, 0.35])
# -> R = 3.60921, C = 0.0182636, P0 = 5.225   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Respiratory mechanics fit: P = [5.2, 9.8, 13.1, 15.9, 18.6, 20.2], Q = [0.5, 0.5, 0.45, 0.4, 0.35, 0.3], V = [0, 0.1, 0.19, 0.27, 0.34, 0.4] | SOLVED | R = 16.7141, C = 0.0216558, P0 = -3.16919 |
| Respiratory mechanics fit: P = [8, 12.5, 16, 19.2, 22, 24.1, 25.5], Q = [0.8, 0.75, 0.7, 0.6, 0.5, 0.4, 0.3], V = [0, 0.08, 0.15, 0.22, 0.27, 0.32, 0.35] | SOLVED | R = 3.60921, C = 0.0182636, P0 = 5.225 |
| Respiratory mechanics fit: Q = [0.5, 0.5, 0.45, 0.4, 0.35, 0.3], V = [0, 0.1, 0.19, 0.27, 0.34, 0.4] | NEEDS_INPUT | bmed_respiratory_mechanics_fit needs P (airway pressures in cmH2O). Example: solve('bmed_respiratory_mechanics_fit', P=[ |
| Respiratory mechanics fit: P = [5.2, 9.8, 13.1, 15.9, 18.6, 20.2], Q = [0.5, 0.5, 0.45, 0.4, 0.35, 0.3], V = [0, 0.1, 0.19, 0.27, 0.34, 0.4], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Goldman-Hodgkin-Katz voltage equation: membrane potential from K, Na and Cl — `bmed_ghk_voltage`

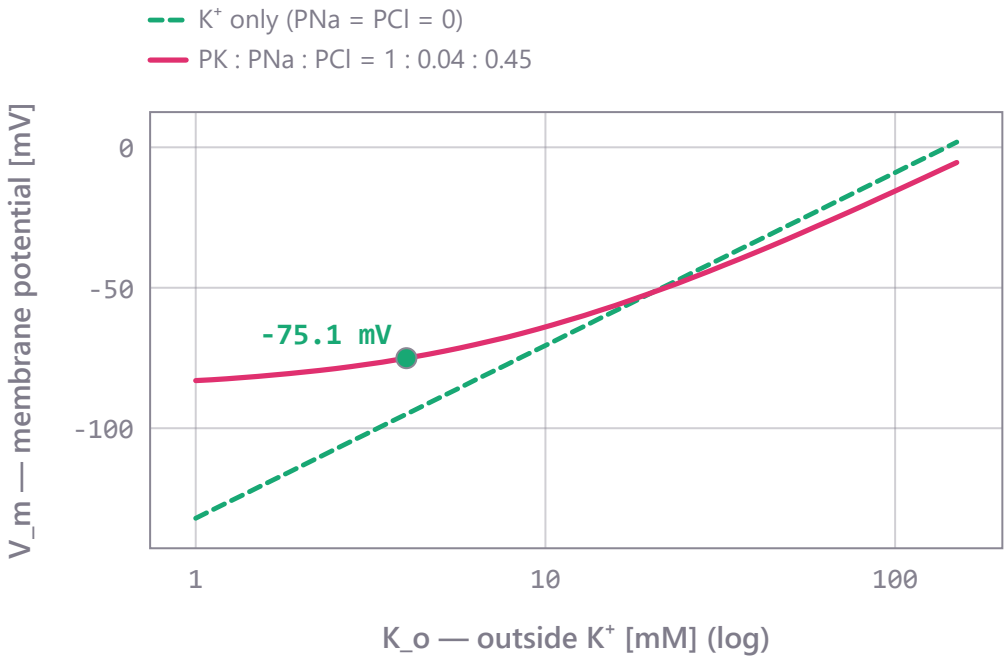
The resting membrane potential of the Goldman-Hodgkin-Katz voltage equation from the relative permeabilities and the inside and outside concentrations of K+, Na+ and Cl-.

$$V_m = \frac{RT}{F} \ln \frac{P_K[K^+]_o + P_{Na}[Na^+]_o + P_{Cl}[Cl^-]_i}{P_K[K^+]_i + P_{Na}[Na^+]_i + P_{Cl}[Cl^-]_o}$$

| input | meaning | condition |
|---|---|---|
| `T` | absolute temperature (or degC) | |
| `PK` | relative permeability to K+ (> 0) | |
| `PNa` | relative permeability to Na+ | |
| `PCl` | relative permeability to Cl- | |
| `K_o` | outside K+ | |
| `K_i` | inside K+ | |
| `Na_o` | outside Na+ | |
| `Na_i` | inside Na+ | |
| `Cl_o` | outside Cl- | |
| `Cl_i` | inside Cl- | |

| output | meaning |
|---|---|
| `V_m` | membrane potential, inside relative to outside |

- **Method:** $V = (RT/F)$ ln((PK K_o + PNa Na_o + PCl Cl_i)/(PK K_i + PNa Na_i + PCl Cl_o))
- **Assumes:** steady state with zero net current; constant field in the membrane, independent ions; only K+, Na+ and Cl- permeate; only ratios of permeabilities matter
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Goldman–Hodgkin–Katz membrane potential against the outside potassium concentration (log axis, 50 runs per line) for the example's other concentrations (K_i 140, Na 145/12, Cl 116/4 mM, T = 310 K): with potassium alone the line is straight (Nernst-like); the sodium and chloride permeabilities bend it at low K_o. Marked: example K_o = 4 mM, V_m = -75.0663 mV.

```
biomedical_science_ai999x.solve('bmed_ghk_voltage', T=310.0, PK=1, PNa=0.04, PCl=0.45, K_o=4.0,
K_i=140.0, Na_o=145.0, Na_i=12.0, Cl_o=116.0, Cl_i=4.0)
# -> V_m = -0.0750663 V   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_ghk_voltage', T=279.45, PK=1, PNa=0.03, PCl=0.1, K_o=20.0,
K_i=400.0, Na_o=440.0, Na_i=50.0, Cl_o=560.0, Cl_i=52.0)
# -> V_m = -0.0596663 V   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| GHK voltage: T = 310 K, PK = 1, PNa = 0.04, PCl = 0.45, K_o = 4 mmol/L, K_i = 140 mmol/L, Na_o = 145 mmol/L, Na_i = 12 mmol/L, Cl_o = 116 mmol/L, Cl_i = 4 mmol/L | SOLVED | V_m = -0.0750663 V |
| GHK voltage: T = 6.3 degC, PK = 1, PNa = 0.03, PCl = 0.1, K_o = 20 mM, K_i = 400 mM, Na_o = 440 mM, Na_i = 50 mM, Cl_o = 560 mM, Cl_i = 52 mM | SOLVED | V_m = -0.0596663 V |
| GHK voltage: PK = 1, PNa = 0.04, PCl = 0.45, K_o = 4 mmol/L, K_i = 140 mmol/L, Na_o = 145 mmol/L, Na_i = 12 mmol/L, Cl_o = 116 mmol/L, Cl_i = 4 mmol/L | NEEDS_INPUT | bmed_ghk_voltage needs T (absolute temperature (or degC)). Example: solve('bmed_ghk_voltage', T='310 K', PK=1, PNa=0.04, |
| GHK voltage: T = 310 K, PK = 1, PNa = 0.04, PCl = 0.45, K_o = 4 mmol/L, K_i = 140 mmol/L, Na_o = 145 mmol/L, Na_i = 12 mmol/L, Cl_o = 116 mmol/L, Cl_i = 4 mmol/L, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

| question | outcome | result |
|---|---|---|
| GHK voltage: T = 36.85 degC, PK = 1, PNa = 0.04, PCl = 0.45, K_o = 4 mmol/L, K_i = 140 mmol/L, Na_o = 145 mmol/L, Na_i = 12 mmol/L, Cl_o = 116 mmol/L, Cl_i = 4 mmol/L | SOLVED | V_m = -0.0750663 V |

### Goldman-Hodgkin-Katz current equation: ionic current density through a membrane — `bmed_ghk_current_density`

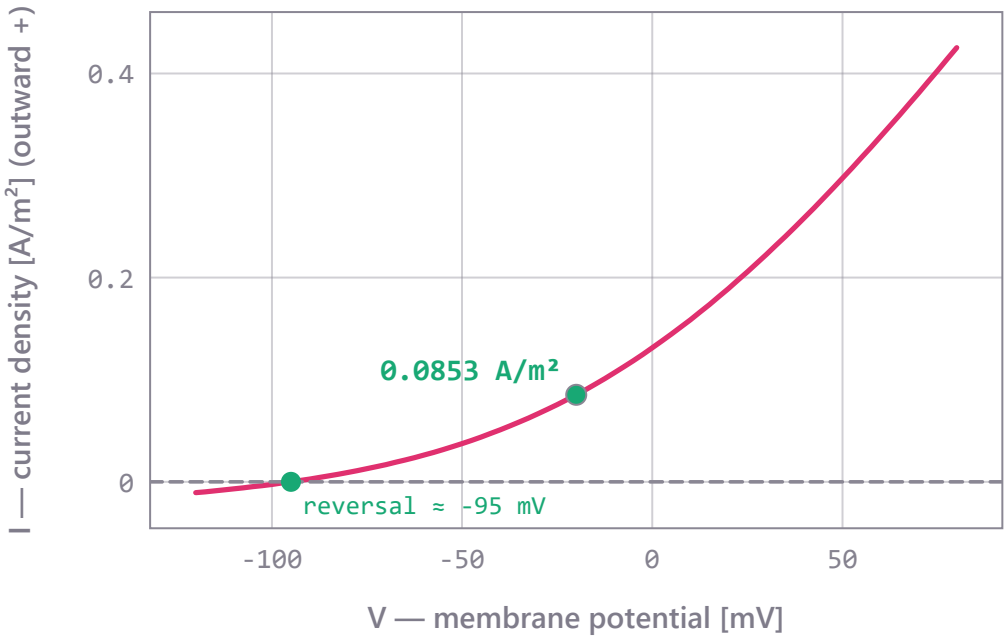
The current density carried by one ion species through a membrane of given permeability at a stated membrane potential, from the GHK current (constant-field) equation; outward current positive.

$$I = P z^2 \frac{F^2 V}{RT} \frac{[C]_i - [C]_o e^{-zFV/RT}}{1 - e^{-zFV/RT}}$$

| input | meaning | condition |
|---|---|---|
| `P` | membrane permeability to the ion | |
| `z` | valence of the ion (+1, +2, -1, ...) | |
| `C_in` | inside concentration | |
| `C_out` | outside concentration | |
| `V` | membrane potential, inside relative to outside | |
| `T` | absolute temperature (or degC) | |

| output | meaning |
|---|---|
| `I` | current density, outward positive |

- **Method:** $I = P\ z^2\ F^2$ V/(RT) (C_in - C_out exp(-u))/(1 - exp(-u)), u = zFV/(RT) (written stably as P z F [C_in g(u) - C_out g(-u)], g(u) = u/(1 - e^-u))
- **Assumes:** constant field across the membrane; independent ion movement; outward current positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Goldman–Hodgkin–Katz current–voltage curve of one ion (z = 1, P = 1.0000e-08 m/s, inside 140 mM, outside 4 mM, T = 310 K), 81 runs from −120 to +80 mV: curved (rectifying) because the concentrations differ. Dot: the reversal potential where the current changes sign (interpolated between runs). Marked: example V = -20 mV, I = 0.0852825 A/m².

```
biomedical_science_ai999x.solve('bmed_ghk_current_density', P=1e-08, z=1, C_in=140.0, C_out=4.0,
V=-0.02, T=310.0)
# -> I = 0.0852825 A/m^2   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_ghk_current_density', P=5e-09, z=2, C_in=0.0001, C_out=2.0,
V=-0.07, T=310.15)
# -> I = -0.0101622 A/m^2   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| GHK current density: P = 1e-8 m/s, z = 1, C_in = 140 mmol/L, C_out = 4 mmol/L, V = -20 mV, T = 310 K | SOLVED | I = 0.0852825 A/m^2 |
| GHK current density: P = 5e-9 m/s, z = 2, C_in = 0.0001 mmol/L, C_out = 2 mmol/L, V = -70 mV, T = 37 degC | SOLVED | I = -0.0101622 A/m^2 |
| GHK current density: z = 1, C_in = 140 mmol/L, C_out = 4 mmol/L, V = -20 mV, T = 310 K | NEEDS_INPUT | bmed_ghk_current_density needs P (membrane permeability to the ion). Example: solve('bmed_ghk_current_density', P='1e-08 |
| GHK current density: P = 1e-8 m/s, z = 1, C_in = 140 mmol/L, C_out = 4 mmol/L, V = -20 mV, T = 310 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| GHK current density: P = 1e-8 m/s, z = 0, C_in = 140 mmol/L, C_out = 4 mmol/L, V = -20 mV, T = 310 K | INVALID_INPUT | Goldman-Hodgkin-Katz current equation: ionic current density through a membrane: z is a nonzero whole number from -3 to |

### Leaky integrate-and-fire neuron: firing rate under constant current — `bmed_lif_firing_rate`

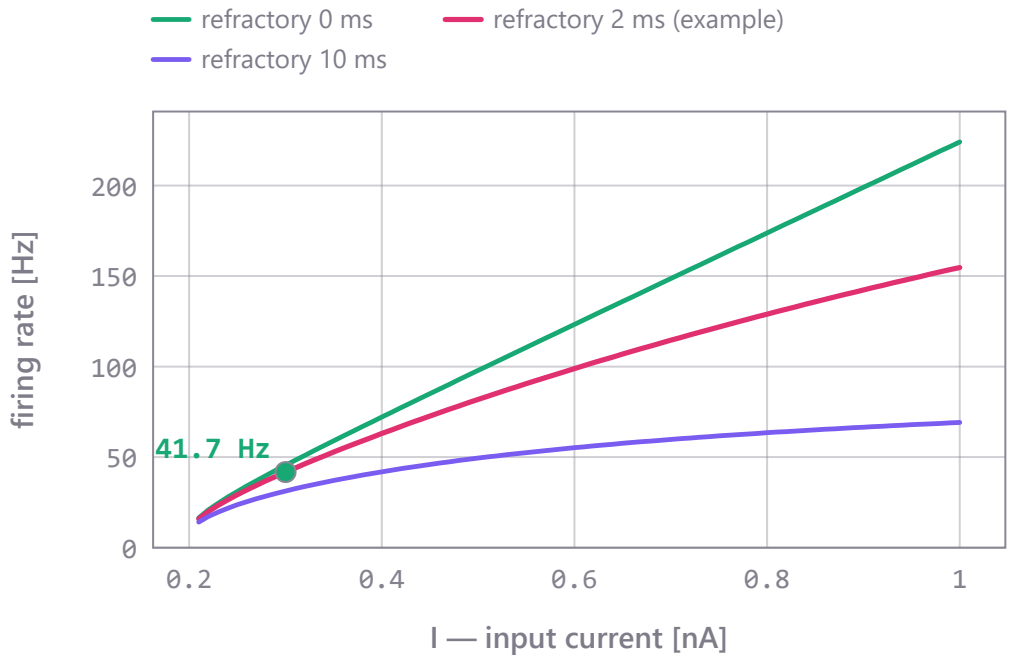
The interspike interval and firing rate of a leaky integrate-and-fire neuron driven by a constant current, with reset and an absolute refractory period.

$$T = t_{ref} + \tau_m \ln \frac{V_\infty - V_{reset}}{V_\infty - V_{th}}, \quad V_\infty = V_{rest} + R_m I$$

| input | meaning | condition |
|---|---|---|
| `tau_m` | membrane time constant | |
| `R_m` | membrane resistance | |
| `I` | constant input current | |
| `V_rest` | resting potential | |
| `V_th` | spike threshold | |
| `V_reset` | reset potential | |
| `t_ref` | absolute refractory period | |

| output | meaning |
|---|---|
| `rate` | firing rate |
| `ISI` | interspike interval |

- **Method:** ISI = t_ref + tau ln((V_inf - V_reset)/(V_inf - V_th)), V_inf = V_rest + R I; rate = 1/ISI
- **Assumes:** V_rest + R $I > V_th$ (otherwise no firing); V_reset < V_th; deterministic constant input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Leaky integrate-and-fire neuron (τm = 20 ms, Rm = 100 MΩ, rest -70 mV, threshold -50 mV, reset -70 mV): firing rate against input current for three refractory periods, 100 currents up to 1 nA (no firing below the threshold current, where the package gives no rate). Marked: example I = 0.3 nA, rate 41.7149 Hz, interspike interval 23.9722 ms.

```
biomedical_science_ai999x.solve('bmed_lif_firing_rate', tau_m=0.02, R_m=100000000.0, I=3e-10,
V_rest=-0.07, V_th=-0.05, V_reset=-0.07, t_ref=0.002)
# -> rate = 41.7149 Hz, ISI = 0.0239722 s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_lif_firing_rate', tau_m=0.01, R_m=50000000.0, I=5e-10,
V_rest=-0.065, V_th=-0.05, V_reset=-0.068, t_ref=0.001)
# -> rate = 88.5254 Hz, ISI = 0.0112962 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| LIF firing rate: tau_m = 20 ms, R_m = 100 Mohm, I = 0.3 nA, V_rest = -70 mV, V_th = -50 mV, V_reset = -70 mV, t_ref = 2 ms | SOLVED | rate = 41.7149 Hz, ISI = 0.0239722 s |
| LIF firing rate: tau_m = 10 ms, R_m = 50 Mohm, I = 500 pA, V_rest = -65 mV, V_th = -50 mV, V_reset = -68 mV, t_ref = 1 ms | SOLVED | rate = 88.5254 Hz, ISI = 0.0112962 s |
| LIF firing rate: R_m = 100 Mohm, I = 0.3 nA, V_rest = -70 mV, V_th = -50 mV, V_reset = -70 mV, t_ref = 2 ms | NEEDS_INPUT | bmed_lif_firing_rate needs tau_m (membrane time constant). Example: solve('bmed_lif_firing_rate', tau_m='0.02 s', R_m='1 |
| LIF firing rate: tau_m = 20 ms, R_m = 100 Mohm, I = 0.3 nA, V_rest = -70 mV, V_th = -50 mV, V_reset = -70 mV, t_ref = 2 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| LIF firing rate: tau_m = 20 ms, R_m = 100 Mohm, I = 0.1 nA, V_rest = -70 mV, V_th = -50 mV, V_reset = -70 mV, t_ref = 2 ms | INVALID_INPUT | Leaky integrate-and-fire neuron: firing rate under constant current: the input does not bring the membrane to threshold: |

## Passive membrane (leaky integrator): voltage at a time under constant current — `bmed_lif_subthreshold_voltage`

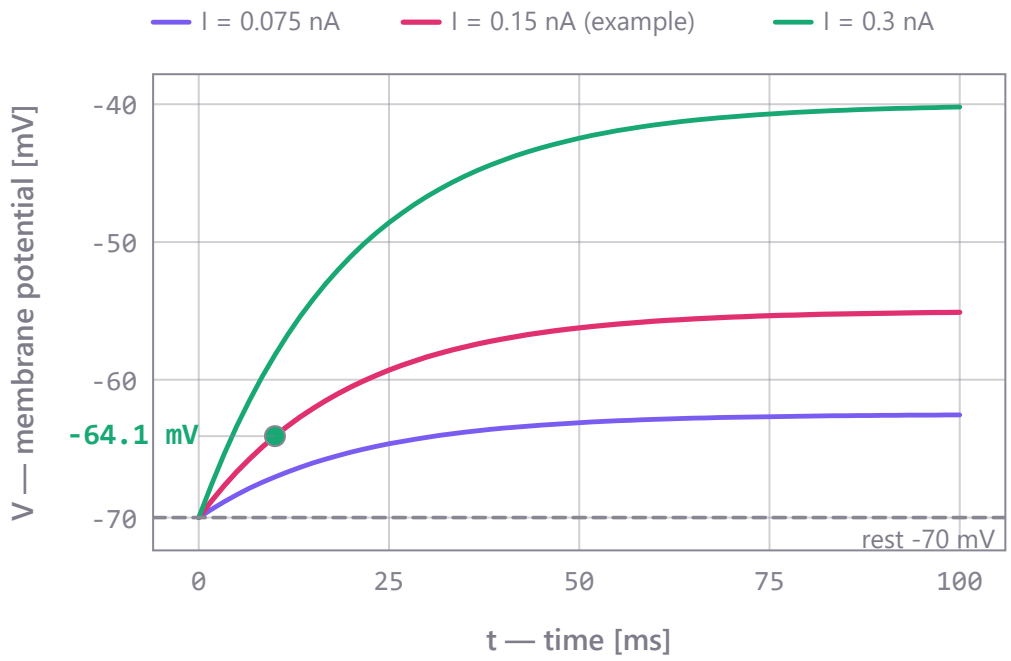
The membrane potential of a passive RC membrane (the subthreshold leaky integrate-and-fire neuron) a time t after a constant current starts, from its initial potential.

$$V(t) = V_\infty + (V_0 - V_\infty)e^{-t/\tau_m}, \quad V_\infty = V_{rest} + R_m I$$

| input | meaning | condition |
|---|---|---|
| `tau_m` | membrane time constant | |
| `R_m` | membrane resistance | |
| `I` | constant input current | |
| `V_rest` | resting potential | |
| `V0` | potential at $t = 0$ | |
| `t` | time | |

| output | meaning |
|---|---|
| V | membrane potential at time t |

- **Method:** tau dV/dt = -(V - V_rest) + R I: V = V_inf + (V0 - V_inf) exp(-t/tau)
- **Assumes:** passive membrane, no spike or threshold; constant current from $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Membrane potential of a leaky integrate-and-fire neuron below threshold for a current step of three sizes (τm = 20 ms, Rm = 100 MΩ, V0 = -70 mV), 61 runs over 100 ms: an exponential charge towards V_rest + Rm·I. Marked: the example at t = 10 ms, V = -64.098 mV.

```
biomedical_science_ai999x.solve('bmed_lif_subthreshold_voltage', tau_m=0.02, R_m=100000000.0,
I=1.5e-10, V_rest=-0.07, V0=-0.07, t=0.01)
# -> V = -0.064098 V   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_lif_subthreshold_voltage', tau_m=0.01, R_m=200000000.0,
I=-5e-11, V_rest=-0.065, V0=-0.06, t=0.025)
# -> V = -0.0737687 V   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Passive membrane voltage: tau_m = 20 ms, R_m = 100 Mohm, I = 0.15 nA, V_rest = -70 mV, V0 = -70 mV, t = 10 ms | SOLVED | V = -0.064098 V |
| Passive membrane voltage: tau_m = 10 ms, R_m = 200 Mohm, I = -50 pA, V_rest = -65 mV, V0 = -60 mV, t = 25 ms | SOLVED | V = -0.0737687 V |
| Passive membrane voltage: R_m = 100 Mohm, I = 0.15 nA, V_rest = -70 | NEEDS_INPUT | bmed_lif_subthreshold_voltage needs tau_m (membrane time constant). Example: |

| question | outcome | result |
|---|---|---|
| mV, V0 = -70 mV, t = 10 ms | | solve('bmed_lif_subthreshold_voltage', tau_ |
| Passive membrane voltage: tau_m = 20 ms, R_m = 100 Mohm, I = 0.15 nA, V_rest = -70 mV, V0 = -70 mV, t = 10 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Passive cable (dendrite or axon): steady-state voltage along it and input resistance — `bmed_cable_steady_state`

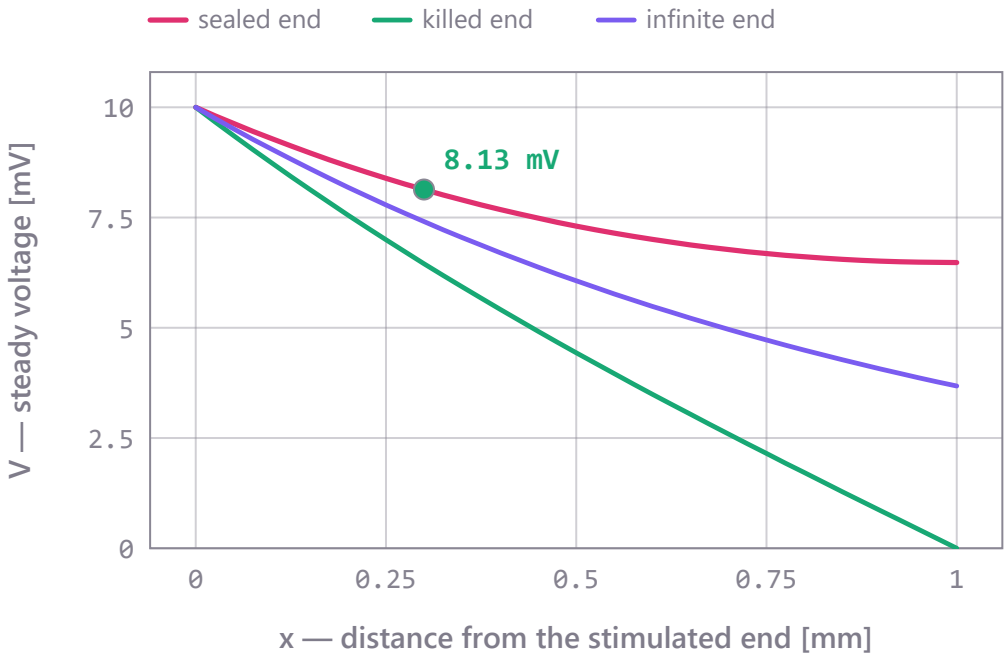
The steady-state voltage at distance x along a passive cylindrical cable held at V0 at $x = 0$, and the input resistance there, for a sealed end, a killed (short-circuited) end or a semi-infinite cable.

$$\lambda = \sqrt{\frac{R_m d}{4 R_i}}, \quad V(x) = V_0 \frac{\cosh((L - x)/\lambda)}{\cosh(L/\lambda)} \ (\text{sealed})$$

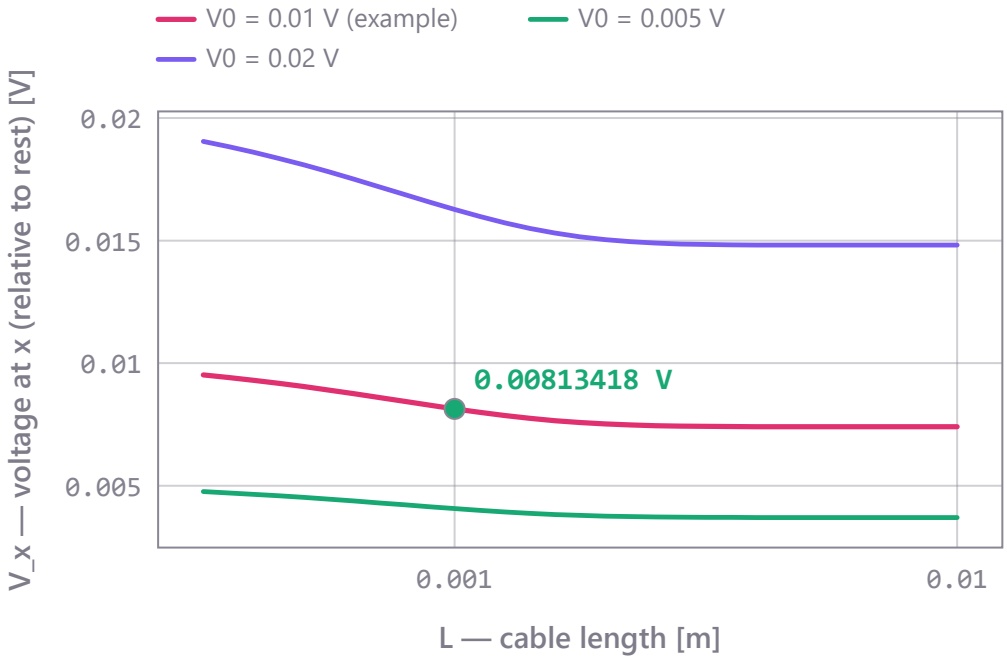
| input | meaning | condition |
|---|---|---|
| `Rm` | specific membrane resistance | |
| `Ri` | axial (cytoplasmic) resistivity | |
| `d` | cable diameter | |
| `x` | distance from the stimulated end | |
| `V0` | steady voltage at $x = 0$ (relative to rest) | |
| `end` | far-end condition | |
| `L` | cable length (sealed or killed end only) | |

| output | meaning |
|---|---|
| `V_x` | voltage at x (relative to rest) |
| `R_in` | input resistance at $x = 0$ |

- **Method:** $\lambda = sqrt$(Rm d/(4 Ri)); sealed V0 cosh((L - x)/lambda)/cosh(L/lambda), killed V0 sinh((L - x)/lambda)/sinh(L/lambda), infinite V0 exp(-x/lambda); $R_i n = r_i$ lambda times coth, tanh or 1
- **Assumes:** passive membrane, steady state; uniform cylinder; L only for a finite cable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Steady voltage along a passive cable (Rm = 2 Ω·m², Ri = 1 Ω·m, d = 2 µm, V0 = 10 mV at x = 0) for a sealed end, a killed (short-circuited) end at L = 1 mm, and an infinitely long cable, 41 runs each. Input resistances: sealed: R_in 417.952 MΩ; killed: R_in 242.423 MΩ; infinite: R_in 318.31 MΩ. Marked: the example (sealed, x = 0.3 mm), V = 8.13418 mV.




V_x (voltage at x (relative to rest)) against L (cable length (sealed or killed end only)) from 0.0001 m to 0.01 m on a logarithmic axis. One curve for each value of V0 (steady voltage at x = 0 (relative to rest)): 0.01 V, 0.005 V, 0.02 V. Other inputs as in the worked example: Rm = 2.0 ohm*m^2, Ri = 1.0 ohm*m, d = 2e-06 m, x = 0.0003 m, end = sealed. 31 points per curve computed by biomedical-science-ai999x; marked: the worked example, L = 0.001 m, V_x = 0.00813418 V.

```
biomedical_science_ai999x.solve('bmed_cable_steady_state', Rm=2.0, Ri=1.0, d=2e-06, x=0.0003,
V0=0.01, end='sealed', L=0.001)
# -> V_x = 0.00813418 V, R_in = 4.17952e+08 ohm   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_cable_steady_state', Rm=1.0, Ri=2.0, d=4e-06, x=0.0005,
V0=-0.005, end='infinite')
# -> V_x = -0.00246534 V, R_in = 1.1254e+08 ohm   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Cable steady state: Rm = 20000 ohm*cm^2, Ri = 100 ohm*cm, d = 2 um, x = 300 um, V0 = 10 mV, end = sealed, L = 1 mm | SOLVED | V_x = 0.00813418 V, R_in = 4.17952e+08 ohm |
| Cable steady state: Rm = 10000 ohm*cm^2, Ri = 200 ohm*cm, d = 4 um, x = 500 um, V0 = -5 mV, end = infinite | SOLVED | V_x = -0.00246534 V, R_in = 1.1254e+08 ohm |
| Cable steady state: Ri = 100 ohm*cm, d = 2 um, x = 300 um, V0 = 10 mV, end = sealed, L = 1 mm | NEEDS_INPUT | bmed_cable_steady_state needs Rm (specific membrane resistance). Example: solve('bmed_cable_steady_state', Rm='2 ohm*m^2 |
| Cable steady state: Rm = 20000 ohm*cm^2, Ri = 100 ohm*cm, d = 2 um, x = 300 um, V0 = 10 mV, end = sealed, L = 1 mm, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Cable steady state: Rm = 20000 ohm*cm^2, Ri = 100 ohm*cm, d = 2 um, x = 300 um, V0 = 10 mV, end = sealed | INVALID_INPUT | Passive cable (dendrite or axon): steady-state voltage along it and input resistance: a finite cable needs its length L. |
| Cable steady state: Rm = 20000 ohm*cm^2, Ri = 100 ohm*cm, d = 2 um, x = 300 um, V0 = 10 mV, end = open, L = 1 mm | INVALID_INPUT | end: one of sealed, killed, infinite; got 'open'. |

## Hodgkin-Huxley model: steady-state gating variables at a membrane potential — `bmed_hh_gating_steady_state`

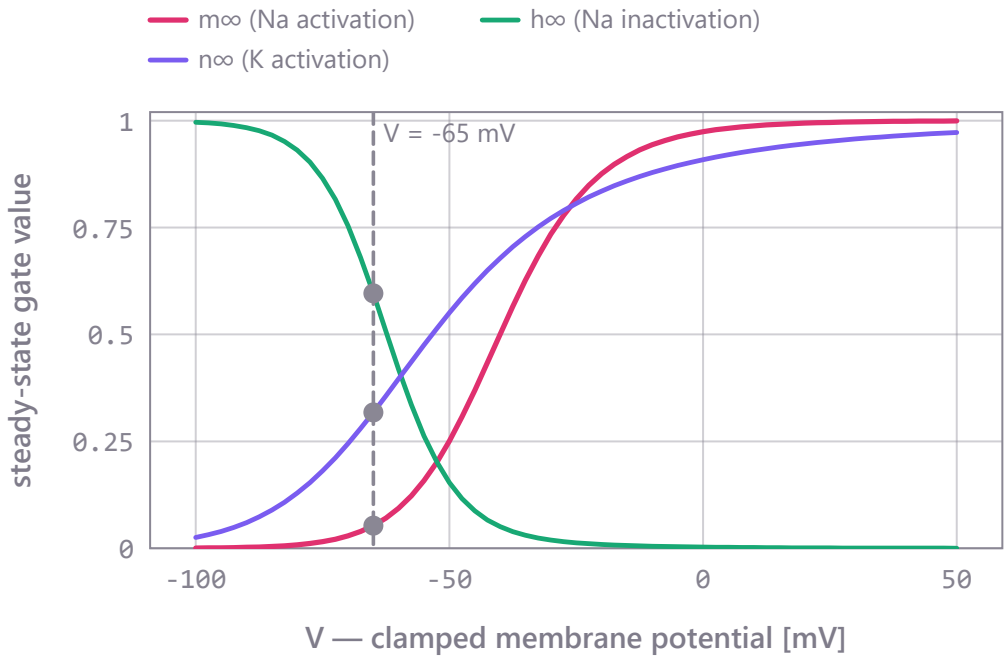
The steady-state values of the Hodgkin-Huxley gating variables m, h and n at a clamped membrane potential, from the 1952 squid-axon rate functions.

$$x_\infty(V) = \frac{\alpha_x(V)}{\alpha_x(V) + \beta_x(V)}$$

| input | meaning | condition |
|---|---|---|
| `V` | clamped membrane potential | |

| output | meaning |
|---|---|
| `m_inf` | Na activation at steady state |
| `h_inf` | Na inactivation at steady state |
| `n_inf` | K activation at steady state |

- **Method:** x_inf = alpha_x / (alpha_x + beta_x) for $x = m$, h, n
- **Assumes:** Hodgkin-Huxley 1952 rate functions, modern convention (rest near -65 mV), 6.3 degC; -150 $mV \leq V$ <= +100 mV
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Hodgkin–Huxley steady-state gating variables against the clamped potential, 61 runs each from −100 to +50 mV: Na activation and K activation rise with depolarisation, Na inactivation falls. Marked: at the example V = -65 mV, m∞ = 0.0529325, h∞ = 0.596121, n∞ = 0.317677.

```
biomedical_science_ai999x.solve('bmed_hh_gating_steady_state', V=-0.065)
# -> m_inf = 0.0529325, h_inf = 0.596121, n_inf = 0.317677   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_hh_gating_steady_state', V=-0.02)
# -> m_inf = 0.875694, h_inf = 0.00894348, n_inf = 0.835178   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Hodgkin-Huxley gating steady state: V = -65 mV | SOLVED | m_inf = 0.0529325, h_inf = 0.596121, n_inf = 0.317677 |
| Hodgkin-Huxley gating steady state: V = -20 mV | SOLVED | m_inf = 0.875694, h_inf = 0.00894348, n_inf = 0.835178 |
| Hodgkin-Huxley gating steady state: | NEEDS_INPUT | bmed_hh_gating_steady_state needs V (clamped membrane potential). Example: solve('bmed_hh_gating_steady_state', V='-0.06 |
| Hodgkin-Huxley gating steady state: V = -65 mV, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Hodgkin-Huxley model: zero-current steady (resting) membrane potential — `bmed_hh_resting_potential`

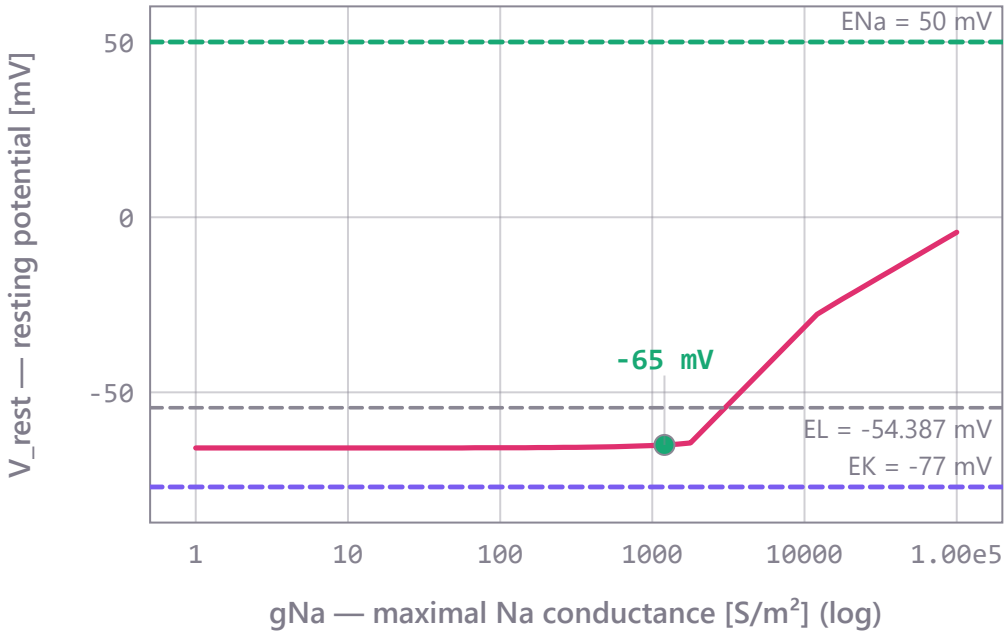
The membrane potential at which the steady-state Na, K and leak currents of the Hodgkin-Huxley model sum to zero (the resting potential), from the maximal conductances and reversal potentials.

$$\bar{g}_{Na} m_\infty^3 h_\infty (V - E_{Na}) + \bar{g}_K n_\infty^4 (V - E_K) + g_L (V - E_L) = 0$$

| input | meaning | condition |
|---|---|---|
| `gNa` | maximal Na conductance | |
| `gK` | maximal K conductance | |
| `gL` | leak conductance | |
| `ENa` | Na reversal potential | |
| `EK` | K reversal potential | |
| `EL` | leak reversal potential | |

| output | meaning |
|---|---|
| `V_rest` | zero-current steady-state potential |

- **Method:** root of gNa m_inf^3 h_inf (V - ENa) + gK n_inf^4 (V - EK) + gL $(V - EL) = 0$ (scan and bisection)
- **Assumes:** Hodgkin-Huxley 1952 rate functions (modern convention), gates at steady state; a unique zero-current potential between -120 and +60 mV (checked); that state stable (Routh-Hurwitz test of the linearised model; otherwise refused); EK < EL < ENa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Zero-current resting potential of the Hodgkin–Huxley membrane against the maximal sodium conductance (log axis, 61 runs; gK = 360, gL = 3 S/m²): it lies between the reversal potentials (inputs, dashed) and moves towards ENa as sodium conductance grows. Marked: the example gNa = 1200 S/m², V_rest = -64.9964 mV.

```
biomedical_science_ai999x.solve('bmed_hh_resting_potential', gNa=1200.0, gK=360.0, gL=3.0,
ENa=0.05, EK=-0.077, EL=-0.054387000000000005)
# -> V_rest = -0.0649964 V   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_hh_resting_potential', gNa=1000.0, gK=400.0, gL=2.0,
ENa=0.055, EK=-0.08, EL=-0.06)
# -> V_rest = -0.0687945 V   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Hodgkin-Huxley resting potential: gNa = 120 mS/cm^2, gK = 36 mS/cm^2, gL = 0.3 mS/cm^2, ENa = 50 mV, EK = -77 mV, EL = -54.387 mV | SOLVED | V_rest = -0.0649964 V |
| Hodgkin-Huxley resting potential: gNa = 100 mS/cm^2, gK = 40 mS/cm^2, gL = 0.2 mS/cm^2, ENa = 55 mV, EK = -80 mV, EL = -60 mV | SOLVED | V_rest = -0.0687945 V |
| Hodgkin-Huxley resting potential: gK = 36 mS/cm^2, gL = 0.3 mS/cm^2, ENa = 50 mV, EK = -77 mV, EL = -54.387 mV | NEEDS_INPUT | bmed_hh_resting_potential needs gNa (maximal Na conductance). Example: solve('bmed_hh_resting_potential', gNa='1200 S/m^ |
| Hodgkin-Huxley resting potential: gNa = 120 mS/cm^2, gK = 36 mS/cm^2, gL = 0.3 mS/cm^2, ENa = 50 mV, EK = -77 mV, EL = -54.387 mV, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Hodgkin-Huxley model: potassium conductance after a voltage-clamp step — `bmed_hh_potassium_conductance_clamp`

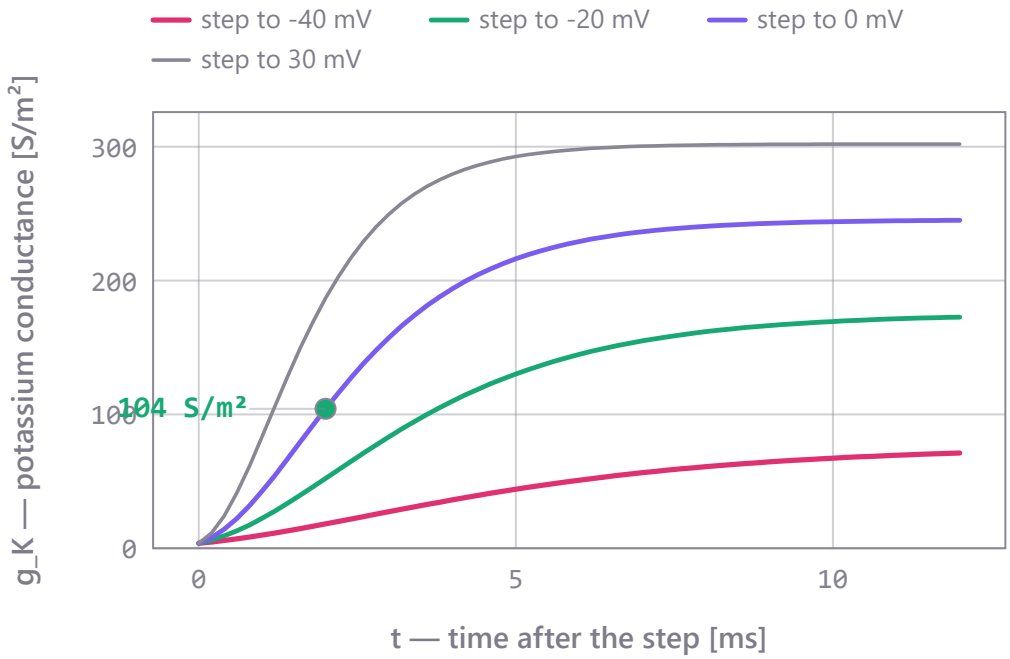
The potassium conductance gK = gK_max $n^4$ and the gate n a time t after the membrane is stepped from a holding potential (at steady state) to a new clamped potential, in the Hodgkin-Huxley model.

$$n(t) = n_\infty - (n_\infty - n_0)e^{-t/\tau_n}, \quad g_K = \bar{g}_K n^4$$

| input | meaning | condition |
|---|---|---|
| `gK_max` | maximal K conductance | |
| `V_hold` | holding potential before the step | |
| `V_step` | clamped potential after the step | |
| `t` | time after the step | |

| output | meaning |
|---|---|
| `g_K` | K conductance at time t |
| `n` | K gate n at time t |

- **Method:** n(t) = n_inf(V1) - (n_inf(V1) - n_inf(V0)) exp(-(alpha_n + beta_n) t); g_K = gK_max $n^4$
- **Assumes:** ideal voltage clamp; gates at steady state at the holding potential; Hodgkin-Huxley 1952 rate functions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Voltage clamp: potassium conductance after a step from -65 mV to four potentials (gK_max = 360 S/m$^2$), 61 runs each over 12 ms: a sigmoid delayed rise, larger and faster for a larger depolarisation. Marked: the example (step to 0 mV, t = 2 ms), g_K = 104.172 S/m$^2$, n = 0.733436.

```
biomedical_science_ai999x.solve('bmed_hh_potassium_conductance_clamp', gK_max=360.0, V_hold=-0.065,
V_step=0.0, t=0.002)
# -> g_K = 104.172 S/m^2, n = 0.733436   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_hh_potassium_conductance_clamp', gK_max=360.0, V_hold=-0.07,
V_step=-0.03, t=0.005)
# -> g_K = 77.5423 S/m^2, n = 0.681254   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Hodgkin-Huxley potassium conductance: gK_max = 36 mS/cm^2, V_hold = -65 mV, V_step = 0 mV, t = 2 ms | SOLVED | g_K = 104.172 S/m^2, n = 0.733436 |
| Hodgkin-Huxley potassium conductance: gK_max = 36 mS/cm^2, V_hold = -70 mV, V_step = -30 mV, t = 5 ms | SOLVED | g_K = 77.5423 S/m^2, n = 0.681254 |
| Hodgkin-Huxley potassium conductance: V_hold = -65 mV, V_step = 0 mV, t = 2 ms | NEEDS_INPUT | bmed_hh_potassium_conductance_clamp needs gK_max (maximal K conductance). Example: solve('bmed_hh_potassium_conductance_ |
| Hodgkin-Huxley potassium conductance: gK_max = 36 mS/cm^2, V_hold = -65 mV, V_step = 0 mV, t = 2 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Postsynaptic potential of a passive membrane driven by an exponentially decaying synaptic current — `bmed_postsynaptic_potential`

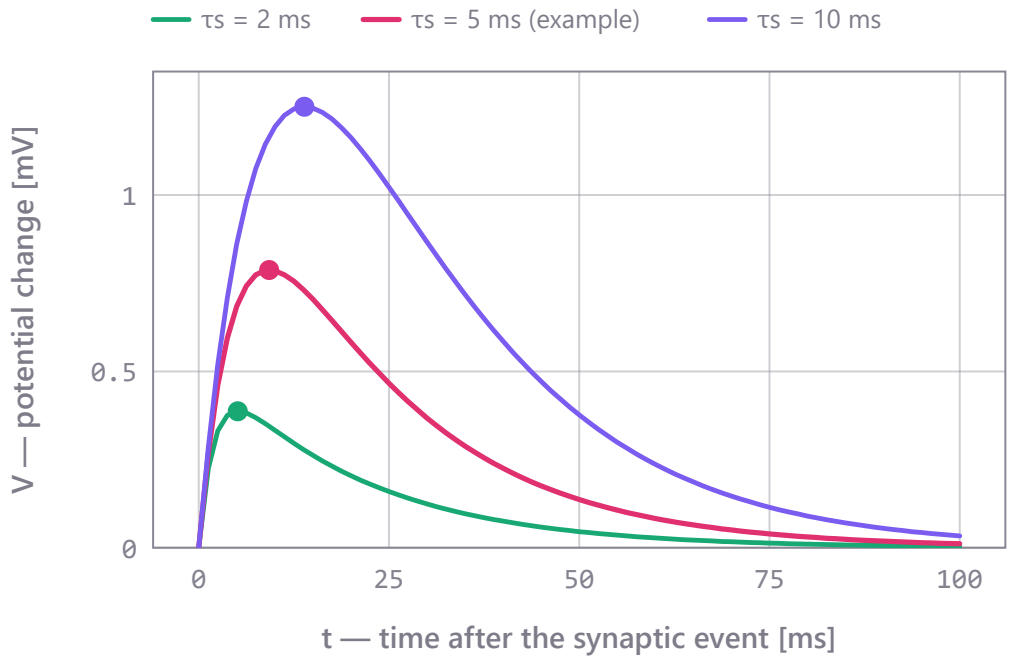
The postsynaptic potential of a passive RC membrane receiving a synaptic current I0 exp(-t/tau_s) at $t = 0$: its value at time t, the time of its peak and the peak value.

$$V(t) = \frac{R I_0 \tau_s}{\tau_s - \tau_m}\left(e^{-t/\tau_s} - e^{-t/\tau_m}\right), \quad t_{peak} = \frac{\tau_m \tau_s}{\tau_s - \tau_m} \ln \frac{\tau_s}{\tau_m}$$

| input | meaning | condition |
|---|---|---|
| `tau_m` | membrane time constant | |
| `tau_s` | synaptic current decay time constant | |
| `R_m` | membrane resistance | |
| `I0` | peak synaptic current (negative for inward) | |
| `t` | time after the synaptic event | |

| output | meaning |
|---|---|
| `V_t` | potential change at time t |
| `t_peak` | time of the peak |
| `V_peak` | peak potential change |

- **Method:** tau_m $dV/dt = -V + R$ I0 exp(-t/tau_s): $V = R$ I0 tau_s/(tau_s - tau_m) (exp(-t/tau_s) - exp(-t/tau_m)); t_peak = tau_m tau_s ln(tau_s/tau_m)/(tau_s - tau_m)
- **Assumes:** passive membrane starting at rest; current jumps to I0 and decays exponentially; $tau_s \neq tau_m$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Postsynaptic potential after one synaptic event (τm = 20 ms, Rm = 100 MΩ, peak current 50 pA) for three synaptic decay times, 81 runs each over 100 ms, the returned peak marked on each. Marked: τs 2 ms: peak 0.387132 mV at 5.11686 ms; τs 5 ms: peak 0.787451 mV at 9.24196 ms; τs 10 ms: peak 1.25 mV at 13.8629 ms.

```
biomedical_science_ai999x.solve('bmed_postsynaptic_potential', tau_m=0.02, tau_s=0.005,
R_m=100000000.0, I0=5e-11, t=0.01)
# -> V_t = 0.000785326 V, t_peak = 0.00924196 s, V_peak = 0.000787451 V   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_postsynaptic_potential', tau_m=0.01, tau_s=0.002,
R_m=200000000.0, I0=-3e-11, t=0.003)
# -> V_t = -0.000776532 V, t_peak = 0.00402359 s, V_peak = -0.000802488 V   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Postsynaptic potential: tau_m = 20 ms, tau_s = 5 ms, R_m = 100 Mohm, I0 = 50 pA, t = 10 ms | SOLVED | V_t = 0.000785326 V, t_peak = 0.00924196 s, V_peak = 0.000787451 V |
| Postsynaptic potential: tau_m = 10 ms, tau_s = 2 ms, R_m = 200 Mohm, I0 = -30 pA, t = 3 ms | SOLVED | V_t = -0.000776532 V, t_peak = 0.00402359 s, V_peak = -0.000802488 V |
| Postsynaptic potential: tau_s = 5 ms, R_m = 100 Mohm, I0 = 50 pA, t = 10 ms | NEEDS_INPUT | bmed_postsynaptic_potential needs tau_m (membrane time constant). Example: solve('bmed_postsynaptic_potential', tau_m='0 |
| Postsynaptic potential: tau_m = 20 ms, tau_s = 5 ms, R_m = 100 Mohm, I0 = 50 pA, t = 10 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## FitzHugh-Nagumo neuron model: fixed point and its stability —

`bmed_fitzhugh_nagumo_fixed_point`

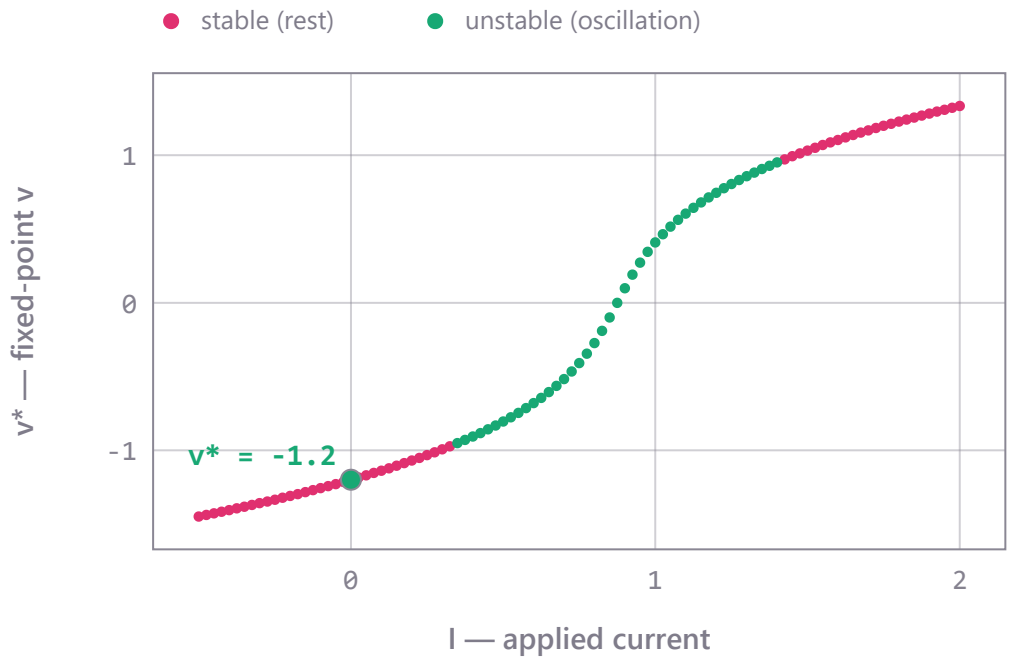
The resting (fixed) point of the FitzHugh-Nagumo model v' = $v - v^3/3 - w + I$, w' = eps (v + a - b w), the trace and determinant of its Jacobian and whether it is stable (resting) or unstable (repetitive firing).

$$v - \frac{v^3}{3} - \frac{v+a}{b} + I = 0, \quad J = \begin{pmatrix} 1 - v^2 & -1 \\ \varepsilon & -\varepsilon b \end{pmatrix}$$

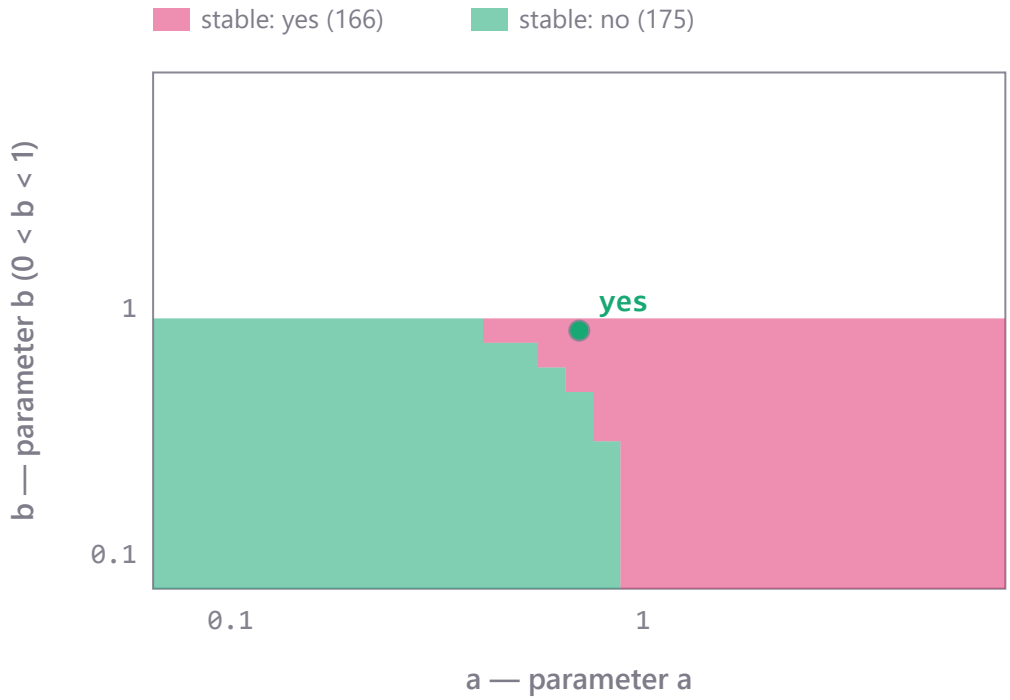
| input | meaning | condition |
|---|---|---|
| `a` | parameter a | |
| `b` | parameter b $(0 < b < 1)$ | |
| `eps` | time-scale ratio eps (> 0) | |
| `I` | applied current (dimensionless) | |

| output | meaning |
|---|---|
| `v_star` | fixed-point v |
| `w_star` | fixed-point w |
| `trace` | trace of the Jacobian |
| `determinant` | determinant of the Jacobian |
| `stable` | true when the fixed point is stable |

- **Method:** v* solves $v - v^3/3 - (v + a)/b + I = 0$ (bisection); w* = (v* + a)/b; J = [[$1 - v$*^2, -1], [eps, -eps b]]; stable when trace < 0 (det > 0 here)
- **Assumes:** dimensionless FitzHugh-Nagumo model; $0 < b < 1$ (a unique fixed point)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Fixed point of the FitzHugh–Nagumo model (a = 0.7, b = 0.8, ε = 0.08) against the applied current, 101 runs, coloured by the stability the package returns: the fixed point loses stability over a window of currents (the neuron fires repetitively there). Marked: example I = 0, v* = -1.19941, w* = -0.62426, trace -0.50258, determinant 0.108069, stable.




true when the fixed point is stable (stable) over a (parameter a) from 0.07 to 7 and b (parameter b (0 < b < 1)) from 0.08 to 8 (logarithmic axes): each of the 31 × 21 cells is one run of biomedical-science-ai999x, coloured by the verdict it returned — yes: 166 cells, no: 175 cells; blank cells: inputs the capability refuses. Other inputs as in the worked example: eps = 0.08, I = 0. Marked: the worked example, a = 0.7, b = 0.8 → yes.

```
biomedical_science_ai999x.solve('bmed_fitzhugh_nagumo_fixed_point', a=0.7, b=0.8, eps=0.08, I=0)
# -> v_star = -1.19941, w_star = -0.62426, trace = -0.50258, determinant = 0.108069, stable = True
(SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_fitzhugh_nagumo_fixed_point', a=0.7, b=0.8, eps=0.08, I=0.5)
# -> v_star = -0.804848, w_star = -0.13106, trace = 0.28822, determinant = 0.0574579, stable =
False   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| FitzHugh-Nagumo fixed point: a = 0.7, b = 0.8, eps = 0.08, I = 0 | SOLVED | v_star = -1.19941, w_star = -0.62426, trace = -0.50258, determinant = 0.108069, stable = True |
| FitzHugh-Nagumo fixed point: a = 0.7, b = 0.8, eps = 0.08, I = 0.5 | SOLVED | v_star = -0.804848, w_star = -0.13106, trace = 0.28822, determinant = 0.0574579, stable = False |
| FitzHugh-Nagumo fixed point: b = 0.8, eps = 0.08, I = 0 | NEEDS_INPUT | bmed_fitzhugh_nagumo_fixed_point needs a (parameter a). Example: solve('bmed_fitzhugh_nagumo_fixed_point', a=0.7, b=0.8, |
| FitzHugh-Nagumo fixed point: a = 0.7, b = 0.8, eps = 0.08, I = 0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| FitzHugh-Nagumo fixed point: a = 0.7, b = 1.5, eps = 0.08, I = 0 | INVALID_INPUT | FitzHugh-Nagumo neuron model: fixed point and its stability: this calculation needs 0 < b < 1 (then the fixed point is u |

## X-ray attenuation through tissue layers (narrow monoenergetic beam) — `bmed_xray_attenuation_layers`

The transmitted fraction and intensity of a narrow monoenergetic X-ray beam crossing successive tissue layers, from each layer's mass attenuation coefficient, density and thickness.

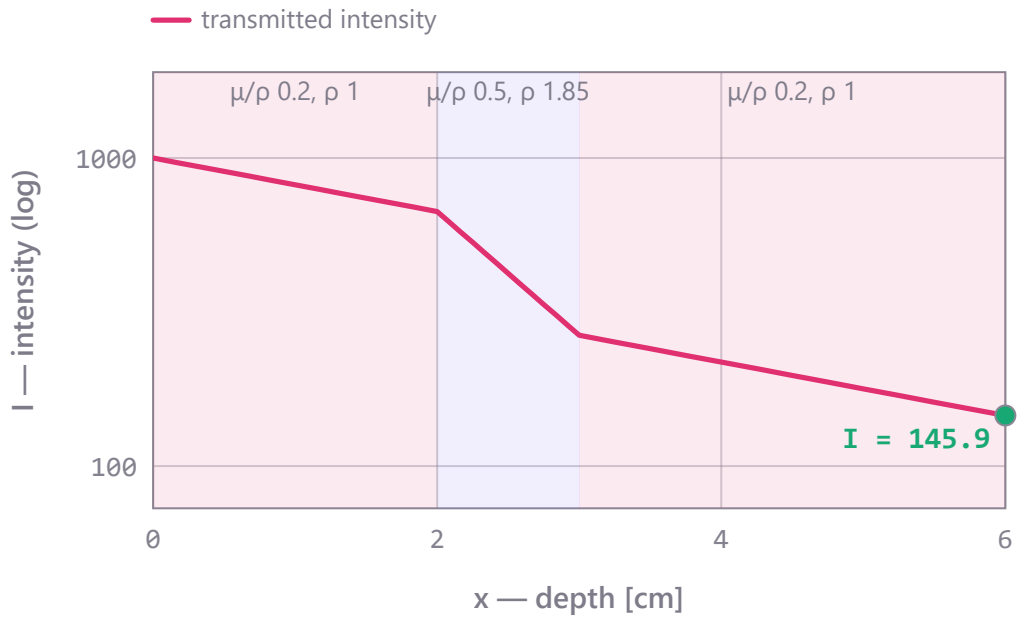
$$\frac{I}{I_0} = \exp\left(-\sum_i \left(\frac{\mu}{\rho}\right)_i \rho_i x_i\right)$$

| input | meaning | condition |
|---|---|---|
| `I0` | incident intensity | |
| `mu_rho` | mass attenuation coefficient of each layer, $cm^2/g$ | |
| `rho` | density of each layer, $g/cm^3$ | |
| `x` | thickness of each layer, cm | |

| output | meaning |
|---|---|
| `T` | transmitted fraction |
| `I` | transmitted intensity |

- **Method:** $T = exp$(-sum (mu/rho)_i rho_i x_i); $I = I_0$ T

- **Assumes:** narrow beam (no scatter reaching the detector); one photon energy; units $cm^2/g$, $g/cm^3$, cm
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




X-ray beam through 3 layers drawn to scale (thicknesses 2, 1, 3 cm, mass attenuation 0.2, 0.5, 0.2 $cm^2/g$, densities 1, 1.85, 1 $g/cm^3$; the example): each point is the package's intensity after the layers crossed so far (log axis, steeper in the denser layer). Marked: I0 = 1000 → I = 145.876, transmitted fraction T = 0.145876.

```
biomedical_science_ai999x.solve('bmed_xray_attenuation_layers', I0=1000.0, mu_rho=[0.2, 0.5, 0.2],
rho=[1.0, 1.85, 1.0], x=[2, 1, 3])
# -> T = 0.145876, I = 145.876 W/m^2   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_xray_attenuation_layers', I0=50.0, mu_rho=[0.18], rho=[1.06],
x=[10])
# -> T = 0.148377, I = 7.41884 W/m^2   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| X-ray attenuation through layers: I0 = 1000 W/m^2, mu_rho = [0.2, 0.5, 0.2], rho = [1, 1.85, 1], x = [2, 1, 3] | SOLVED | T = 0.145876, I = 145.876 W/m^2 |
| X-ray attenuation through layers: I0 = 50 W/m^2, mu_rho = [0.18], rho = [1.06], x = [10] | SOLVED | T = 0.148377, I = 7.41884 W/m^2 |
| X-ray attenuation through layers: mu_rho = [0.2, 0.5, 0.2], rho = [1, 1.85, 1], x = [2, 1, 3] | NEEDS_INPUT | bmed_xray_attenuation_layers needs I0 (incident intensity). Example: solve('bmed_xray_attenuation_layers', I0='1000 W/m^ |
| X-ray attenuation through layers: I0 = 1000 W/m^2, mu_rho = [0.2, 0.5, 0.2], rho = [1, 1.85, 1], x = [2, 1, 3], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### MRI spin-echo signal from T1, T2, TR and TE (simplified model) — `bmed_mri_spin_echo_signal`

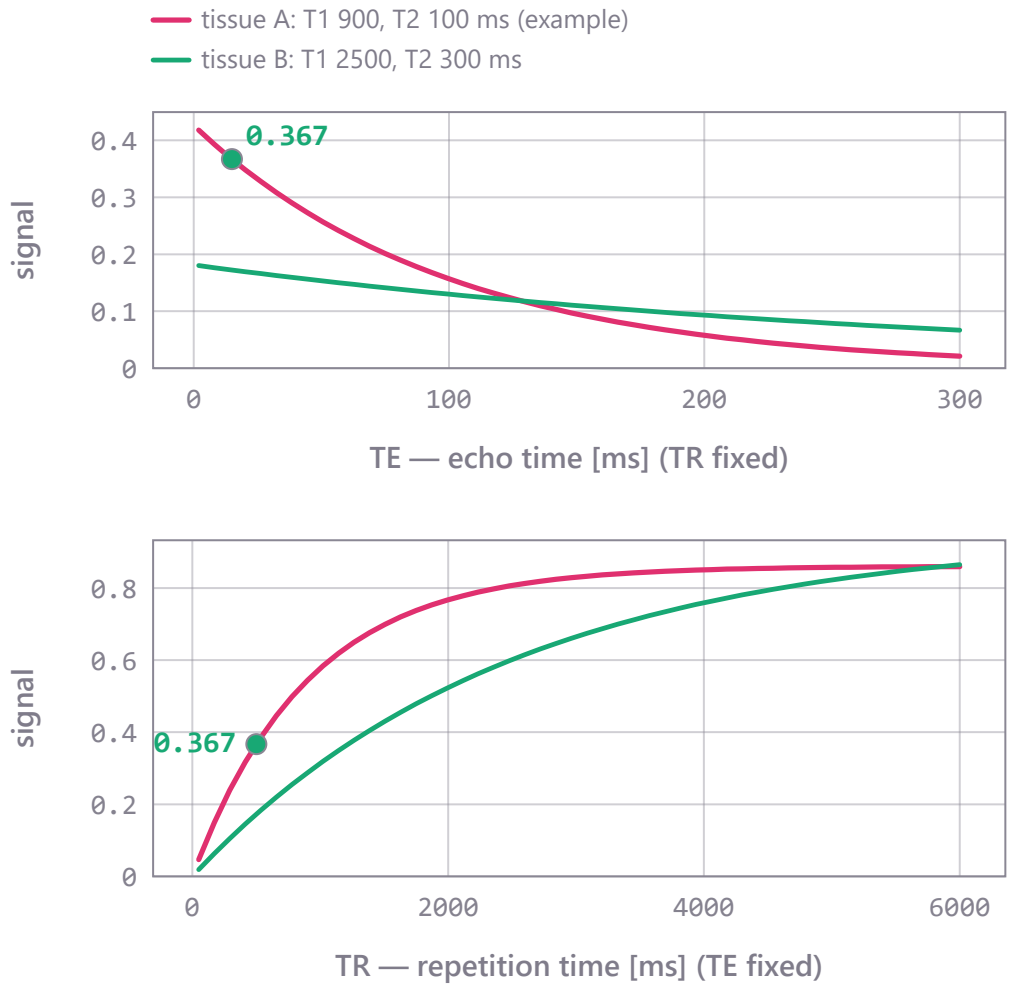
The steady-state spin-echo signal $S = M_0$ (1 - exp(-TR/T1)) exp(-TE/T2) of a tissue with relaxation times T1 and T2, for repetition time TR and echo time TE.

$$S = M_0 \left(1 - e^{-TR/T_1}\right) e^{-TE/T_2}$$

| input | meaning | condition |
|---|---|---|
| `M0` | equilibrium magnetisation (relative proton density) | |
| `TR` | repetition time | |
| `TE` | echo time | |
| `T1` | longitudinal relaxation time | |
| `T2` | transverse relaxation time | |

| output | meaning |
|---|---|
| `S` | signal in the units of M0 |

- **Method:** $S = M_0$ (1 - exp(-TR/T1)) exp(-TE/T2)
- **Assumes:** 90 degree excitations every TR, transverse magnetisation spoiled before each; ideal refocusing without effect on Mz (TE << TR); $TE < TR$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Spin-echo signal of two tissues (A: the example's T1 = 900 ms, T2 = 100 ms; B: a chosen long-T1, long-T2 tissue, T1 = 2500 ms, T2 = 300 ms; M0 = 1) against echo time at TR = 500 ms (top) and against repetition time at TE = 15 ms (bottom), 50 runs per line: short TR and TE weight the image by T1, long ones by T2, and the curves cross. Marked: the example signal S = 0.366874.

```
biomedical_science_ai999x.solve('bmed_mri_spin_echo_signal', M0=1, TR=0.5, TE=0.015, T1=0.9,
T2=0.1)
# -> S = 0.366874   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_spin_echo_signal', M0=0.8, TR=3.0, TE=0.1, T1=2.5,
T2=0.25)
# -> S = 0.374739   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Spin-echo signal: M0 = 1, TR = 500 ms, TE = 15 ms, T1 = 900 ms, T2 = 100 ms | SOLVED | S = 0.366874 |
| Spin-echo signal: M0 = 0.8, TR = 3 s, TE = 100 ms, T1 = 2.5 s, T2 = 250 ms | SOLVED | S = 0.374739 |
| Spin-echo signal: TR = 500 ms, TE = 15 ms, T1 = 900 ms, T2 = 100 ms | NEEDS_INPUT | bmed_mri_spin_echo_signal needs M0 (equilibrium magnetisation (relative proton density)). Example: solve('bmed_mri_spin_ |

| question | outcome | result |
|---|---|---|
| Spin-echo signal: M0 = 1, TR = 500 ms, TE = 15 ms, T1 = 900 ms, T2 = 100 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Spin-echo signal: M0 = 1, TR = 10 ms, TE = 15 ms, T1 = 900 ms, T2 = 100 ms | INVALID_INPUT | MRI spin-echo signal from T1, T2, TR and TE (simplified model): TE must be shorter than TR. |

## MRI inversion recovery: longitudinal magnetisation at TI and the null point — `bmed_mri_inversion_recovery`

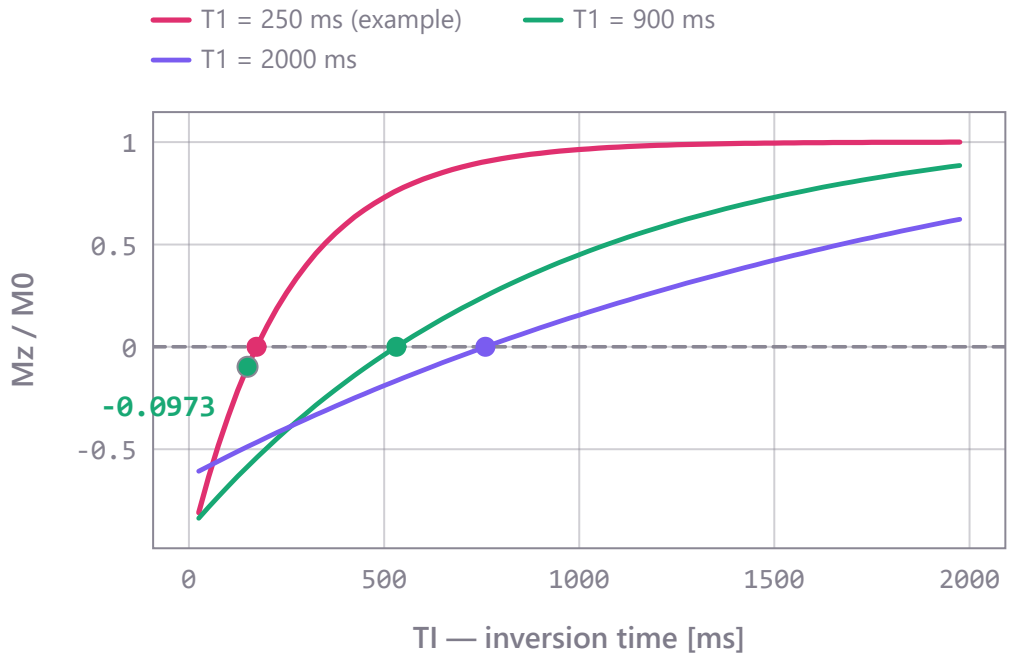
The longitudinal magnetisation (fraction of M0, signed) at inversion time TI in a repeated inversion-recovery sequence with repetition time TR, and the inversion time that nulls the tissue.

$$\frac{M_z(TI)}{M_0} = 1 - 2e^{-TI/T_1} + e^{-TR/T_1}, \quad TI_{null} = T_1 \ln \frac{2}{1 + e^{-TR/T_1}}$$

| input | meaning | condition |
|---|---|---|
| `T1` | longitudinal relaxation time | |
| `TR` | repetition time | |
| `TI` | inversion time | |

| output | meaning |
|---|---|
| `Mz_ratio` | $Mz/M_0$ at TI (negative before the null) |
| `TI_null` | inversion time of zero signal |

- **Method:** Mz(TI)/M0 = 1 - 2 exp(-TI/T1) + exp(-TR/T1); null at $TI = T_1$ ln(2/(1 + exp(-TR/T1)))
- **Assumes:** ideal 180 degree inversion and 90 degree readout at TI; readout leaves $Mz = 0$; steady state, $TI < TR$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Inversion recovery: longitudinal magnetisation against inversion time (TR = 2000 ms) for three T1 values, 81 runs each; dots: each tissue's null point TI_null returned by the package (choosing that TI suppresses the tissue). Marked: example T1 = 250 ms at TI = 150 ms, Mz/M0 = -0.0972878; its null at 173.203 ms.

```
biomedical_science_ai999x.solve('bmed_mri_inversion_recovery', T1=0.25, TR=2.0, TI=0.15)
# -> Mz_ratio = -0.0972878, TI_null = 0.173203 s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_inversion_recovery', T1=4.0, TR=9.0, TI=2.5)
# -> Mz_ratio = 0.0348764, TI_null = 2.37176 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Inversion recovery: T1 = 250 ms, TR = 2000 ms, TI = 150 ms | SOLVED | Mz_ratio = -0.0972878, TI_null = 0.173203 s |
| Inversion recovery: T1 = 4 s, TR = 9 s, TI = 2.5 s | SOLVED | Mz_ratio = 0.0348764, TI_null = 2.37176 s |
| Inversion recovery: TR = 2000 ms, TI = 150 ms | NEEDS_INPUT | bmed_mri_inversion_recovery needs T1 (longitudinal relaxation time). Example: solve('bmed_mri_inversion_recovery', T1='0 |
| Inversion recovery: T1 = 250 ms, TR = 2000 ms, TI = 150 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## MRI spoiled gradient-echo signal (Ernst equation) — `bmed_mri_spoiled_gradient_echo`

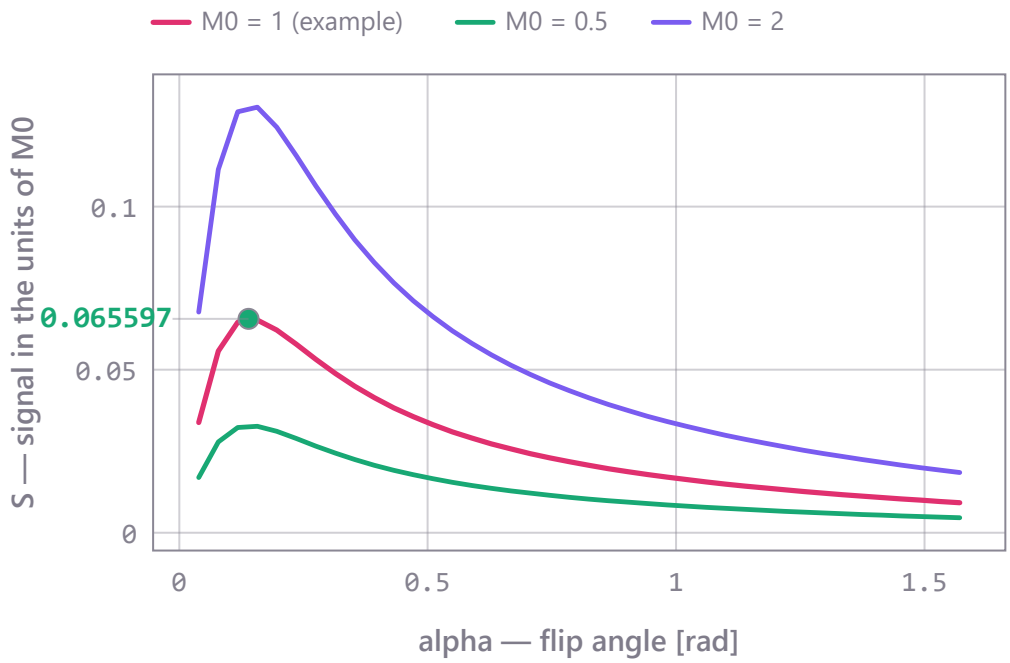
The steady-state signal of a spoiled gradient-echo sequence with flip angle alpha, repetition time TR and echo time TE for a tissue with T1 and $T_2$.

$$S = M_0 \sin\alpha \, \frac{1 - E_1}{1 - E_1 \cos\alpha} \, e^{-TE/T_2^*}, \quad E_1 = e^{-TR/T_1}$$

| input | meaning | condition |
|---|---|---|
| `M0` | equilibrium magnetisation (relative proton density) | |
| `TR` | repetition time | |
| `TE` | echo time | |
| `T1` | longitudinal relaxation time | |
| `T2star` | effective transverse relaxation time T2* | |
| `alpha` | flip angle | |

| output | meaning |
|---|---|
| `S` | signal in the units of M0 |

- **Method:** $S = M_0$ sin(alpha) (1 - E1)/(1 - cos(alpha) E1) exp($-TE/T_2$*), $E_1 = e^{\frac{-TR}{T_1}}$
- **Assumes:** ideal spoiling of transverse magnetisation before each pulse; steady state; $0 < \alpha \leq 90$ deg, $TE < TR$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




S (signal in the units of M0) against alpha (flip angle) from 0 rad to 1.5708 rad. One curve for each value of M0 (equilibrium magnetisation (relative proton density)): 1, 0.5, 2. Other inputs as in the worked example: TR = 0.01 s, TE = 0.003 s, T1 = 1.0 s, T2star = 0.04 s. 40 points per curve computed by biomedical-science-ai999x; marked: the worked example, alpha = 0.139626 rad, S = 0.065597.

```
biomedical_science_ai999x.solve('bmed_mri_spoiled_gradient_echo', M0=1, TR=0.01, TE=0.003, T1=1.0,
T2star=0.04, alpha=0.13962634015954636)
# -> S = 0.065597   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_spoiled_gradient_echo', M0=0.9, TR=0.02, TE=0.002,
T1=0.6, T2star=0.03, alpha=0.5235987755982988)
# -> S = 0.0850011   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Spoiled gradient-echo signal: M0 = 1, TR = 10 ms, TE = 3 ms, T1 = 1000 ms, T2star = 40 ms, alpha = 8 deg | SOLVED | S = 0.065597 |
| Spoiled gradient-echo signal: M0 = 0.9, TR = 20 ms, TE = 2 ms, T1 = 600 ms, T2star = 30 ms, alpha = 30 deg | SOLVED | S = 0.0850011 |
| Spoiled gradient-echo signal: TR = 10 ms, TE = 3 ms, T1 = 1000 ms, T2star = 40 ms, alpha = 8 deg | NEEDS_INPUT | bmed_mri_spoiled_gradient_echo needs M0 (equilibrium magnetisation (relative proton density)). Example: solve('bmed_mri_ |
| Spoiled gradient-echo signal: M0 = 1, TR = 10 ms, TE = 3 ms, T1 = 1000 ms, T2star = 40 ms, alpha = 8 deg, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Ultrasound reflection and transmission at a tissue interface (normal incidence) — `bmed_ultrasound_interface`

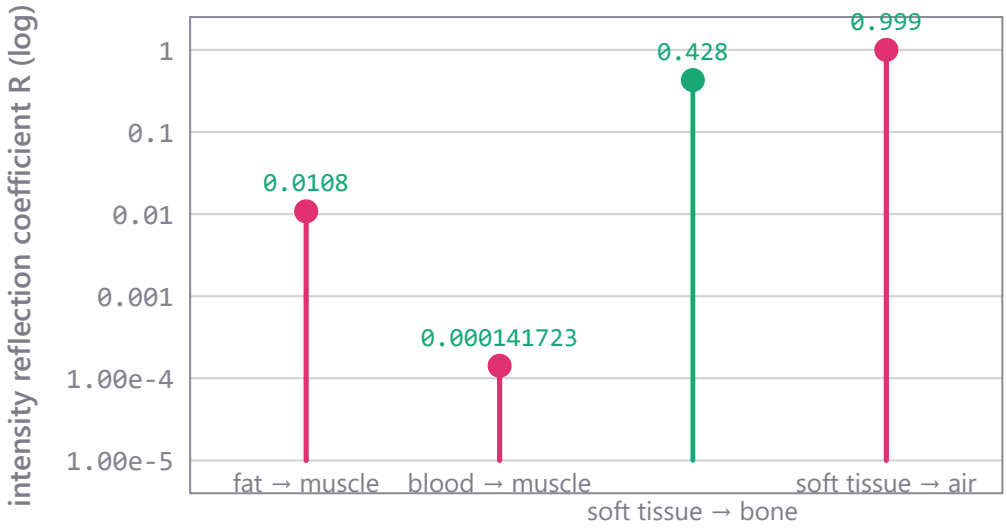
The intensity reflection and transmission coefficients and the pressure amplitude coefficients of an ultrasound wave meeting a flat interface between two media at normal incidence.

$$R = \left(\frac{Z_2 - Z_1}{Z_2 + Z_1}\right)^2, \quad T = \frac{4 Z_1 Z_2}{(Z_1 + Z_2)^2}$$

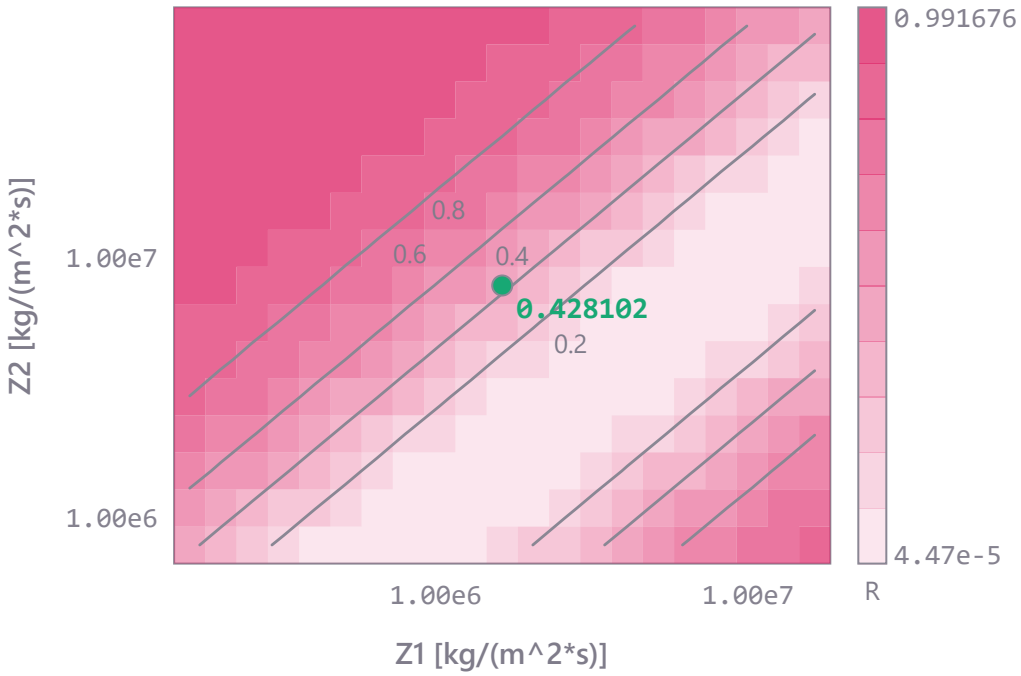
| input | meaning | condition |
|---|---|---|
| `Z1` | acoustic impedance of the first medium | |
| `Z2` | acoustic impedance of the second medium | |

| output | meaning |
|---|---|
| `R` | intensity reflection coefficient |
| `T` | intensity transmission coefficient |
| `r_p` | pressure reflection coefficient |
| `t_p` | pressure transmission coefficient |

- **Method:** $r = (Z_2 - Z_1)/(Z_2 + Z_1)$, $t = 2\ Z_2/(Z_1 + Z_2)$; $R = r^2$, $T = 4$ Z1 $Z_2/(Z_1 + Z_2)^2$
- **Assumes:** normal incidence on a flat interface; lossless media, plane waves
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Intensity reflection at four tissue interfaces (log axis), one run each with stated acoustic impedances (MRayl: fat 1.38, muscle 1.70, soft tissue 1.63, blood 1.66, bone 7.8, air 0.00043): fat/muscle R = 0.0107944; blood/muscle R = 0.000141723; soft tissue/bone R = 0.428102; soft tissue/air R = 0.998945. The example soft tissue → bone (Z1 = 1.63e+06, Z2 = 7.8e+06 kg/(m²·s)) gives R = 0.428102, T = 0.571898; air reflects almost everything, hence coupling gel.




R (intensity reflection coefficient) over Z1 (acoustic impedance of the first medium) from 163000 kg/(m^2*s) to 1.6300e+07 kg/(m^2*s) and Z2 (acoustic impedance of the second medium) from 780000 kg/(m^2*s) to 7.8000e+07 kg/(m^2*s) (logarithmic axes): each of the 21 × 15 cells is one run of biomedical-science-ai999x, shaded by its value (light 4.4700e-05 to dark 0.991676); iso-lines at R = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, Z1 = 1.63e+06 kg/(m^2*s), Z2 = 7.8e+06 kg/(m^2*s), R = 0.428102.

```
biomedical_science_ai999x.solve('bmed_ultrasound_interface', Z1=1630000.0, Z2=7800000.0)
# -> R = 0.428102, T = 0.571898, r_p = 0.654295, t_p = 1.65429   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_ultrasound_interface', Z1=1380000.0, Z2=429.0)
# -> R = 0.998757, T = 0.00124271, r_p = -0.999378, t_p = 0.000621546   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Ultrasound interface reflection: Z1 = 1.63 MRayl, Z2 = 7.8 MRayl | SOLVED | R = 0.428102, T = 0.571898, r_p = 0.654295, t_p = 1.65429 |
| Ultrasound interface reflection: Z1 = 1.38e6 kg/(m^2*s), Z2 = 429 kg/(m^2*s) | SOLVED | R = 0.998757, T = 0.00124271, r_p = -0.999378, t_p = 0.000621546 |
| Ultrasound interface reflection: Z2 = 7.8 MRayl | NEEDS_INPUT | bmed_ultrasound_interface needs Z1 (acoustic impedance of the first medium). Example: solve('bmed_ultrasound_interface', |
| Ultrasound interface reflection: Z1 = 1.63 MRayl, Z2 = 7.8 MRayl, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Ultrasound transmission through a matching layer (normal incidence) — `bmed_ultrasound_matching_layer`

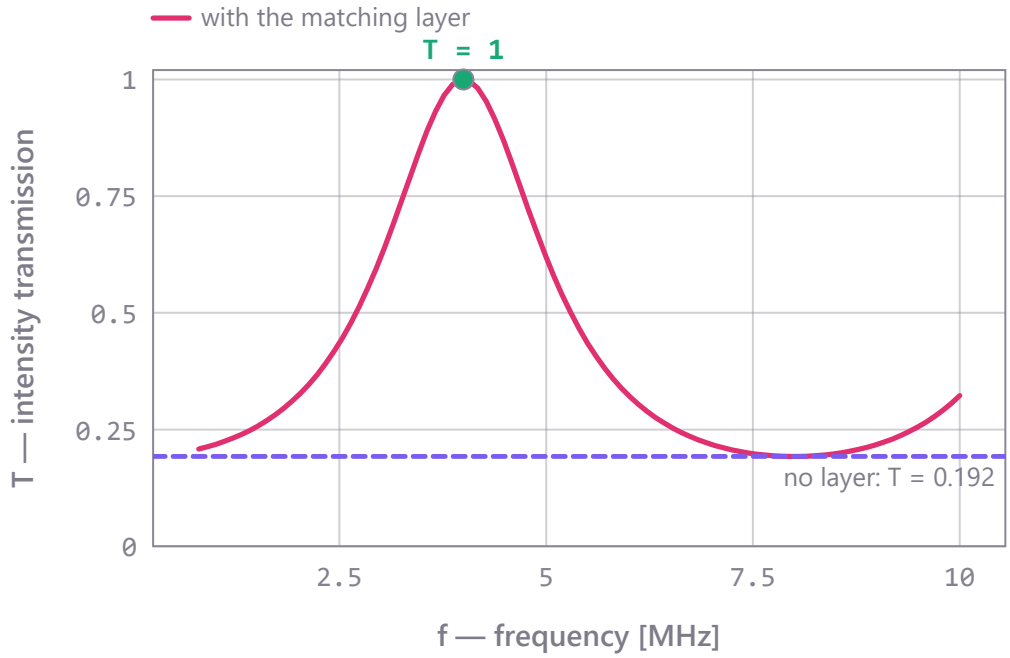
The intensity transmission coefficient from a transducer (impedance Z1) through a matching layer (impedance Zm, thickness d, sound speed c_m) into tissue (Z2) at frequency f, and the layer's phase thickness.

$$T = \frac{4 Z_1 Z_2}{(Z_1 + Z_2)^2 \cos^2 kd + \left(Z_m + \frac{Z_1 Z_2}{Z_m}\right)^2 \sin^2 kd}$$

| input | meaning | condition |
|---|---|---|
| `Z1` | impedance of the source medium (transducer) | |
| `Zm` | impedance of the matching layer | |
| `Z2` | impedance of the load medium (tissue) | |
| `d` | layer thickness | |
| `c_m` | sound speed in the layer | |
| `f` | frequency | |

| output | meaning |
|---|---|
| `T` | intensity transmission coefficient |
| `kd` | phase thickness of the layer (rad) |

- **Method:** $T = 4$ Z1 Z2 / [$(Z_1 + Z_2)^2$ $cos^2$(kd) + ($Zm + Z_1$ Z2/Zm)^2 $sin^2$(kd)], $k = 2$ pi f / c_m
- **Assumes:** normal incidence, plane waves, lossless layers; semi-infinite source and load media
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Transmission from a transducer (Z1 = 30 MRayl) into tissue (Z2 = 1.6 MRayl) through a matching layer (Zm = 6.9 MRayl, d = 0.15 mm, c = 2400 m/s) against frequency, 91 runs; the line is the package's direct-interface transmission without a layer. Marked: example f = 4 MHz, T = 0.999983, phase thickness kd = 1.5708 rad (a quarter wave is π/2).

```
biomedical_science_ai999x.solve('bmed_ultrasound_matching_layer', Z1=30000000.0, Zm=6900000.0,
Z2=1600000.0, d=0.00015, c_m=2400.0, f=4000000.0)
# -> T = 0.999983, kd = 1.5708   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_ultrasound_matching_layer', Z1=30000000.0, Zm=3000000.0,
Z2=1500000.0, d=0.0002, c_m=2000.0, f=2000000.0)
# -> T = 0.464142, kd = 1.25664   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Ultrasound matching layer: Z1 = 30 MRayl, Zm = 6.9 MRayl, Z2 = 1.6 MRayl, d = 0.15 mm, c_m = 2400 m/s, f = 4 MHz | SOLVED | T = 0.999983, kd = 1.5708 |
| Ultrasound matching layer: Z1 = 30 MRayl, Zm = 3 MRayl, Z2 = 1.5 MRayl, d = 0.2 mm, c_m = 2000 m/s, f = 2 MHz | SOLVED | T = 0.464142, kd = 1.25664 |
| Ultrasound matching layer: Zm = 6.9 MRayl, Z2 = 1.6 MRayl, d = 0.15 mm, c_m = 2400 m/s, f = 4 MHz | NEEDS_INPUT | bmed_ultrasound_matching_layer needs Z1 (impedance of the source medium (transducer)). Example: solve('bmed_ultrasound_m |
| Ultrasound matching layer: Z1 = 30 MRayl, Zm = 6.9 MRayl, Z2 = 1.6 MRayl, d = 0.15 mm, c_m = 2400 m/s, f = 4 MHz, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### Imaging resolution: MTF of a Gaussian point-spread function — `bmed_gaussian_psf_mtf`

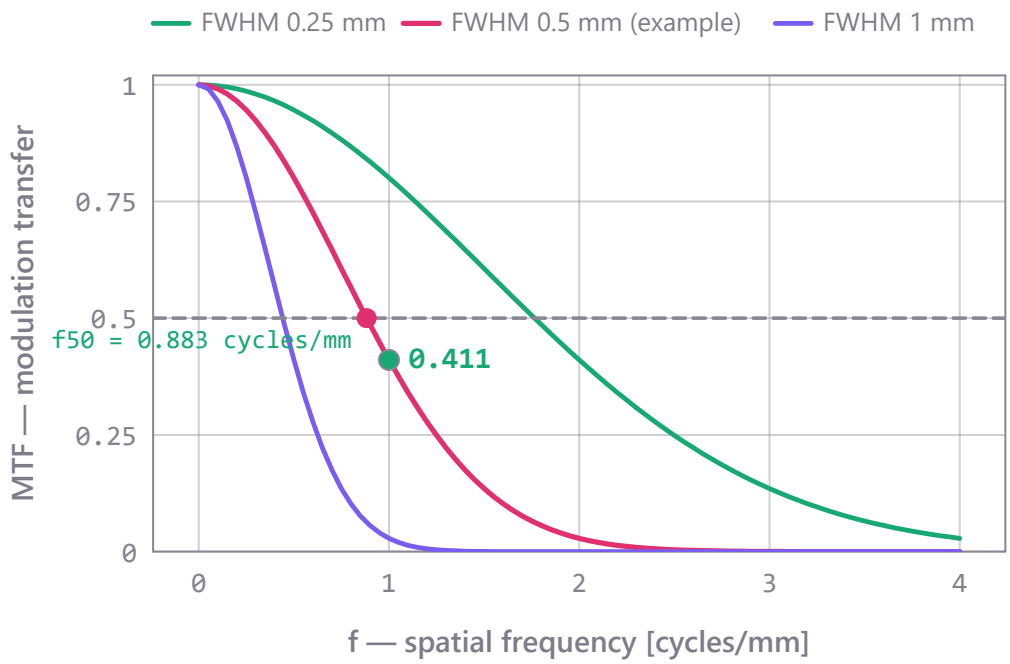
The modulation transfer function at a spatial frequency of an imaging system with a Gaussian point-spread function of stated FWHM, and the frequency at which the MTF falls to 50 %.

$$\mathrm{MTF}(f) = e^{-2\pi^2\sigma^2 f^2}, \quad \sigma = \frac{\mathrm{FWHM}}{2\sqrt{2\ln 2}}$$

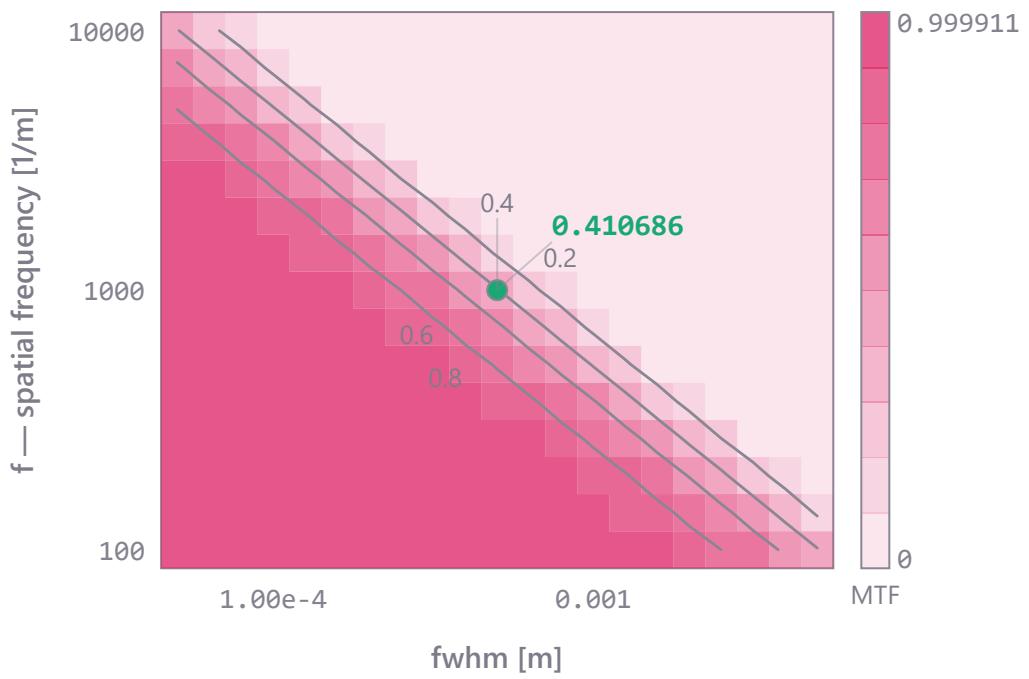
| input | meaning | condition |
|---|---|---|
| `fwhm` | full width at half maximum of the point-spread function | |
| `f` | spatial frequency (cycles per length) | |

| output | meaning |
|---|---|
| `MTF` | modulation transfer at f |
| `f50` | spatial frequency where the MTF is 0.5 |

- **Method:** sigma = FWHM/(2 sqrt(2 ln 2)); MTF = exp(-2 $pi^2$ $\sigma^2$ f^2); $f50 = sqrt(\ln 2/2)/(pi$ sigma)
- **Assumes:** Gaussian, shift-invariant point-spread function; one dimension (a line-spread function)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Modulation transfer function of a Gaussian point-spread function for three widths (FWHM), 81 runs each up to 4 cycles/mm: a sharper system keeps contrast to higher frequencies. Marked: the example FWHM 0.5 mm at f = 1 cycles/mm, MTF = 0.410686, and its 50 % frequency f50 = 0.882542 cycles/mm.




MTF (modulation transfer at f) over fwhm (full width at half maximum of the point-spread function) from 5.0000e-05 m to 0.005 m and f (spatial frequency (cycles per length)) from 100 1/m to 10000 1/m (logarithmic axes): each of the 21 × 15 cells is one run of biomedical-science-ai999x, shaded by its value (light 0 to dark 0.999911); iso-lines at MTF = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, fwhm = 0.0005 m, f = 1000 1/m, MTF = 0.410686.

```
biomedical_science_ai999x.solve('bmed_gaussian_psf_mtf', fwhm=0.0005, f=1000.0)
# -> MTF = 0.410686, f50 = 882.542 1/m   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_gaussian_psf_mtf', fwhm=0.004, f=100.0)
# -> MTF = 0.565778, f50 = 110.318 1/m   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Gaussian PSF MTF: fwhm = 0.5 mm, f = 1 1/mm | SOLVED | MTF = 0.410686, f50 = 882.542 1/m |
| Gaussian PSF MTF: fwhm = 4 mm, f = 0.1 1/mm | SOLVED | MTF = 0.565778, f50 = 110.318 1/m |
| Gaussian PSF MTF: f = 1 1/mm | NEEDS_INPUT | bmed_gaussian_psf_mtf needs fwhm (full width at half maximum of the point-spread function). Example: solve('bmed_gaussia |
| Gaussian PSF MTF: fwhm = 0.5 mm, f = 1 1/mm, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Diffusion MRI: apparent diffusion coefficient from signals at several b-values — `bmed_mri_adc_fit`

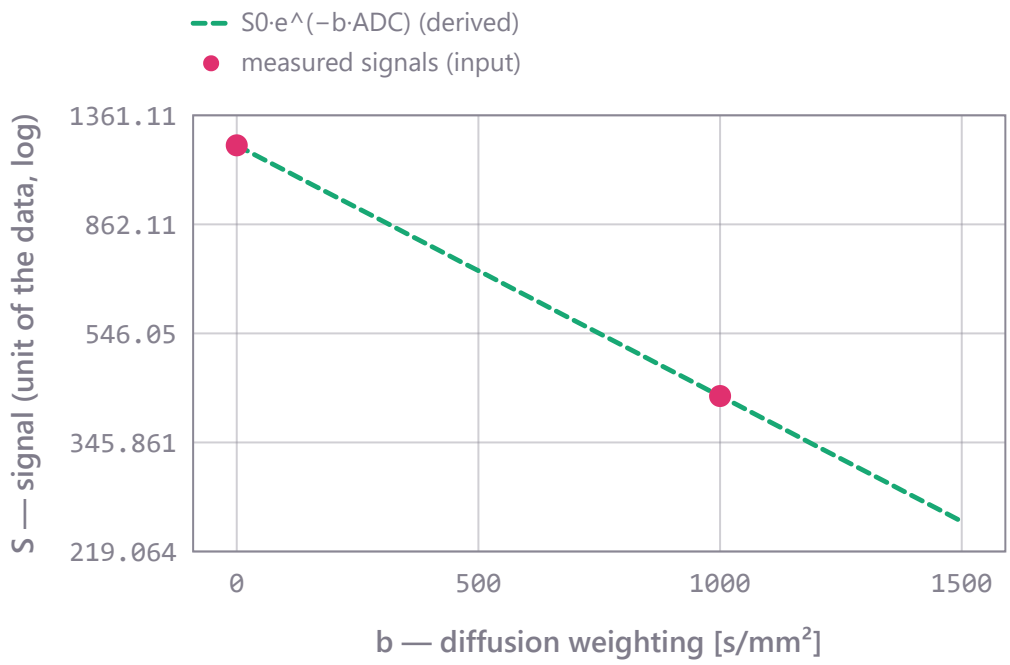
The apparent diffusion coefficient and the $b = 0$ signal from diffusion-weighted signals at two or more b-values, by a least-squares fit of ln S against b (mono-exponential decay).

$$\ln S = \ln S_0 - b \cdot ADC$$

| input | meaning | condition |
|---|---|---|
| b | b-values in $s/mm^2$ | |
| S | signals at those b-values (any one unit) | |

| output | meaning |
|---|---|
| ADC | apparent diffusion coefficient, $mm^2/s$ |
| S0 | fitted signal at $b = 0$ (unit of S) |

- **Method:** least-squares line ln $S = ln$ S0 - ADC b
- **Assumes:** mono-exponential decay $S = S_0$ exp(-b ADC); unweighted least squares on ln S; b in $s/mm^2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Diffusion-weighted signals of the example at b = 0, 1000 s/mm² (input, log axis) and the mono-exponential decay drawn from the returned ADC = 0.00104982 mm²/s and S0 = 1200 (derived), extended to b = 1500.

```
biomedical_science_ai999x.solve('bmed_mri_adc_fit', b=[0, 1000], S=[1200, 420])
# -> ADC = 0.00104982, S0 = 1200   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_adc_fit', b=[0, 200, 500, 1000], S=[980, 820, 620, 390])
# -> ADC = 0.000923418, S0 = 983.024   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| ADC fit: b = [0, 1000], S = [1200, 420] | SOLVED | ADC = 0.00104982, S0 = 1200 |
| ADC fit: b = [0, 200, 500, 1000], S = [980, 820, 620, 390] | SOLVED | ADC = 0.000923418, S0 = 983.024 |

| question | outcome | result |
|---|---|---|
| ADC fit: S = [1200, 420] | NEEDS_INPUT | bmed_mri_adc_fit needs b (b-values in s/mm^2). Example: solve('bmed_mri_adc_fit', b=[0, 1000], S=[1200, 420]). |
| ADC fit: b = [0, 1000], S = [1200, 420], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

### MRI multi-echo T2 estimate (log-linear fit) — `bmed_mri_t2_fit`

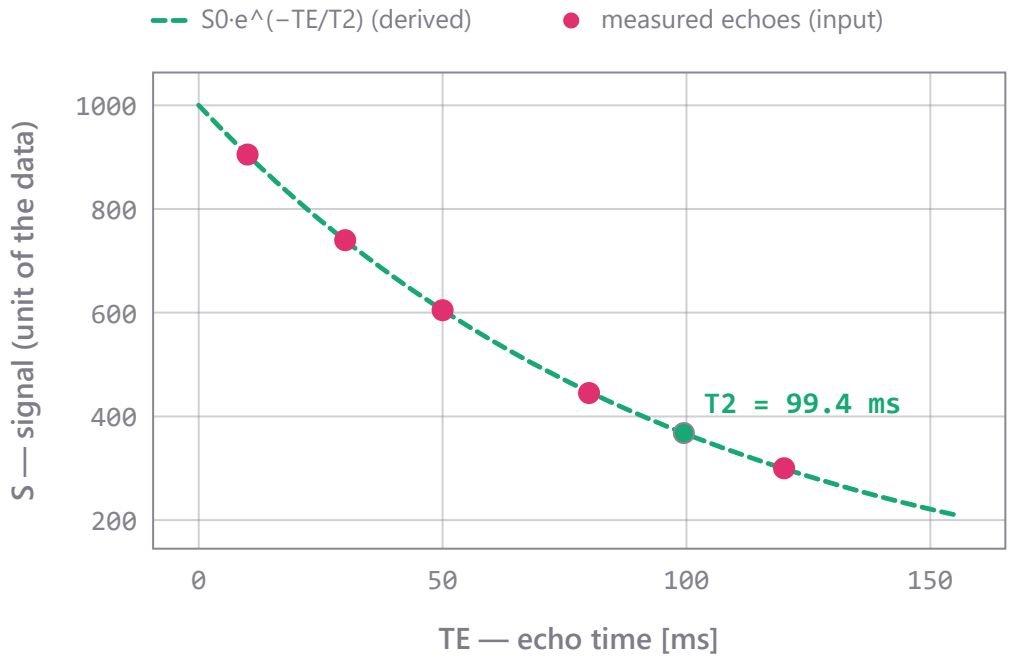
The transverse relaxation time T2 and the extrapolated signal at $TE = 0$ from signals at several echo times, by a least-squares fit of ln S against TE.

$$\ln S = \ln S_0 - \frac{TE}{T_2}$$

| input | meaning | condition |
|---|---|---|
| `TE` | echo times in ms | |
| `S` | signals at those echo times (any one unit) | |

| output | meaning |
|---|---|
| `T2` | transverse relaxation time, ms |
| `S0` | fitted signal at $TE = 0$ (unit of S) |

- **Method:** least-squares line ln $S = ln\, S_0 - TE/T_2$
- **Assumes:** mono-exponential decay $S = S_0$ exp(-TE/T2); unweighted least squares on ln S; TE in ms
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The example's 5 spin-echo signals (input) and the decay S0·exp(−TE/T2) drawn from the returned T2 = 99.4459 ms and S0 = 999.81 (derived). Marked: at TE = T2 the signal is S0/e.

```
biomedical_science_ai999x.solve('bmed_mri_t2_fit', TE=[10, 30, 50, 80, 120], S=[905, 740, 605, 445, 300])
# -> T2 = 99.4459, S0 = 999.81   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_t2_fit', TE=[20, 100], S=[800, 260])
# -> T2 = 71.1788, S0 = 1059.54   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Multi-echo T2 fit: TE = [10, 30, 50, 80, 120], S = [905, 740, 605, 445, 300] | SOLVED | T2 = 99.4459, S0 = 999.81 |
| Multi-echo T2 fit: TE = [20, 100], S = [800, 260] | SOLVED | T2 = 71.1788, S0 = 1059.54 |
| Multi-echo T2 fit: S = [905, 740, 605, 445, 300] | NEEDS_INPUT | bmed_mri_t2_fit needs TE (echo times in ms). Example: solve('bmed_mri_t2_fit', TE=[10, 30, 50, 80, 120], S=[905, 740, 60 |
| Multi-echo T2 fit: TE = [10, 30, 50, 80, 120], S = [905, 740, 605, 445, 300], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |

## Signal-to-noise ratio gained by signal averaging — `bmed_snr_averaging`

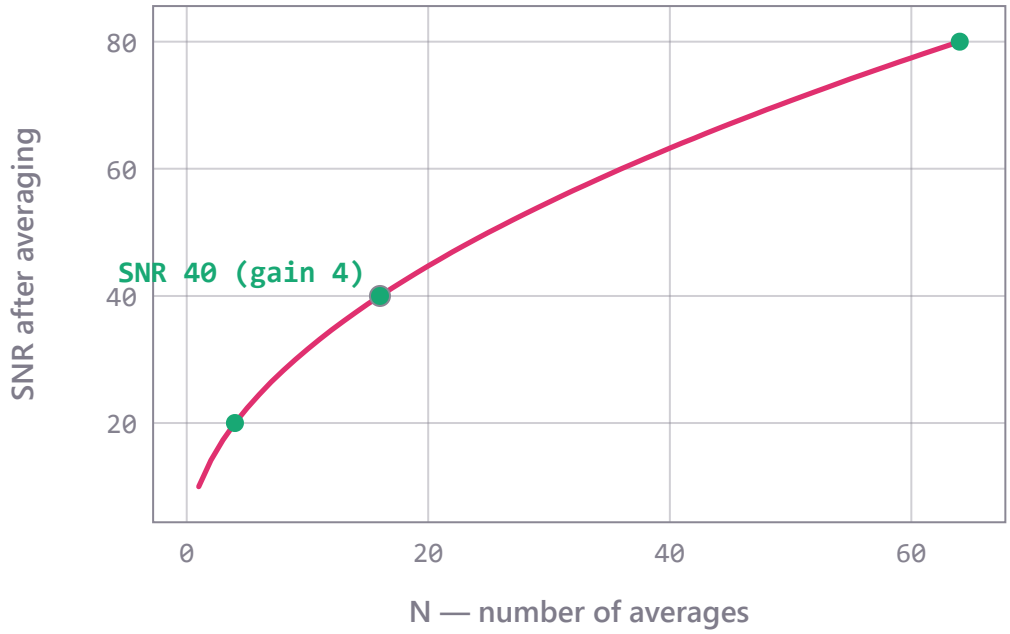
The signal-to-noise ratio after averaging N acquisitions with independent noise, SNR_N = SNR_1 sqrt(N), or the smallest number of averages that reaches a target SNR.

$$SNR_N = SNR_1\sqrt{N}$$

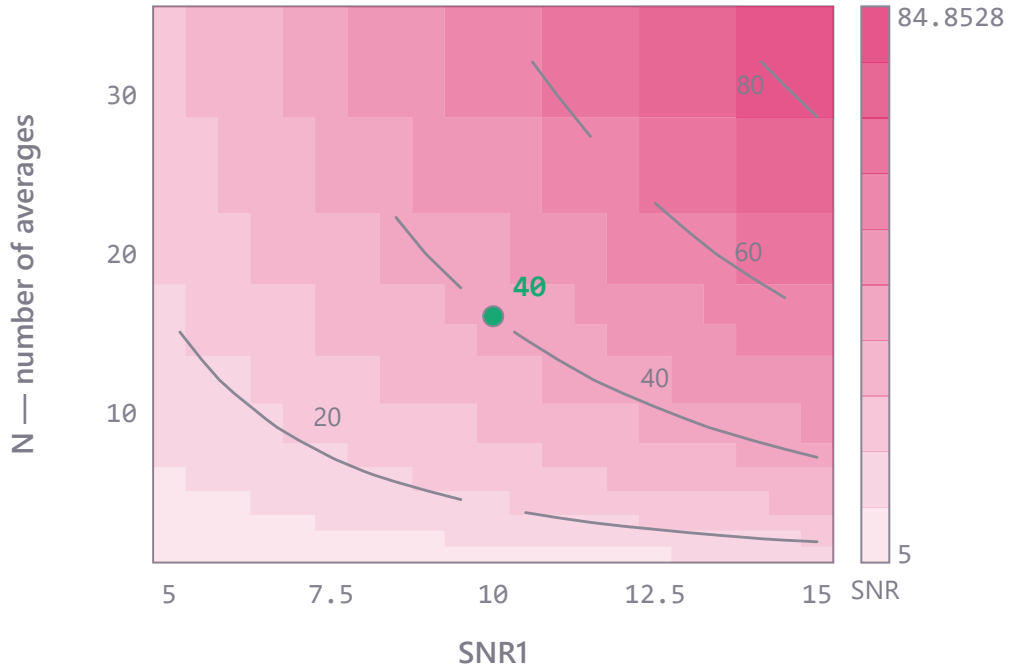
| input | meaning | condition |
|---|---|---|
| `SNR1` | signal-to-noise ratio of one acquisition | |
| `N` | number of averages (give N or SNR_target) | |
| `SNR_target` | target SNR (give N or SNR_target) | |

| output | meaning |
|---|---|
| `SNR` | SNR after averaging |
| `gain` | SNR gain sqrt(N) |
| `N` | number of averages |

- **Method:** SNR_N = SNR_1 sqrt(N); N needed = ceil((SNR_target/SNR_1)^2)
- **Assumes:** independent, zero-mean noise of equal variance in every acquisition; an identical signal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Signal-to-noise ratio after averaging N acquisitions of single-acquisition SNR 10, one run per N from 1 to 64: it grows with √N, so 4, 16 and 64 averages (dots) double it each time. Marked: the example N = 16, SNR = 40, gain 4.




SNR (SNR after averaging) over SNR1 (signal-to-noise ratio of one acquisition) from 5 to 15 and N (number of averages (give N or SNR_target)) from 1 to 32: each of the 21 × 13 cells is one run of biomedical-science-ai999x, shaded by its value (light 5 to dark 84.8528); iso-lines at SNR = 20, 40, 60, 80. Marked: the worked example, SNR1 = 10, N = 16, SNR = 40.

```
biomedical_science_ai999x.solve('bmed_snr_averaging', SNR1=10, N=16)
# -> SNR = 40, gain = 4, N = 16   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_snr_averaging', SNR1=5, SNR_target=20)
# -> SNR = 20, gain = 4, N = 16   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SNR averaging: SNR1 = 10, N = 16 | SOLVED | SNR = 40, gain = 4, N = 16 |
| SNR averaging: SNR1 = 5, SNR_target = 20 | SOLVED | SNR = 20, gain = 4, N = 16 |
| SNR averaging: N = 16 | NEEDS_INPUT | bmed_snr_averaging needs SNR1 (signal-to-noise ratio of one acquisition). Example: solve('bmed_snr_averaging', SNR1=10, |
| SNR averaging: SNR1 = 10, N = 16, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| SNR averaging: SNR1 = 10, N = 16, SNR_target = 50 | INVALID_INPUT | Signal-to-noise ratio gained by signal averaging: give the number of averages N or a target SNR (one of them). |

### MRI Ernst angle (the flip angle of the largest spoiled gradient-echo signal) — `bmed_mri_ernst_angle`

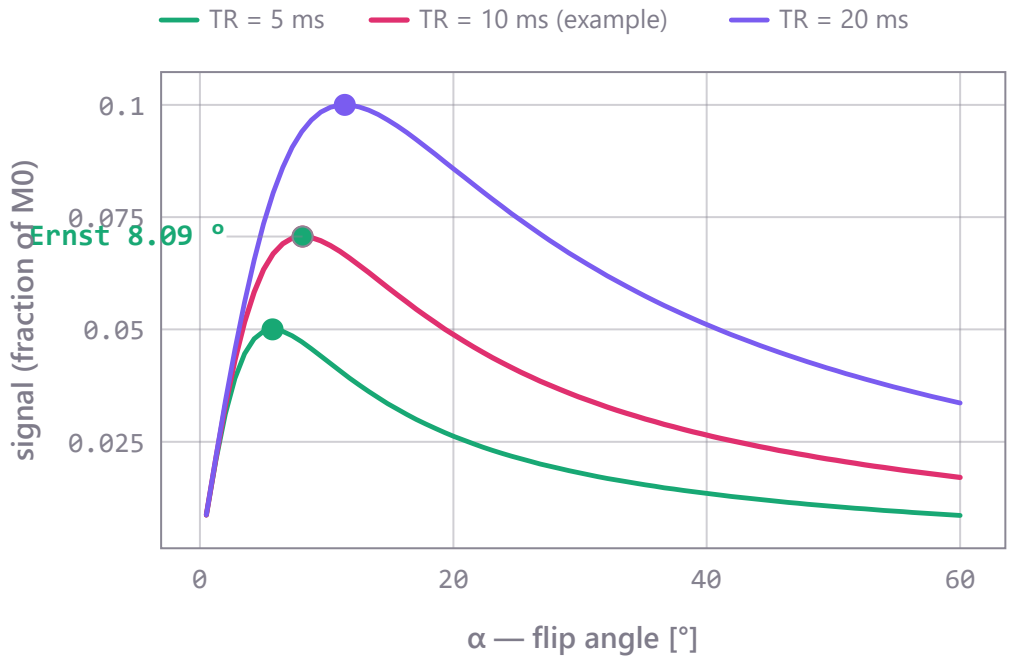
The Ernst angle alpha_E = arccos(exp(-TR/T1)) that maximises the steady-state signal of a spoiled gradient-echo sequence, and that largest signal as a fraction of M0 (T2* decay excluded).

$$\alpha_E = \arccos\left(e^{-TR/T_1}\right)$$

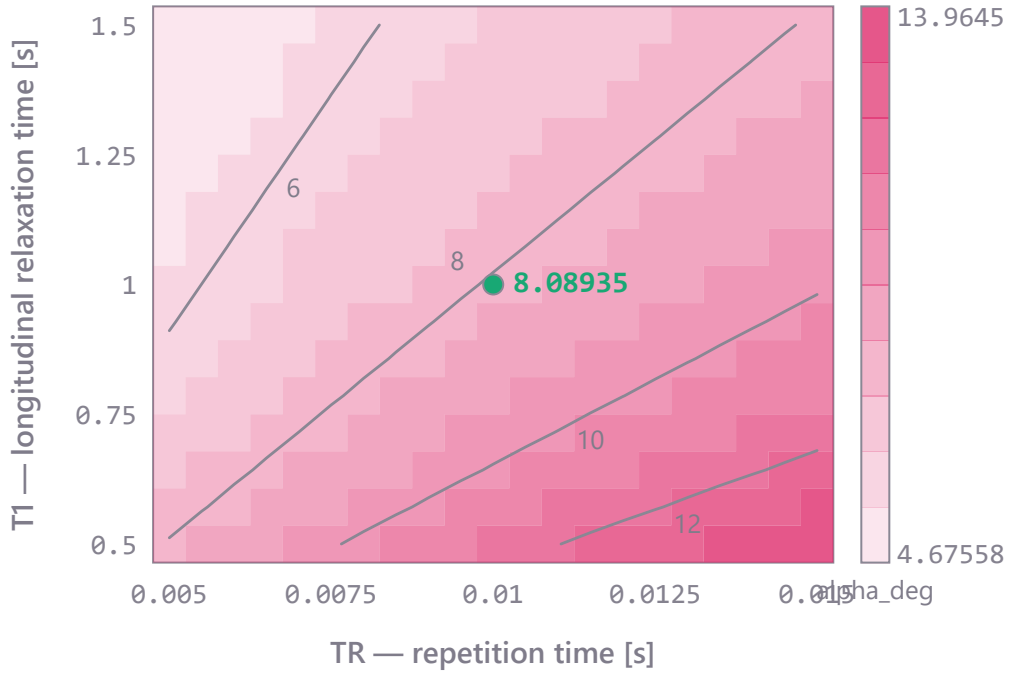
| input | meaning | condition |
|---|---|---|
| `TR` | repetition time | |
| `T1` | longitudinal relaxation time | |

| output | meaning |
|---|---|
| `alpha` | Ernst angle |
| `alpha_deg` | Ernst angle in degrees |
| `S_rel` | signal at the Ernst angle, fraction of M0 (no T2* decay) |

- **Method:** cos(alpha_E) = exp(-TR/T1); $S = sin$(alpha)(1 - E1)/(1 - E1 cos alpha)
- **Assumes:** ideal spoiling, steady state; TR, $T_1 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Spoiled gradient-echo signal against flip angle (T1 = 1000 ms, three repetition times, no T2* decay: TE = 1 µs, T2* = 1000 s), 80 runs each from the package's spoiled gradient echo; dots: the Ernst angle and its signal returned by this capability for each TR, at the top of each curve. Marked: the example TR = 10 ms, Ernst angle 8.08935°, signal 0.0707104 M0.




alpha_deg (Ernst angle in degrees) over TR (repetition time) from 0.005 s to 0.015 s and T1 (longitudinal relaxation time) from 0.5 s to 1.5 s: each of the 21 × 15 cells is one run of biomedical-science-ai999x, shaded by its value (light 4.67558 to dark 13.9645); iso-lines at alpha_deg = 6, 8, 10, 12. Marked: the worked example, TR = 0.01 s, T1 = 1 s, alpha_deg = 8.08935.

```
biomedical_science_ai999x.solve('bmed_mri_ernst_angle', TR=0.01, T1=1.0)
# -> alpha = 0.141186 rad, alpha_deg = 8.08935, S_rel = 0.0707104   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_mri_ernst_angle', TR=0.5, T1=0.9)
# -> alpha = 0.959715 rad, alpha_deg = 54.9876, S_rel = 0.52043   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Ernst angle: TR = 10 ms, T1 = 1000 ms | SOLVED | alpha = 0.141186 rad, alpha_deg = 8.08935, S_rel = 0.0707104 |
| Ernst angle: TR = 500 ms, T1 = 900 ms | SOLVED | alpha = 0.959715 rad, alpha_deg = 54.9876, S_rel = 0.52043 |
| Ernst angle: T1 = 1000 ms | NEEDS_INPUT | bmed_mri_ernst_angle needs TR (repetition time). Example: solve('bmed_mri_ernst_angle', TR='0.01 s', T1='1 s'). |
| Ernst angle: TR = 10 ms, T1 = 1000 ms, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Ernst angle: TR = -10 ms, T1 = 1000 ms | INVALID_INPUT | MRI Ernst angle (the flip angle of the largest spoiled gradient-echo signal): TR and T1 must be a positive number. |

### SEIR epidemic model: peak prevalence and the time of the peak — `bmed_epi_seir_peak`

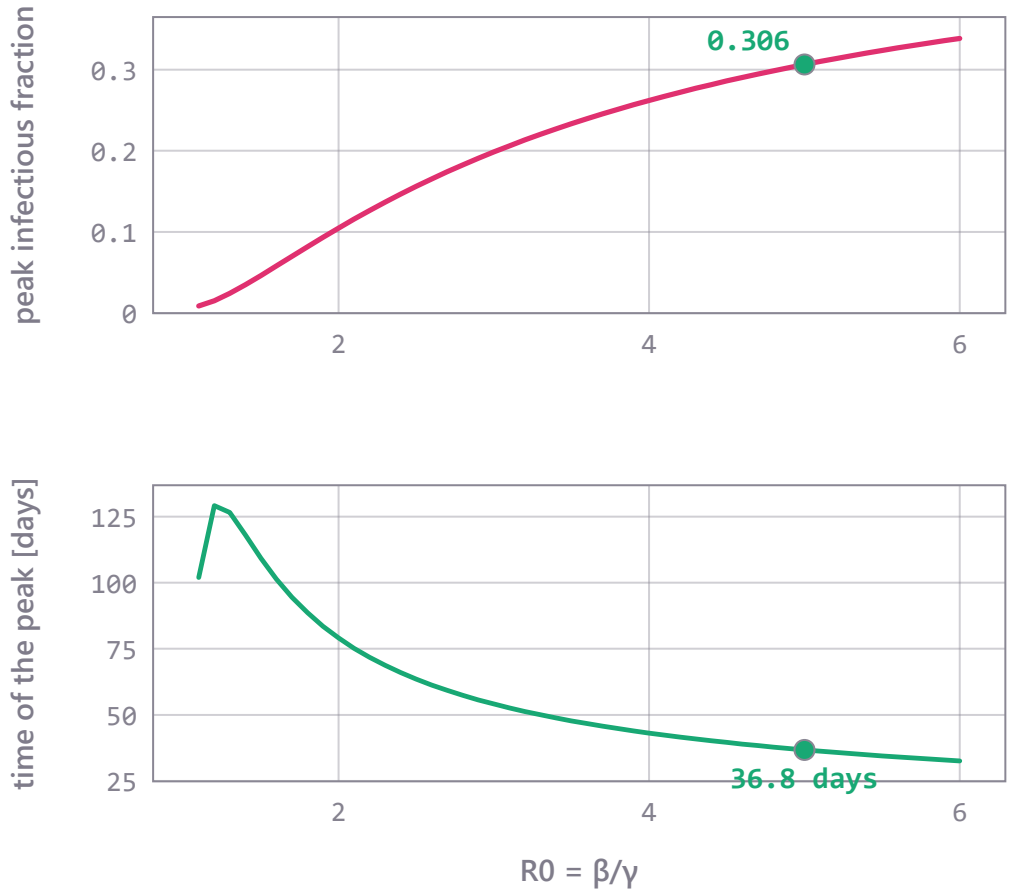
The largest infectious fraction of an SEIR epidemic and the time at which it occurs, from the transmission, incubation and recovery rates and the initial fractions.

$$\dot{s} = -\beta s i,\ \dot{e} = \beta s i - \sigma e,\ \dot{i} = \sigma e - \gamma i; \quad t_{peak} :\ \sigma e = \gamma i$$

| input | meaning | condition |
|---|---|---|
| `beta` | transmission rate | |
| `sigma` | rate of leaving the latent stage (1/latent period) | |
| `gamma` | recovery rate | |
| `s0` | initial susceptible fraction | |
| `e0` | initial exposed (latent) fraction | |
| `i0` | initial infectious fraction | |

| output | meaning |
|---|---|
| `t_peak` | time of the peak of the infectious fraction |
| `i_max` | peak infectious fraction |
| `s_at_peak` | susceptible fraction at the peak |
| `e_at_peak` | exposed fraction at the peak |

- **Method:** Taylor-series integration (order 26, steps set by the series' tail) of the SEIR system to the first sign change of sigma e - gamma i, the root located on the series
- **Assumes:** closed population, homogeneous mixing, frequency-dependent transmission; R0 $s_0 > 1$, $e_0 + i_0 > 0$; the peak is the first maximum of i after $t = 0$ and exceeds i0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Peak of an SEIR epidemic against R0 = β/γ (β varied, infectious period 10 days, latent period 5 days, s0 = 0.99, e0 = 0.008, i0 = 0.002), 50 runs from 1.1 to 6: a higher R0 gives a higher, earlier peak. Marked: the example β = 0.5 1/day (R0 = 5), peak fraction 0.30625 at 36.7734 days.

```
biomedical_science_ai999x.solve('bmed_epi_seir_peak', beta=5.787037037037037e-06,
sigma=2.3148148148148148e-06, gamma=1.1574074074074074e-06, s0=0.99, e0=0.008, i0=0.002)
# -> t_peak = 3.17722e+06 s, i_max = 0.30625, s_at_peak = 0.122176, e_at_peak = 0.153125   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_epi_seir_peak', beta=3.472222222222222e-06,
sigma=5.787037037037037e-06, gamma=1.736111111111111e-06, s0=0.95, e0=0.01, i0=0.005)
# -> t_peak = 3.52552e+06 s, i_max = 0.110337, s_at_peak = 0.475211, e_at_peak = 0.033101
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| SEIR peak: beta = 0.5 1/day, sigma = 0.2 1/day, gamma = 0.1 1/day, s0 = 0.99, e0 = 0.008, i0 = 0.002 | SOLVED | t_peak = 3.17722e+06 s, i_max = 0.30625, s_at_peak = 0.122176, e_at_peak = 0.153125 |
| SEIR peak: beta = 0.3 1/day, sigma = 0.5 1/day, gamma = 0.15 1/day, s0 = 0.95, e0 = 0.01, i0 = 0.005 | SOLVED | t_peak = 3.52552e+06 s, i_max = 0.110337, s_at_peak = 0.475211, e_at_peak = 0.033101 |
| SEIR peak: sigma = 0.2 1/day, gamma = 0.1 1/day, s0 = 0.99, e0 = 0.008, i0 = 0.002 | NEEDS_INPUT | bmed_epi_seir_peak needs beta (transmission rate). Example: solve('bmed_epi_seir_peak', beta='5.78704e-06 1/s', sigma='2 |
| SEIR peak: beta = 0.5 1/day, sigma = 0.2 1/day, gamma = 0.1 1/day, s0 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: |

| question | outcome | result |
|---|---|---|
| = 0.99, e0 = 0.008, i0 = 0.002, zz = 1 | | biomedical_science_ai999x.templat |
| SEIR peak: beta = 0.1 1/day, sigma = 0.2 1/day, gamma = 0.2 1/day, s0 = 0.99, e0 = 0.008, i0 = 0.002 | INVALID_INPUT | SEIR epidemic model: peak prevalence and the time of the peak: no epidemic peak: R0 s0 must exceed 1. |

### Body mass index (a calculation, not medical advice) — `bmed_bmi`

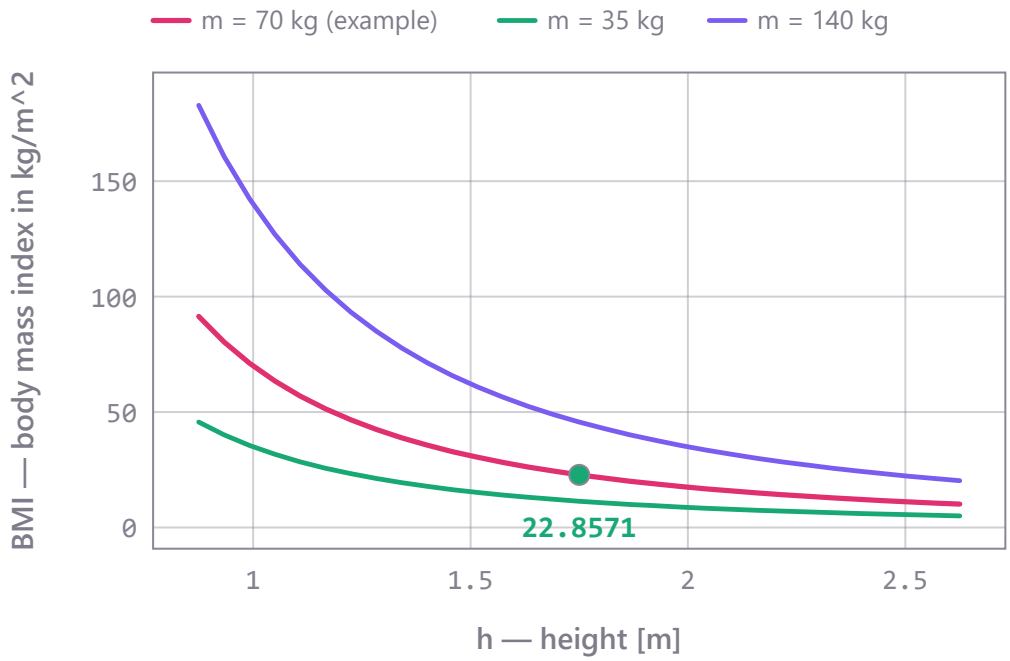
The body mass index BMI = m/h^2 $(kg/m^2)$ and BMI prime (BMI/25) as a calculation from mass and height; no category or health advice is given.

$$\mathrm{BMI} = \frac{m}{h^2}$$

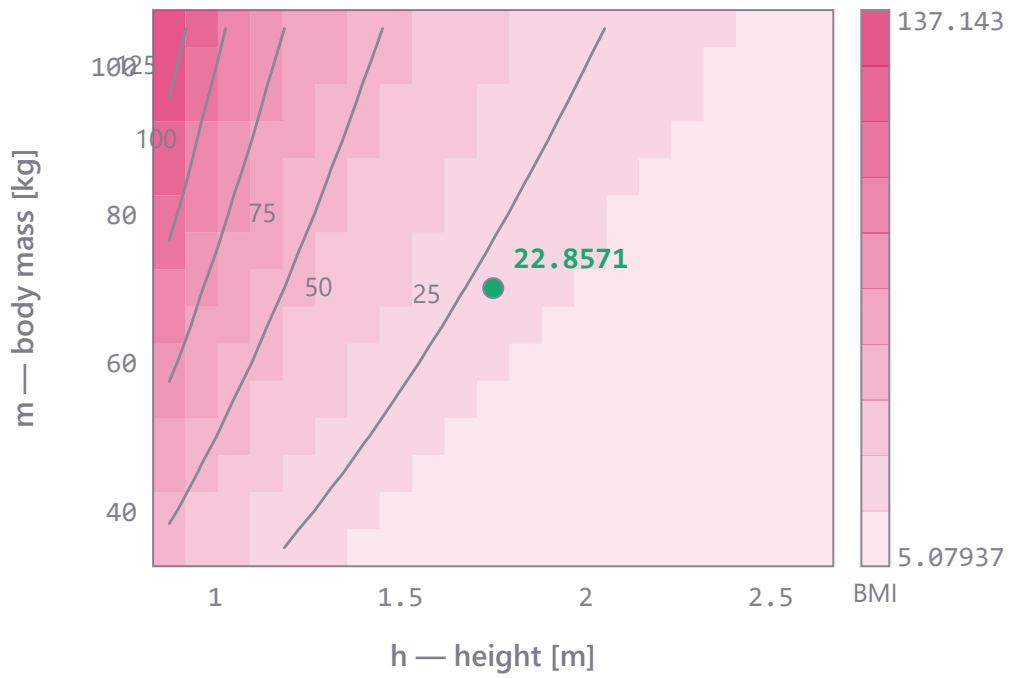
| input | meaning | condition |
|---|---|---|
| `m` | body mass | |
| `h` | height | |

| output | meaning |
|---|---|
| `BMI` | body mass index in $kg/m^2$ |
| `BMI_prime` | BMI/25 |

- **Method:** BMI = m/h^2
- **Assumes:** a calculation only: not medical advice; $1 \le m \le 700$ kg, $0.3 \le h \le 2.8$ m
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




BMI (body mass index in kg/m^2) against h (height) from 0.875 m to 2.625 m. One curve for each value of m (body mass): 70 kg, 35 kg, 140 kg. 31 points per curve computed by biomedical-science-ai999x; marked: the worked example, h = 1.75 m, BMI = 22.8571.




BMI (body mass index in kg/m^2) over h (height) from 0.875 m to 2.625 m and m (body mass) from 35 kg to 105 kg: each of the 21 × 15 cells is one run of biomedical-science-ai999x, shaded by its value (light 5.07937 to dark 137.143); iso-lines at BMI = 25, 50, 75, 100, 125. Marked: the worked example, h = 1.75 m, m = 70 kg, BMI = 22.8571.

```
biomedical_science_ai999x.solve('bmed_bmi', m=70.0, h=1.75)
# -> BMI = 22.8571, BMI_prime = 0.914286   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_bmi', m=90.0, h=1.8)
# -> BMI = 27.7778, BMI_prime = 1.11111   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Body mass index: m = 70 kg, h = 1.75 m | SOLVED | BMI = 22.8571, BMI_prime = 0.914286 |
| Body mass index: m = 90 kg, h = 180 cm | SOLVED | BMI = 27.7778, BMI_prime = 1.11111 |
| Body mass index: h = 1.75 m | NEEDS_INPUT | bmed_bmi needs m (body mass). Example: solve('bmed_bmi', m='70 kg', h='1.75 m'). |
| Body mass index: m = 70 kg, h = 1.75 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Body mass index: m = 70 kg, h = 5 m | INVALID_INPUT | Body mass index (a calculation, not medical advice): the height must lie between 0.3 and 2.8 m and the mass between 1 an |

## Basal metabolic rate by a stated regression (a calculation, not medical advice) — `bmed_bmr`

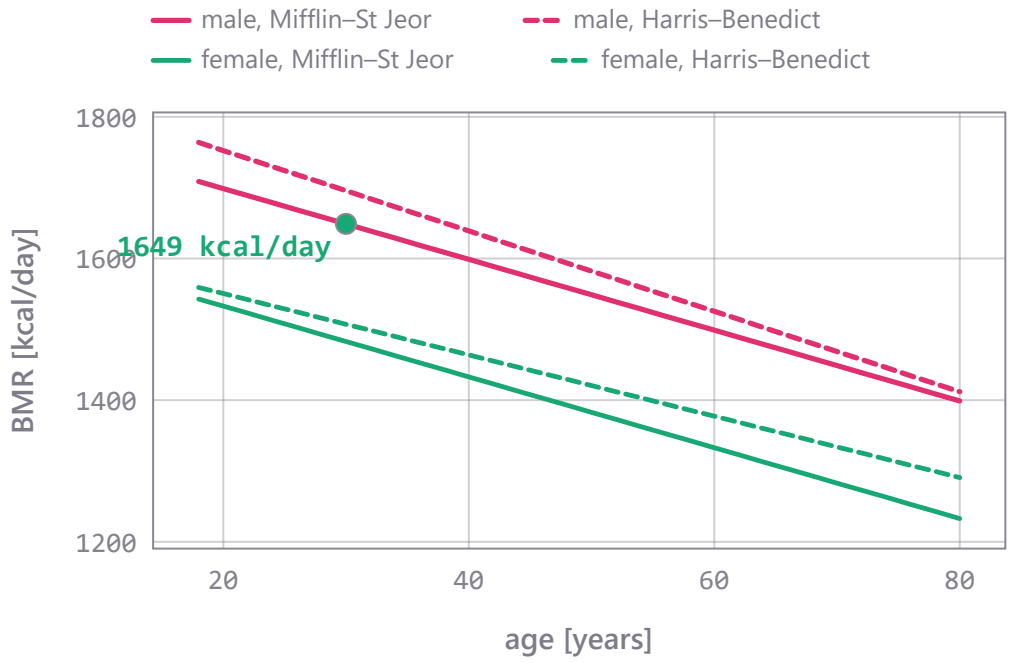
The basal metabolic rate estimated by the Mifflin-St Jeor (1990) or the revised Harris-Benedict (Roza and Shizgal 1984) regression from mass, height, age and sex, in kcal/day, W and MJ/day; a model calculation, not dietary or medical advice.

$$\mathrm{BMR} = 10\,m + 6.25\,h - 5\,a + s$$

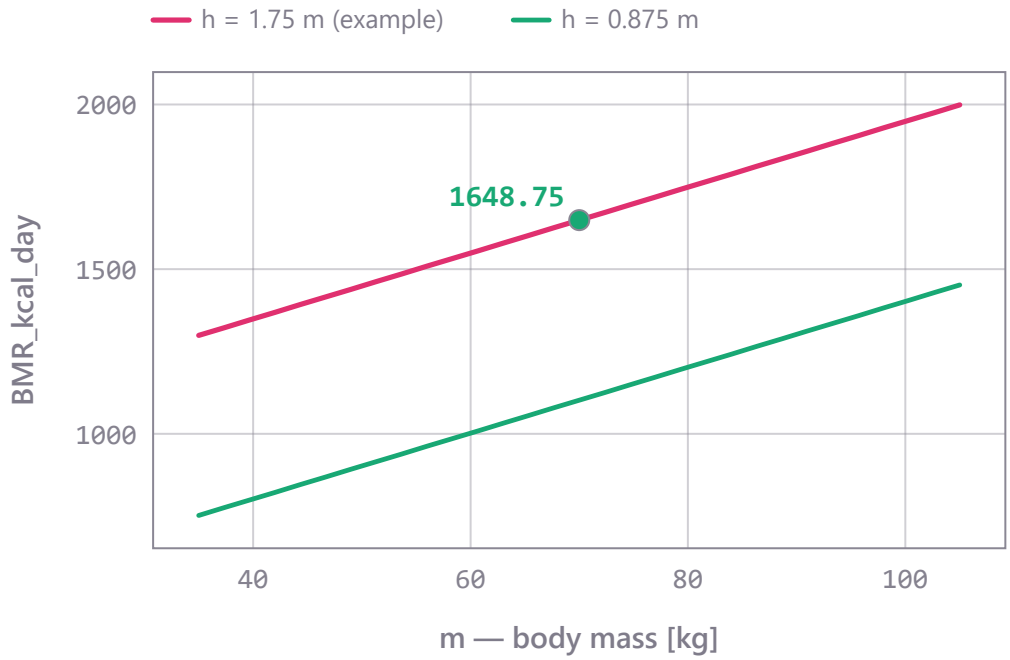
| input | meaning | condition |
|---|---|---|
| `m` | body mass | |
| `h` | height | |
| `age` | age in years (adults) | |
| `sex` | sex (the equations have a male and a female form) | |
| `equation` | regression: mifflin_st_jeor or harris_benedict_revised | |

| output | meaning |
|---|---|
| `BMR_kcal_day` | basal metabolic rate in kcal/day |
| `BMR_W` | basal metabolic rate (W) |
| `BMR_MJ_day` | basal metabolic rate in MJ/day |

- **Method:** Mifflin-St Jeor: 10 $m + 6.25\ h_c m - 5$ age + 5 (male) $or - 161$ (female); revised Harris-Benedict: $88.362 + 13.397\ m + 4.799\ h_c m - 5.677$ age (male), $447.593 + 9.247\ m + 3.098\ h_c m - 4.330$ age (female)
- **Assumes:** a population regression, not a measurement and not advice; adults 18 to 120 years; 1 kcal = 4.184 kJ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Basal metabolic rate against age for a body of m = 70 kg and h = 1.75 m, for both sexes and both regressions (32 ages each): it falls steadily with age and is lower for the female form. Marked: the example (male, age 30, Mifflin–St Jeor) BMR = 1648.75 kcal/day (79.8422 W).




BMR_kcal_day (basal metabolic rate in kcal/day) against m (body mass) from 35 kg to 105 kg. One curve for each value of h (height): 1.75 m, 0.875 m. Other inputs as in the worked example: age = 30, sex = male, equation = mifflin_st_jeor. 31 points per curve computed by biomedical-science-ai999x; marked: the worked example, m = 70 kg, BMR_kcal_day = 1648.75.

```
biomedical_science_ai999x.solve('bmed_bmr', m=70.0, h=1.75, age=30, sex='male',
equation='mifflin_st_jeor')
# -> BMR_kcal_day = 1648.75, BMR_W = 79.8422 W, BMR_MJ_day = 6.89837   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_bmr', m=60.0, h=1.6500000000000001, age=45, sex='female',
equation='harris_benedict_revised')
# -> BMR_kcal_day = 1318.73, BMR_W = 63.8609 W, BMR_MJ_day = 5.51758   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Basal metabolic rate: m = 70 kg, h = 175 cm, age = 30, sex = male, equation = mifflin_st_jeor | SOLVED | BMR_kcal_day = 1648.75, BMR_W = 79.8422 W, BMR_MJ_day = 6.89837 |
| Basal metabolic rate: m = 60 kg, h = 165 cm, age = 45, sex = female, equation = harris_benedict_revised | SOLVED | BMR_kcal_day = 1318.73, BMR_W = 63.8609 W, BMR_MJ_day = 5.51758 |
| Basal metabolic rate: h = 175 cm, age = 30, sex = male, equation = mifflin_st_jeor | NEEDS_INPUT | bmed_bmr needs m (body mass). Example: solve('bmed_bmr', m='70 kg', h='1.75 m', age=30, sex='male', equation='mifflin_st |
| Basal metabolic rate: m = 70 kg, h = 175 cm, age = 30, sex = male, equation = mifflin_st_jeor, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Basal metabolic rate: m = 70 kg, h = 175 cm, age = 10, sex = male, equation = mifflin_st_jeor | INVALID_INPUT | Basal metabolic rate by a stated regression (a calculation, not medical advice): the equations are stated for adults: ag |

### CT number in Hounsfield units — `bmed_hounsfield_units`

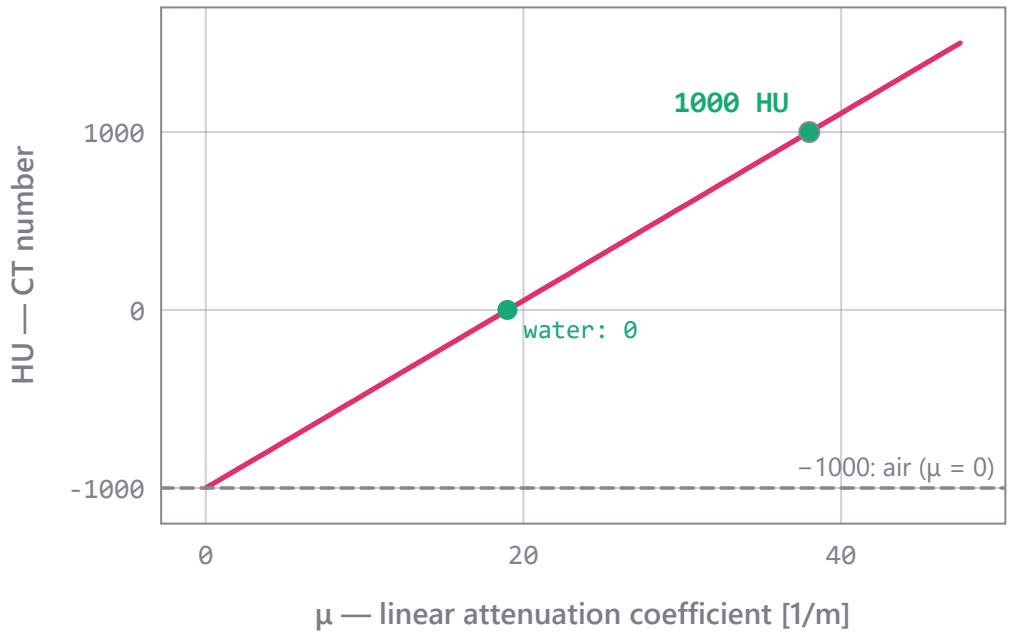
The CT number $HU = 1000$ (mu - mu_water)/(mu_water - mu_air) of a material from its linear attenuation coefficient and those of water and air (air 0 by default, then 1000 (mu - mu_water)/mu_water).

$$HU = 1000\,\frac{\mu - \mu_{water}}{\mu_{water} - \mu_{air}}$$

| input | meaning | condition |
|---|---|---|
| `mu` | linear attenuation coefficient of the material | |
| `mu_water` | linear attenuation coefficient of water | |
| `mu_air` | linear attenuation coefficient of air (default 0) | |

| output | meaning |
|---|---|
| `HU` | CT number in Hounsfield units |
| `mu_rel` | mu/mu_water |

- **Method:** $HU = 1000$ (mu - mu_water)/(mu_water - mu_air)
- **Assumes:** coefficients at the same effective energy; 0 <= mu_air < mu_water
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




The Hounsfield scale: CT number against linear attenuation coefficient for μ_water = 19 1/m (μ_air = 0), 51 runs from 0.2 % to 250 % of μ_water: a straight line that heads to −1000 HU as μ goes to zero (air, dashed: the scale's definition) and passes 0 HU at water (dot, the package's value). Marked: the example μ = 38 1/m, HU = 1000 (μ/μ_water = 2).

```
biomedical_science_ai999x.solve('bmed_hounsfield_units', mu=38.0, mu_water=19.0)
# -> HU = 1000, mu_rel = 2   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_hounsfield_units', mu=20.0, mu_water=19.0, mu_air=0.02)
# -> HU = 52.687, mu_rel = 1.05263   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Hounsfield units: mu = 0.38 1/cm, mu_water = 0.19 1/cm | SOLVED | HU = 1000, mu_rel = 2 |
| Hounsfield units: mu = 0.2 1/cm, mu_water = 0.19 1/cm, mu_air = 0.0002 1/cm | SOLVED | HU = 52.687, mu_rel = 1.05263 |
| Hounsfield units: mu_water = 0.19 1/cm | NEEDS_INPUT | bmed_hounsfield_units needs mu (linear attenuation coefficient of the material). Example: solve('bmed_hounsfield_units', |
| Hounsfield units: mu = 0.38 1/cm, mu_water = 0.19 1/cm, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Hounsfield units: mu = -0.38 1/cm, mu_water = 0.19 1/cm | INVALID_INPUT | CT number in Hounsfield units: the attenuation coefficients must be a positive number. |

## Larmor (precession) frequency in a magnetic field — `bmed_larmor_frequency`

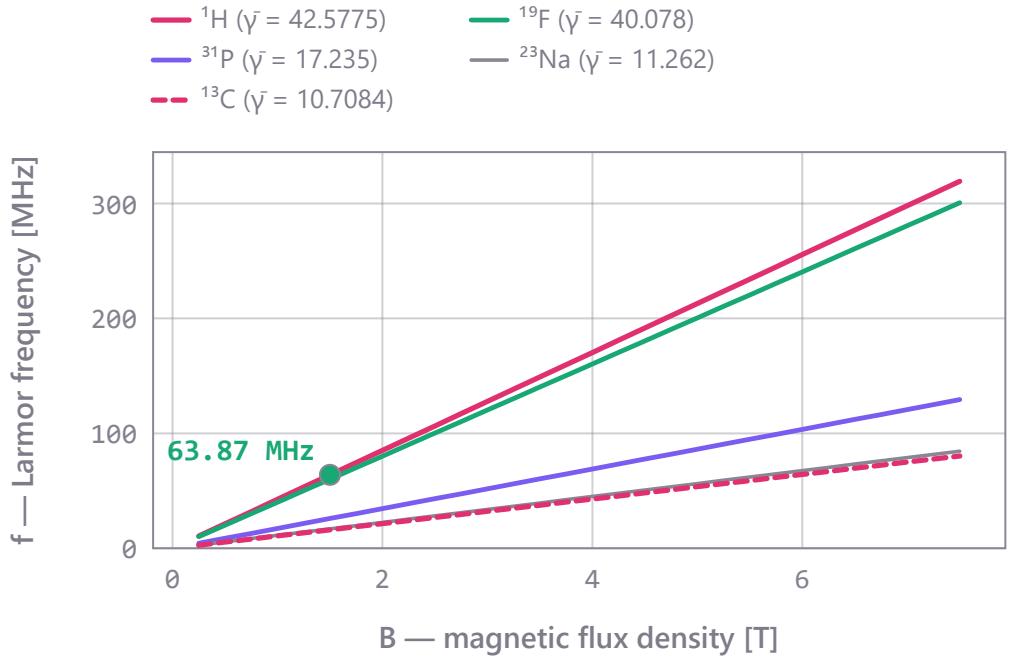
The Larmor frequency f = (gamma/2 pi) B of nuclear spins in a field B: protons by default (42.577478461 MHz/T, CODATA 2022), or another nucleus with its stated $\gamma/2\,\pi$ in MHz/T.

$$f = \frac{\gamma}{2\pi} B_0$$

| input | meaning | condition |
|---|---|---|
| `B` | magnetic flux density (T) | |
| `gamma_bar` | reduced gyromagnetic ratio $\gamma/2\,\pi$ in MHz/T (default: the proton) | |

| output | meaning |
|---|---|
| `f` | Larmor frequency |
| `f_MHz` | Larmor frequency in MHz |
| `omega` | angular Larmor frequency |

- **Method:** f = (gamma/2 pi) B
- **Assumes:** $B > 0$; bare nucleus (no chemical shielding)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




Larmor frequency against field strength (0–7.5 T, 31 runs per line) for five nuclei, each given by its reduced gyromagnetic ratio $\bar{\gamma}$ in MHz/T (stated inputs: $^{1}$H 42.577478, $^{19}$F 40.078, $^{31}$P 17.235, $^{23}$Na 11.262, $^{13}$C 10.7084). Marked: the example, protons at B = 1.5 T, f = 63.8662 MHz.

```
biomedical_science_ai999x.solve('bmed_larmor_frequency', B=1.5)
# -> f = 6.38662e+07 Hz, f_MHz = 63.8662, omega = 4.01283e+08 rad/s   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_larmor_frequency', B=3.0, gamma_bar=17.235)
# -> f = 5.1705e+07 Hz, f_MHz = 51.705, omega = 3.24872e+08 rad/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Larmor frequency: B = 1.5 T | SOLVED | f = 6.38662e+07 Hz, f_MHz = 63.8662, omega = 4.01283e+08 rad/s |
| Larmor frequency: B = 3 T, gamma_bar = 17.235 | SOLVED | f = 5.1705e+07 Hz, f_MHz = 51.705, omega = 3.24872e+08 rad/s |
| Larmor frequency: | NEEDS_INPUT | bmed_larmor_frequency needs B (magnetic flux density (T)). Example: solve('bmed_larmor_frequency', B='1.5 T'). |
| Larmor frequency: B = 1.5 T, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Larmor frequency: B = -3 T | INVALID_INPUT | Larmor (precession) frequency in a magnetic field: the magnetic field must be a positive number. |

## Ultrasound wavelength in tissue — `bmed_ultrasound_wavelength`

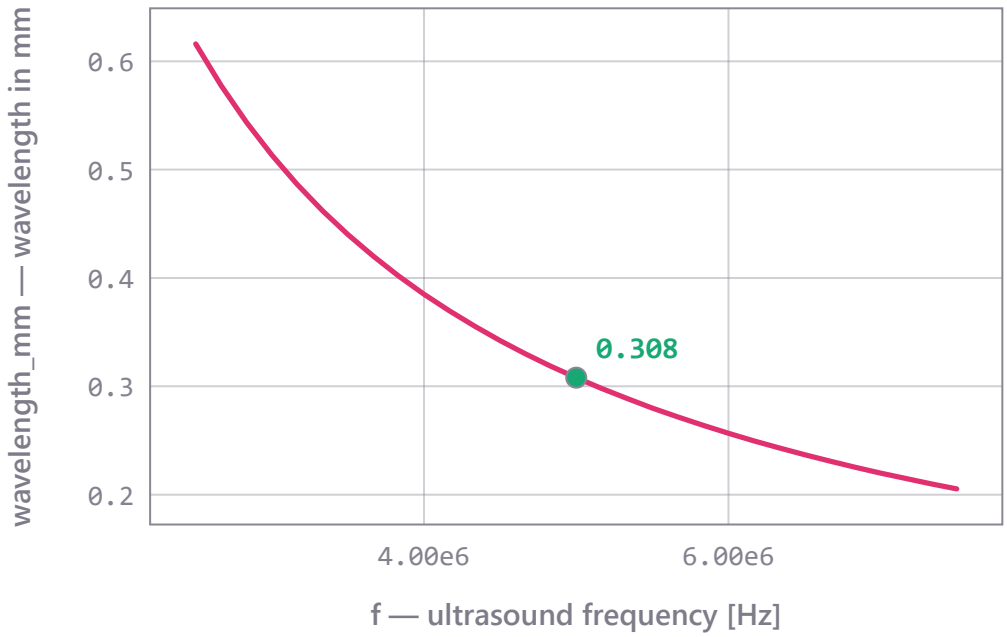
The wavelength $\lambda = c/f$ of ultrasound of frequency f in a medium with speed of sound c (1540 m/s, the soft-tissue convention, by default), with the period and wavenumber.

$$\lambda = \frac{c}{f}$$

| input | meaning | condition |
|---|---|---|
| `f` | ultrasound frequency | |
| `c` | speed of sound (default 1540 m/s) | |

| output | meaning |
|---|---|
| `wavelength` | wavelength |
| `wavelength_mm` | wavelength in mm |
| `period` | period $1/f$ |
| `k` | wavenumber 2 pi/lambda |

- **Method:** $\lambda = c/f$
- **Assumes:** f, $c > 0$; $c = 1540$ m/s unless stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




wavelength_mm (wavelength in mm) against f (ultrasound frequency) from 2.5e+06 Hz to 7.5e+06 Hz. 31 points computed by biomedical-science-ai999x; marked: the worked example, f = 5e+06 Hz, wavelength_mm = 0.308.

```
biomedical_science_ai999x.solve('bmed_ultrasound_wavelength', f=5000000.0)
# -> wavelength = 0.000308 m, wavelength_mm = 0.308, period = 2e-07 s, k = 20400 1/m   (SOLVED)
```

```
biomedical_science_ai999x.solve('bmed_ultrasound_wavelength', f=3500000.0, c=1580.0)
# -> wavelength = 0.000451429 m, wavelength_mm = 0.451429, period = 2.85714e-07 s, k = 13918.4 1/m
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Ultrasound wavelength: f = 5 MHz | SOLVED | wavelength = 0.000308 m, wavelength_mm = 0.308, period = 2e-07 s, k = 20400 1/m |

| question | outcome | result |
|---|---|---|
| Ultrasound wavelength: f = 3.5 MHz, c = 1580 m/s | SOLVED | wavelength = 0.000451429 m, wavelength_mm = 0.451429, period = 2.85714e-07 s, k = 13918.4 1/m |
| Ultrasound wavelength: | NEEDS_INPUT | bmed_ultrasound_wavelength needs f (ultrasound frequency). Example: solve('bmed_ultrasound_wavelength', f='5e+06 Hz'). |
| Ultrasound wavelength: f = 5 MHz, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: biomedical_science_ai999x.templat |
| Ultrasound wavelength: f = 0 MHz | INVALID_INPUT | Ultrasound wavelength in tissue: the frequency and the speed of sound must be a positive number. |